

Machine Learning Beyond the Cleveland Dataset:

Cross-cohort Coronary Disease Prediction Using Expanded Clinical Features

School: University of San Diego

Program: Master's Science in Applied Data Science

Course: ADS 503 Applied Predictive Modeling

Authors: Darren Chen, Jun Clemente, Graham Ward

Date: 6/23/2025

Table of Contents

Abstract.....	3
Introduction.....	4
Methodology.....	5
Data Acquisition and Description.....	5
Data Wrangling and Preprocessing.....	6
Exploratory Data Analysis.....	6
Data Splitting and Resampling Strategies.....	8
Model Development.....	8
Validation, Tuning, and Testing Strategies.....	10
Results.....	11
Conclusion.....	11
References.....	13
Appendix.....	14

Abstract

The goal of this study was to develop a predictive model for identifying coronary heart disease using patient data from four different medical centers around the globe. Leveraging a complete 76-feature heart disease data set from the UCI Machine Learning Repository, records from the Veterans Administration in Long Beach, the Hungarian Institute of Cardiology, the University Hospital in Zurich, and the Cleveland Clinic underwent merging, preprocessing, and then underwent rigorous modeling. A comprehensive exploratory data analysis (EDA), data cleaning process, and imputation procedures were performed to handle extensive missing values and features with high correlations to avoid impacting model performance and minimizing the amount of bias and variance the models produce. Multiple classification models were developed to include Logistic Regression, Random Forest, Partial Least Squares Discriminant Analysis (PLS-DA), K-Nearest Neighbors (KNN), Penalized Logistic Regression (Lasso, Ridge, and ElasticNet). These models were trained and evaluated using a 5-fold cross-validation resampling strategy and were compared in terms of performance based primarily on Receiver Operating Characteristics - Area Under the Curve (ROC-AUC), however other classification metrics, including Accuracy, Sensitivity, Specificity were taken into account in order to determine the best performing model. The results indicate the robust combined data set and statistical implementations are competitive in terms of performance to previous models and offer a benchmark for AUC-ROC performance as well. These findings highlight the critical implications to not only identify, but to predict, and take preemptive actions to mitigate coronary heart disease development.

Introduction

Coronary heart disease remains the leading global cause of death as of 2021 (WHO, 2024). This marks over three decades of heart disease being the global leading cause of death (Di Cesare et al., 2024) and over 100 years of coronary heart disease being the leading cause of death within the United States (American Heart Association, 2024). This statistic highlights the urgency to develop reliable predictive models which facilitate early detection or flag individuals who are at elevated risks for developing heart disease if their current lifestyles persist.

Previously, predictive model development for coronary heart disease has focused on simplified data sets of 14 features and typically have centered around performing the work on just the Cleveland subset of data. These previous approaches offer the benefit of accessibility and a complete data set for modeling purposes but omit 62 potential valuable predictor information from the entire data set. Additionally, they focus on one specific subset of data, which is those patients who were treated at the Chicago Clinic, which is a medical center that is in a technologically developed, first world country, which could not be a complete representative sample of coronary heart disease on a global scale. With these limitations in mind, the current study employs the entire raw 76 feature predictor set from each of the four geographically separated reporting locations, Long Beach Veterans Administration (USA), Hungarian Institute of Cardiology (Hungary), University Hospital of Zurich (Switzerland), and finally the Cleveland Clinic (USA) and then applies preprocessing, feature selection, data cleaning, and feature engineering for a combined final data set of 899 observations across 39 predictive features.

The primary goal of the study is to enhance predictive accuracy of coronary heart disease by employing a richer and detailed feature set, which we hope will lead to improved performance

metrics across the multiple classification machine learning algorithms developed. Specifically, the study focuses on the following models, Logistic Regression, Random Forest, PLS-DA, KNN, and Regularization-Based Penalized Classification Models (Lasso, Ridge, and ElasticNet).

Substantial challenges arose in order to complete this analysis on the expanded data set, including substantial missingness of key variables, multicollinearity, and noise that needed to be addressed. The outcomes seek to validate methodological choices and to provide clinically meaningful recommendations for the integration of new predictive models and features to improve the detection and mitigation of coronary heart disease.

Methodology

Data Acquisition and Description

The data for this study was sourced from the University of California Irvine (UCI) Machine Learning repository's Heart Disease data set (Janosi et al, 1988). The original, comprehensive data set comprises 76 clinical features, spanning four distinct medical facilities that collected the unstructured data. The complete merged data set consisted of 899 observations and 76 features. The final data set used in modeling after preprocessing, feature selection, and feature engineering was trimmed down to a final 39 features for prediction. The file *heart-disease.names* contains a data dictionary with all the feature definitions.

Data Wrangling and Preprocessing

The individual locations were initially ingested as separate data files which took the form of unstructured data, representing 10 rows of data for each observation. Data was not stored with headers and had to be separated by identifying a common unused *name* feature which separated each individual record. It is important to note that for the full records from the Cleveland data set there were data corruption issues regarding the ingestion of the full 303 patient records as indicated in the '*heart-disease.names*' text file. Of the 303 patient records from the Cleveland data set, only 282 remaining were not corrupted or missing.

Exploratory Data Analysis

EDA was performed to investigate the relationships between each feature and the target variable. During univariate analysis, the distribution patterns and missingness of each feature were examined through histograms, density plots, and bar plots. Missingness was visualized using an UpSet plot as seen in Figure 1 below. The predictors *ca*, *thal*, and *thaltime* were missing together more often suggesting that missingness may not be random. Most predictors showed right-skewed distributions, therefore missing values were imputed with the median. For each numerical feature with missing data, an additional column was created to flag missingness. Missing data in categorical features were imputed with "Missing".

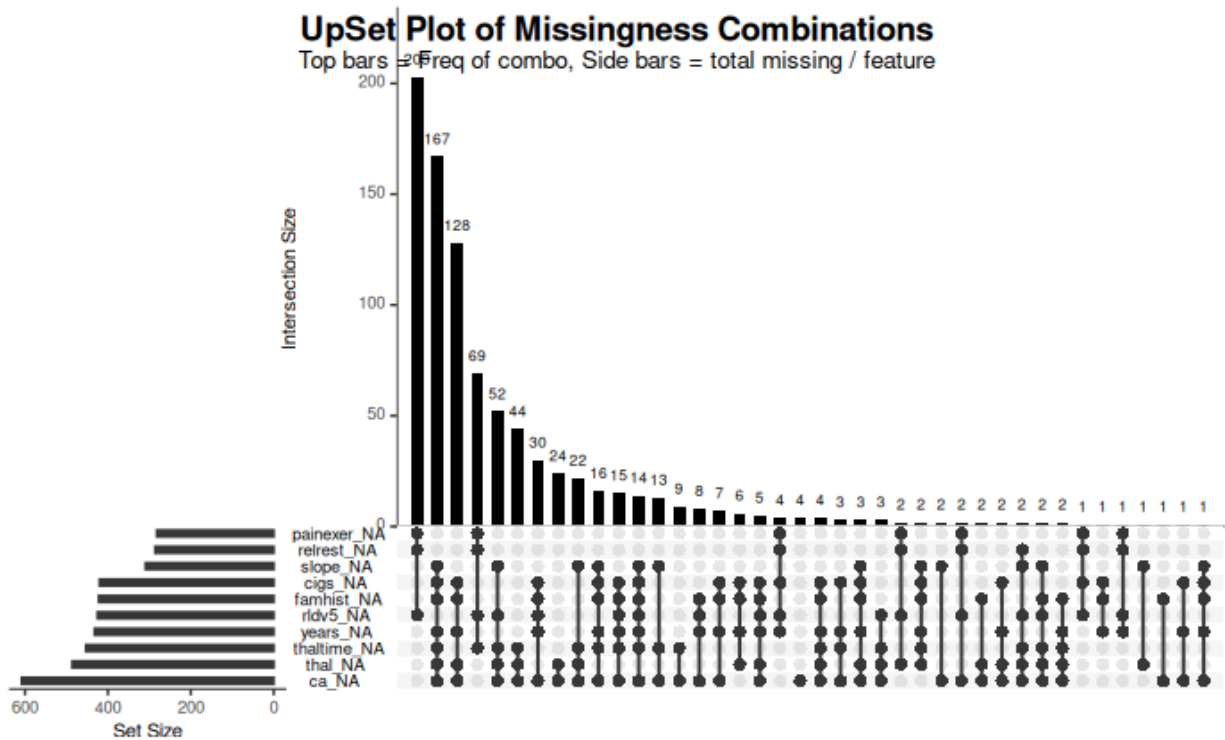


Figure 1: UpSet Plot

Bivariate analysis included visualizations using boxplots and statistical methods to determine statistical significance of each numerical predictor for heart disease. Categorical features were analyzed visually with barplots to see distributions and determine collinearity. Observed collinearity was verified statistically using Cramer's V association test. After EDA, columns that were deemed non-significant or exhibited multicollinearity were dropped, resulting in the final modeling dataset containing 39 features and 899 observations. Finally to prepare the data for a binary classification problem a target feature *has_disease* was developed classifying the original five-level target feature and converting it into a binary classification problem as a factor with two levels, No and Yes.

Data Splitting and Resampling Strategies

A train-test split was performed first, separating out the testing set for final performance validation. Cross-validation was conducted exclusively within the training set to ensure rigorous tuning and robust estimation of model performance during the training phase.

The models were validated using a 5-fold cross-validation approach on the training dataset, optimizing primarily for ROC-AUC to maximize the discriminative ability between patients with and without heart disease. Accuracy, sensitivity, and specificity were also computed to provide additional context for clinical relevance.

Model Development

Based on the dataset characteristics, exploratory insights, and ultimate data science problem of classifying patients with heart disease, the following classification models were chosen to provide robust predictive performance. The first is logistic regression, which is one of the fundamental classification methods and thus serves as a baseline model for its simplicity. The logistic regression assumes a linear relationship and has the power to estimate probabilities for the binary outcomes as well making it a powerful tool to classify the binary outcome.

The next model leveraged in order to account for any nonlinearity and establish an ensemble model of decision trees was a random forest model that was trained on the data. Random Forests help to reduce variance, by training on the predictions from multiple bootstrapped samples of the data and further reducing variance by using random feature subsets at the splits in each decision tree that is developed.

PLS-DA was also selected as the overall dimensionality of the data set increased and to mitigate any concerns regarding multicollinearity. PLS-DA reduces the number of predictors by combining them into components which capture maximal covariance between the features and binary outcome. It was determined that the optimal number of components was to be five, when evaluated on a range of values from one through ten. While this study focused on the binary outcome, future works can extend this to the raw full 5-factor level classification that the heart disease data is encoded in.

KNN for classification is a powerful tool which predicts the new sample based on the K-closest samples from the training set. KNN is effective at modeling decision boundaries based on the training data structures and then assigning new observations according to the training space. The ability to control the level of K allows us to easily balance model bias and variance and see the results over a range of K values. This allows us to choose the optimal value of K and apply it to our prediction models.

Finally, to build off the baseline logistic regression models and to mitigate any multicollinearity, penalization terms were applied to the logistic regression model via regularization in order to improve the model's fit to the training data. Lasso, Ridge, and Elastic Net penalties were applied which penalize large coefficient magnitudes to improve the model generalizability. Lasso (L1) has the potential to perform feature selection by shrinking the coefficients down to zero, while Ridge (L2) addresses multicollinearity by distributing the penalties across the features. Finally, Elastic Net performs the advantages of both Lasso and Ridge penalties and acts as a way to balance model complexity.

Validation, Tuning, and Testing Strategies

Data preprocessing and hyperparameter tuning were conducted using a systematic grid search approach, as illustrated by in the appendix code section block entitled *global_parameter_tuning*.

Model-specific tuning strategies were as follows:

- **Logistic Regression:** Centering and scaling of predictors; no hyperparameter tuning needed beyond baseline fitting.
- **Random Forest:** Evaluated using the number of variables randomly sampled as candidates at each split and the number of trees to ensure robust generalization.
- **Partial Least Squares Discriminant Analysis (PLS-DA):** Optimized number of latent components (ncomp) ranging from 1 to 10 to handle correlated features efficiently.
- **K-Nearest Neighbors (KNN):** The optimal number of neighbors (k) was determined by systematically testing odd values from 1 to 25, selecting the value that maximized cross-validation performance.
- **Penalized Regression Models (Lasso, Ridge, Elastic Net):** Hyperparameters for regularization strength (lambda) and the balance between L1 and L2 penalties (alpha) were tuned through an extensive grid search to manage feature selection and multicollinearity effectively.

In each case, the tuning parameters chosen maximized model generalizability and predictive power, ensuring robust performance on unseen data. Models were subsequently evaluated on the held-out test data to confirm their predictive validity.

Results

Model performance results are summarized below in Table 1.

Table 1
Heart Disease Model Performance Comparison

Model	Accuracy	Sensitivity	Specificity	ROC-AUC
Logistic Regression	0.749	0.727	0.755	0.834
Random Forest	0.821	0.798	0.85	0.906
PLS-DA	0.804	0.798	0.812	0.911
K-Nearest Neighbors	0.821	0.848	0.787	0.902
Lasso Penalization	0.760	0.737	0.787	0.884
Ridge Penalization	0.799	0.808	0.787	0.909
Elastic Net Penalization	0.760	0.737	0.787	0.884

When evaluating model performance using ROC-AUC as the primary metric, PLS-DA emerged as the top performer with an AUC of 0.911, indicating excellent discriminative ability in identifying heart disease. While Random Forest achieved the highest specificity at 0.85 and shared the highest accuracy at 0.821, its interpretability is limited due to its ensemble nature. On the other hand, K-Nearest Neighbors (KNN), which also tied for highest accuracy and demonstrated the highest sensitivity at 0.848, offers greater transparency in decision-making and is more interpretable than Random Forest. Overall, although PLS-DA is our top predictor of choice based on AUC, KNN may be favored in settings where model interpretability is critical.

Conclusion

This study aimed to investigate machine learning models to effectively identify and predict coronary heart disease through the use of an expanded feature and geographically diverse data set beyond the commonly applied Cleveland data set. By incorporating multiple locations

from around the world, the aim is to create a more holistic, effective predictive model, which is representative of the true population of heart disease rather than focusing solely on the Cleveland Clinic. Final model performance for the strongest predictive model on the expanded feature and geographical data set can be attributed to the PLS-DA model with 5 components that had an AUC of 0.911 and the KNN model tuned to 21 neighbors which had an AUC of 0.902.

Despite our rigorous application of data preprocessing and implementation of machine learning models, several limitations of the study exist that need to be addressed. Due to considerable missingness of data in the expanded data set, imputation methods were applied to create as robust and full of a data set as possible, however this inherently leaves room for bias to enter the model. It is recommended that future research explore more advanced imputation methods to include machine learning and model-based approaches, which can lead to enhanced predictive performance. Furthermore, future research can extend to the full five-factor level classification which can provide a ranking for identifying severity levels of coronary heart disease outside of just a binary classification setting.

In summary, this study highlighted potential benefits of employing the expanded clinical data set to predict coronary heart disease which is the leading cause of death globally (WHO, 2024). Our results establish baseline standards for AUC-ROC performance metrics for future works to compare as well as seeing marginal improvements (2% improvement on Random Forest) in accuracy compared to previous studies.

References

- American Heart Association. (2024, January 24). *More than half of U.S. adults don't know heart disease is leading cause of death, despite 100-year reign*. American Heart Association.
<https://newsroom.heart.org/news/more-than-half-of-u-s-adults-dont-know-heart-disease-is-leading-cause-of-death-despite-100-year-reign>
- Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, J.-J., Sandhu, S., Guppy, K. H., Lee, S., & Froelicher, V. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. *The American Journal of Cardiology*, 64(5), 304–310. [https://doi.org/10.1016/0002-9149\(89\)90524-9](https://doi.org/10.1016/0002-9149(89)90524-9)
- Di Cesare, M., Perel, P., Taylor, S., Kabudula, C., Bixby, H., Gaziano, T. A., Vaca McGhie, D., Mwangi, J., Pervan, B., Narula, J., Pineiro, D., & Pinto, F. J. (2024). The Heart of the World. *Global Heart*, 19(1). <https://doi.org/10.5334/gh.1288>
- Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988, June 30). *UCI Machine Learning Repository*. Archive.ics.uci.edu. <https://archive.ics.uci.edu/dataset/45/heart+disease>
- Rehman, I., Kerndt, C. C., & Rehman, A. (2023, January 27). *Anatomy, Thorax, Heart Left Anterior Descending (LAD) Artery*. Nih.gov; StatPearls Publishing.
<http://www.ncbi.nlm.nih.gov/books/NBK482375/>
- World Health Organization. (2024, August 7). *The Top 10 Causes of Death*. World Health Organization.
<https://www.who.int/news-room/fact-sheets/detail/the-top-10-causes-of-death>

Appendix

Appendix Table of Contents

Data Wrangling	1
EDA	7
Modeling	142

1_data_extraction

2025-06-09

Introduction

This document will be responsible for extracting all of the raw data from the `long-beach-va.data`, `hungarian.data`, `switzerland.data`, and `cleveland.data` files containing the raw data sets and merging them into one data set. Once this has been accomplished Exploratory Data Analysis (EDA) will be performed on the entire merged data. Comprising **899 patient records** with **76 attributes**, including the target variable. The data is stored in an unconventional format: although it's a plain text file with space-separated values, **each patient record spans 10 rows**, and there are no header names provided in the data set specifically. The end of each record is indicated by the final field labeled `"name"`.

Importing Libraries

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2     3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
```

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(readr)
```

```
library(dplyr)
```

```
library(here)
```

```
## here() starts at /home/rstudio/Documents/ads503_project_g1
```

```
library(caret)
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'caret'
```

```
##
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## lift
```

```

# set the column names per the heart-disease.names text file
full_cols <- c(
  "id",      # 1. patient ID
  "ccf",     # 2. ccf (social security #; now dummy)
  "age",     # 3. age in years
  "sex",     # 4. sex (1=male; 0=female)
  "painloc", # 5. chest pain location (1=substernal; 0=otherwise)
  "painexer", # 6. provoked by exertion (1=yes; 0=no)
  "relrest", # 7. relieved after rest (1=yes; 0=no)
  "pncaden", # 8. sum of 5, 6, and 7
  "cp",      # 9. chest pain type (1-4)
  "trestbps", # 10. resting blood pressure (mm Hg)
  "htn",     # 11. history of hypertension (1=yes; 0=no)
  "chol",    # 12. serum cholesterol (mg/dL)
  "smoke",   # 13. is a smoker (1=yes; 0=no)
  "cigs",    # 14. cigarettes per day
  "years",   # 15. years as a smoker
  "fbs",     # 16. fasting blood sugar > 120 mg/dL (1=yes; 0=no)
  "dm",      # 17. diabetes history (1=yes; 0=no)
  "famhist", # 18. family history of CAD (1=yes; 0=no)
  "restecg", # 19. resting ECG results (0-2)
  "ekgmo",   # 20. month of exercise ECG reading
  "ekgday",  # 21. day of exercise ECG reading
  "ekgyr",   # 22. year of exercise ECG reading
  "dig",     # 23. digitalis used during ECG (1=yes; 0=no)
  "prop",    # 24. beta blocker used (1=yes; 0=no)
  "nitr",    # 25. nitrates used (1=yes; 0=no)
  "pro",     # 26. calcium channel blocker used (1=yes; 0=no)
  "diuretic", # 27. diuretic used during ECG (1=yes; 0=no)
  "proto",   # 28. exercise protocol (1=Bruce, 2=Kottus, ... 12=arm ergometer)
  "thaldur", # 29. duration of exercise test (minutes)
  "thaltime", # 30. time when ST depression was noted
  "met",     # 31. mets achieved
  "thalach", # 32. maximum heart rate achieved
  "thalrest", # 33. resting heart rate
  "tpeakbps", # 34. peak exercise BP (first part)
  "tpeakbpd", # 35. peak exercise BP (second part)
  "dummy",   # 36. (unused)
  "trestbpd", # 37. resting blood pressure (duplicate)
  "exang",   # 38. exercise-induced angina (1=yes; 0=no)
  "xhypo",   # 39. (1=yes; 0=no)
  "oldpeak", # 40. ST depression relative to rest
  "slope",   # 41. slope of the peak exercise ST segment (1-3)
  "rldv5",   # 42. height at rest
  "rldv5e",  # 43. height at peak exercise
  "ca",      # 44. number of major vessels (0-3) seen on fluoroscopy
  "restckm", # 45. irrelevant
  "exerckm", # 46. irrelevant
  "restef",  # 47. rest radionuclide ejection fraction
  "restwm",  # 48. rest wall motion abnormality (0-3)
  "exeref",  # 49. exercise radionuclide ejection fraction
  "exerwm",  # 50. exercise wall motion (0-3)
  "thal",    # 51. thalassemia (3=normal; 6=fixed; 7=reversible)
)

```



```

"thalsev", # 52. not used
"thalpul", # 53. not used
"earlobe", # 54. not used
"cmo",     # 55. month of cardiac cath
"cday",    # 56. day of cardiac cath
"cyr",     # 57. year of cardiac cath
"num",     # 58. diagnosis (0-4)
"lmt",     # 59. left main coronary artery
"ladprox", # 60. left anterior descending proximal
"laddist", # 61. left anterior descending distal
"diag",    # 62. diagonal
"cxmain",  # 63. circumflex main
"ramus",   # 64. ramus
"om1",     # 65. obtuse marginal 1
"om2",     # 66. obtuse marginal 2
"rcaprox", # 67. right coronary artery proximal
"rcadist", # 68. right coronary artery distal
"lvx1",    # 69. not used
"lvx2",    # 70. not used
"lvx3",    # 71. not used
"lvx4",    # 72. not used
"lvf",     # 73. not used
"cathef",  # 74. not used
"junk",    # 75. not used
"name"     # 76. last name (dummy string)
)

```

Data Importing

VA Long Beach

```

# using readLines, to read in each line of the long-beach-va.data file
# read_table, read_csv proved difficult to use to extract
# outline is to use the occurrence of 'name' to distinguish unique observations
# extract all the lines
va_raw_lines <- readLines(here("Data/Raw", "long-beach-va.data"))
# merge all lines into one string
va_all_text <- paste(va_raw_lines, collapse = " ")
# split each observation on name
va_obs <- strsplit(va_all_text, " name ")[[1]]

va_record_list <- map(va_obs, ~ {
  toks <- strsplit(.x, "\\s+")[[1]] # split strings at whitespace
  toks <- toks[toks != ""] #empty token removal
  stopifnot(length(toks) == 75)
  data.frame(matrix(toks, nrow = 1),
              stringsAsFactors = FALSE)
})
# bind the rows
va_df_raw <- bind_rows(va_record_list)
# assign column names but leave out name
colnames(va_df_raw) <- full_cols[1:75]
# add name column and location column in

```

```
va_df_raw$name <- "name"
va_df_raw$location <- "VA Long Beach"
```

Hungary

```
lines <- readLines(here("Data/Raw", "hungarian.data"))

# Step 2: Combine every 10 lines per patient
patient_blocks <- split(lines, ceiling(seq_along(lines) / 10))

# Step 3: Parse each block into numeric + name
records <- map(patient_blocks, function(block) {
  tokens <- str_split(paste(block, collapse = " "), "\\s+")[[1]]
  name_value <- tail(tokens, 1) # last value is the name
  numeric_values <- as.numeric(tokens[-length(tokens)])
  c(numeric_values, name_value) # combine numeric + name
})

# Step 4: Combine into data frame
hu_df_raw <- do.call(rbind, records) %>%
  as.data.frame(stringsAsFactors = FALSE)
hu_df_raw <- hu_df_raw %>% slice(-295) ## sliced out the blank row 295th
#dim(hu_df_raw)

# Step 5: Assign column names
colnames(hu_df_raw) <- full_cols

# Step 6: Replace -9 with NA
hu_df_raw <- hu_df_raw %>%
  mutate(across(where(is.character), ~ na_if(.x, "-9"))) %>% # character cols
  mutate(across(where(is.numeric), ~ na_if(.x, -9))) %>% # numeric columns
  mutate(location = "Hungary")

#head(hu_df_raw)
```

Switzerland

```
# Generative AI used to write code to read in .data file
# Step 1: Read the file
lines <- readLines(here("Data/Raw", "switzerland.data"))

# Step 2: Combine every 10 lines per patient
patient_blocks <- split(lines, ceiling(seq_along(lines) / 10))

# Step 3: Parse each block into numeric + name
records <- map(patient_blocks, function(block) {
  tokens <- str_split(paste(block, collapse = " "), "\\s+")[[1]]
  name_value <- tail(tokens, 1) # last value is the name
  numeric_values <- as.numeric(tokens[-length(tokens)])
  c(numeric_values, name_value) # combine numeric + name
})

# Step 4: Combine into data frame
```

```

sw_df_raw <- do.call(rbind, records) %>%
  as.data.frame(stringsAsFactors = FALSE)

# Step 5: Assign column names
colnames(sw_df_raw) <- full_cols

# Step 6: Replace -9 with NA
sw_df_raw <- sw_df_raw %>%
  mutate(across(where(is.character), ~ na_if(.x, "-9"))) %>% # character columns
  mutate(across(where(is.numeric), ~ na_if(.x, -9))) %>% # numeric columns
  mutate(location = "switzerland")

#head(sw_df_raw)

```

Cleveland

```

# using readLines, to read in each line of the cleveland.data file
# read_table, read_csv proved difficult to use to extract
# outline is to use the occurrence of 'name' to distinguish unique observations
# extract all the lines
clvd_raw_lines <- readLines(here("Data/Raw", "cleveland.data"),
  warn = FALSE)

#clvd_raw_lines
# merge all lines into one string
clvd_all_text <- paste(clvd_raw_lines, collapse = " ")
# remove the corrupted tail end
clvd_all_text <- substr(clvd_all_text,
  start = 1,
  stop = 56933)

#clvd_all_text
# split each observation on name
clvd_obs <- strsplit(clvd_all_text, " name ")[[1]]
#clvd_obs
# map obs to record list
clvd_record_list <- map(clvd_obs, ~ {
  toks <- strsplit(.x, "\\s+")[1] # split strings at whitespace
  toks <- toks[toks != ""] #empty token removal
  stopifnot(length(toks) == 75)
  data.frame(matrix(toks, nrow = 1),
    stringsAsFactors = FALSE)
})
# bind the rows
clvd_df_raw <- bind_rows(clvd_record_list)
# assign column names but leave out name
colnames(clvd_df_raw) <- full_cols[1:75]
# add name column and location column in
clvd_df_raw$name <- "name"
clvd_df_raw$location <- "Cleveland"

```

Data Merging

```
# bind the rows and merge the df on top of each other
merged_df <- bind_rows(clvd_df_raw,
                       hu_df_raw,
                       sw_df_raw,
                       va_df_raw)

# merged_df

# write the merged raw file out to the raw data folder as 'merged_df_raw.csv'
# export to file
write.csv(merged_df, here("Data/Raw", "merged_df_raw.csv"),
          row.names = FALSE)
```

2_EDA

2025-06-11

Data Cleaning and Exploratory Data Analysis

This R markdown document will be used to perform Exploratory Data Analysis (EDA) on the merged_df_raw that was developed in the previous workbook 1_data_extraction.Rmd.

Data Importing

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(readr)
library(ggplot2)
library(grid)
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(here)
```

```
## here() starts at /home/rstudio/Documents/ads503_project_g1
```

```
library(naniar)
library(VIM)
```

```
## Loading required package: colorspace
## VIM is ready to use.
##
```

```

## Suggestions and bug-reports can be submitted at: https://github.com/statistikat/VIM/issues
##
## Attaching package: 'VIM'
##
## The following object is masked from 'package:datasets':
##
##     sleep
library(skimr)

##
## Attaching package: 'skimr'
##
## The following object is masked from 'package:naniar':
##
##     n_complete
library(stringr)
library(rcompanion)

merged_df <- read_csv(here('Data/Raw', 'merged_df_raw.csv'))

## Rows: 899 Columns: 77
## -- Column specification -----
## Delimiter: ","
## chr (2): name, location
## dbl (75): id, ccf, age, sex, painloc, painexer, relrest, pncaden, cp, trestb...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
head(merged_df)

## # A tibble: 6 x 77
##       id    ccf    age    sex painloc painexer relrest pncaden    cp trestbps    htn
##   <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl>   <dbl> <dbl>
## 1     1     0    63     1     -9     -9     -9     -9     1    145     1
## 2     2     0    67     1     -9     -9     -9     -9     4    160     1
## 3     3     0    67     1     -9     -9     -9     -9     4    120     1
## 4     4     0    37     1     -9     -9     -9     -9     3    130     0
## 5     6     0    41     0     -9     -9     -9     -9     2    130     1
## 6     7     0    56     1     -9     -9     -9     -9     2    120     1
## # i 66 more variables: chol <dbl>, smoke <dbl>, cigs <dbl>, years <dbl>,
## #   fbs <dbl>, dm <dbl>, famhist <dbl>, restecg <dbl>, ekgmo <dbl>,
## #   ekgday <dbl>, ekgyr <dbl>, dig <dbl>, prop <dbl>, nitr <dbl>, pro <dbl>,
## #   diuretic <dbl>, proto <dbl>, thaldur <dbl>, thaltime <dbl>, met <dbl>,
## #   thalach <dbl>, thalrest <dbl>, tpeakbps <dbl>, tpeakbpd <dbl>, dummy <dbl>,
## #   trestbpd <dbl>, exang <dbl>, xhypo <dbl>, oldpeak <dbl>, slope <dbl>,
## #   rldv5 <dbl>, rldv5e <dbl>, ca <dbl>, restckm <dbl>, exerckm <dbl>, ...

```

Categorical Conversions

```

## categorical features and their levels and labels
#sex
merged_df$sex <- factor(merged_df$sex,
                        levels = c(0, 1),

```

```

        labels = c("Female", "Male")
    )

#painloc
#unique(merged_df$painloc)
merged_df$painloc <- factor(merged_df$painloc,
    levels = c(0, 1),
    labels = c("Otherwise", "Substernal")
)

#painexer
#unique(merged_df$painexer)
merged_df$painexer <- factor(merged_df$painexer,
    levels = c(0, 1),
    labels = c("Otherwise", "Provoked by Exertion")
)

#relrest
#unique(merged_df$relrest)
merged_df$relrest <- factor(merged_df$relrest,
    levels = c(0, 1),
    labels = c("Otherwise", "Relieved after rest")
)

#cp
#unique(merged_df$cp)
merged_df$cp <- factor(merged_df$cp,
    levels = c(1, 2, 3, 4),
    labels = c("Typical Angina", "Atypical Angina",
        "Non-anginal Pain", "Asymptomatic"),
    ordered = TRUE)

#smoke
#unique(merged_df$smoke)
merged_df$smoke <- factor(merged_df$smoke,
    levels = c(0, 1),
    labels = c("No", "Yes")
)

#fbs
#unique(merged_df$fbs)
merged_df$fbs <- factor(merged_df$fbs,
    levels = c(0, 1),
    labels = c("No", "Yes")
)

#dm
#unique(merged_df$dm)
merged_df$dm <- factor(merged_df$dm,
    levels = c(0, 1),
    labels = c("No", "Yes")
)

#famhist
#unique(merged_df$famhist)
merged_df$famhist <- factor(merged_df$famhist,
    levels = c(0, 1),
    labels = c("No", "Yes")
)

#restecg
#unique(merged_df$restecg)

```

```

merged_df$restecg <- factor(merged_df$restecg,
  levels = c(0, 1, 2),
  labels = c("Normal", "Abnormal", "Hypertrophy"),
  ordered = TRUE)

#dig
#unique(merged_df$dig)
merged_df$dig <- factor(merged_df$dig,
  levels = c(0, 1),
  labels = c("No", "Yes")
)

#prop
#unique(merged_df$prop)
# find the number of 22's a feature outside the range/
#sum(merged_df$prop == 22, na.rm = TRUE)
# convert the 22 to NA
merged_df <- merged_df %>%
  mutate(prop = ifelse(prop == 22, NA, prop))
#create factor
merged_df$prop <- factor(merged_df$prop,
  levels = c(0, 1),
  labels = c("No", "Yes")
)

#nitr
#unique(merged_df$nitr)
merged_df$nitr <- factor(merged_df$nitr,
  levels = c(0, 1),
  labels = c("No", "Yes")
)

#pro
#unique(merged_df$pro)
merged_df$pro <- factor(merged_df$pro,
  levels = c(0, 1),
  labels = c("No", "Yes")
)

#diuretic
#unique(merged_df$diuretic)
merged_df$diuretic <- factor(merged_df$diuretic,
  levels = c(0, 1),
  labels = c("No", "Yes")
)

#proto
#unique(merged_df$proto)
merged_df$proto <- factor(merged_df$proto,
  levels = c(0, 1),
  labels = c("No", "Yes")
)

#exang
#unique(merged_df$exang)
merged_df$exang <- factor(merged_df$exang,
  levels = c(0, 1),
  labels = c("No", "Yes")
)

#xhypo

```



```

#unique(merged_df$xhypo)
merged_df$xhypo <- factor(merged_df$xhypo,
  levels = c(0, 1),
  labels = c("No", "Yes")
)

#slope
#unique(merged_df$slope)
# check count of 0 valuyes
#sum(merged_df$slope == 0, na.rm = TRUE)
# replace 0 with NA
merged_df <- merged_df %>%
  mutate(slope = ifelse(slope == 0, NA, slope))
# factor
merged_df$slope <- factor(merged_df$slope,
  levels = c(1, 2, 3),
  labels = c("Upsloping", "Flat", "Downsloping"),
  ordered = TRUE
)

#restwm
#unique(merged_df$restwm)
# check count of 0 valuyes
# factor
merged_df$restwm <- factor(merged_df$restwm,
  levels = c(0, 1, 2, 3),
  labels = c("None", "Mild/Moderate", "Moderate/Severe",
    "Akinesis"),
  ordered = TRUE
)

#thal
#unique(merged_df$thal)
# examine 1, 2, 5 values which are not in the documentation
# check count of 1,2,5 values
#sum(merged_df$thal == 1, na.rm = TRUE)
#sum(merged_df$thal == 2, na.rm = TRUE)
#sum(merged_df$thal == 5, na.rm = TRUE)
# replace 1, 2, 5 with na
merged_df <- merged_df %>%
  mutate(thal = ifelse(thal == 1, NA, thal)) %>%
  mutate(thal = ifelse(thal == 2, NA, thal)) %>%
  mutate(thal = ifelse(thal == 4, NA, thal)) %>%
  mutate(thal = ifelse(thal == 5, NA, thal))

```

Data Cleaning

This below section of code starts by removing columns that are unused as indicated by the `heart-disease.names` text file which serves as a data dictionary and source for information on all the data sets. Once the explicitly unnecessary columns are removed, we convert the -9's and -9.0's into NA values and set a threshold for feature removal of if the feature is missing more than 60% of the observations it is immediately dropped. Once these features have been removed we focus on removing near zero variance features.

```

# remove unnecessary columns as indicated in heart-disease.names
# cols marked as not used or dummy variables
not_used_dummy_var <- c(
  "id", "ccf", "dummy", "restckm", "exerckm", "thalsev", "thalpul",

```

```

"earlobe", "lvx1", "lvx2", "lvx3", "lvx4", "lvf", "cathef", "junk", "name"
)

df_cleaning <- merged_df %>%
  select(-all_of(not_used_dummy_var))

# convert -9 to NA
df_cleaning <- df_cleaning %>%
  mutate(across(where(is.character), ~ na_if(.x, "-9"))) %>% # character columns
  mutate(across(where(is.numeric), ~ na_if(.x, -9))) # numeric columns
#Get NA count
sum(is.na(df_cleaning))

## [1] 15881

#get dimensions of the data
df_cleaning %>% dim

## [1] 899 61

#reexamine the head
df_cleaning %>% head()

## # A tibble: 6 x 61
##   age sex   painloc painexer relrest pncaden cp   trestbps   htn   chol smoke
##   <dbl> <fct> <fct> <fct> <fct> <dbl> <ord> <dbl> <dbl> <dbl> <fct>
## 1 63 Male <NA> <NA> <NA> NA Typi~ 145 1 233 <NA>
## 2 67 Male <NA> <NA> <NA> NA Asym~ 160 1 286 <NA>
## 3 67 Male <NA> <NA> <NA> NA Asym~ 120 1 229 <NA>
## 4 37 Male <NA> <NA> <NA> NA Non~ 130 0 250 <NA>
## 5 41 Female <NA> <NA> <NA> NA Atyp~ 130 1 204 <NA>
## 6 56 Male <NA> <NA> <NA> NA Atyp~ 120 1 236 <NA>
## # i 50 more variables: cigs <dbl>, years <dbl>, fbs <fct>, dm <fct>,
## # famhist <fct>, restecg <ord>, ekgmo <dbl>, ekgday <dbl>, ekgyr <dbl>,
## # dig <fct>, prop <fct>, nitr <fct>, pro <fct>, diuretic <fct>, proto <fct>,
## # thaldur <dbl>, thaltime <dbl>, met <dbl>, thalach <dbl>, thalrest <dbl>,
## # tpeakbps <dbl>, tpeakbpd <dbl>, trestbpd <dbl>, exang <fct>, xhypo <fct>,
## # oldpeak <dbl>, slope <ord>, rldv5 <dbl>, rldv5e <dbl>, ca <dbl>,
## # restef <dbl>, restwm <ord>, exeref <dbl>, exerwm <dbl>, thal <dbl>, ...

# calculate percentage missing
missing_threshold <- 60
missing_pct <- df_cleaning %>%
  summarise(across(everything(), ~ mean(is.na(.)) * 100)) %>%
  pivot_longer(everything(), names_to = "column", values_to = "missing_percent") %>%
  arrange(desc(missing_percent))

# filter by col missing
missing_cols <- missing_pct %>%
  filter(missing_percent > missing_threshold)
var_missing_gt_60 <- missing_cols$column

missing_cols

## # A tibble: 12 x 2
##   column missing_percent

```

```
##      <chr>          <dbl>
## 1 pncaden          100
## 2 exeref           99.8
## 3 exerwm           99.4
## 4 restef           96.9
## 5 restwm           96.7
## 6 dm               89.4
## 7 smoke            74.4
## 8 ca               67.6
## 9 proto            64.0
## 10 om2             63.6
## 11 ramus           63.1
## 12 diag            62.1

# drop cols missing greater than threshold
df_cleaning <- df_cleaning %>%
  select(-all_of(var_missing_gt_60))
dim(df_cleaning)

## [1] 899 49

# check near zero variance vars
nzv_cols <- nearZeroVar(df_cleaning %>%
  select(-num), saveMetrics = TRUE)
nzv_cols %>% filter(nzv == TRUE)

##      freqRatio percentUnique zeroVar  nzv
## dig      27.65517      0.2224694  FALSE TRUE
## xhypo     37.22727      0.2224694  FALSE TRUE

# get column names of nzv
nzv_names <- nzv_cols %>%
  filter(nzv == TRUE) %>%
  rownames()
nzv_names

## [1] "dig"  "xhypo"

# remove nzv variables
df_cleaning <- df_cleaning %>%
  select(-all_of(nzv_names))

# add back ca column for EDA
df_cleaning$ca <- merged_df$ca
df_cleaning$ca <- na_if(df_cleaning$ca, -9)

dim(df_cleaning)

## [1] 899 48
```

In the process of removing variables with >60% missing data, the `ca` variable, which represents *Major vessels via fluoroscopy* was removed due to having 67.6% missing data. This variable was part of the *cleveland-14* set. It has re-added to the final data set after *nzv* variables were removed.

Export cleaned dataset

```
# export to file
write.csv(df_cleaning, here("Data/Clean", "heart_disease_cleaned.csv"), row.names = FALSE)
saveRDS(df_cleaning, here("Data/Clean", "heart_disease_cleaned.rds"))
```

Exploratory Data Analysis

```
df <- readRDS(here("Data/Clean", "heart_disease_cleaned.rds"))
# head(df)

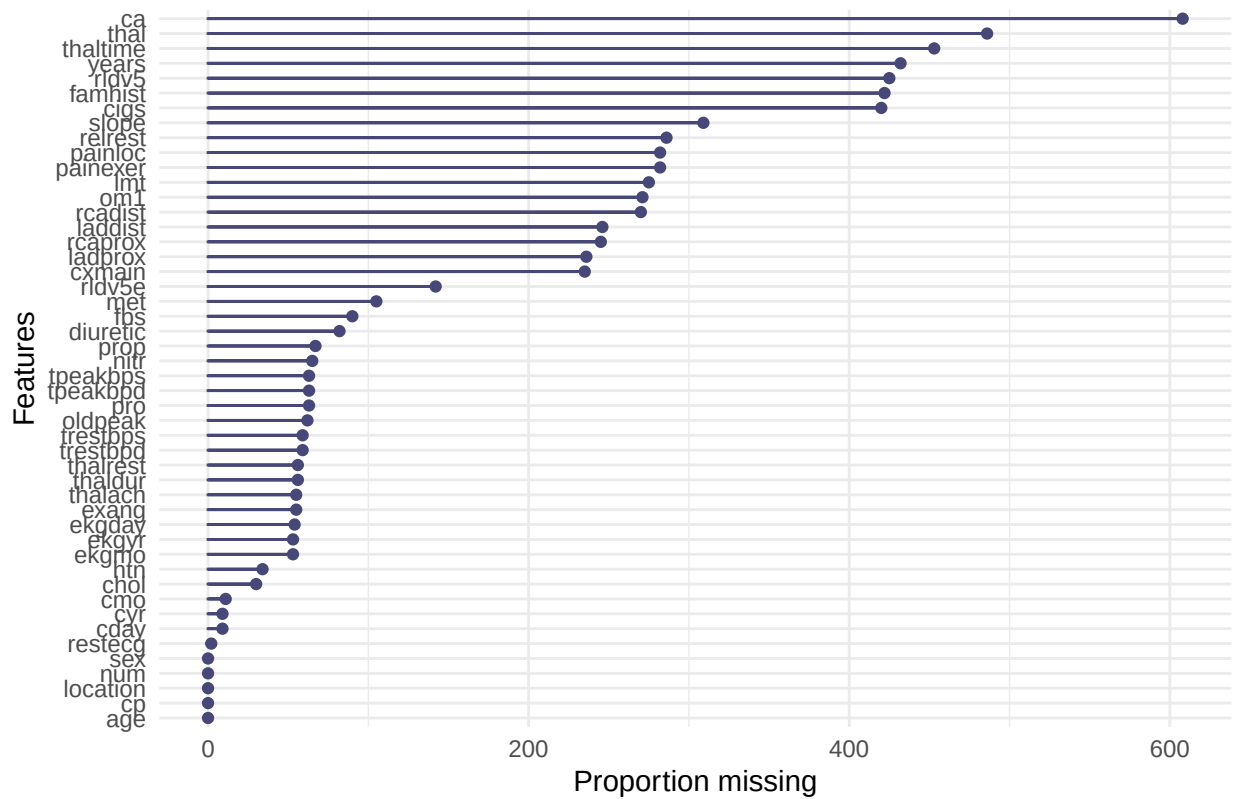
missing_var <- df %>%
  summarise(across(everything(), ~ mean(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "missing_pct")

missing_var %>%
  filter(missing_pct > 0) %>%
  arrange(desc(missing_pct))
```

```
## # A tibble: 43 x 2
##   variable missing_pct
##   <chr>          <dbl>
## 1 ca              0.676
## 2 thal            0.541
## 3 thaltime        0.504
## 4 years           0.481
## 5 rldv5           0.473
## 6 famhist         0.469
## 7 cigs            0.467
## 8 slope           0.344
## 9 relrest         0.318
## 10 painloc        0.314
## # i 33 more rows
```

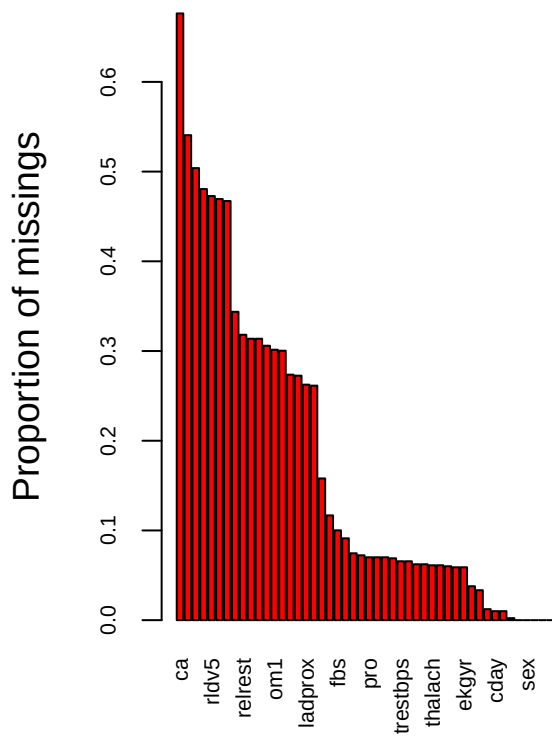
```
# bar chart of missingness per feature
gg_miss_var(df) +
  labs(
    title = "Missing Data per Feature",
    x = "Features",
    y = "Proportion missing"
  )
```

Missing Data per Feature

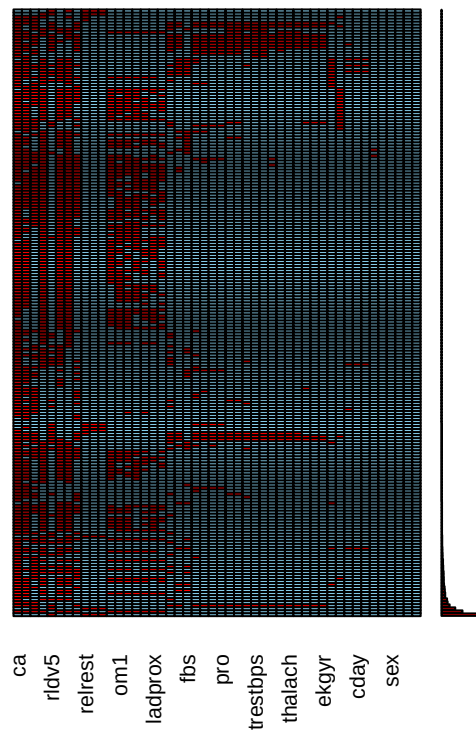


```
# missingness matrix
aggr(df, numbers = TRUE, sortVars = TRUE, cex.axis = 0.7,
     main = "Missingness Matrix",
     sub = "Red = missing, Blue = observed")
```

```
## Warning in plot.aggr(res, ...): not enough vertical space to display
## frequencies (too many combinations)
```



Combinations



```
##
## Variables sorted by number of missings:
## Variable      Count
##      ca 0.676307008
##      thal 0.540600667
## thaltime 0.503893215
##      years 0.480533927
##      rldv5 0.472747497
## famhist 0.469410456
##      cigs 0.467185762
##      slope 0.343715239
##      relrest 0.318131257
##      painloc 0.313681869
## painexer 0.313681869
##      lmt 0.305895439
##      om1 0.301446051
##      rcadist 0.300333704
##      laddist 0.273637375
##      rcaprox 0.272525028
##      ladprox 0.262513904
##      cxmain 0.261401557
##      rldv5e 0.157953281
##      met 0.116796440
##      fbs 0.100111235
##      diuretic 0.091212458
##      prop 0.074527253
```

```
##      nitr 0.072302558
##      pro 0.070077864
## tpeakbps 0.070077864
## tpeakbpd 0.070077864
##   oldpeak 0.068965517
## trestbps 0.065628476
## trestbpd 0.065628476
##   thaldur 0.062291435
## thalrest 0.062291435
##   thalach 0.061179088
##     exang 0.061179088
##   ekgday 0.060066741
##   ekgmo 0.058954394
##   ekgyr 0.058954394
##     htn 0.037819800
##   chol 0.033370412
##     cmo 0.012235818
##   cday 0.010011123
##   cyr 0.010011123
## restecg 0.002224694
##   age 0.000000000
##   sex 0.000000000
##   cp 0.000000000
##   num 0.000000000
## location 0.000000000
```

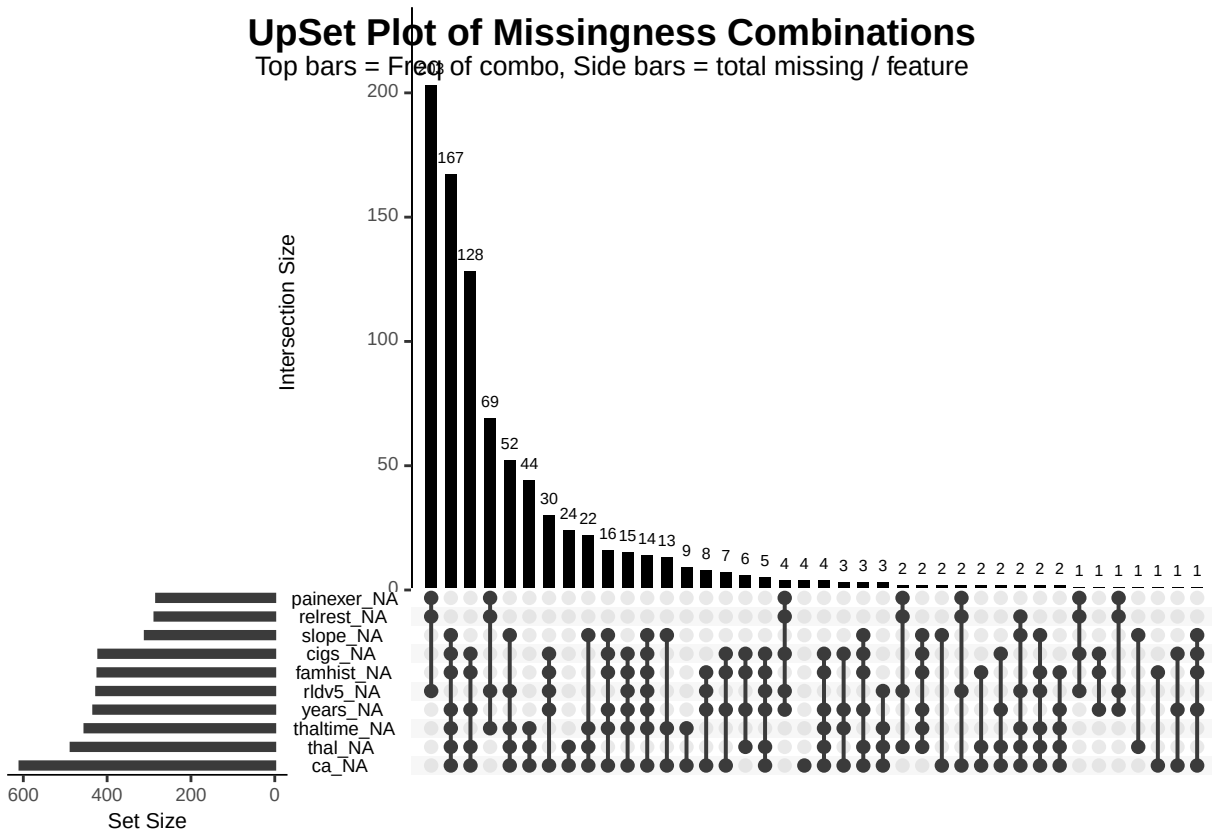
```
# missingness correlation / upset plot
```

```
gg_miss_upset(df, nsets = 10, main.bar.color = "black")
```

```
# Add a title manually
```

```
grid.text("UpSet Plot of Missingness Combinations", y = 0.95, gp = gpar(fontsize = 14, fontface = "bold"
```

```
grid.text("Top bars = Freq of combo, Side bars = total missing / feature", y = 0.91, gp = gpar(fontsize
```



Variables Missing > 40%

- ca: cleveland-14 variable - represents # of major vessels (0 - 3) colored by fluoroscopy.
- thal: cleveland-14 variable - results from thallium stress test
- thalttime: time duration for thallium stress test
- years: number of years as a smoker
- rldv5: height at rest
- famhist: family history of coronary artery disease
- cigs: cigarettes smoked per day

Upset

The UpSet plot shows the intersections of variable sets, similar to a Venn Diagram. This plot shows the combinations of variables that have missing values and the total of those combinations.

- based on upset table, ca, thal, and thalttime are missing together
- missingness might not be random

Custom Functions

```
# function for showing histogram
show_histogram <- function(dataframe, feature) {
  ggplot(dataframe, aes(x = {{feature}})) +
    geom_histogram(binwidth = 1, fill = "steelblue", color = "white") +
    labs(
      title = paste("Histogram of", rlang::as_label(enquo(feature))),

```



```

    x = rlang::as_label(enquo(feature)),
    y = "Count"
  ) +
  theme_minimal()
}
# function for showing density plot
show_densityplot <- function(dataframe, feature) {
  ggplot(df, aes(x = {{feature}})) +
    geom_density(fill = "darkorange", alpha = 0.5) +
    labs(
      title = paste("Density Plot of", rlang::as_label(enquo(feature))),
      x = rlang::as_label(enquo(feature)),
      y = "Count"
    ) +
    theme_minimal()
}

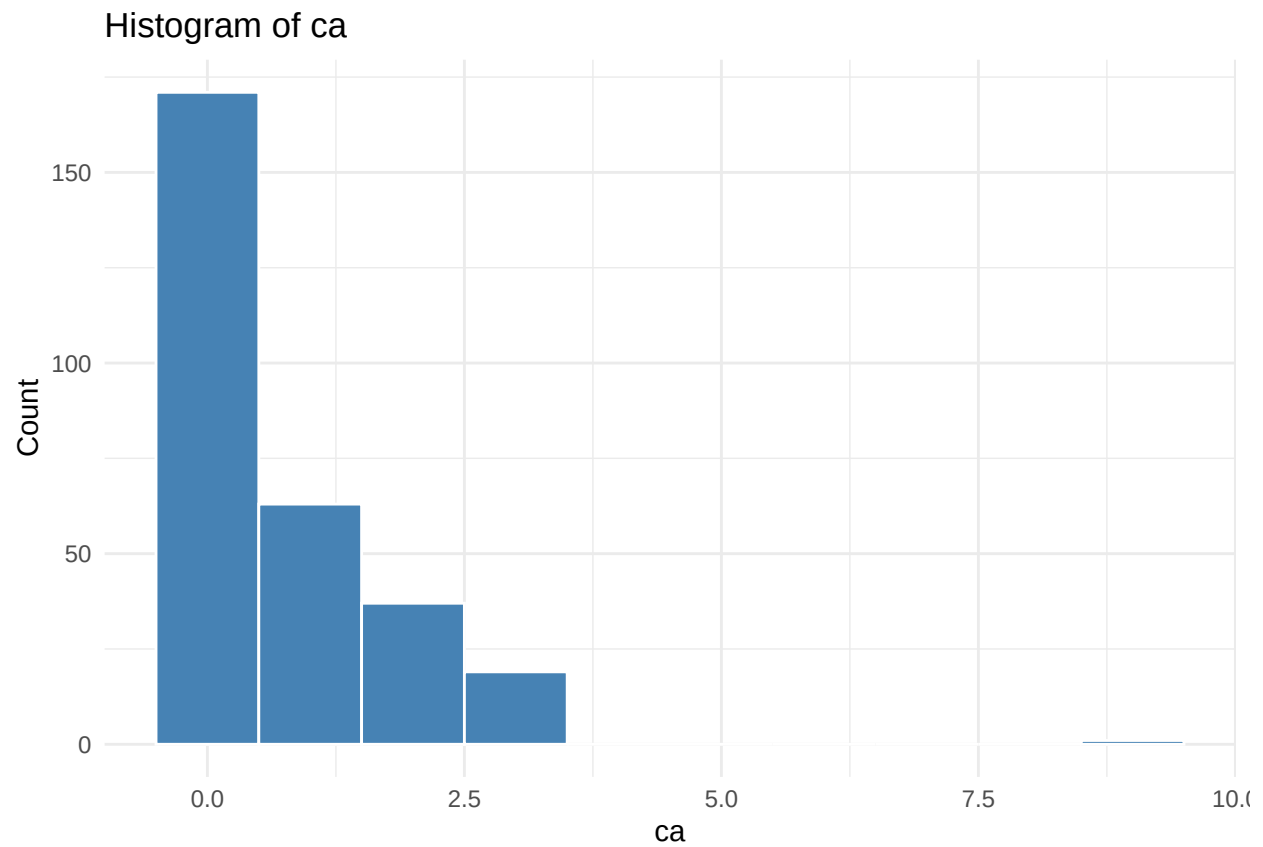
show_barplot <- function(dataframe, feature) {
  ggplot(dataframe, aes(x = {{feature}})) +
    geom_bar(fill = "steelblue") +
    labs(
      title = paste("Bar Plot of", rlang::as_label(enquo(feature))),
      x = rlang::as_label(enquo(feature)),
      y = "Count"
    ) +
    theme_minimal()
}

```

Feature: ca

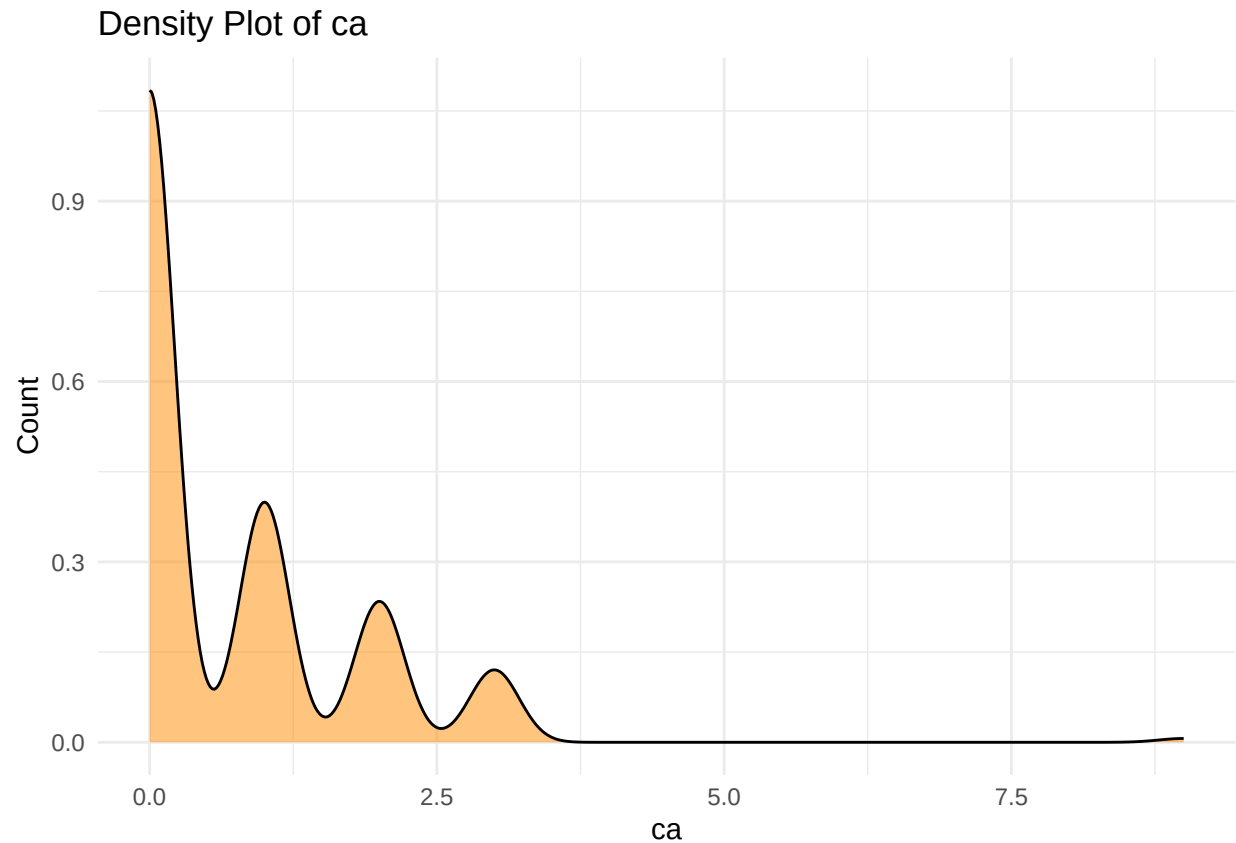
```
show_histogram(df, ca)
```

```
## Warning: Removed 608 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



```
show_densityplot(df, ca)
```

```
## Warning: Removed 608 rows containing non-finite outside the scale range  
## (`stat_density()`).
```



```
# impute with median and missingness flag
df <- df %>%
  mutate(
    ca_missing = ifelse(is.na(ca), 1, 0),
    ca_imputed = ifelse(is.na(ca), median(ca, na.rm = TRUE), ca)
  )

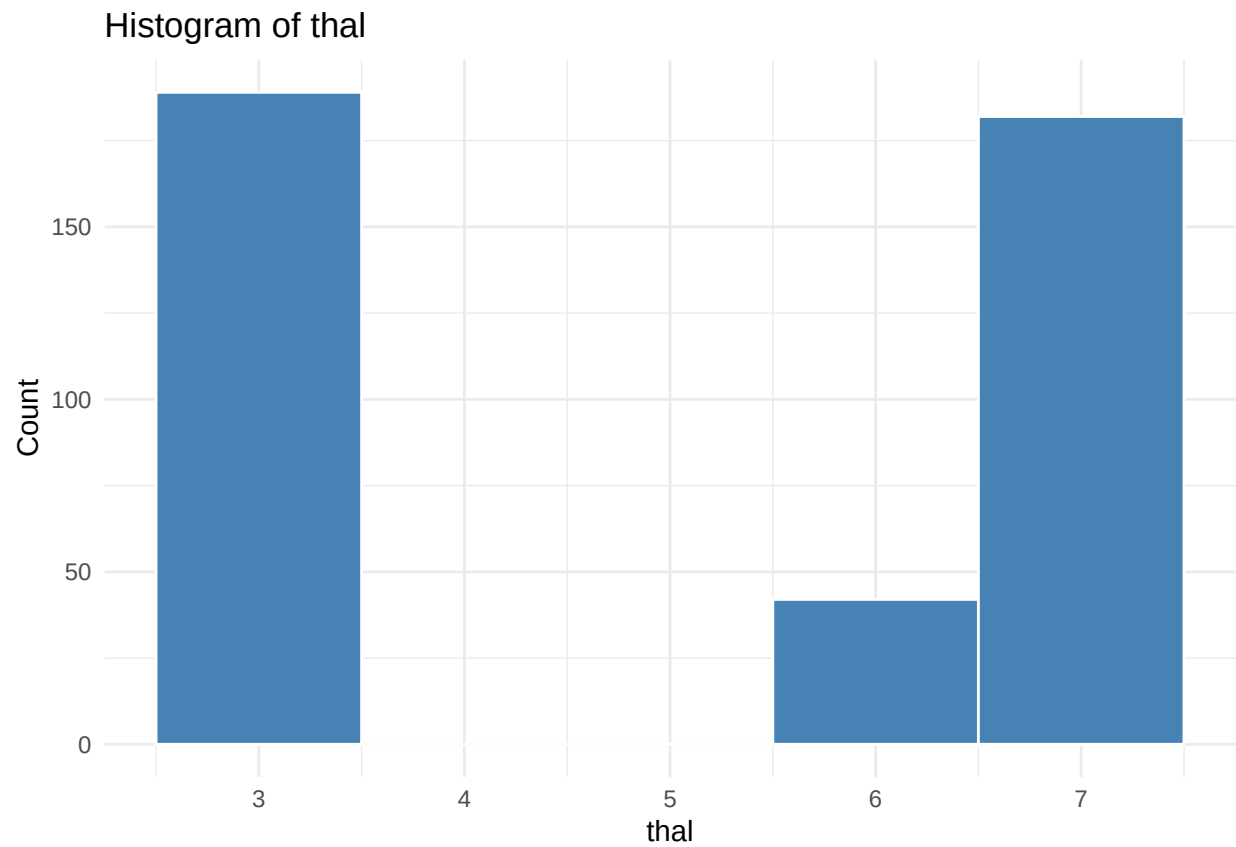
# drop imputed column
df <- df %>%
  select(-ca)
```

Feature ca has a right skew therefore median was used for imputation.

Feature: thal

```
show_histogram(df, thal)
```

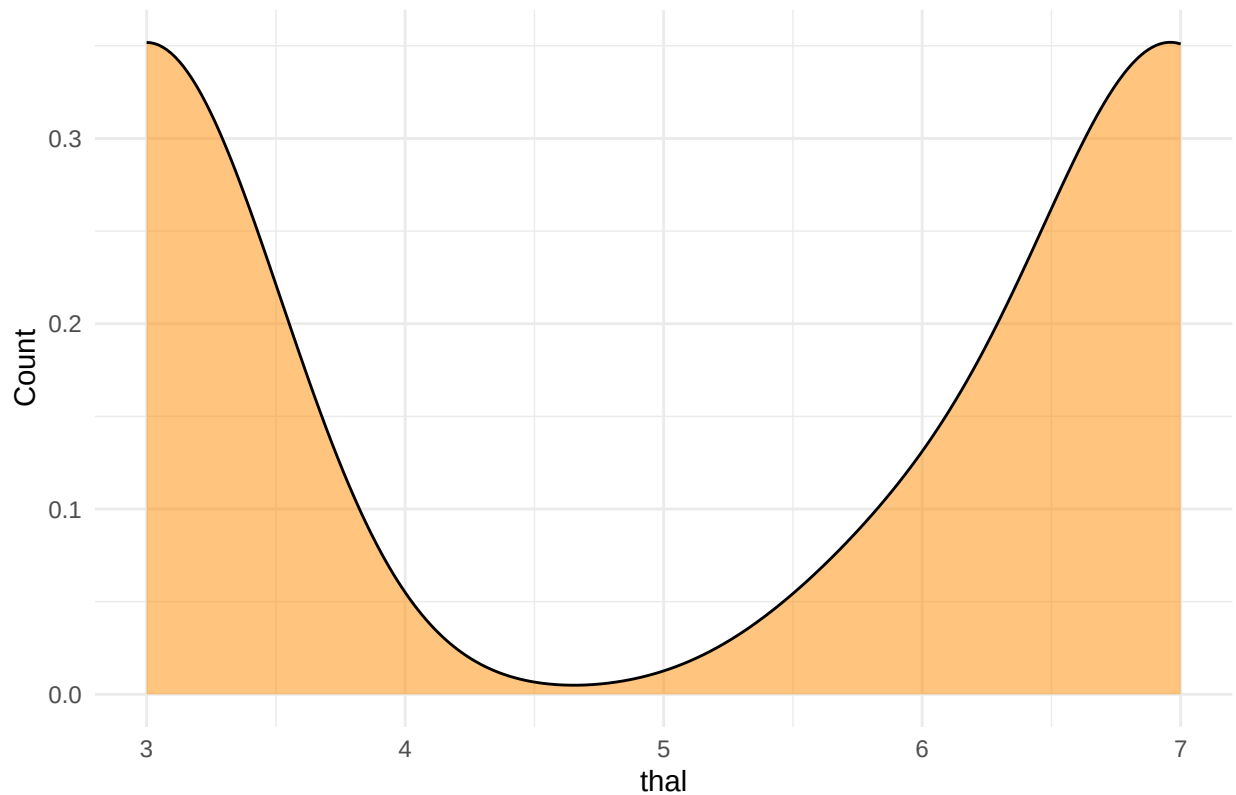
```
## Warning: Removed 486 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



```
show_densityplot(df, thal)
```

```
## Warning: Removed 486 rows containing non-finite outside the scale range  
## (`stat_density()`).
```

Density Plot of thal



```
table(df$thal)
```

```
##
##    3    6    7
## 189   42  182
```

```
# impute with mode and missingness flag
```

```
df <- df %>%
  mutate(
    thal_missing = ifelse(is.na(df$thal), 1, 0),
    thal_imputed = ifelse(is.na(df$thal), 3, df$thal) # mode imputation
  )
```

```
# drop imputed column
```

```
df <- df %>%
  select(-thal)
```

Feature: thaltime

```
summary(df$thaltime)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
##      0.00   3.00   6.00   5.69   8.00  20.00   453
```

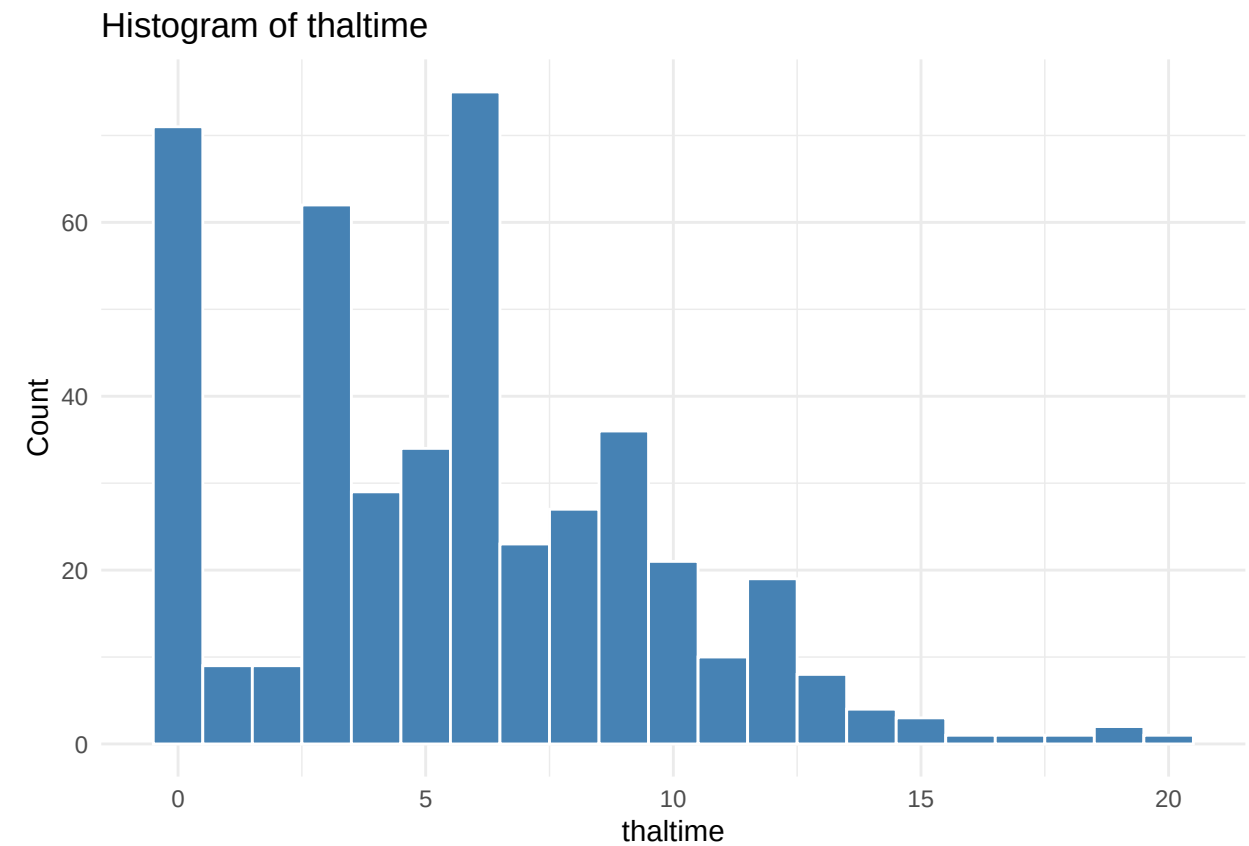
```
sort(table(df$thaltime), decreasing = TRUE)
```

```
##
##      0      6      3      9      8     10      4     12      5     7.5     5.5      7     4.5      2     11      1
```

```
## 70 65 57 28 20 19 17 17 16 11 10 9 8 7 6 5
## 3.5 6.5 8.5 13 1.5 8.6 15 2.5 3.6 4.8 5.1 5.8 6.3 7.4 7.8 9.3
## 5 5 4 4 3 3 3 2 2 2 2 2 2 2 2 2
## 9.5 13.2 14 19 0.5 0.7 3.8 4.2 4.6 4.7 5.2 5.4 6.4 6.8 7.9 9.4
## 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1
## 10.2 10.5 10.6 10.7 11.1 11.3 11.7 11.8 12.6 13.5 14.3 14.5 15.8 17.5 17.8 20
## 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

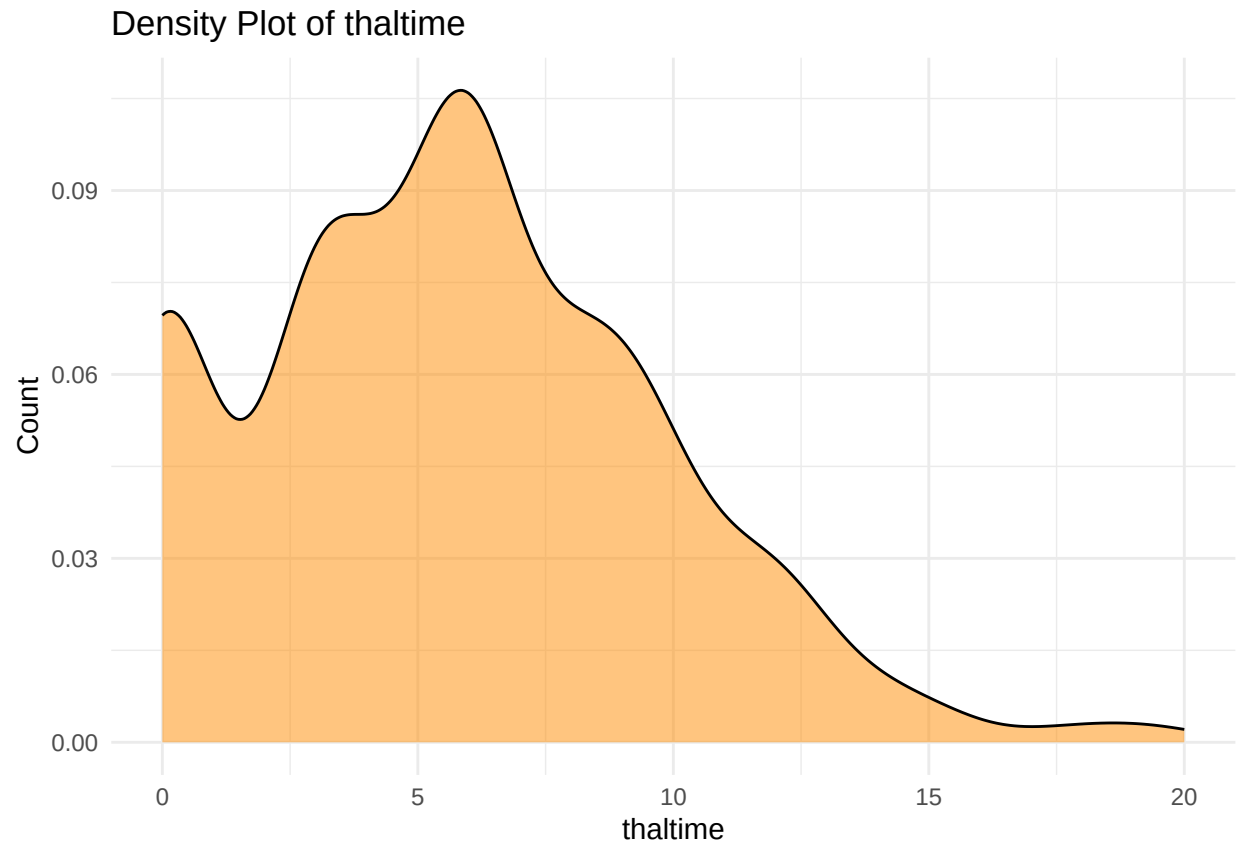
```
# histogram
show_histogram(df, thaltime)
```

```
## Warning: Removed 453 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



```
show_densityplot(df, thaltime)
```

```
## Warning: Removed 453 rows containing non-finite outside the scale range
## (`stat_density()`).
```



```
# impute with median + missingness flag
df <- df %>%
  mutate(
    thaltime_missing = ifelse(is.na(thaltime), 1, 0),
    thaltime_imputed = ifelse(is.na(thaltime), median(thaltime, na.rm = TRUE), thaltime)
  )

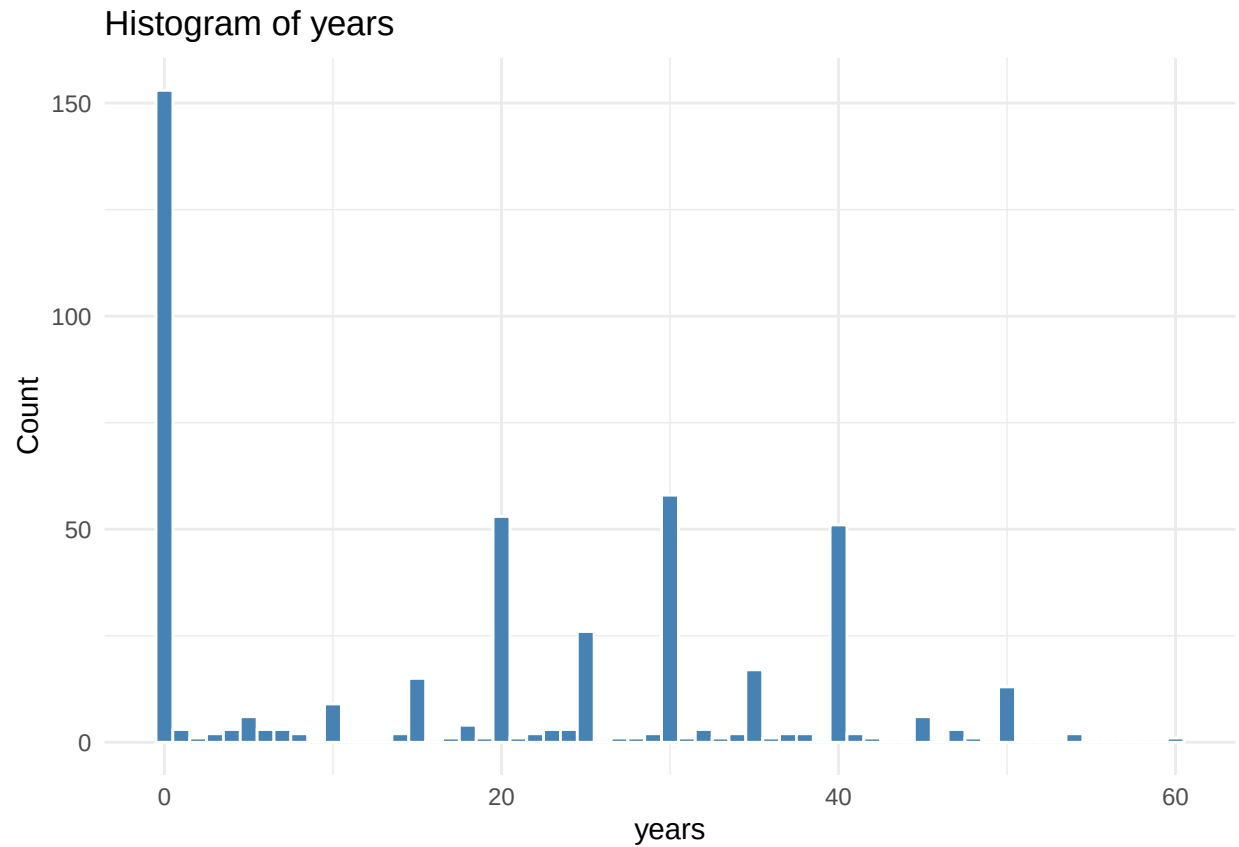
# drop imputed column
df <- df %>%
  select(-thaltime)
```

Feature thaltime showed a right skew. Imputed with median.

Features: years and cigs

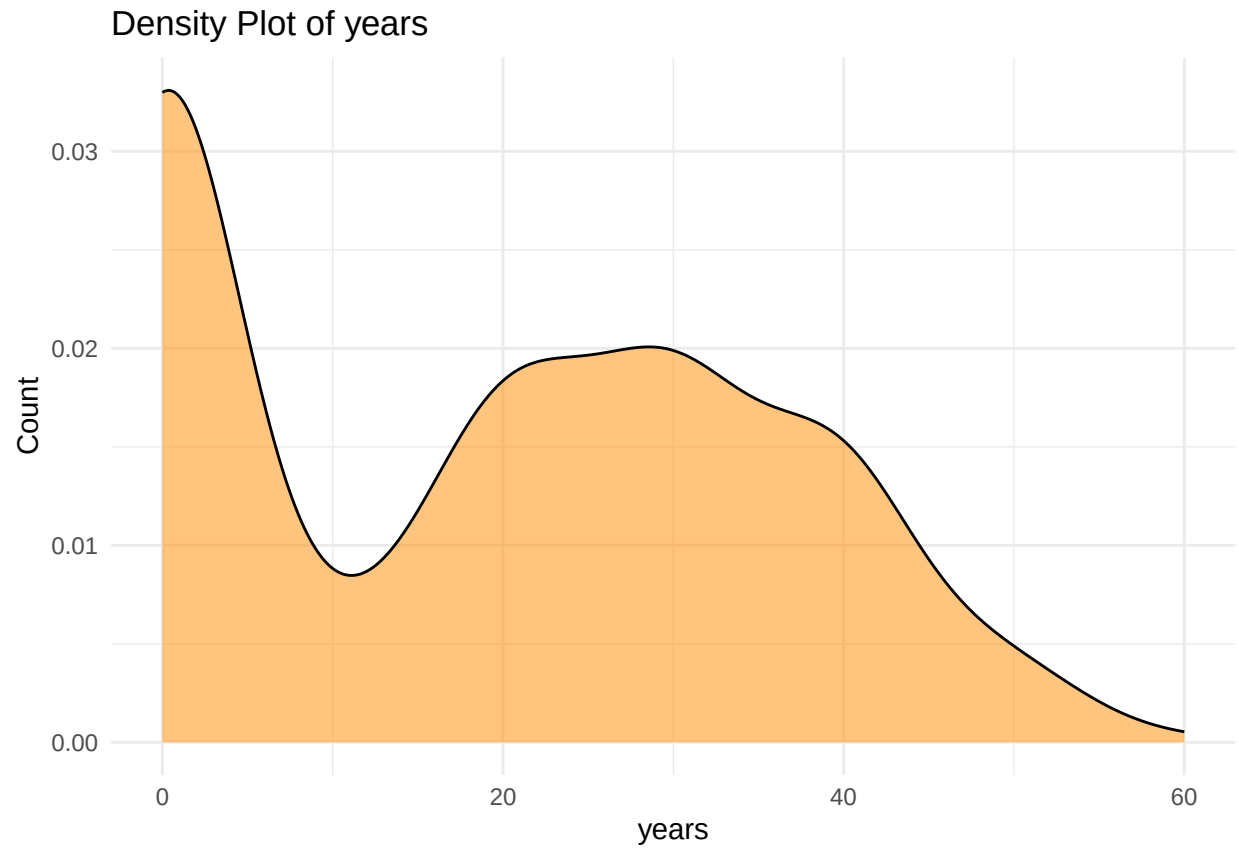
```
show_histogram(df, years)
```

```
## Warning: Removed 432 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



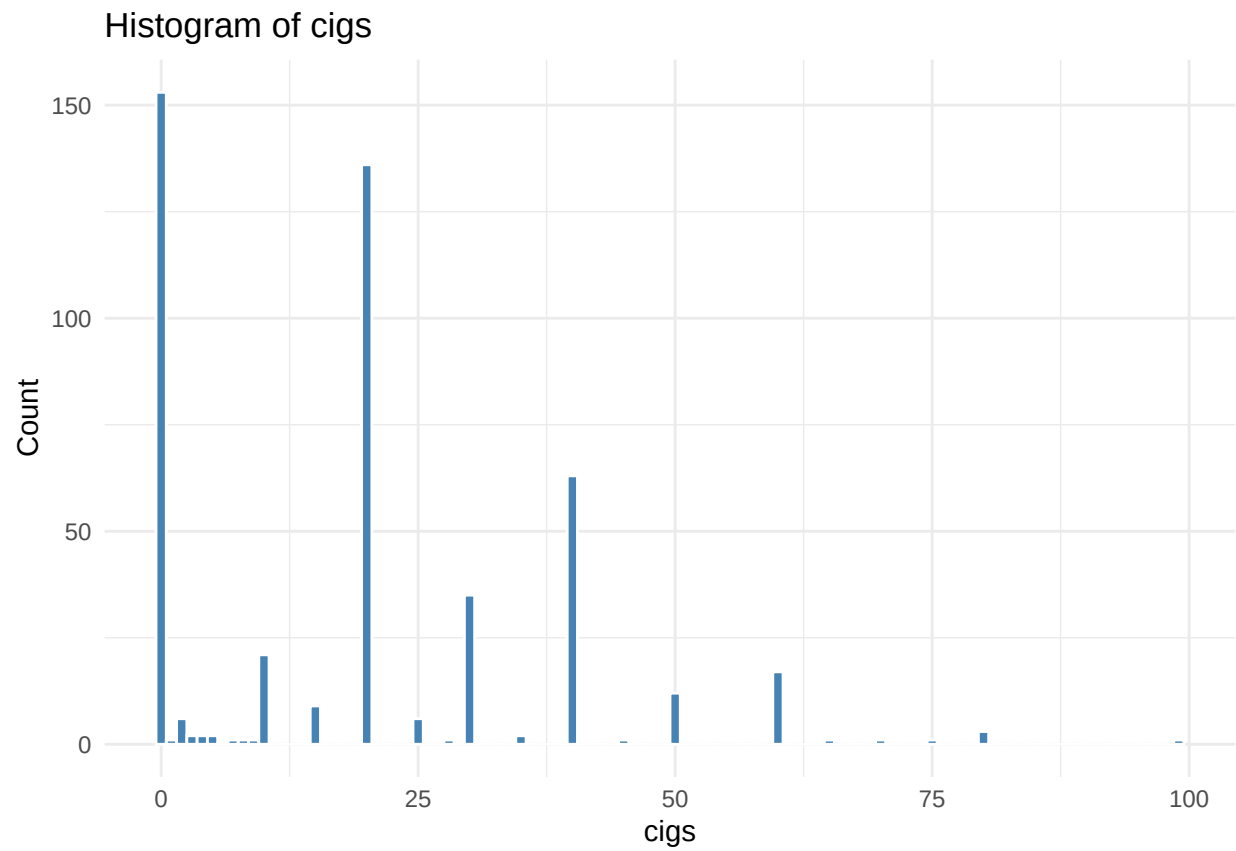
```
show_densityplot(df, years)
```

```
## Warning: Removed 432 rows containing non-finite outside the scale range  
## (`stat_density()`).
```

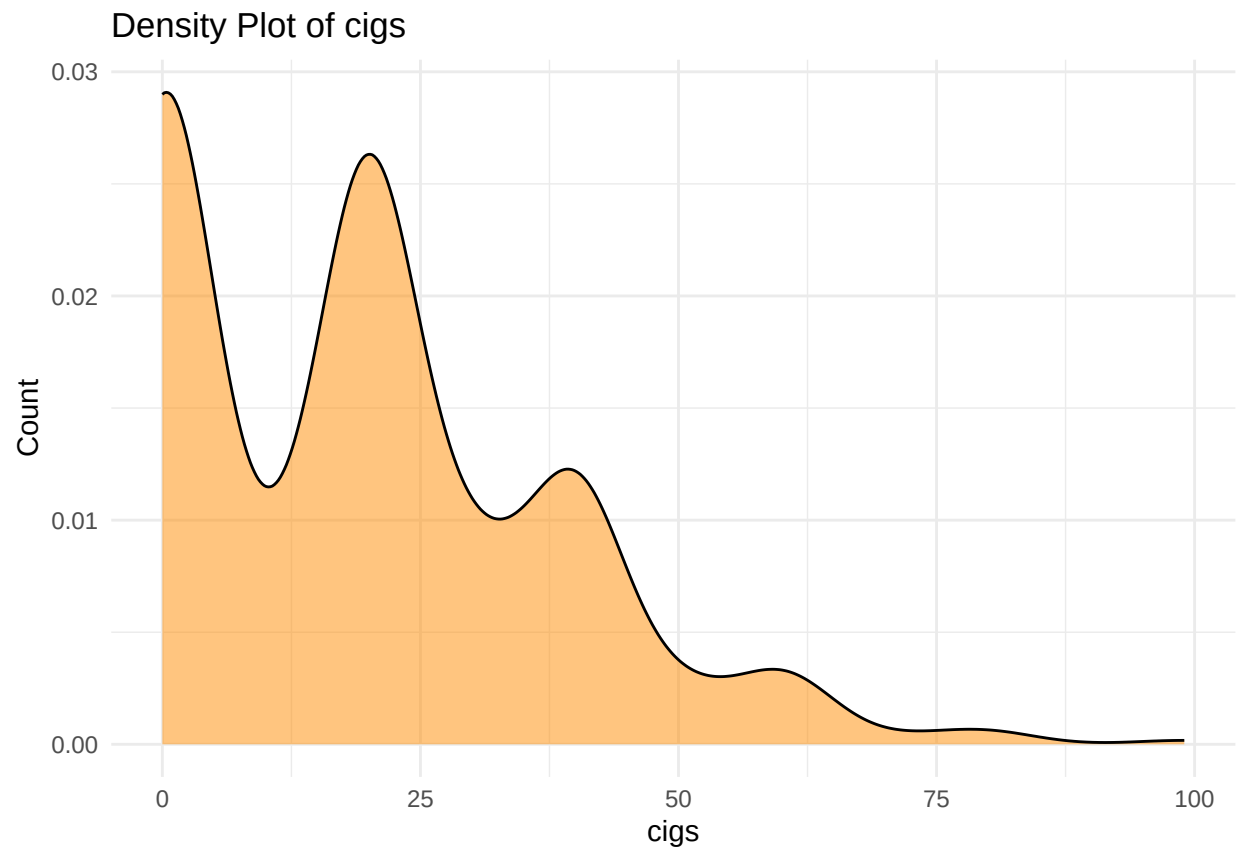
```
show_histogram(df, cigs)
```

```
## Warning: Removed 420 rows containing non-finite outside the scale range  
## (`stat_bin()`).
```



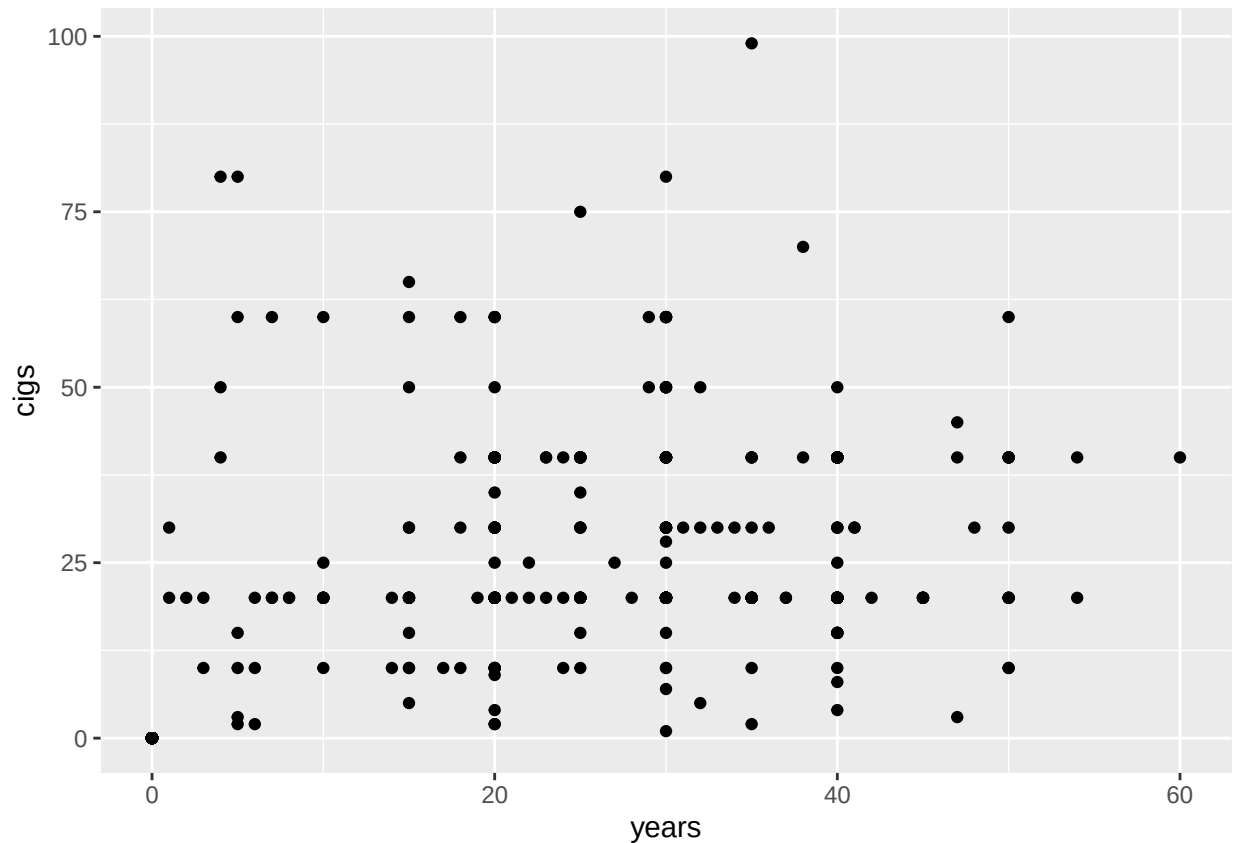
```
show_densityplot(df, cigs)
```

```
## Warning: Removed 420 rows containing non-finite outside the scale range
## (`stat_density()`).
```



```
ggplot(df, aes(x = years, y = cigs)) + geom_point()
```

```
## Warning: Removed 435 rows containing missing values or values outside the scale range  
## (`geom_point()`).
```



```
corr_val <- cor(df$years, df$cigs, use = "complete.obs", method = "pearson")
cat("Correlation value:", corr_val)
```

```
## Correlation value: 0.5890923
```

```
# create smoke_profile feature
# generative AI was used to create this code to create the smoke_profile feature
```

```
df <- df %>%
  mutate(
    # Categorize smoking duration
    years_cat = case_when(
      is.na(years) ~ "missing",
      years == 0 ~ "non",
      years <= 10 ~ "short",
      years <= 20 ~ "medium",
      years <= 40 ~ "long",
      TRUE ~ "very_long"
    ),

    # Categorize cigarette intensity
    cigs_cat = case_when(
      is.na(cigs) ~ "missing",
      cigs == 0 ~ "non",
      cigs <= 10 ~ "light",
      cigs <= 20 ~ "moderate",
      cigs <= 40 ~ "heavy",
      TRUE ~ "very_heavy"
    )
  )
```

```

    ),

    # Combine into single profile
    smoke_profile = case_when(
      years_cat == "non" & cigs_cat == "non" ~ "non",
      years_cat == "missing" | cigs_cat == "missing" ~ "missing",
      TRUE ~ paste0(cigs_cat, "_", years_cat)
    ),

    # Optional: convert to factor
    smoke_profile = factor(smoke_profile)
  )

# drop helper columns
df <- df %>%
  select(-c(cigs_cat, years_cat))

# drop imputed column
df <- df %>%
  select(-c(years, cigs))

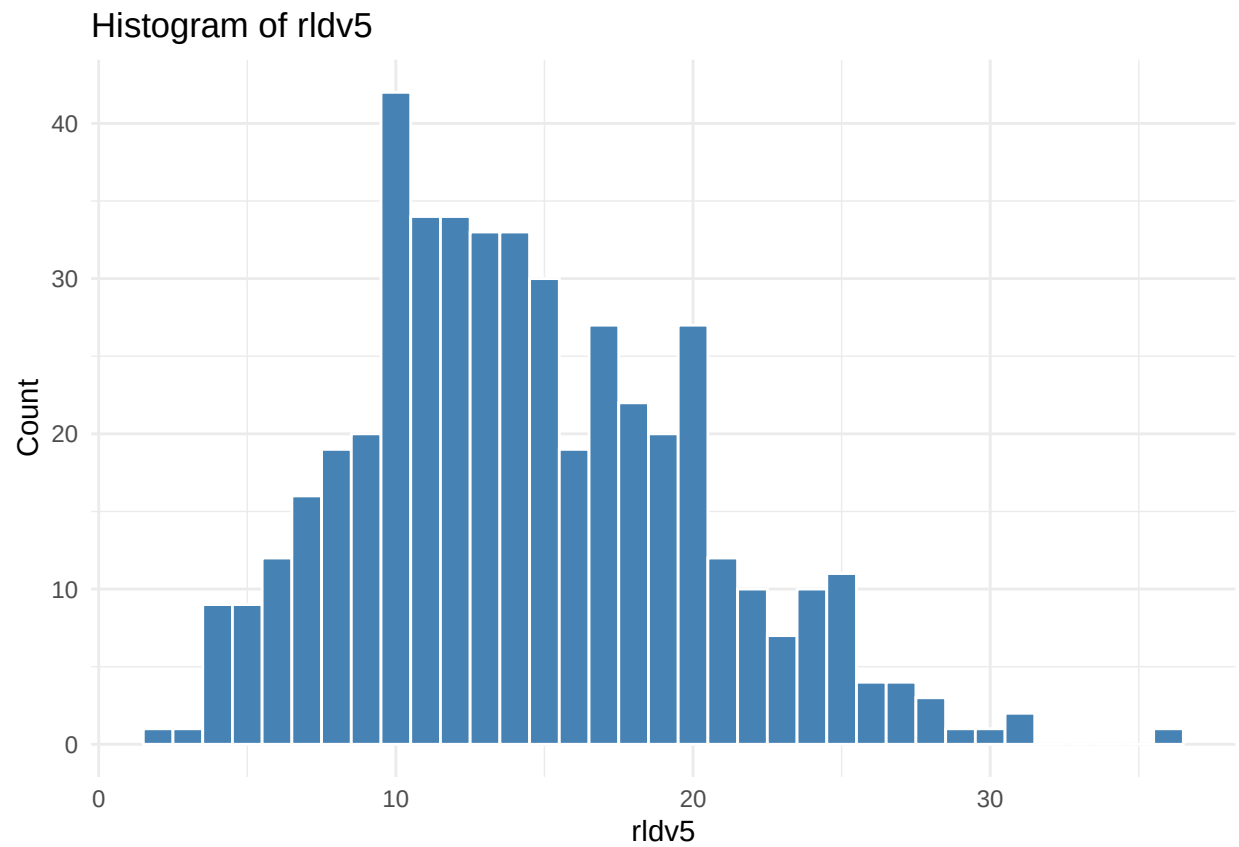
```

Features `years` and `cigs` show moderate collinearity (Pearson correlation = 0.59). Due to missing data, a `smoke_profile` was engineered to combine the two columns.

Feature: `rldv5`

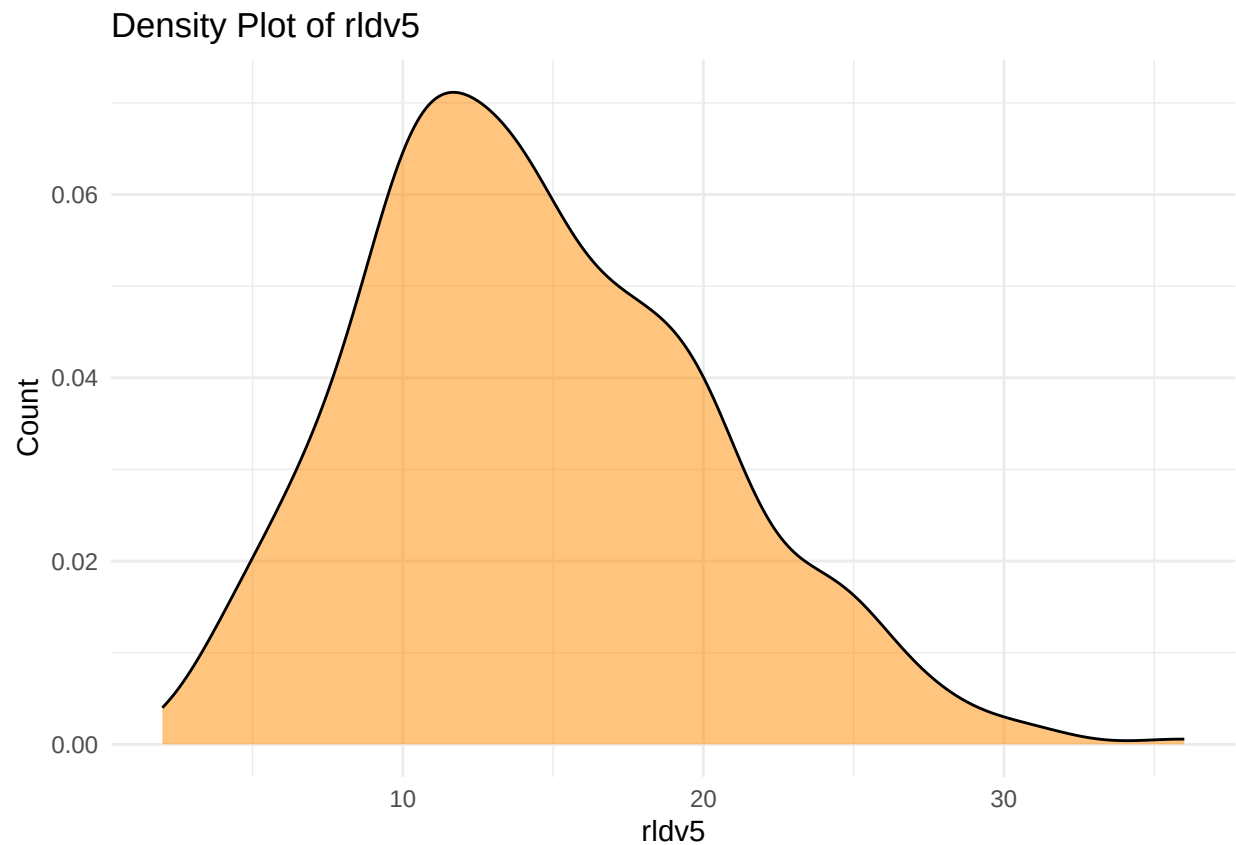
```
show_histogram(df, rldv5)
```

```
## Warning: Removed 425 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



```
show_densityplot(df, rldv5)
```

```
## Warning: Removed 425 rows containing non-finite outside the scale range  
## (`stat_density()`).
```

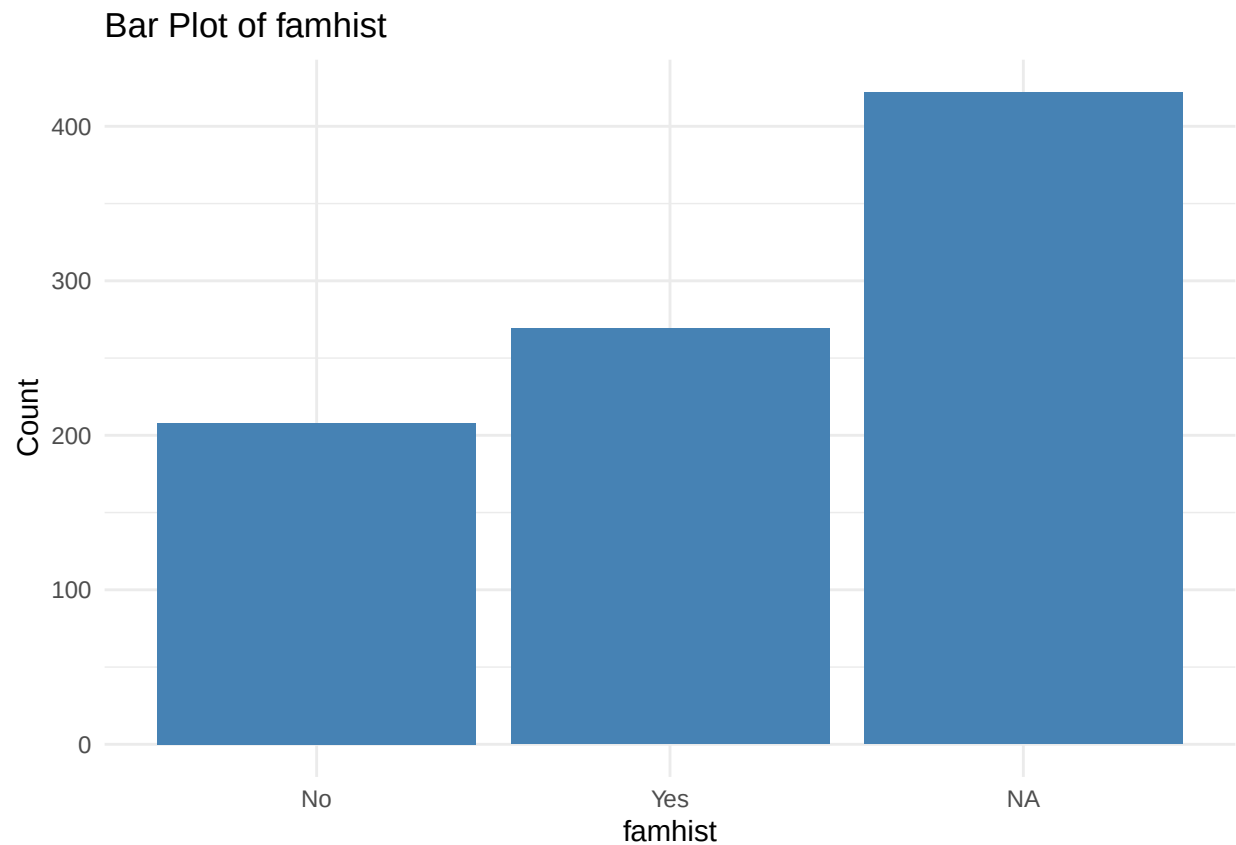


```
df <- df %>%  
  mutate(  
    rldv5_missing = ifelse(is.na(rldv5), 1, 0),  
    rldv5_imputed = ifelse(is.na(rldv5), median(rldv5, na.rm = TRUE), rldv5)  
  )  
  
# drop imputed column  
df <- df %>%  
  select(-rldv5)
```

Feature `rldv5` has a right skew therefore imputed with median.

Feature: famhist

```
show_barplot(df, famhist)
```



```
unique(df$famhist)
```

```
## [1] Yes  No  <NA>
```

```
## Levels: No Yes
```

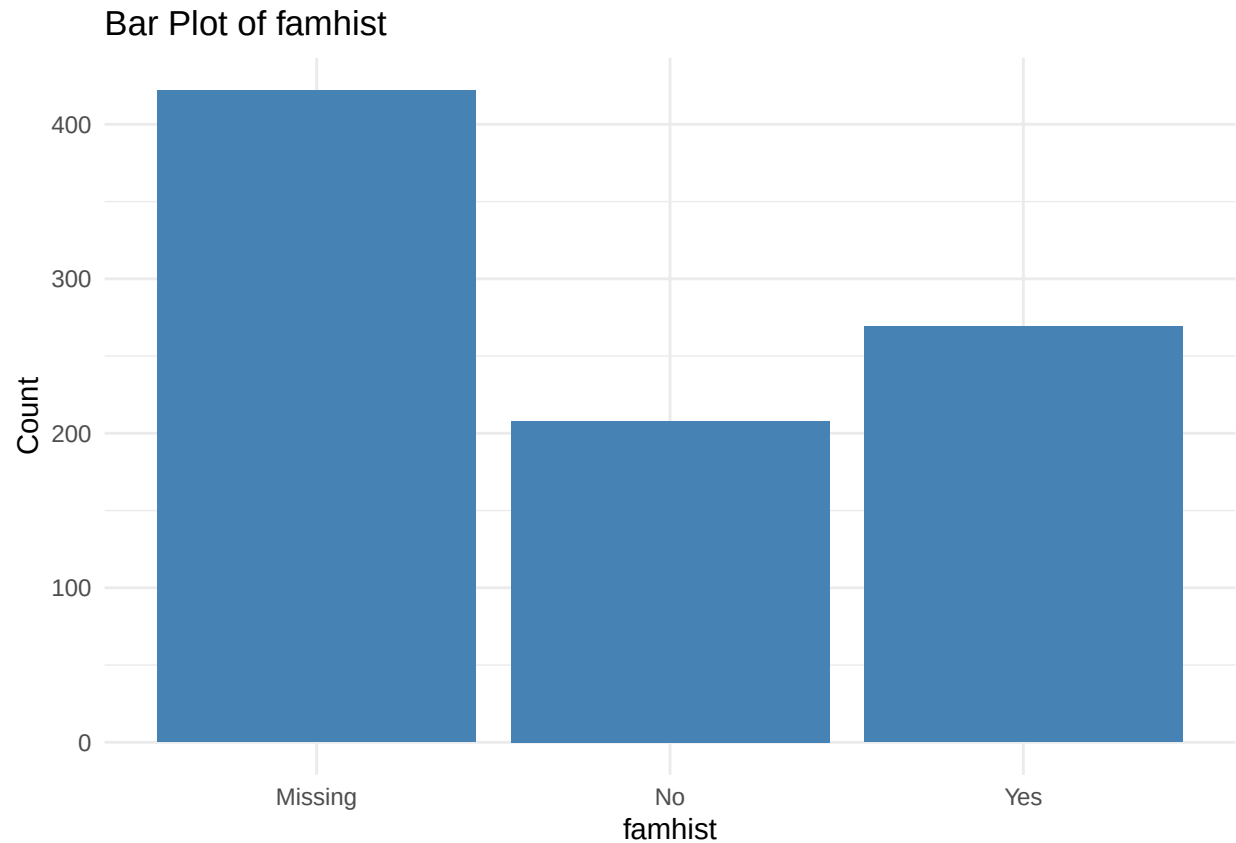
```
# replace NA with "Missing"
```

```
df$famhist <- as.character(df$famhist)
```

```
df$famhist[is.na(df$famhist)] <- "Missing"
```

```
df$famhist <- factor(df$famhist)
```

```
show_barplot(df, famhist)
```

Missing values in `famhist` were imputed with 'missing'.

```
# list of cols
# missing_gt40 <- c('ca', 'thal', 'thalttime', 'years', 'rldv5', 'famhist', 'cigs')
imputed_cols <-c('ca_imputed', 'ca_missing', 'thal_imputed', 'thal_missing',
                 'thalttime_missing', 'thalttime_imputed', 'smoke_profile',
                 'rldv5_missing', 'rldv5_imputed', 'famhist')
# drop date / time columns
drop_cols <- c('cmo', 'cday', 'cyr', 'ekgmo', 'ekgday', 'ekgyr')
df1 <- df %>%
  select(-all_of(drop_cols))

# compare dimensions
dim(df_cleaning)
```

Dropping original columns after imputation

```
## [1] 899 48
```

```
dim(df)
```

```
## [1] 899 51
```

```
dim(df1)
```

```
## [1] 899 45
```

```
colnames(df1)
```

```
## [1] "age"          "sex"          "painloc"      "painexer"
## [5] "relrest"      "cp"           "trestbps"     "htn"
## [9] "chol"         "fbs"          "famhist"      "restecg"
## [13] "prop"         "nitr"         "pro"          "diuretic"
## [17] "thaldur"      "met"          "thalach"      "thalrest"
## [21] "tpeakbps"     "tpeakbpd"     "trestbpd"     "exang"
## [25] "oldpeak"      "slope"        "rldv5e"       "num"
## [29] "lmt"          "ladprox"      "laddist"      "cxmain"
## [33] "om1"          "rcaprox"      "rcadist"      "location"
## [37] "ca_missing"   "ca_imputed"   "thal_missing" "thal_imputed"
## [41] "thalttime_missing" "thalttime_imputed" "smoke_profile" "rldv5_missing"
## [45] "rldv5_imputed"
```

```
head(df1)
```

```
## # A tibble: 6 x 45
##   age sex   painloc painexer relrest cp   trestbps   htn chol fbs   famhist
##   <dbl> <fct> <fct>   <fct>   <fct> <ord>   <dbl> <dbl> <dbl> <fct> <fct>
## 1  63 Male <NA>   <NA>   <NA> Typi~   145   1   233 Yes Yes
## 2  67 Male <NA>   <NA>   <NA> Asym~   160   1   286 No  Yes
## 3  67 Male <NA>   <NA>   <NA> Asym~   120   1   229 No  Yes
## 4  37 Male <NA>   <NA>   <NA> Non~   130   0   250 No  Yes
## 5  41 Female <NA> <NA>   <NA> Atyp~   130   1   204 No  Yes
## 6  56 Male <NA>   <NA>   <NA> Atyp~   120   1   236 No  Yes
## # i 34 more variables: restecg <ord>, prop <fct>, nitr <fct>, pro <fct>,
## # diuretic <fct>, thaldur <dbl>, met <dbl>, thalach <dbl>, thalrest <dbl>,
## # tpeakbps <dbl>, tpeakbpd <dbl>, trestbpd <dbl>, exang <fct>, oldpeak <dbl>,
## # slope <ord>, rldv5e <dbl>, num <dbl>, lmt <dbl>, ladvprox <dbl>,
## # laddist <dbl>, cxmain <dbl>, om1 <dbl>, rcaprox <dbl>, rcadist <dbl>,
## # location <chr>, ca_missing <dbl>, ca_imputed <dbl>, thal_missing <dbl>,
## # thal_imputed <dbl>, thalttime_missing <dbl>, thalttime_imputed <dbl>, ...
```

Missing < 40%

Binary Variables

```
# list binary var
df1 %>%
  summarise(across(everything(), ~ n_distinct(na.omit(.)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "n_unique") %>%
  filter(n_unique == 2)
```

```
## # A tibble: 21 x 2
##   variable n_unique
##   <chr>      <int>
## 1 sex        2
## 2 painloc    2
## 3 painexer   2
## 4 relrest    2
## 5 htn        2
## 6 fbs        2
## 7 prop       2
## 8 nitr       2
```

```
## 9 pro 2
## 10 diuretic 2
## # i 11 more rows

# get binary var names
binary_vars <- df1 %>%
  select(where(~ n_distinct(na.omit(.)) == 2)) %>%
  names()

# convert binary vars to factors
df1[binary_vars] <- lapply(df1[binary_vars], as.factor)
# convert sex to male/female
#df$sex <- factor(df$sex, levels = c(0, 1), labels = c("female", "male"))

head(df1)

## # A tibble: 6 x 45
##   age sex painloc painexer relrest cp trestbps htn chol fbs famhist
##   <dbl> <fct> <fct> <fct> <fct> <ord> <dbl> <fct> <dbl> <fct> <fct>
## 1 63 Male <NA> <NA> <NA> Typi~ 145 1 233 Yes Yes
## 2 67 Male <NA> <NA> <NA> Asym~ 160 1 286 No Yes
## 3 67 Male <NA> <NA> <NA> Asym~ 120 1 229 No Yes
## 4 37 Male <NA> <NA> <NA> Non~ 130 0 250 No Yes
## 5 41 Female <NA> <NA> <NA> Atyp~ 130 1 204 No Yes
## 6 56 Male <NA> <NA> <NA> Atyp~ 120 1 236 No Yes
## # i 34 more variables: restecg <ord>, prop <fct>, nitr <fct>, pro <fct>,
## # diuretic <fct>, thaldur <dbl>, met <dbl>, thalach <dbl>, thalrest <dbl>,
## # tpeakbps <dbl>, tpeakbpd <dbl>, trestbpd <dbl>, exang <fct>, oldpeak <dbl>,
## # slope <ord>, rldv5e <dbl>, num <dbl>, lmt <dbl>, ladprox <fct>,
## # laddist <fct>, cxmain <fct>, om1 <fct>, rcaprox <fct>, readist <fct>,
## # location <chr>, ca_missing <fct>, ca_imputed <dbl>, thal_missing <fct>,
## # thal_imputed <dbl>, thaltime_missing <fct>, thaltime_imputed <dbl>, ...
```

Numerical Variables

```
numeric_vars <- df1 %>%
  select(where(is.numeric)) %>%
  names()

numeric_na <- df1 %>%
  select(where(is.numeric)) %>%
  select(where(~ any(is.na(.)))) %>%
  names()

numeric_vars

## [1] "age" "trestbps" "chol" "thaldur"
## [5] "met" "thalach" "thalrest" "tpeakbps"
## [9] "tpeakbpd" "trestbpd" "oldpeak" "rldv5e"
## [13] "num" "lmt" "ca_imputed" "thal_imputed"
## [17] "thaltime_imputed" "rldv5_imputed"

df1[numeric_na]
```

```
## # A tibble: 899 x 12
##   trestbps chol thaldur met thalach thalrest tpeakbps tpeakbpd trestbpd
```

```
##      <dbl> <dbl>    <dbl> <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
## 1      145  233     10.5  13      150      60      190      90      85
## 2      160  286      9.5  13      108      64      160      90      90
## 3      120  229      8.5  10      129      78      140      80      80
## 4      130  250     13     17      187      84      195      68      78
## 5      130  204      7      9      172      71      160      74      86
## 6      120  236    11.3  16      178      73      165      70      75
## 7      140  268      6      7      160      83      180      84     100
## 8      120  354      9     10      163      84      165      80      80
## 9      130  254      8      9      147      75      120      70     105
## 10     140  203     5.5   7      155      86      185     120     100
## # i 889 more rows
## # i 3 more variables: oldpeak <dbl>, rldv5e <dbl>, lmt <dbl>

# impute w median and create missingness columns
for (var in numeric_na) {
  median_val <- median(df1[[var]], na.rm = TRUE)
  df1[[paste0(var, "_missing")]] <- ifelse(is.na(df1[[var]]), 1, 0)
  df1[[paste0(var, "_imputed")]] <- ifelse(is.na(df1[[var]]), median_val, df1[[var]])
}

# drop imputed columns
df1 <- df1 %>%
  select(-all_of(numeric_na))
dim(df1)

## [1] 899 57
```

Numerical variables were imputed with the median value. Missing flags were also created for these variables.

Categorical Variables

```
categorical_na <- df1 %>%
  select(where(~ is.factor(.) || is.character(.))) %>%
  select(where(~ any(is.na(.)))) %>%
  names()

# impute with Missing
for (var in categorical_na) {
  df1[[var]] <- as.character(df1[[var]])
  df1[[var]][is.na(df1[[var]])] <- "Missing"
  df1[[var]] <- factor(df1[[var]])
}
dim(df1)
```

```
## [1] 899 57
```

```
anyNA(df1)
```

```
## [1] FALSE
```

For NAs in categorical variables, NAs were replaced with 'Missing'.

Create binary for target (num)

```
# create binary target 0 = no disease, 1 - 3 = varying levels
df1 <- df1 %>%
```

```
mutate(
  has_disease = factor(ifelse(num == 0, "No", "Yes"), levels = c("No", "Yes"))
)

head(df1)

## # A tibble: 6 x 58
##   age sex   painloc painexer relrest cp   htn   fbs   famhist restecg prop
##   <dbl> <fct> <fct> <fct> <fct> <ord> <fct> <fct> <fct> <fct> <fct>
## 1    63 Male   Missing Missing Missing Typic~ 1    Yes   Yes   Hypert~ No
## 2    67 Male   Missing Missing Missing Asymp~ 1    No    Yes   Hypert~ Yes
## 3    67 Male   Missing Missing Missing Asymp~ 1    No    Yes   Hypert~ Yes
## 4    37 Male   Missing Missing Missing Non-a~ 0    No    Yes   Normal  Yes
## 5    41 Female Missing Missing Missing Atypi~ 1    No    Yes   Hypert~ No
## 6    56 Male   Missing Missing Missing Atypi~ 1    No    Yes   Normal  No
## # i 47 more variables: nitr <fct>, pro <fct>, diuretic <fct>, exang <fct>,
## # slope <fct>, num <dbl>, ladprox <fct>, laddist <fct>, cxmain <fct>,
## # om1 <fct>, rcaprox <fct>, rcadist <fct>, location <chr>, ca_missing <fct>,
## # ca_imputed <dbl>, thal_missing <fct>, thal_imputed <dbl>,
## # thaltime_missing <fct>, thaltime_imputed <dbl>, smoke_profile <fct>,
## # rldv5_missing <fct>, rldv5_imputed <dbl>, trestbps_missing <dbl>,
## # trestbps_imputed <dbl>, chol_missing <dbl>, chol_imputed <dbl>, ...
```

Binary target column for the target variable num was created. The column num was not removed from the dataset.

Export finalized dataset

```
# export final dataset
saveRDS(df1, here("Data/Clean", "heart_disease_final.rds"))
```

Univariate Analysis

```
knitr::opts_chunk$set(echo = TRUE)
```

```
df <- readRDS(here("Data/Clean", "heart_disease_final.rds"))
head(df)
```

```
## # A tibble: 6 x 58
##   age sex   painloc painexer relrest cp   htn   fbs   famhist restecg prop
##   <dbl> <fct> <fct> <fct> <fct> <ord> <fct> <fct> <fct> <fct> <fct>
## 1    63 Male   Missing Missing Missing Typic~ 1    Yes   Yes   Hypert~ No
## 2    67 Male   Missing Missing Missing Asymp~ 1    No    Yes   Hypert~ Yes
## 3    67 Male   Missing Missing Missing Asymp~ 1    No    Yes   Hypert~ Yes
## 4    37 Male   Missing Missing Missing Non-a~ 0    No    Yes   Normal  Yes
## 5    41 Female Missing Missing Missing Atypi~ 1    No    Yes   Hypert~ No
## 6    56 Male   Missing Missing Missing Atypi~ 1    No    Yes   Normal  No
## # i 47 more variables: nitr <fct>, pro <fct>, diuretic <fct>, exang <fct>,
## # slope <fct>, num <dbl>, ladprox <fct>, laddist <fct>, cxmain <fct>,
## # om1 <fct>, rcaprox <fct>, rcadist <fct>, location <chr>, ca_missing <fct>,
## # ca_imputed <dbl>, thal_missing <fct>, thal_imputed <dbl>,
## # thaltime_missing <fct>, thaltime_imputed <dbl>, smoke_profile <fct>,
## # rldv5_missing <fct>, rldv5_imputed <dbl>, trestbps_missing <dbl>,
## # trestbps_imputed <dbl>, chol_missing <dbl>, chol_imputed <dbl>, ...
```

```
# separate target from variables
target_vars <- c("num", "has_disease")
y_df <- df %>%
  select(all_of(target_vars))

X_df <- df %>%
  select(-all_of(target_vars))
```

Numerical vars

```
num_vars <- X_df %>%
  select(where(is.numeric)) %>%
  names()

# summary
X_df %>%
  select(all_of(num_vars)) %>%
  summary()
```

```
##      age      ca_imputed    thal_imputed  thaltime_imputed
##  Min.   :28.00  Min.   :0.0000  Min.    :3.00  Min.    : 0.000
##  1st Qu.:47.00  1st Qu.:0.0000  1st Qu.:3.00  1st Qu.: 6.000
##  Median :54.00  Median :0.0000  Median :3.00  Median : 6.000
##  Mean   :53.48  Mean   :0.2258  Mean    :3.95  Mean    : 5.846
##  3rd Qu.:60.00  3rd Qu.:0.0000  3rd Qu.:3.00  3rd Qu.: 6.000
##  Max.   :77.00  Max.   :9.0000  Max.    :7.00  Max.   :20.000
##  rldv5_imputed  trestbps_missing  trestbps_imputed  chol_missing
##  Min.    : 2.00  Min.    :0.00000  Min.    : 0      Min.    :0.00000
##  1st Qu.:13.00  1st Qu.:0.00000  1st Qu.:120      1st Qu.:0.00000
##  Median :14.00  Median :0.00000  Median :130      Median :0.00000
##  Mean    :14.21  Mean    :0.06563  Mean    :132      Mean    :0.03337
##  3rd Qu.:14.00  3rd Qu.:0.00000  3rd Qu.:140      3rd Qu.:0.00000
##  Max.    :36.00  Max.    :1.00000  Max.    :200      Max.    :1.00000
##  chol_imputed  thaldur_missing  thaldur_imputed  met_missing
##  Min.    : 0.0  Min.    :0.00000  Min.    : 1.000  Min.    :0.0000
##  1st Qu.:177.5  1st Qu.:0.00000  1st Qu.: 6.000  1st Qu.:0.0000
##  Median :224.0  Median :0.00000  Median : 8.100  Median :0.0000
##  Mean    :199.6  Mean    :0.06229  Mean    : 8.621  Mean    :0.1168
##  3rd Qu.:267.0  3rd Qu.:0.00000  3rd Qu.:10.300  3rd Qu.:0.0000
##  Max.    :603.0  Max.    :1.00000  Max.    :24.000  Max.    :1.0000
##  met_imputed  thalach_missing  thalach_imputed  thalrest_missing
##  Min.    : 2.00  Min.    :0.00000  Min.    : 60.0  Min.    :0.00000
##  1st Qu.: 6.00  1st Qu.:0.00000  1st Qu.:120.0  1st Qu.:0.00000
##  Median : 7.00  Median :0.00000  Median :140.0  Median :0.00000
##  Mean    :15.38  Mean    :0.06118  Mean    :137.5  Mean    :0.06229
##  3rd Qu.:10.00  3rd Qu.:0.00000  3rd Qu.:155.0  3rd Qu.:0.00000
##  Max.    :200.00  Max.    :1.00000  Max.    :202.0  Max.    :1.00000
##  thalrest_imputed  tpeakbps_missing  tpeakbps_imputed  tpeakbpd_missing
##  Min.    : 37.00  Min.    :0.00000  Min.    : 84.0  Min.    :0.00000
##  1st Qu.: 65.00  1st Qu.:0.00000  1st Qu.:158.0  1st Qu.:0.00000
##  Median : 74.00  Median :0.00000  Median :170.0  Median :0.00000
##  Mean    : 75.39  Mean    :0.07008  Mean    :171.5  Mean    :0.07008
##  3rd Qu.: 84.00  3rd Qu.:0.00000  3rd Qu.:190.0  3rd Qu.:0.00000
```

```
## Max. :139.00 Max. :1.00000 Max. :240.0 Max. :1.00000
## tpeakbpd_imputed trestbpd_missing trestbpd_imputed oldpeak_missing
## Min. : 11.00 Min. :0.00000 Min. : 0.00 Min. :0.00000
## 1st Qu.: 80.00 1st Qu.:0.00000 1st Qu.: 80.00 1st Qu.:0.00000
## Median : 88.00 Median :0.00000 Median : 80.00 Median :0.00000
## Mean : 87.34 Mean :0.06563 Mean : 83.29 Mean :0.06897
## 3rd Qu.: 98.00 3rd Qu.:0.00000 3rd Qu.: 90.00 3rd Qu.:0.00000
## Max. :134.00 Max. :1.00000 Max. :120.00 Max. :1.00000
## oldpeak_imputed rldv5e_missing rldv5e_imputed lmt_missing
## Min. : -2.6000 Min. :0.000 Min. : 2.00 Min. :0.0000
## 1st Qu.: 0.0000 1st Qu.:0.000 1st Qu.: 13.00 1st Qu.:0.0000
## Median : 0.5000 Median :0.000 Median : 19.00 Median :0.0000
## Mean : 0.8449 Mean :0.158 Mean : 49.24 Mean :0.3059
## 3rd Qu.: 1.5000 3rd Qu.:0.000 3rd Qu.: 88.00 3rd Qu.:1.0000
## Max. : 6.2000 Max. :1.000 Max. :270.00 Max. :1.0000
## lmt_imputed
## Min. : 0.000
## 1st Qu.: 1.000
## Median : 1.000
## Mean : 1.225
## 3rd Qu.: 1.000
## Max. :162.000
```

*# Example: just the *_imputed columns (exclude *_missing)*

```
imputed_vars <- names(X_df)[str_detect(names(X_df), "_imputed") & !str_detect(names(X_df), "_missing")]
```

```
X_df %>%
```

```
  select(all_of(imputed_vars)) %>%
```

```
  pivot_longer(cols = everything(), names_to = "variable", values_to = "value") %>%
```

```
  ggplot(aes(x = variable, y = value)) +
```

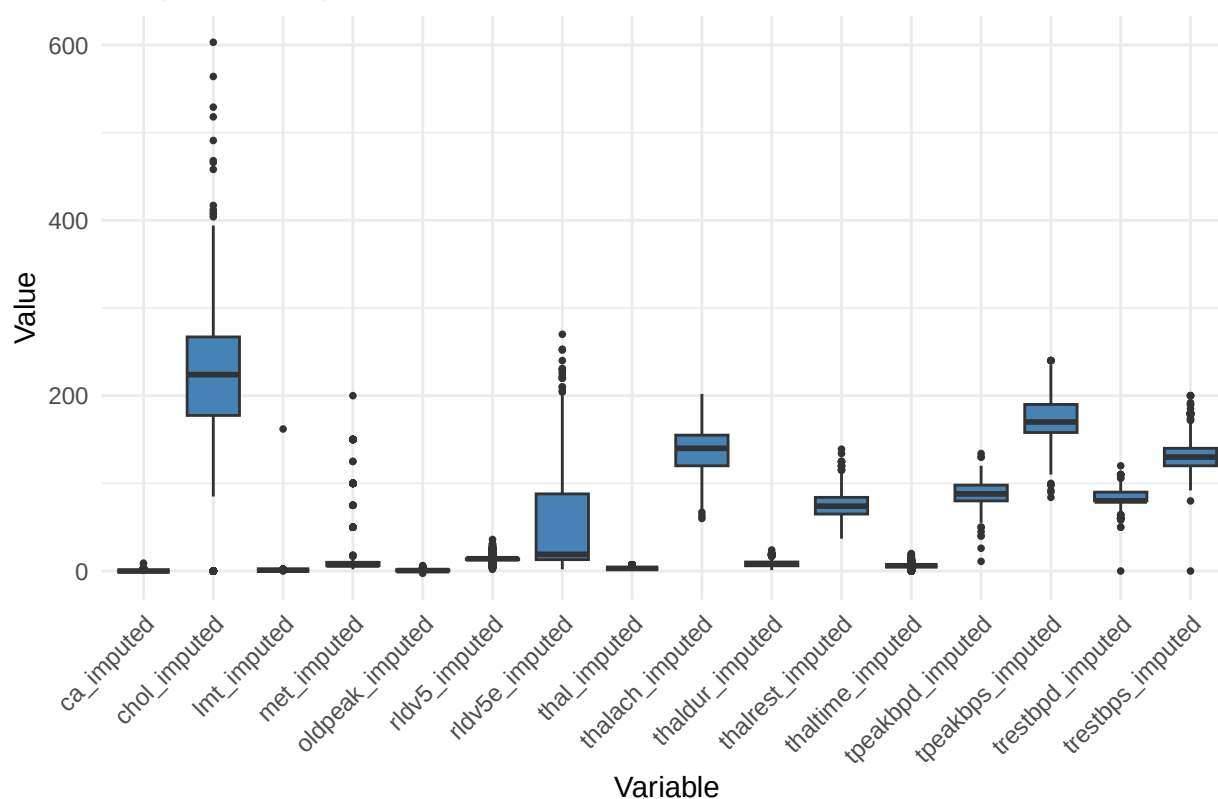
```
  geom_boxplot(fill = "steelblue", outlier.shape = 16, outlier.size = 1) +
```

```
  theme_minimal() +
```

```
  labs(title = "Boxplots of Imputed Numeric Variables", x = "Variable", y = "Value") +
```

```
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Boxplots of Imputed Numeric Variables



```
# summary statistics
X_df %>%
  select(where(is.numeric)) %>%
  skimr::skim()
```

Table 1: Data summary

Name	Piped data
Number of rows	899
Number of columns	29
Column type frequency:	
numeric	29
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
age	0	1	53.48	9.44	28.0	47.0	54.0	60.0	77.0	
ca_imputed	0	1	0.23	0.68	0.0	0.0	0.0	0.0	9.0	
thal_imputed	0	1	3.95	1.66	3.0	3.0	3.0	3.0	7.0	
thaltime_imputed	0	1	5.85	2.82	0.0	6.0	6.0	6.0	20.0	
rldv5_imputed	0	1	14.21	4.14	2.0	13.0	14.0	14.0	36.0	
trestbpd_missing	0	1	0.07	0.25	0.0	0.0	0.0	0.0	1.0	

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
trestbps_imputed	0	1	131.96	18.52	0.0	120.0	130.0	140.0	200.0	
chol_missing	0	1	0.03	0.18	0.0	0.0	0.0	0.0	1.0	
chol_imputed	0	1	199.60	110.04	0.0	177.5	224.0	267.0	603.0	
thaldur_missing	0	1	0.06	0.24	0.0	0.0	0.0	0.0	1.0	
thaldur_imputed	0	1	8.62	3.63	1.0	6.0	8.1	10.3	24.0	
met_missing	0	1	0.12	0.32	0.0	0.0	0.0	0.0	1.0	
met_imputed	0	1	15.38	29.08	2.0	6.0	7.0	10.0	200.0	
thalach_missing	0	1	0.06	0.24	0.0	0.0	0.0	0.0	1.0	
thalach_imputed	0	1	137.46	25.17	60.0	120.0	140.0	155.0	202.0	
thalrest_missing	0	1	0.06	0.24	0.0	0.0	0.0	0.0	1.0	
thalrest_imputed	0	1	75.39	14.27	37.0	65.0	74.0	84.0	139.0	
tpeakbps_missing	0	1	0.07	0.26	0.0	0.0	0.0	0.0	1.0	
tpeakbps_imputed	0	1	171.53	24.82	84.0	158.0	170.0	190.0	240.0	
tpeakbpd_missing	0	1	0.07	0.26	0.0	0.0	0.0	0.0	1.0	
tpeakbpd_imputed	0	1	87.34	14.21	11.0	80.0	88.0	98.0	134.0	
trestbpd_missing	0	1	0.07	0.25	0.0	0.0	0.0	0.0	1.0	
trestbpd_imputed	0	1	83.29	9.95	0.0	80.0	80.0	90.0	120.0	
oldpeak_missing	0	1	0.07	0.25	0.0	0.0	0.0	0.0	1.0	
oldpeak_imputed	0	1	0.84	1.05	-2.6	0.0	0.5	1.5	6.2	
rldv5e_missing	0	1	0.16	0.36	0.0	0.0	0.0	0.0	1.0	
rldv5e_imputed	0	1	49.24	56.87	2.0	13.0	19.0	88.0	270.0	
lmt_missing	0	1	0.31	0.46	0.0	0.0	0.0	1.0	1.0	
lmt_imputed	0	1	1.22	5.37	0.0	1.0	1.0	1.0	162.0	

Categorical vars

```
cat_vars <- X_df %>%
  select(where(~ is.factor(.) || is.character(.))) %>%
  names()

# relative frequency tables
for (var in cat_vars) {
  cat("----", var, "----\n")
  print(prop.table(table(X_df[[var]])))
  cat("\n")
}
```

```
## ---- sex ----
##
##      Female      Male
## 0.2091212 0.7908788
##
## ---- painloc ----
##
##      Missing  Otherwise  Substernal
## 0.31368187 0.05450501 0.63181313
##
## ---- painexer ----
##
##              Missing      Otherwise Provoked by Exertion
##              0.3136819      0.2791991      0.4071190
##
```

```

## ---- relrest ----
##
##           Missing           Otherwise Relieved after rest
##           0.3181313         0.2235818         0.4582870
##
## ---- cp ----
##
##   Typical Angina  Atypical Angina Non-anginal Pain  Asymptomatic
##           0.05005562         0.18576196         0.22469410         0.53948832
##
## ---- htn ----
##
##           0           1   Missing
## 0.5038932 0.4582870 0.0378198
##
## ---- fbs ----
##
##   Missing           No           Yes
## 0.1001112 0.7497219 0.1501669
##
## ---- famhist ----
##
##   Missing           No           Yes
## 0.4694105 0.2313682 0.2992214
##
## ---- restecg ----
##
##   Abnormal Hypertrophy   Missing   Normal
## 0.196885428 0.202447164 0.002224694 0.598442714
##
## ---- prop ----
##
##   Missing           No           Yes
## 0.07452725 0.68743048 0.23804227
##
## ---- nitr ----
##
##   Missing           No           Yes
## 0.07230256 0.68075640 0.24694105
##
## ---- pro ----
##
##   Missing           No           Yes
## 0.07007786 0.76974416 0.16017798
##
## ---- diuretic ----
##
##   Missing           No           Yes
## 0.09121246 0.80645161 0.10233593
##
## ---- exang ----
##
##   Missing           No           Yes
## 0.06117909 0.57174638 0.36707453

```

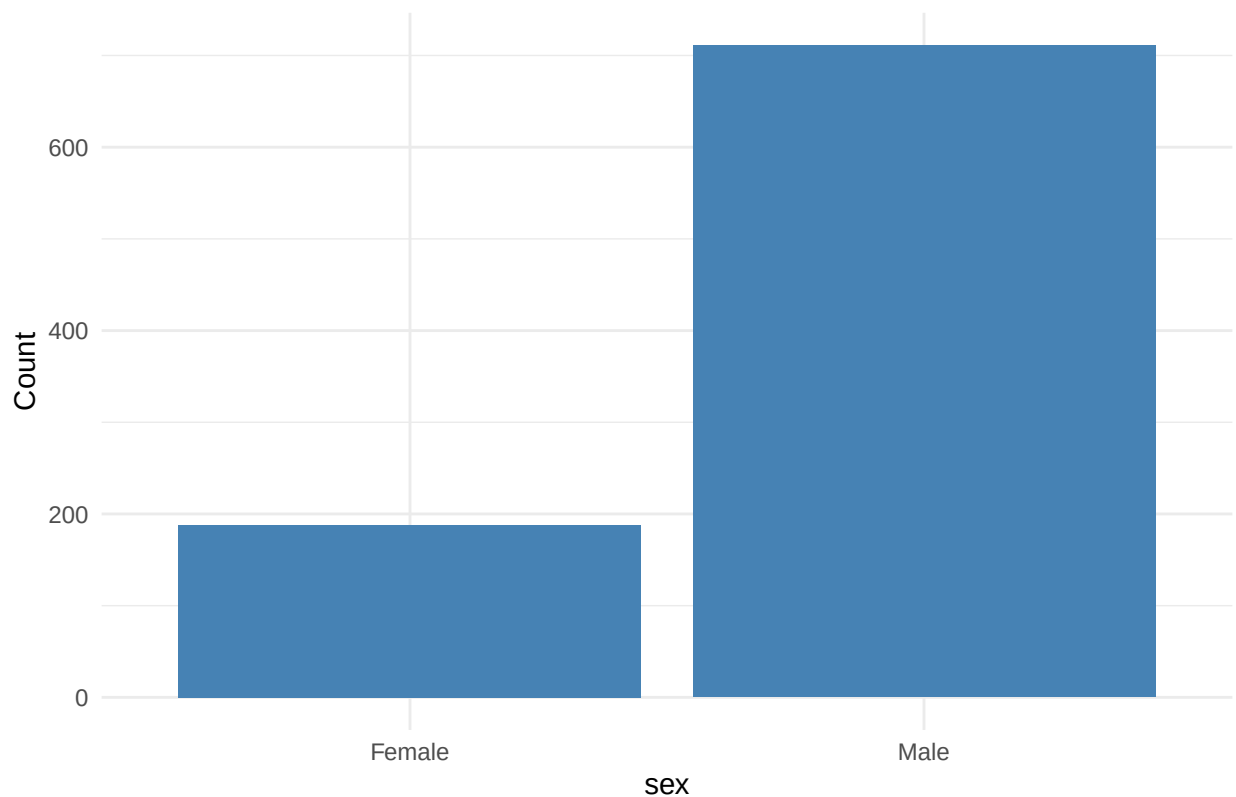
```

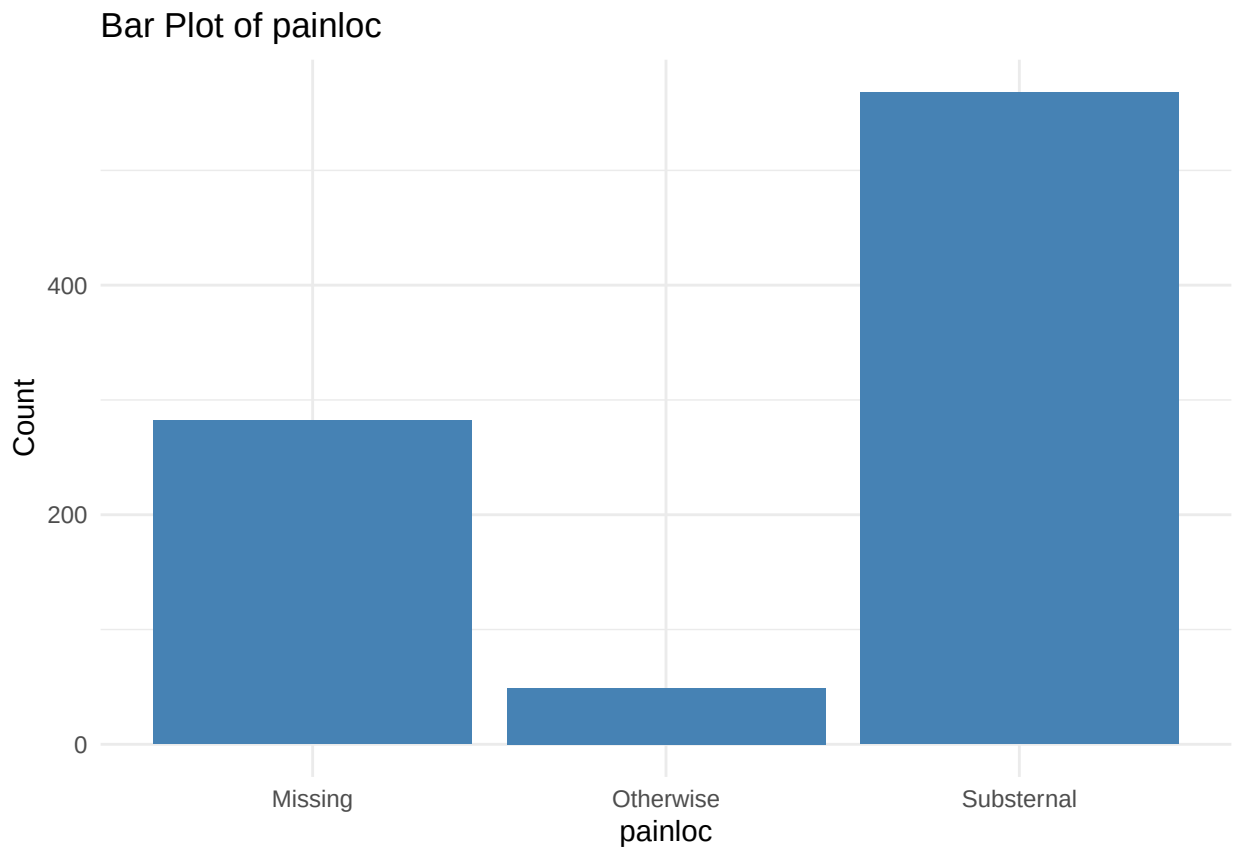
##
## ---- slope ----
##
## Downsloping      Flat      Missing  Upsloping
## 0.06674082 0.37152392 0.34371524 0.21802002
##
## ---- ladprox ----
##
##          1          2  Missing
## 0.4961068 0.2413793 0.2625139
##
## ---- laddist ----
##
##          1          2  Missing
## 0.5461624 0.1802002 0.2736374
##
## ---- cxmain ----
##
##          1          2  Missing
## 0.5194661 0.2191324 0.2614016
##
## ---- om1 ----
##
##          1          2  Missing
## 0.5750834 0.1234705 0.3014461
##
## ---- rcaprox ----
##
##          1          2  Missing
## 0.4783092 0.2491657 0.2725250
##
## ---- rcadist ----
##
##          1          2  Missing
## 0.5795328 0.1201335 0.3003337
##
## ---- location ----
##
##      Cleveland      Hungary  switzerland VA Long Beach
##      0.3136819      0.3270300      0.1368187      0.2224694
##
## ---- ca_missing ----
##
##          0          1
## 0.323693 0.676307
##
## ---- thal_missing ----
##
##          0          1
## 0.4593993 0.5406007
##
## ---- thaltime_missing ----
##
##          0          1

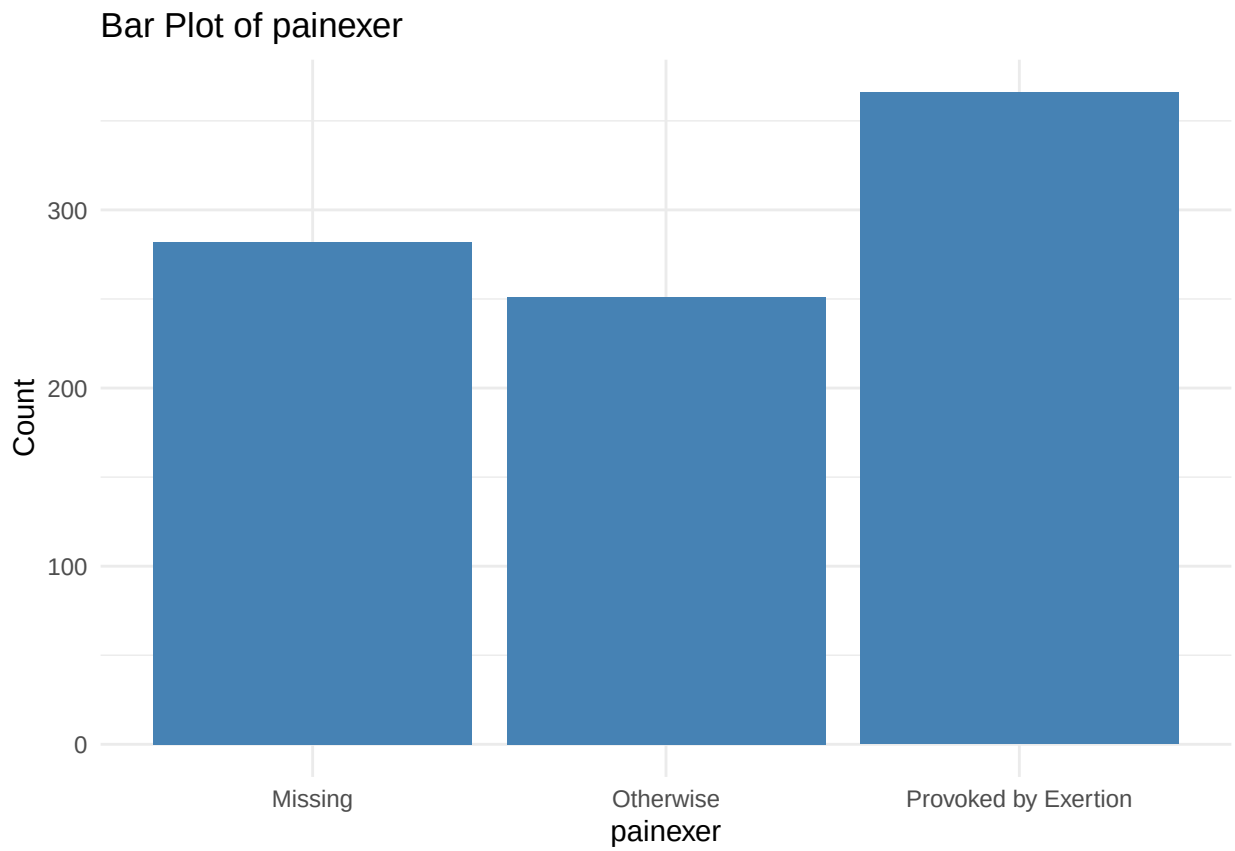
```

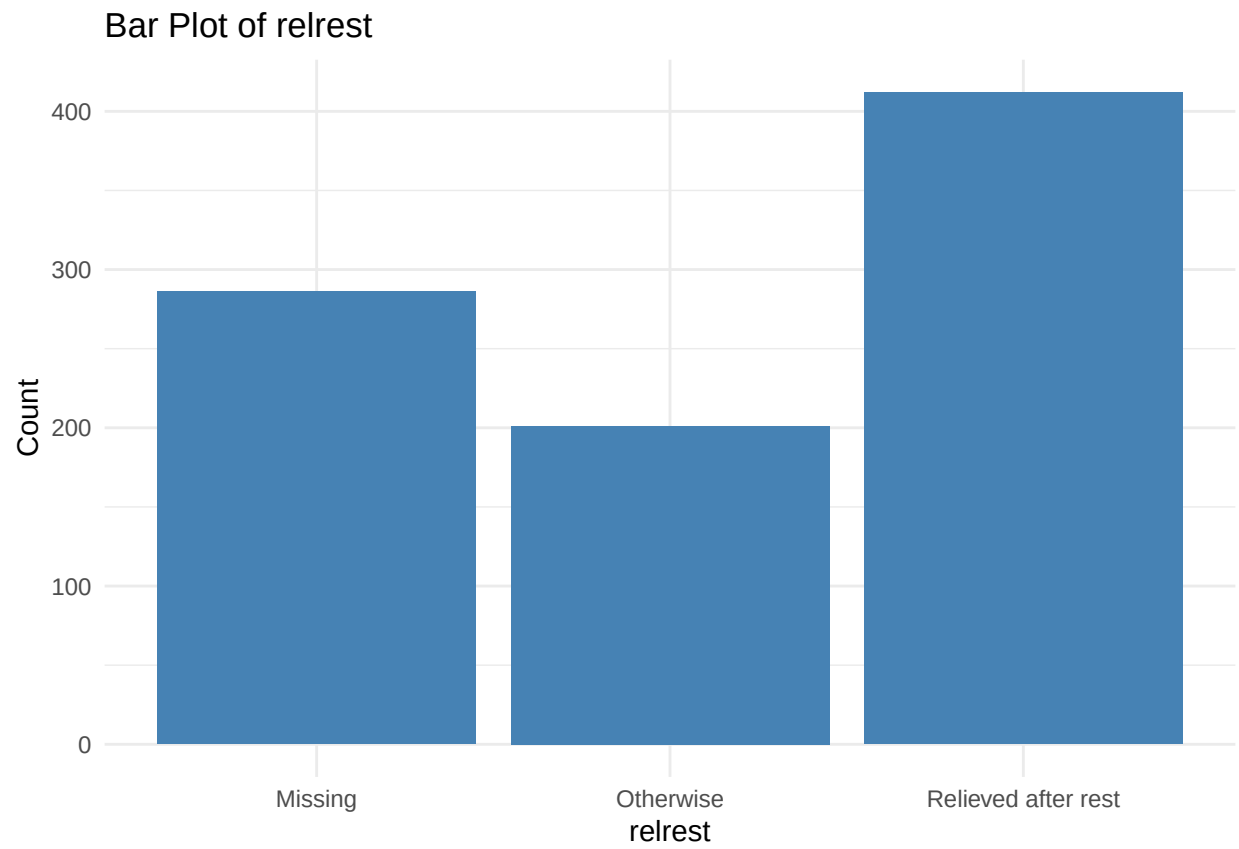
```
## 0.4961068 0.5038932
##
## ---- smoke_profile ----
##
##      heavy_long      heavy_medium      heavy_short
##      0.073414905      0.027808676      0.003337041
##      heavy_very_long      light_long      light_medium
##      0.012235818      0.013348165      0.015572859
##      light_short      light_very_long      missing
##      0.007786429      0.003337041      0.483870968
##      moderate_long      moderate_medium      moderate_short
##      0.087875417      0.032258065      0.016685206
##      moderate_very_long      non      very_heavy_long
##      0.013348165      0.170189099      0.021134594
##      very_heavy_medium      very_heavy_short      very_heavy_very_long
##      0.008898776      0.006674082      0.002224694
##
## ---- rldv5_missing ----
##
##      0      1
## 0.5272525 0.4727475
```

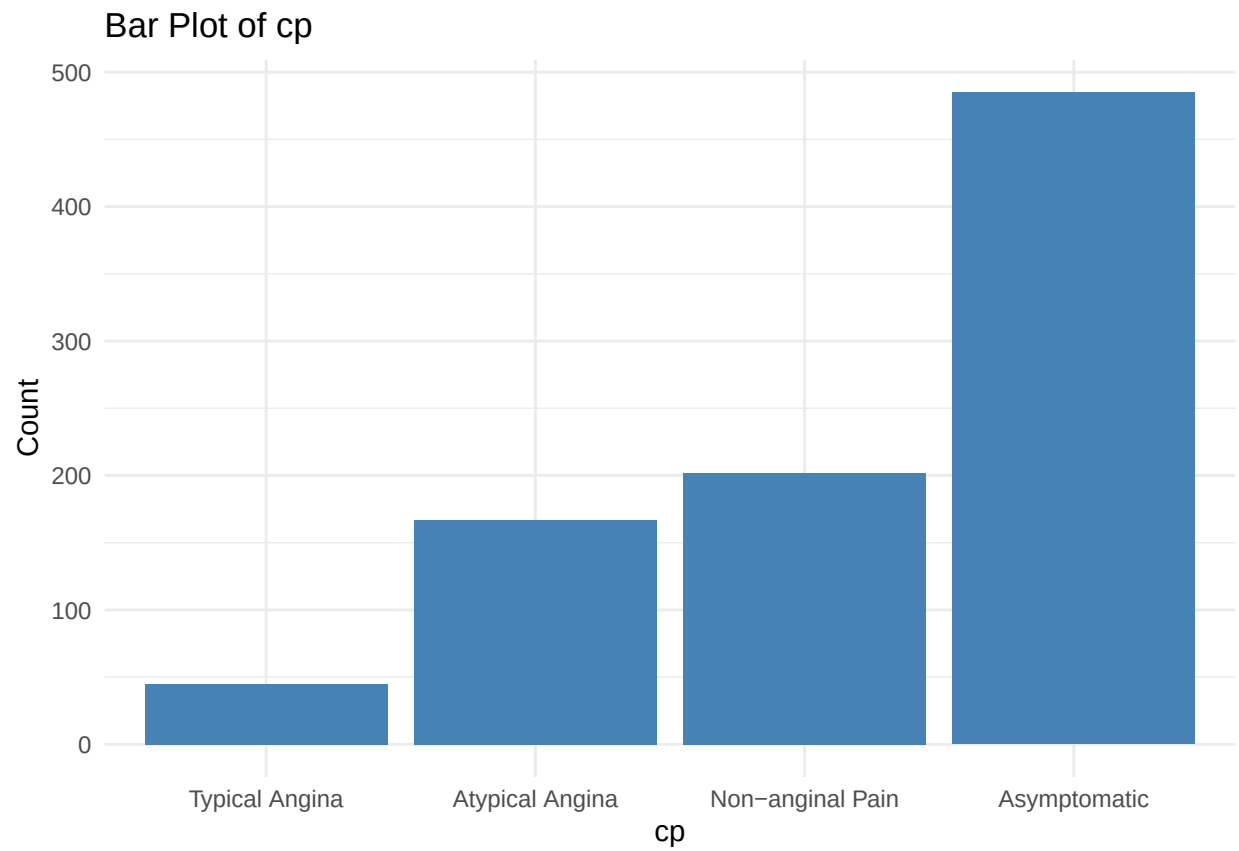
Bar Plot of sex

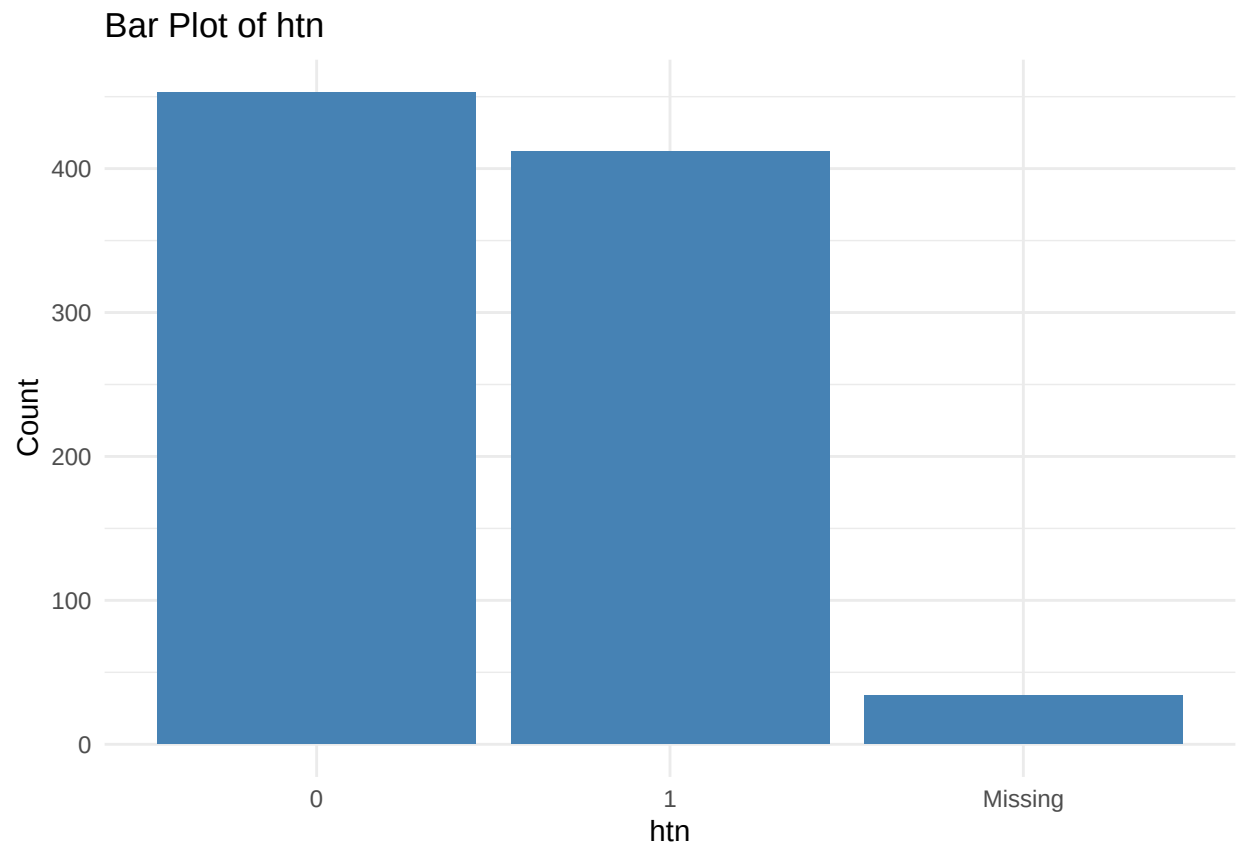


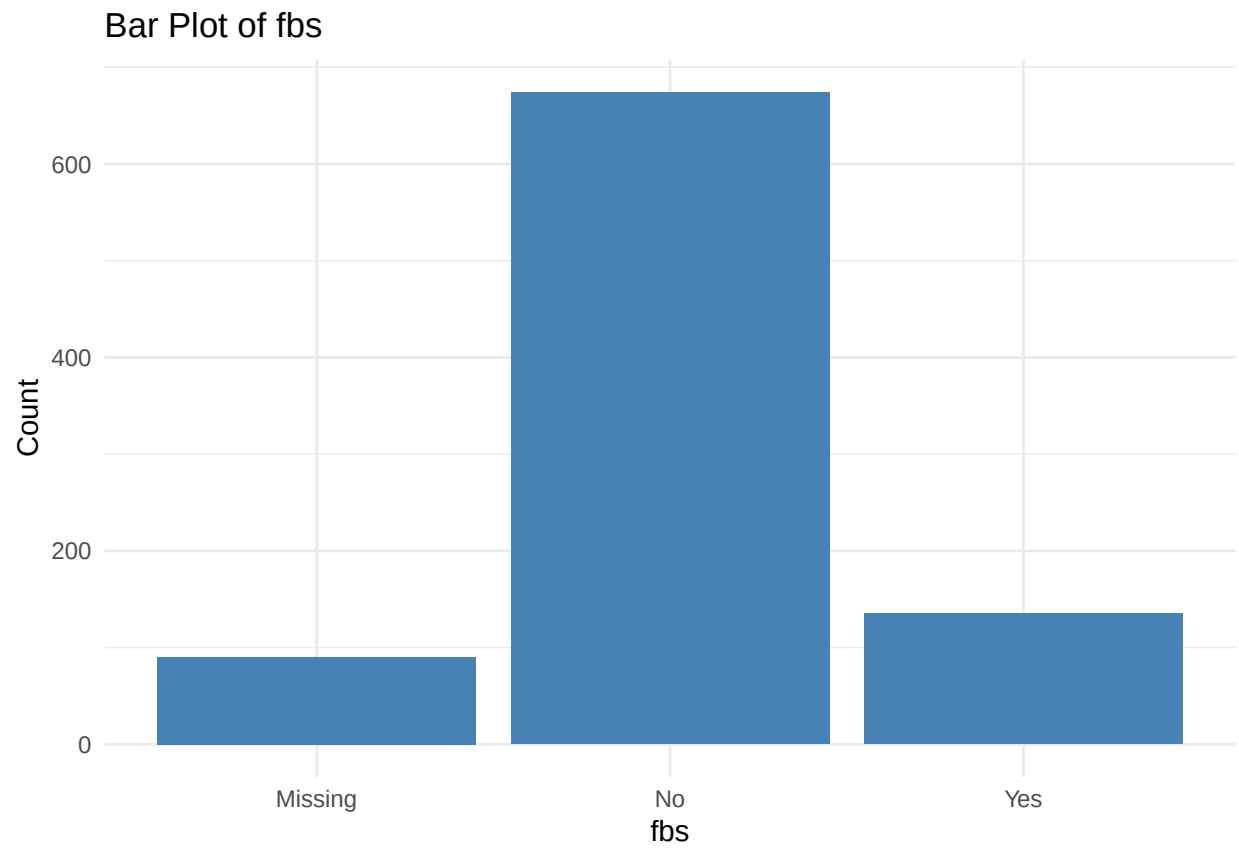


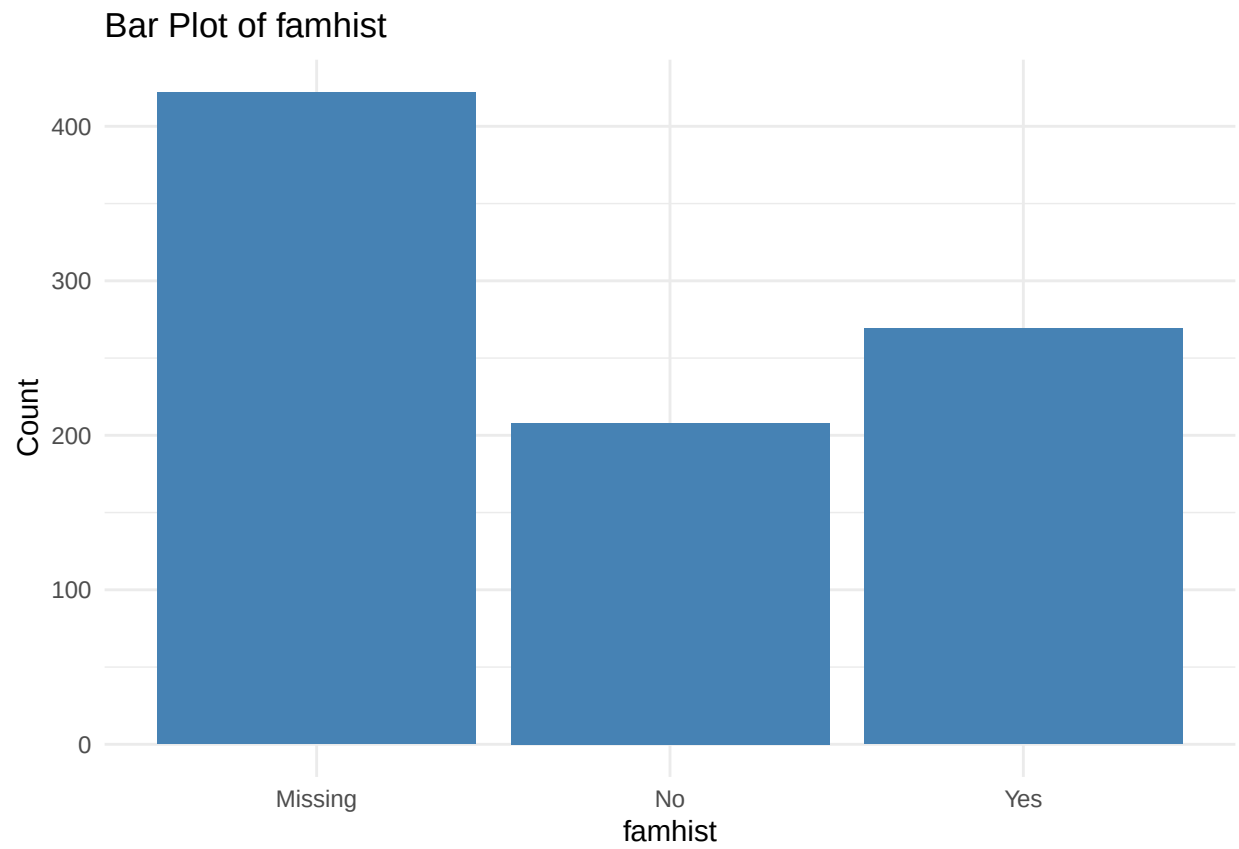


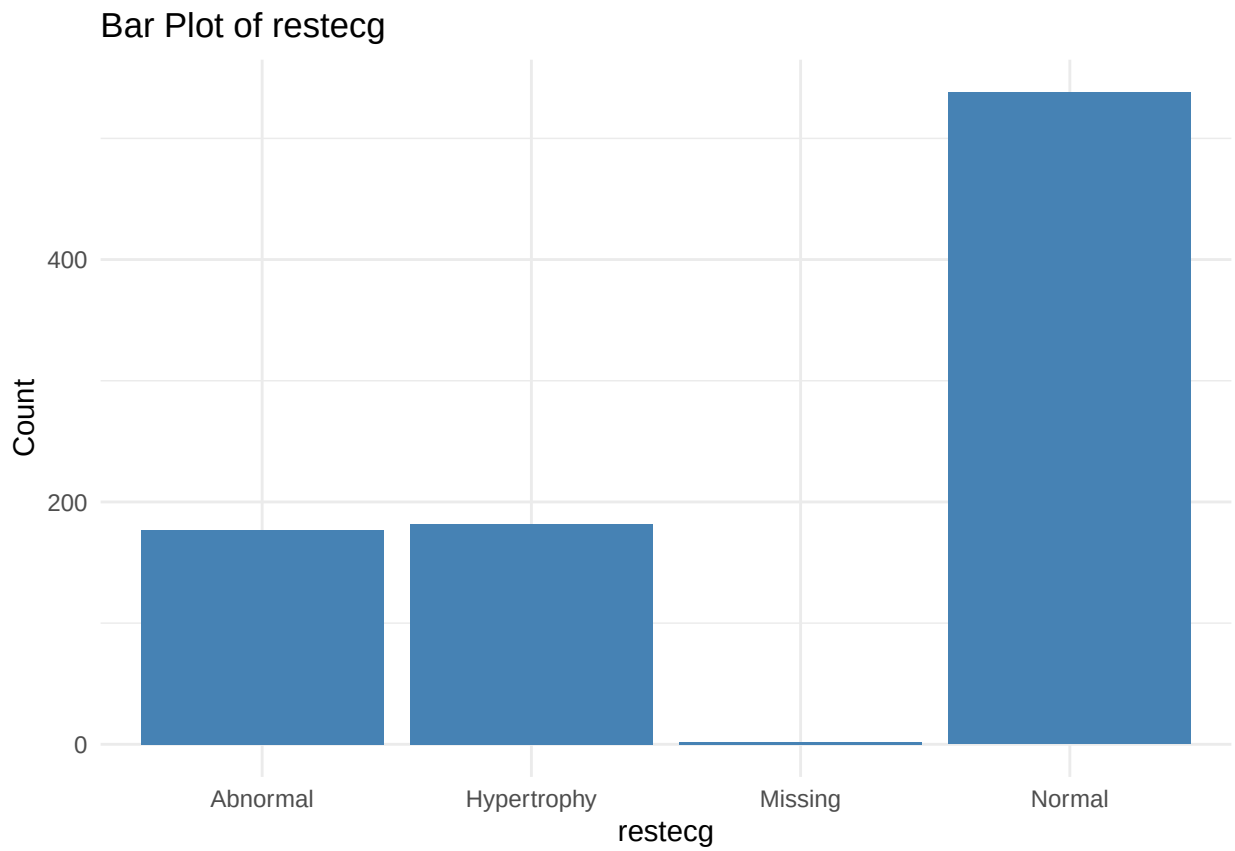


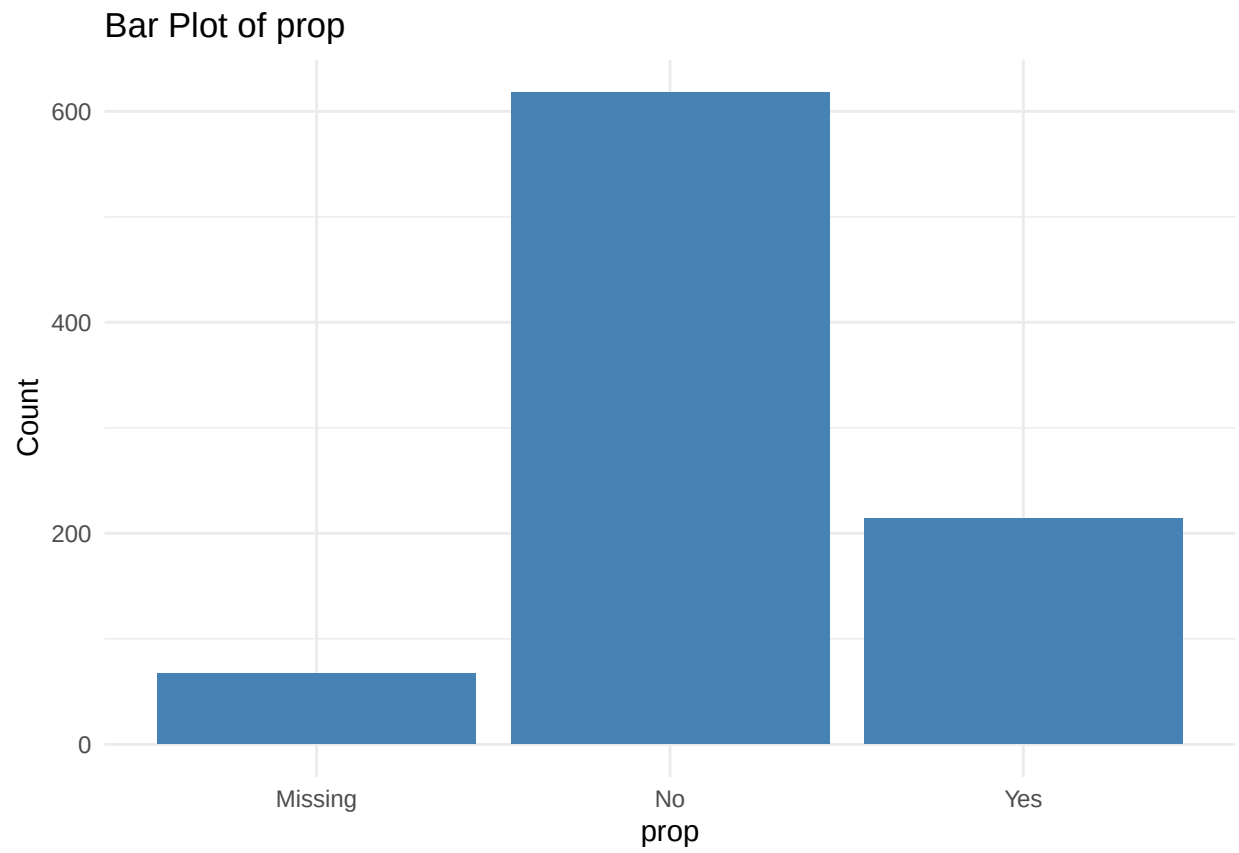


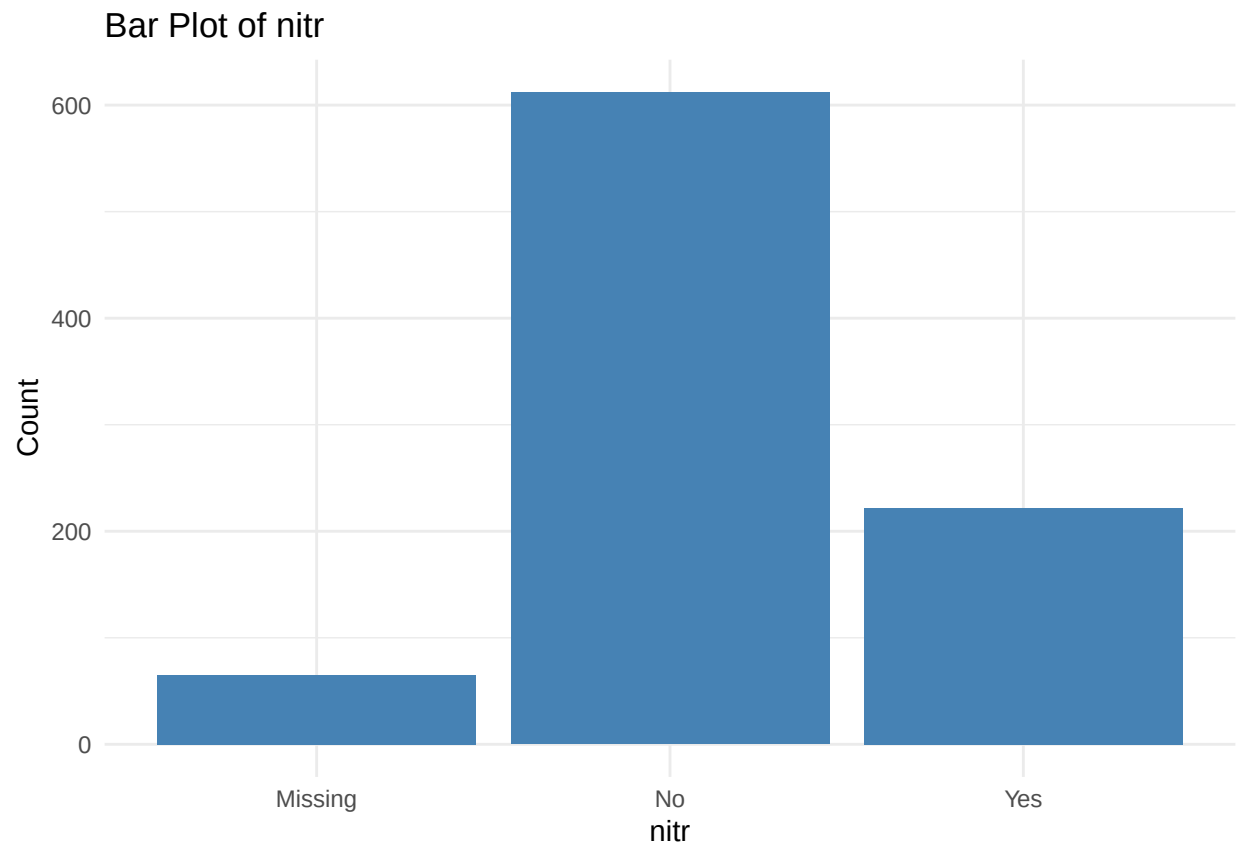


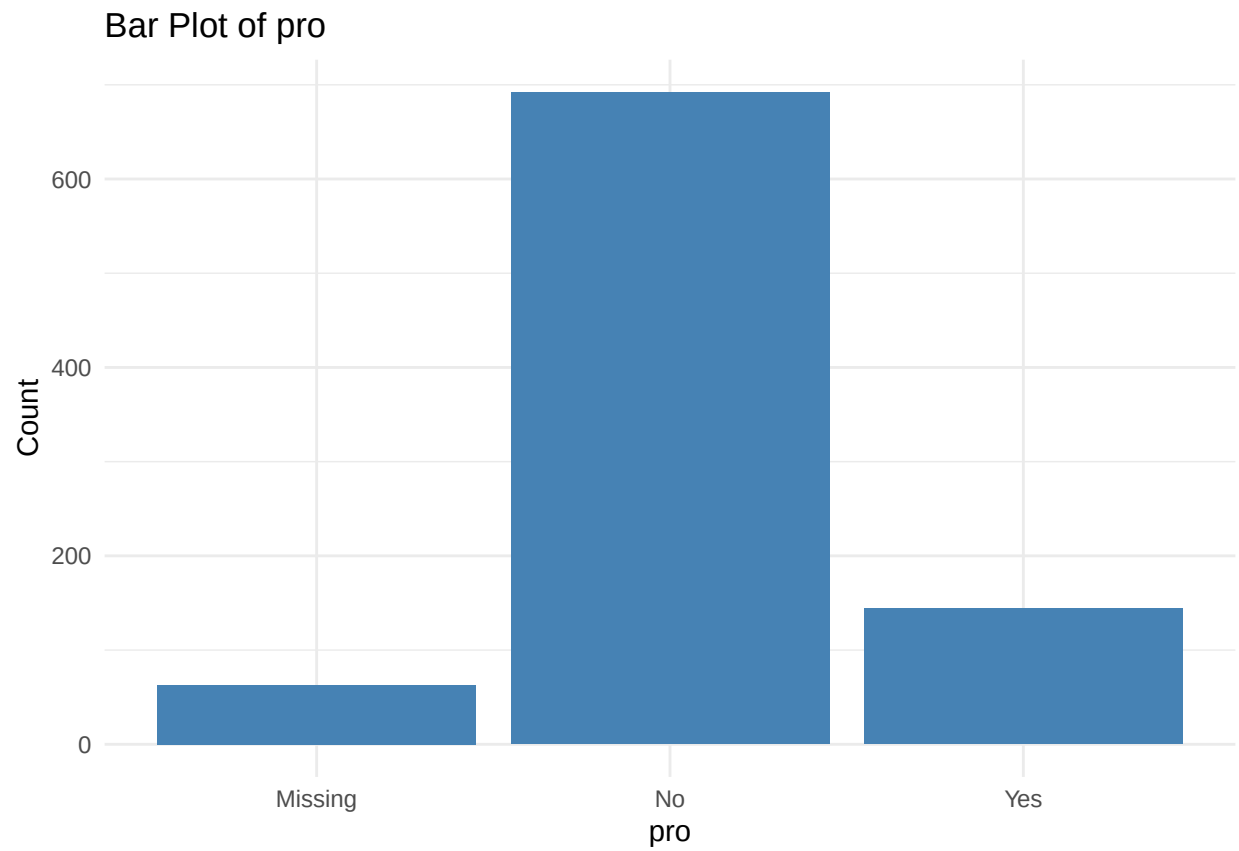


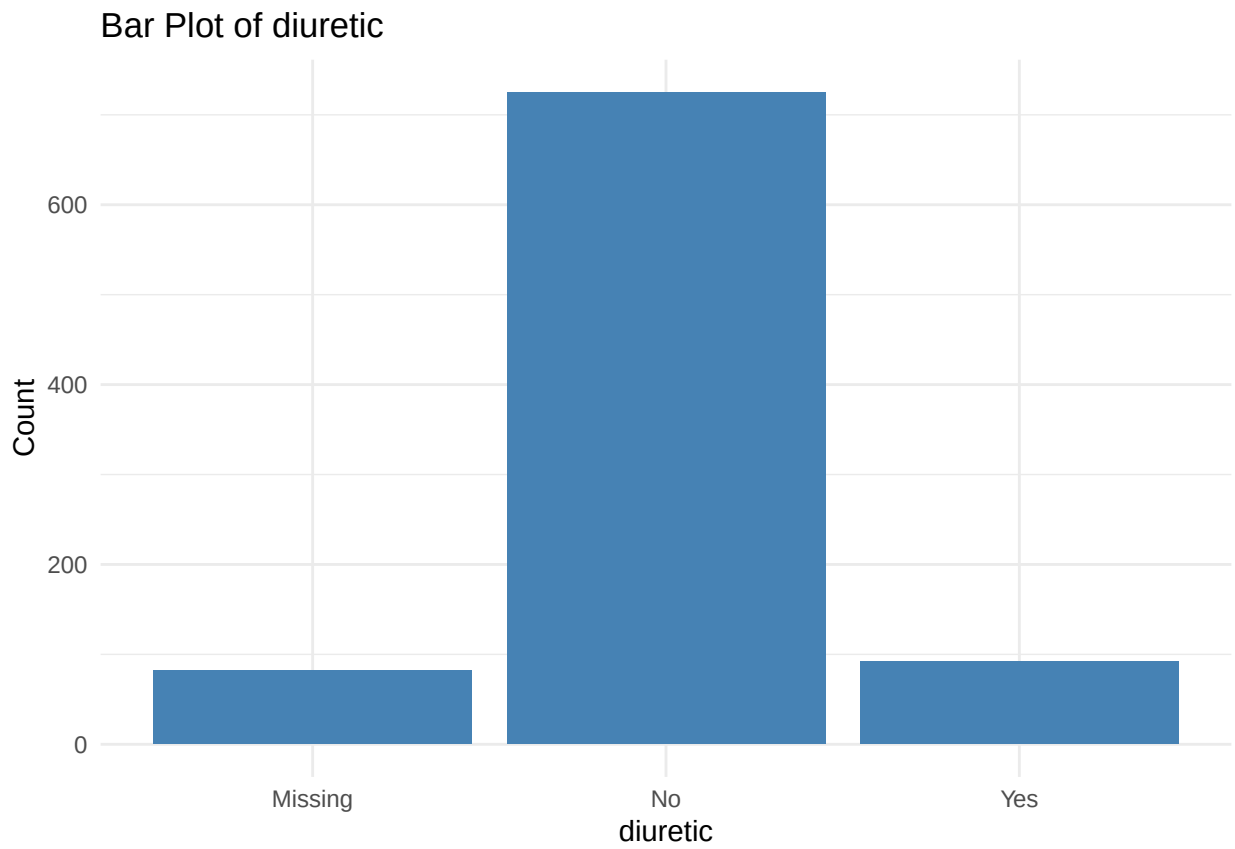


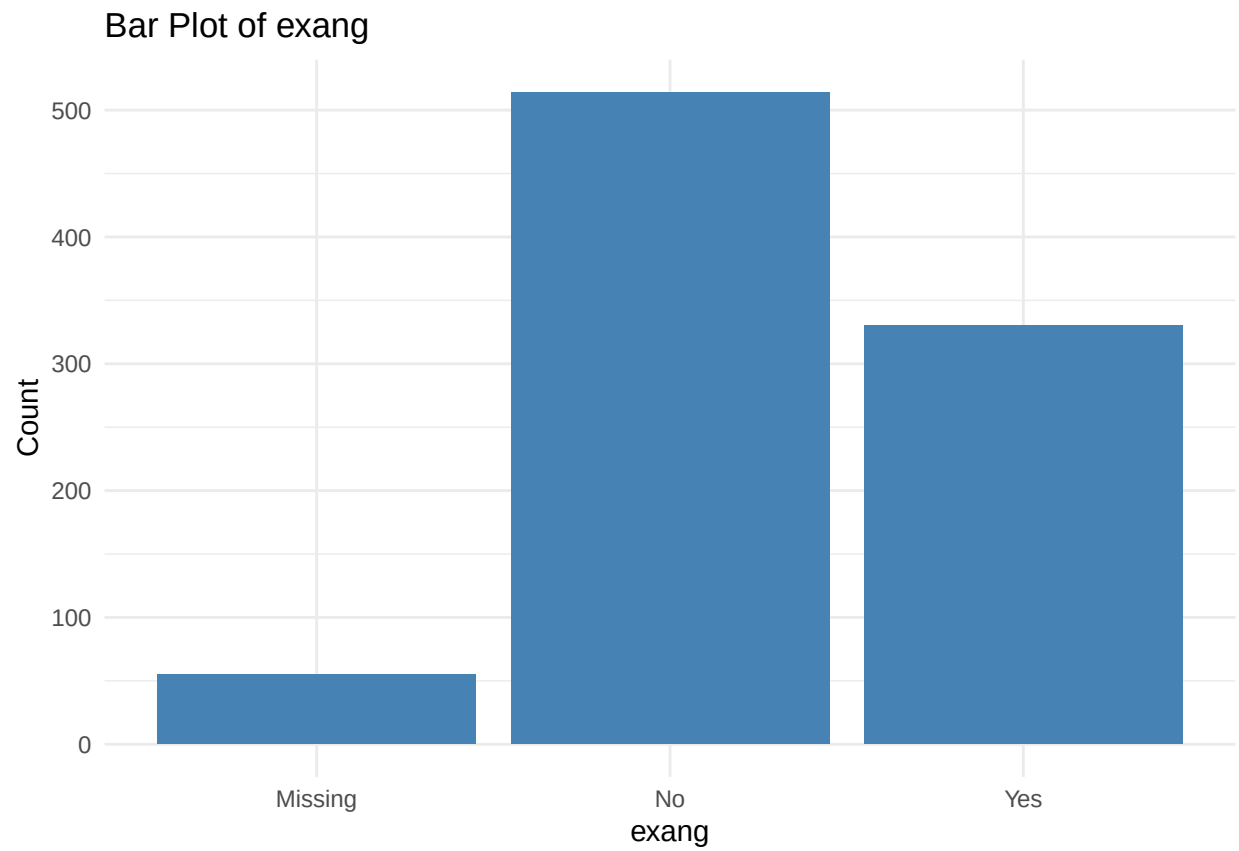


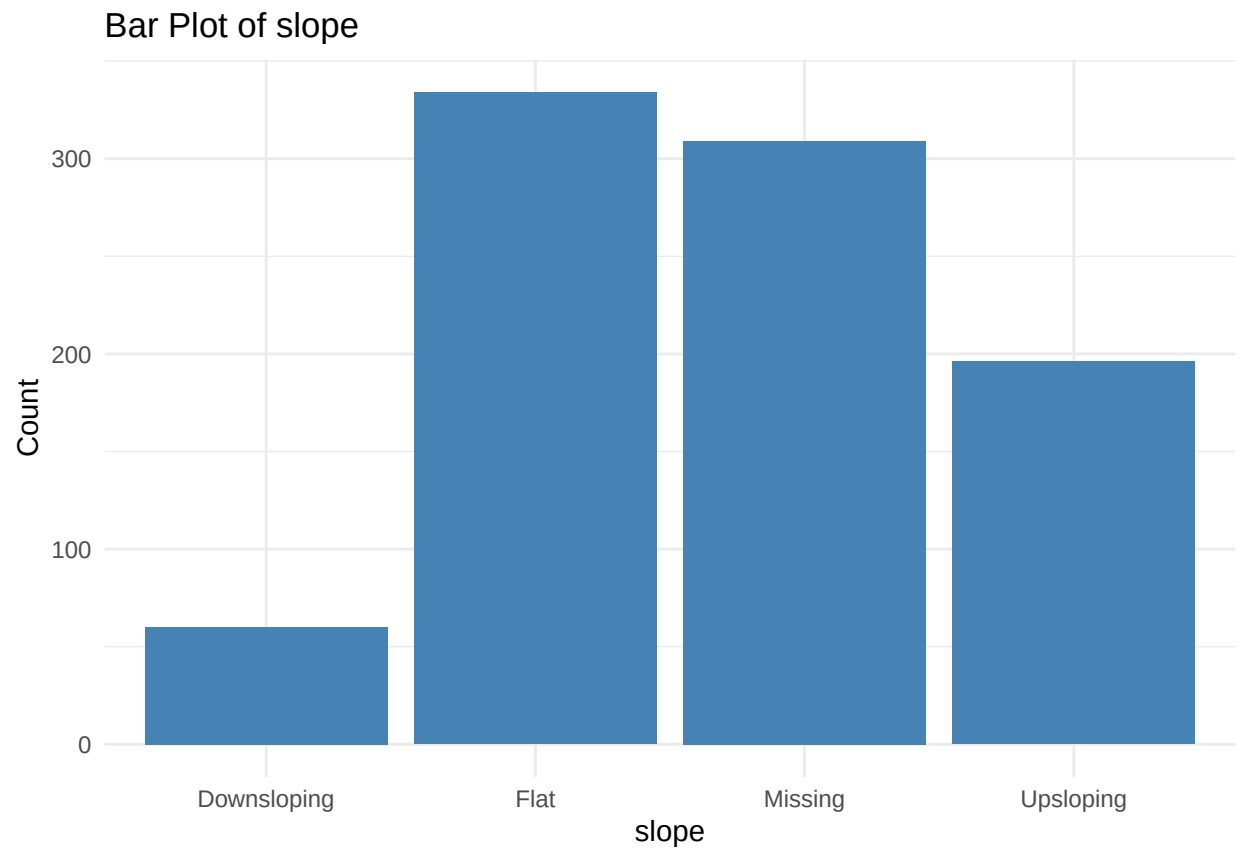


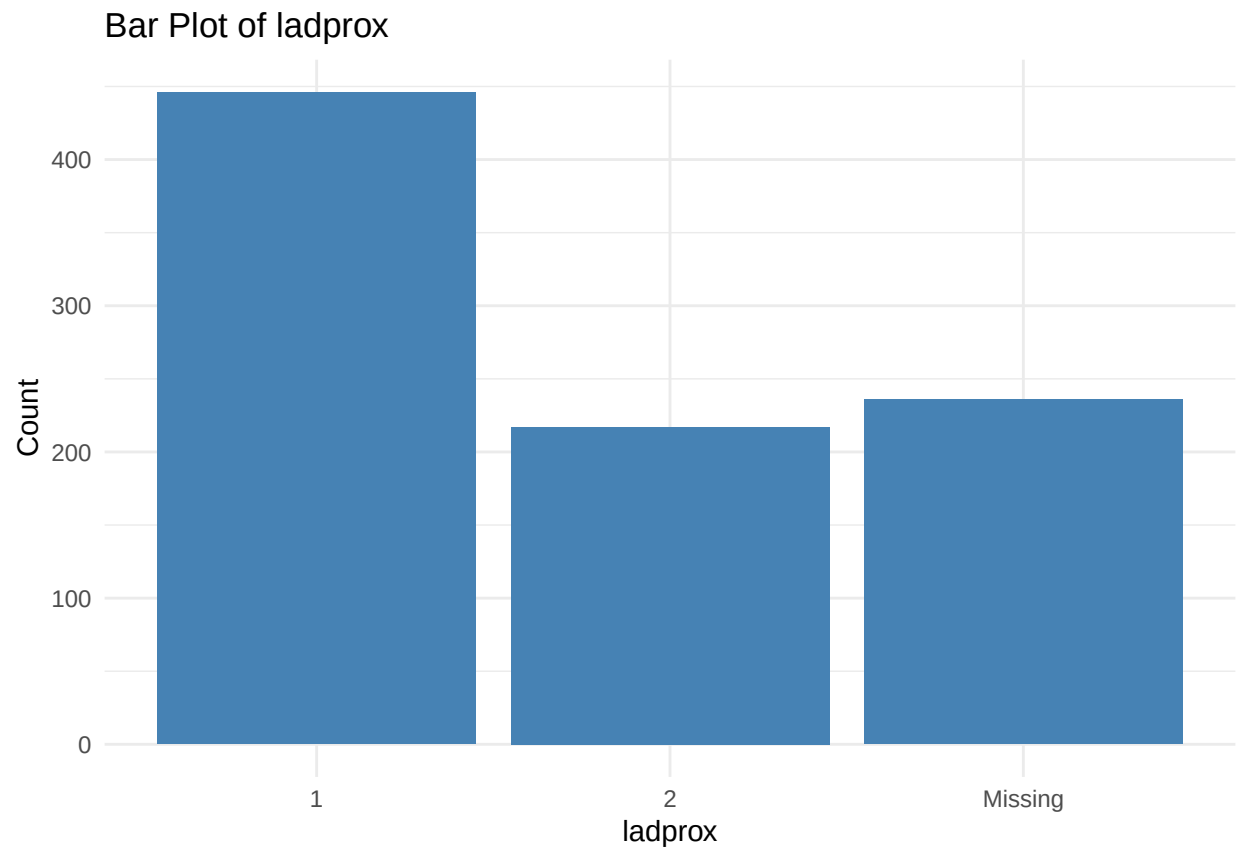


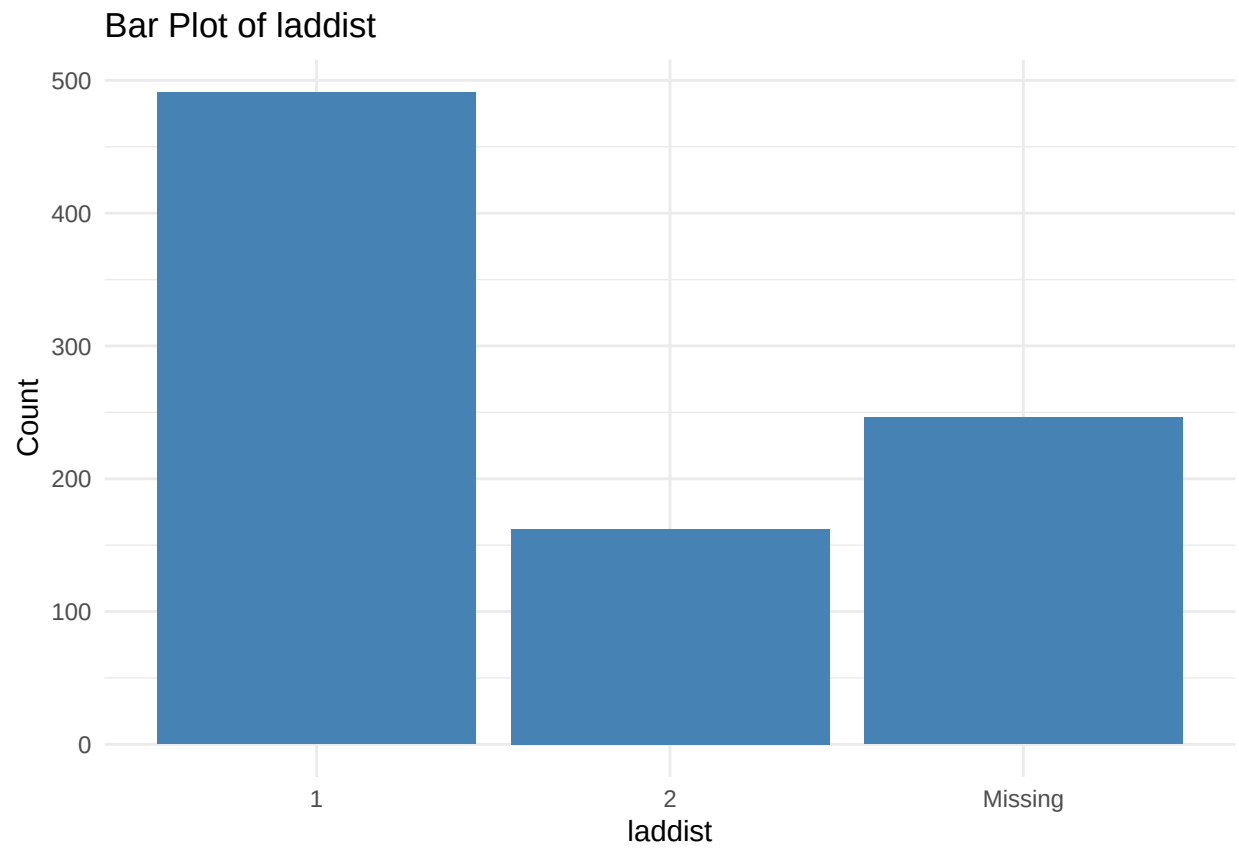


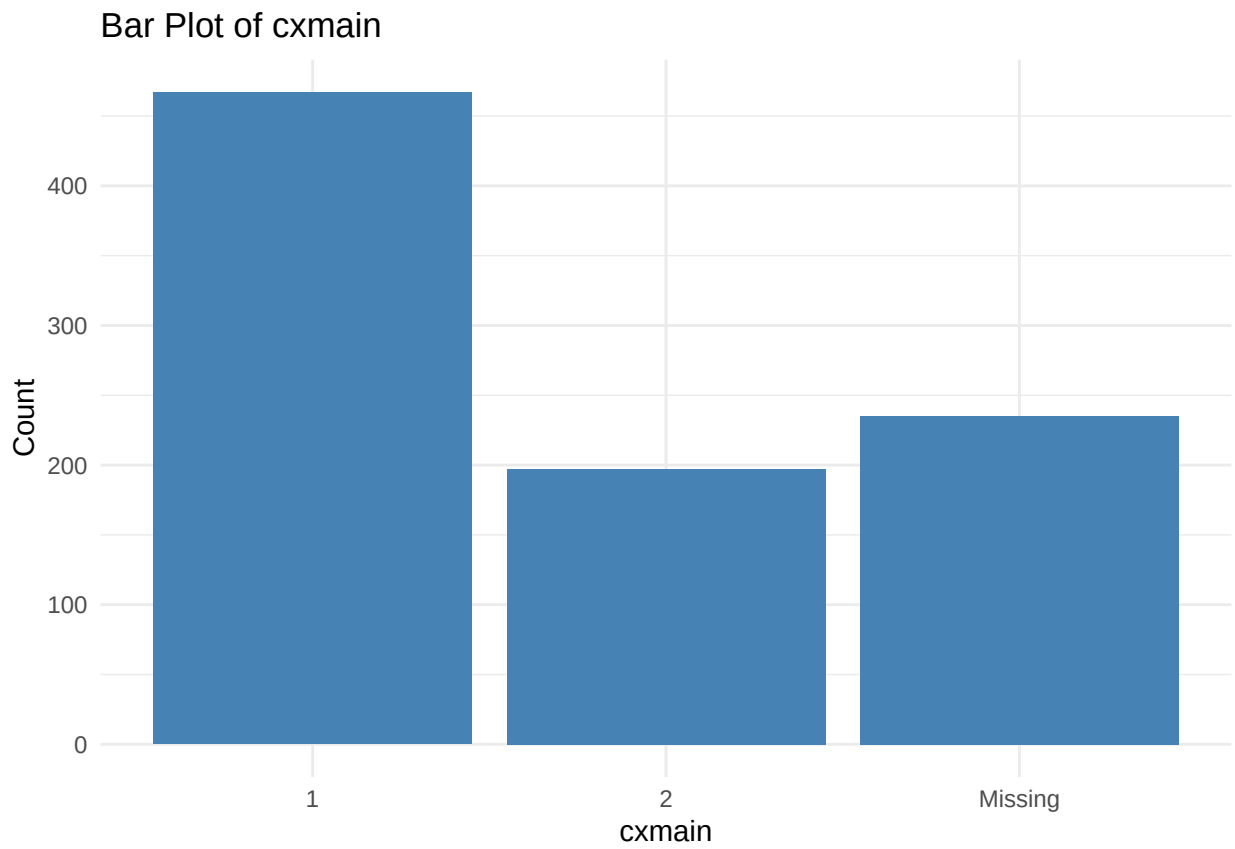


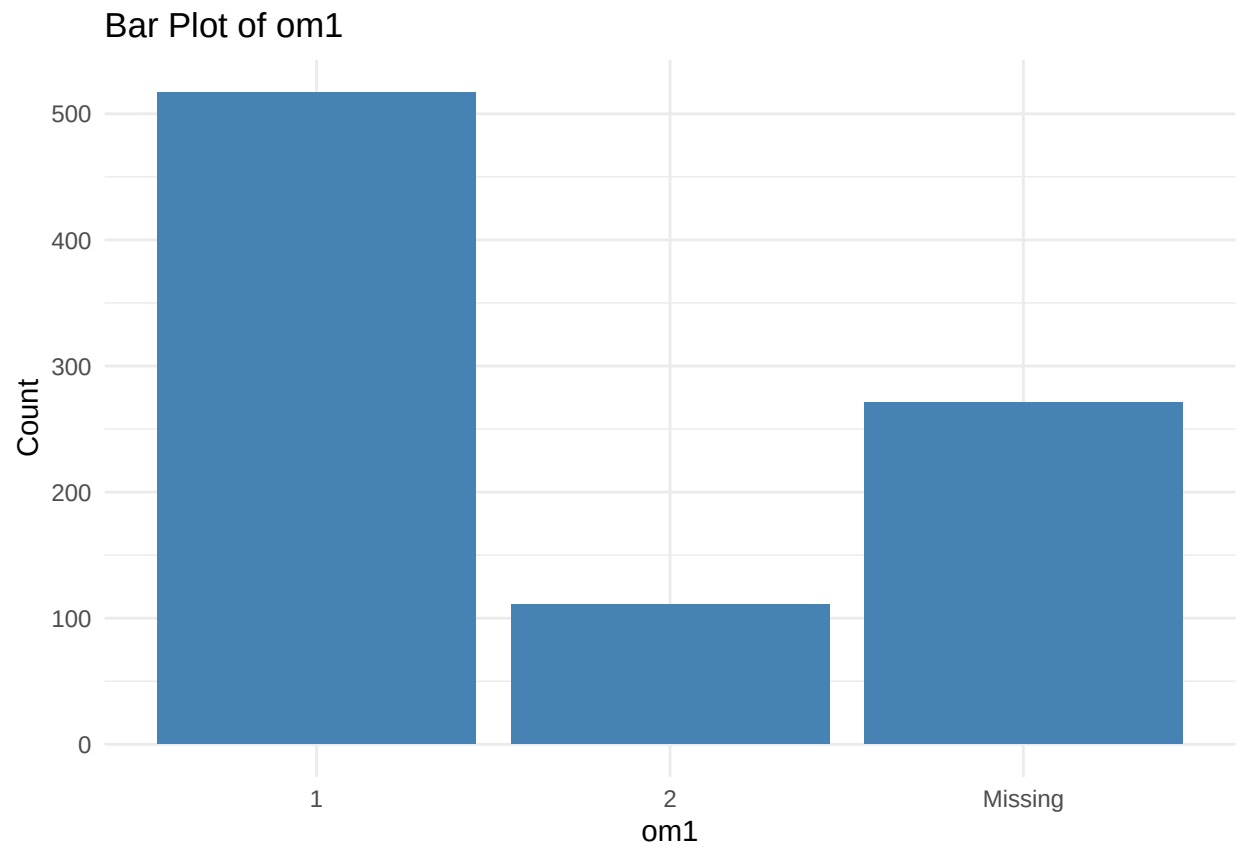


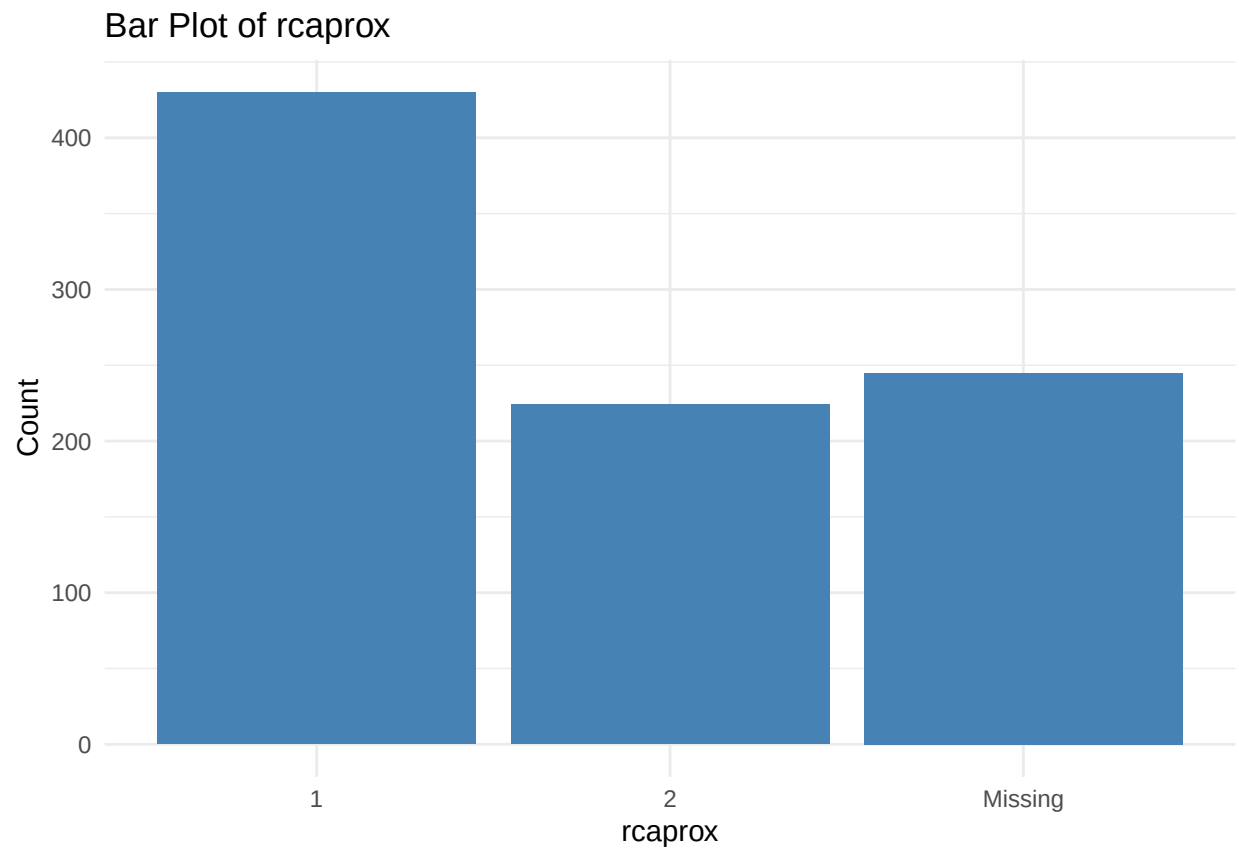


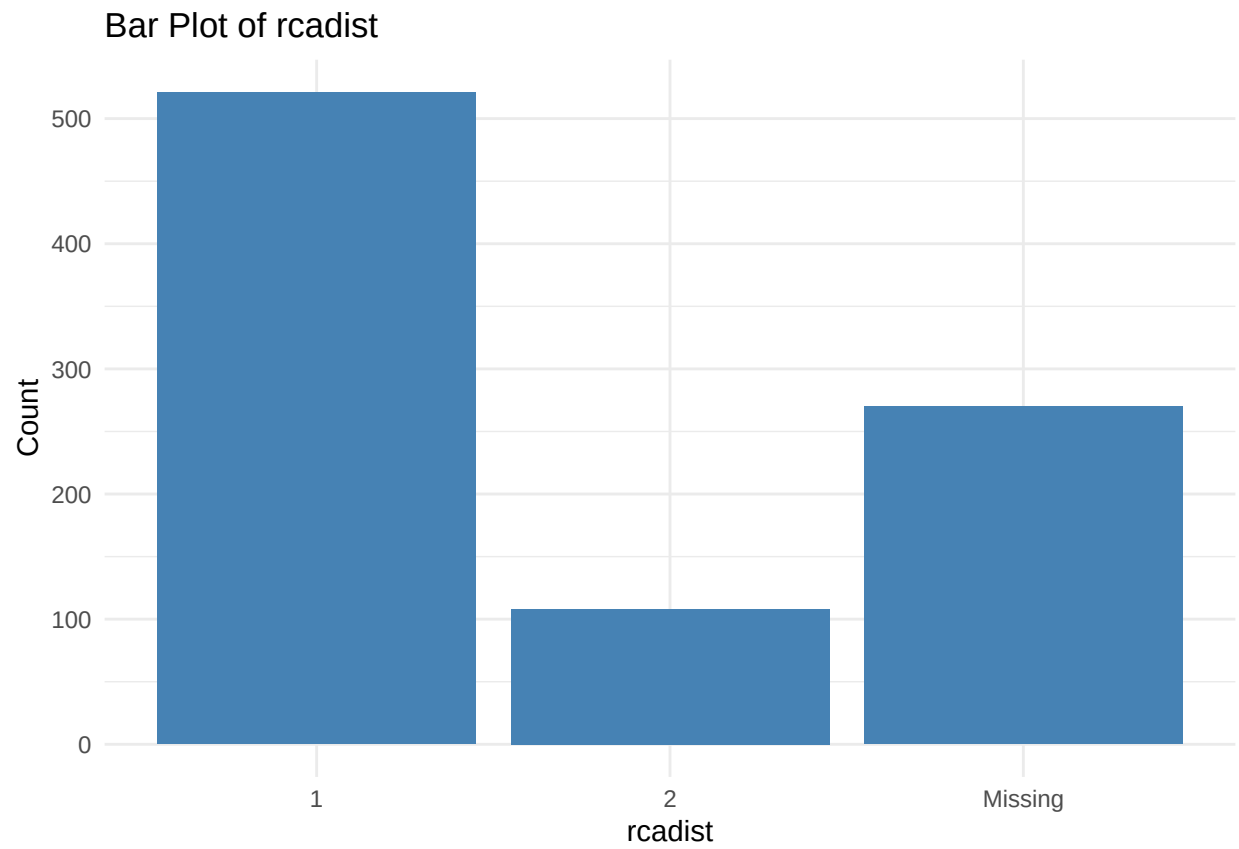


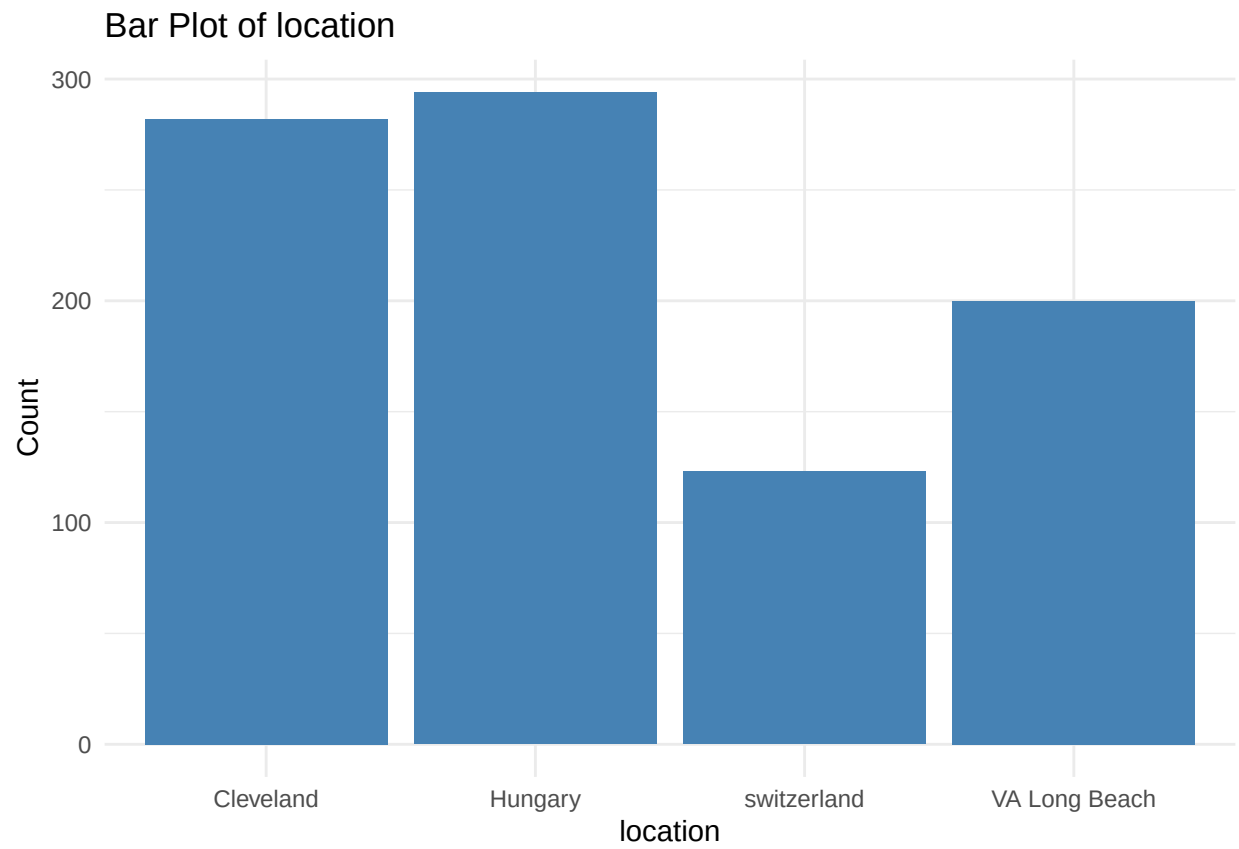


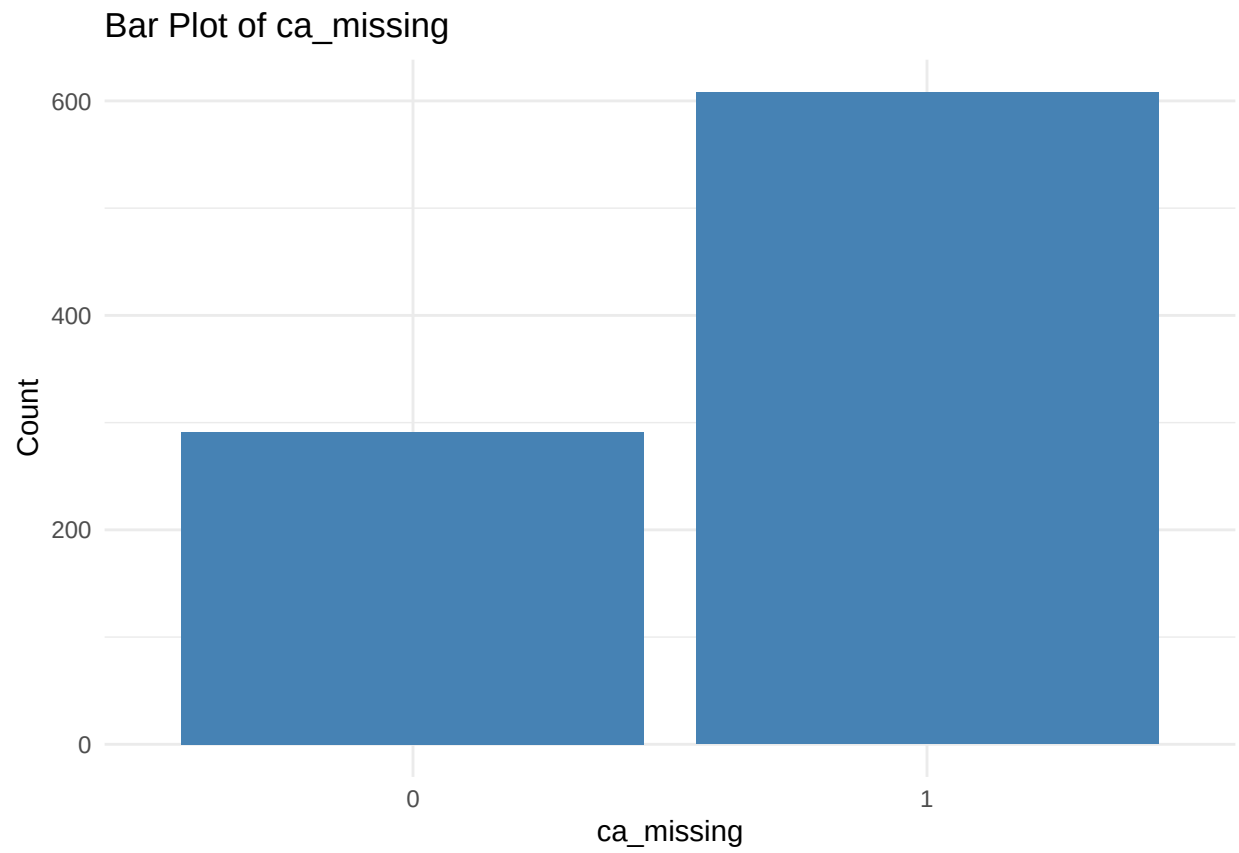


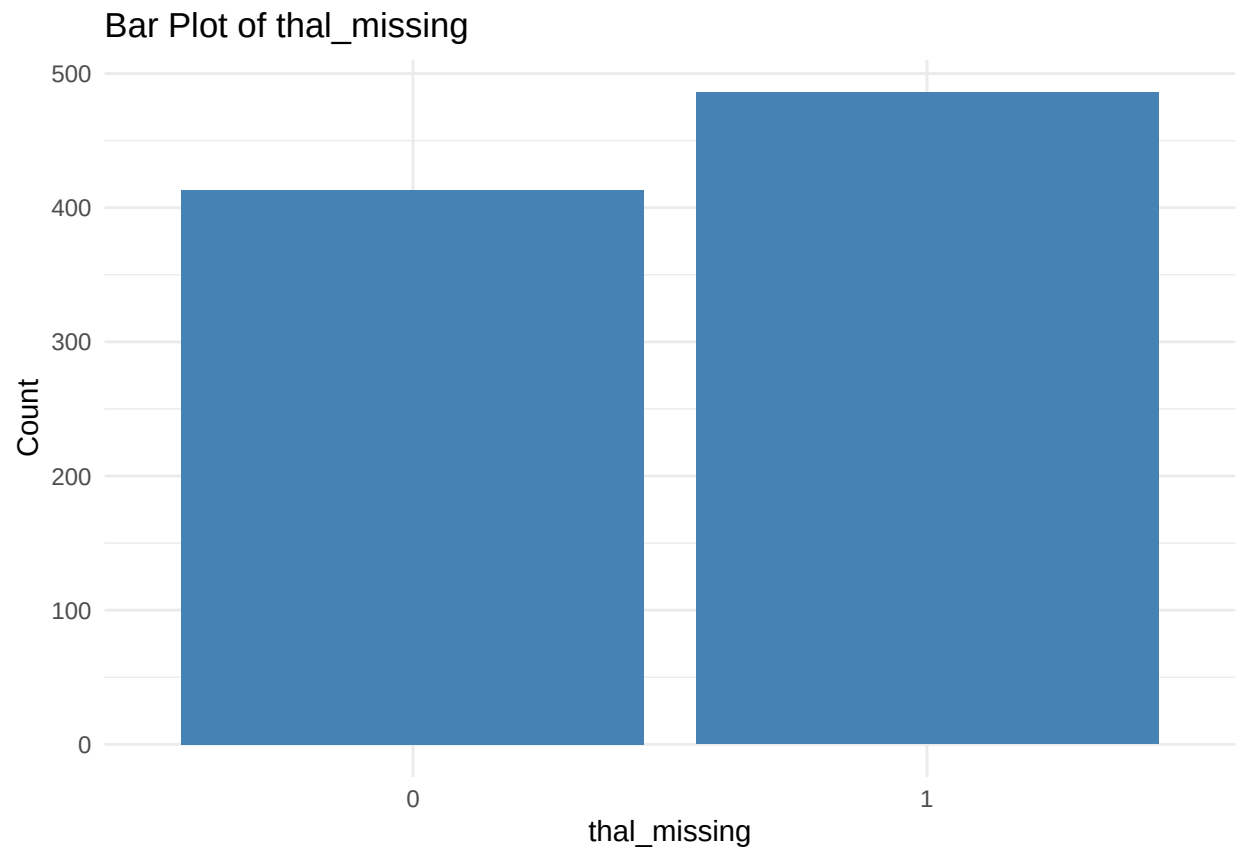


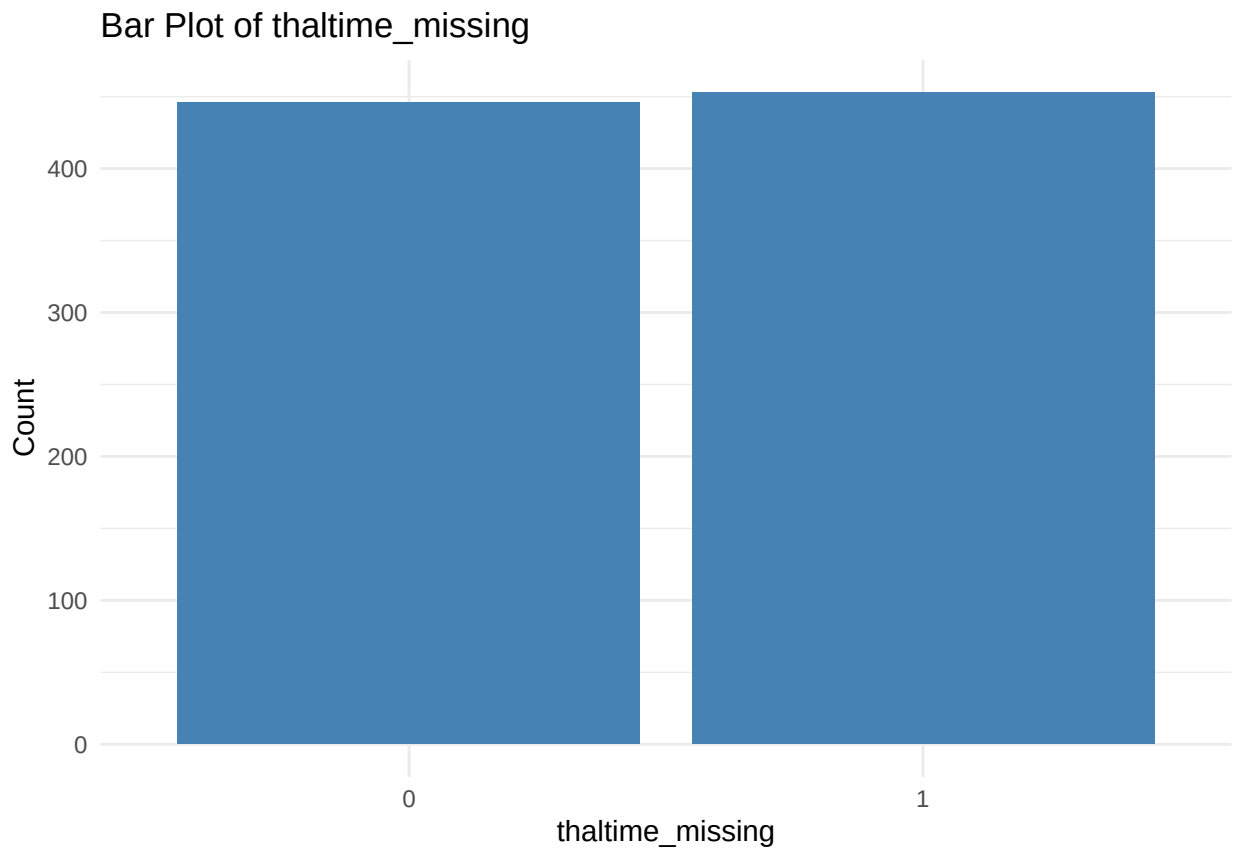


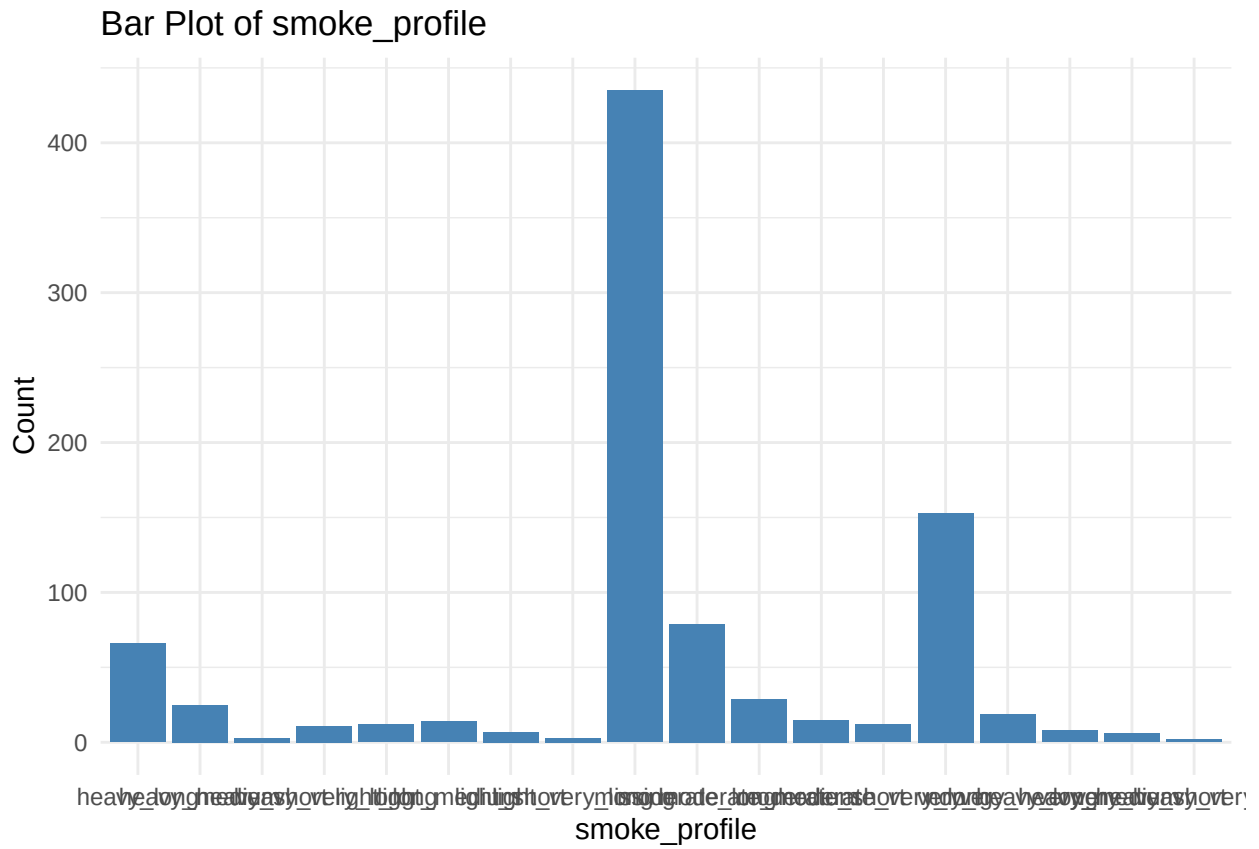




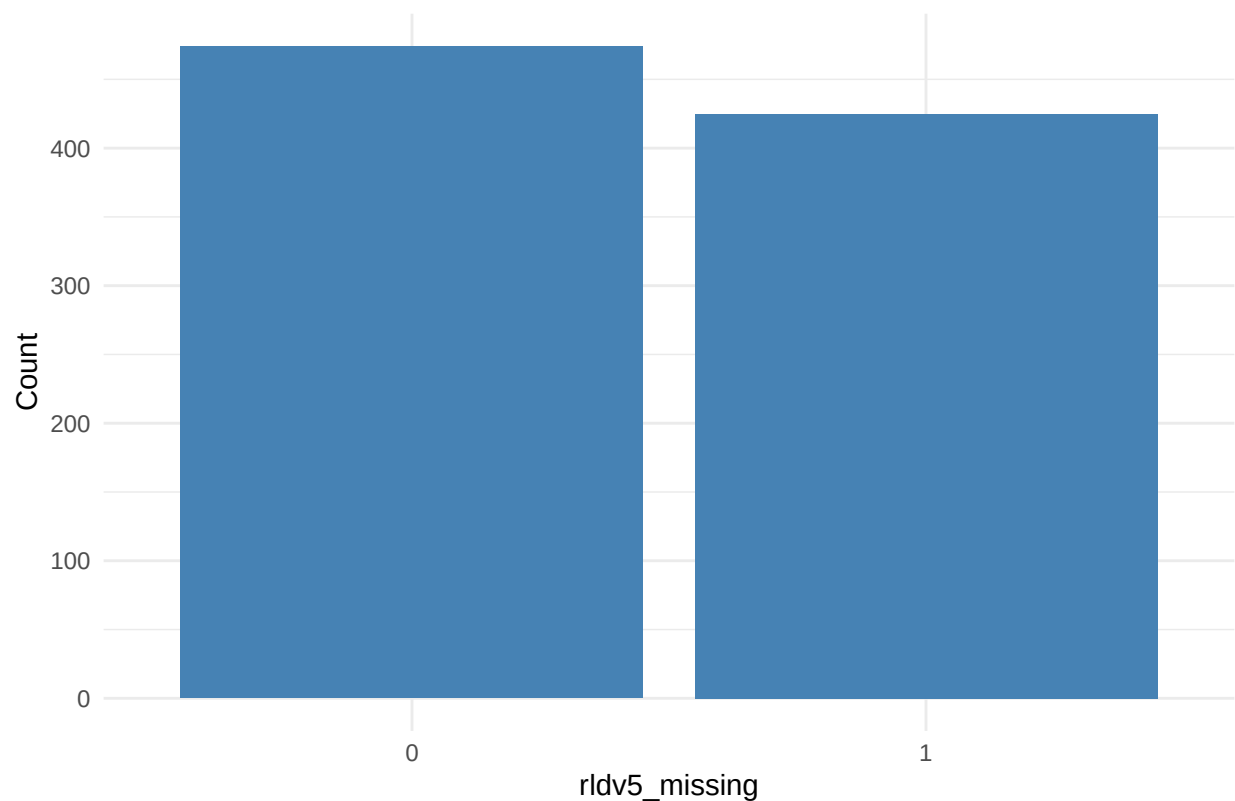








Bar Plot of rldv5_missing



Feature Types

```
X_df %>%  
  summarise(  
    numeric = sum(sapply(., is.numeric)),  
    categorical = sum(sapply(., is.factor))  
  )
```

```
## # A tibble: 1 x 2  
##   numeric categorical  
##   <int>      <int>  
## 1      29         26
```

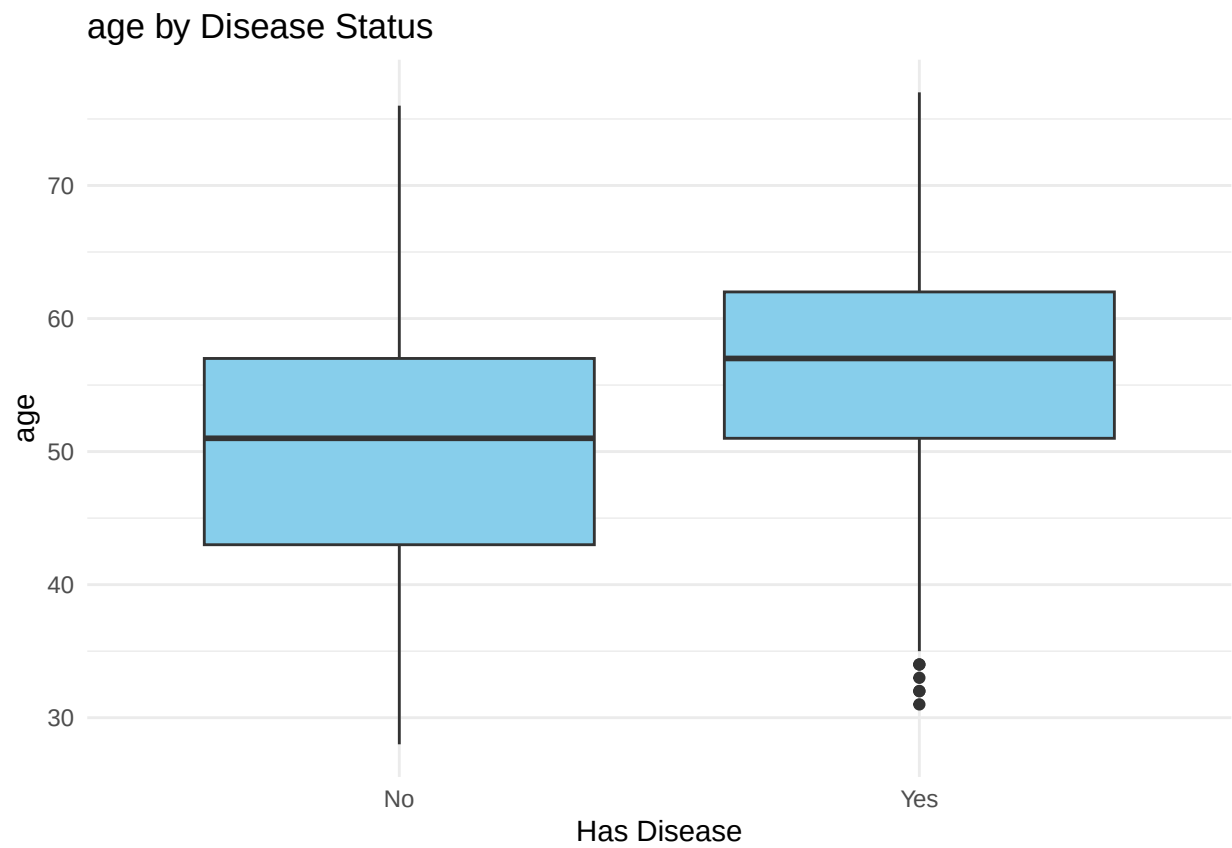
Bivariate Analysis

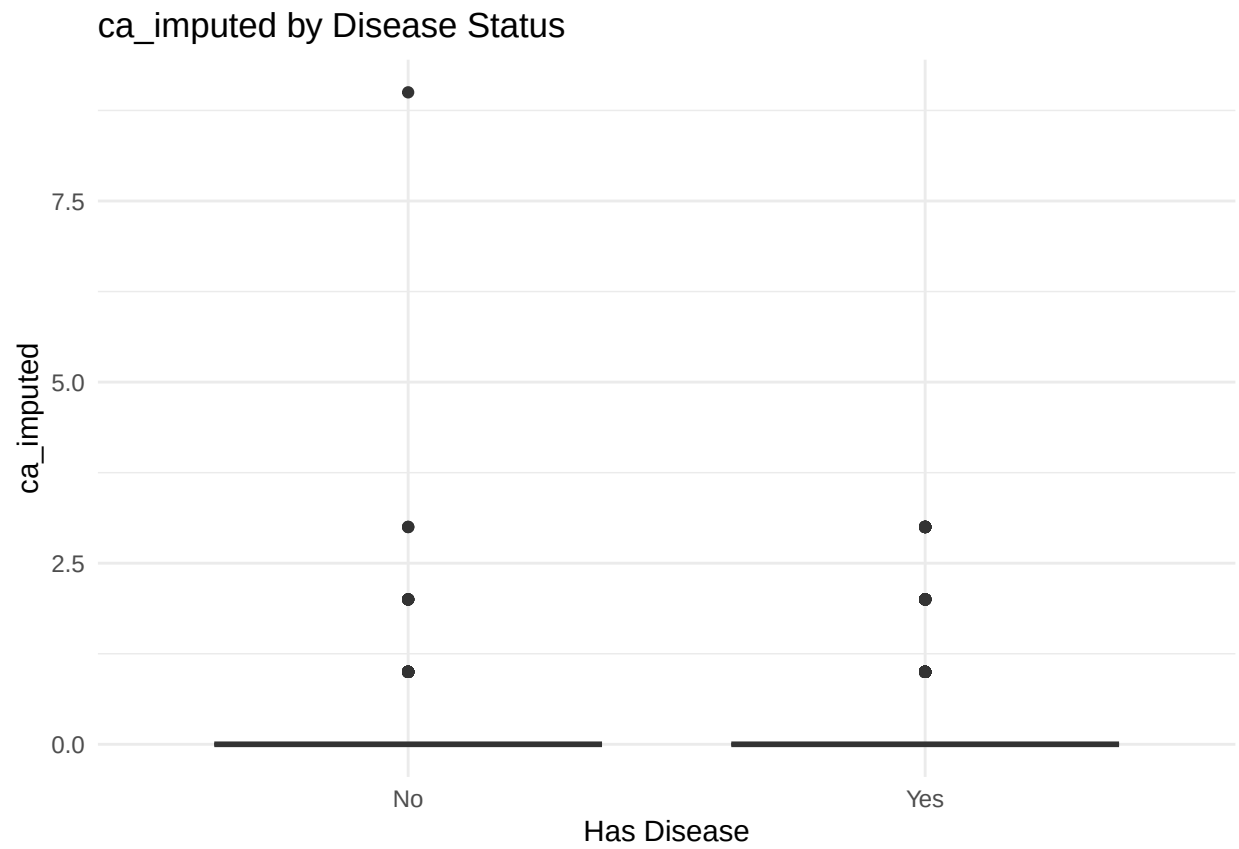
Numerical Vars

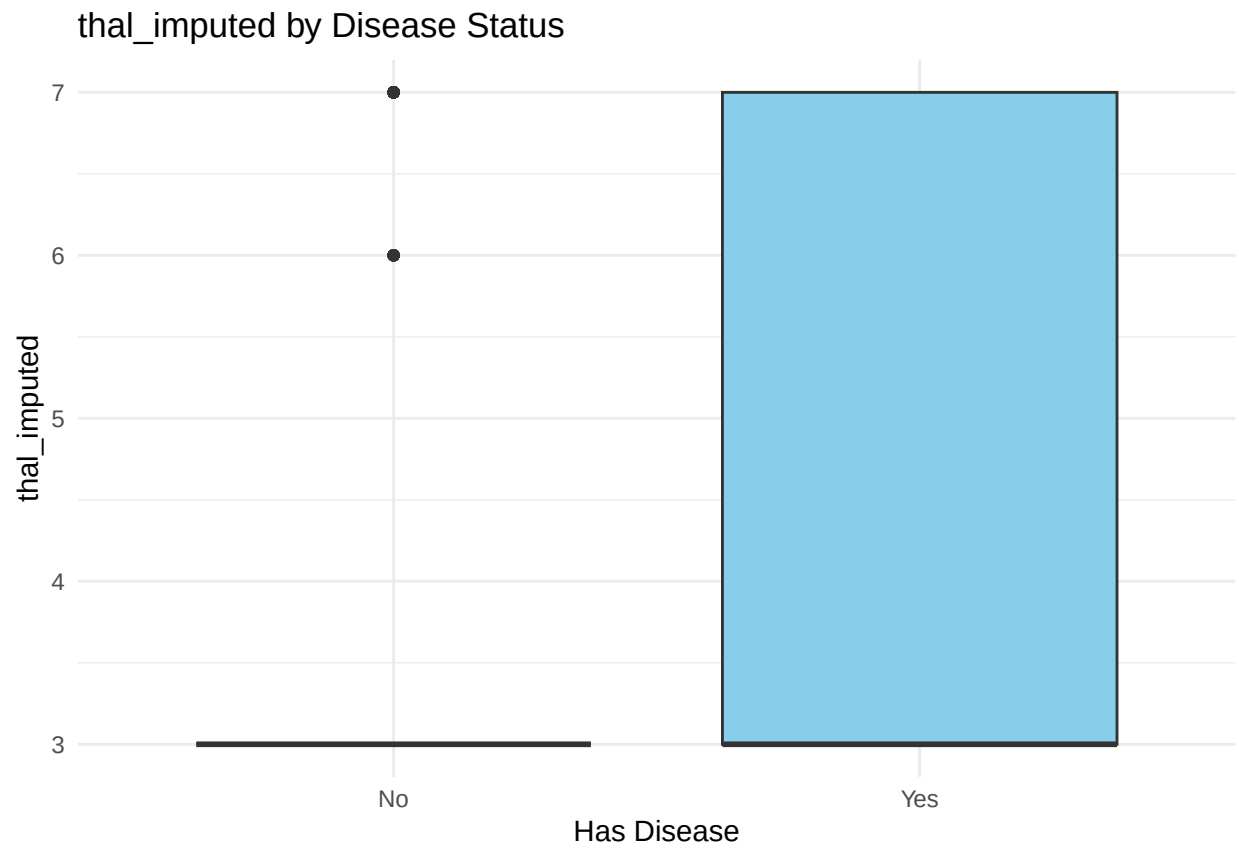
Box Plots

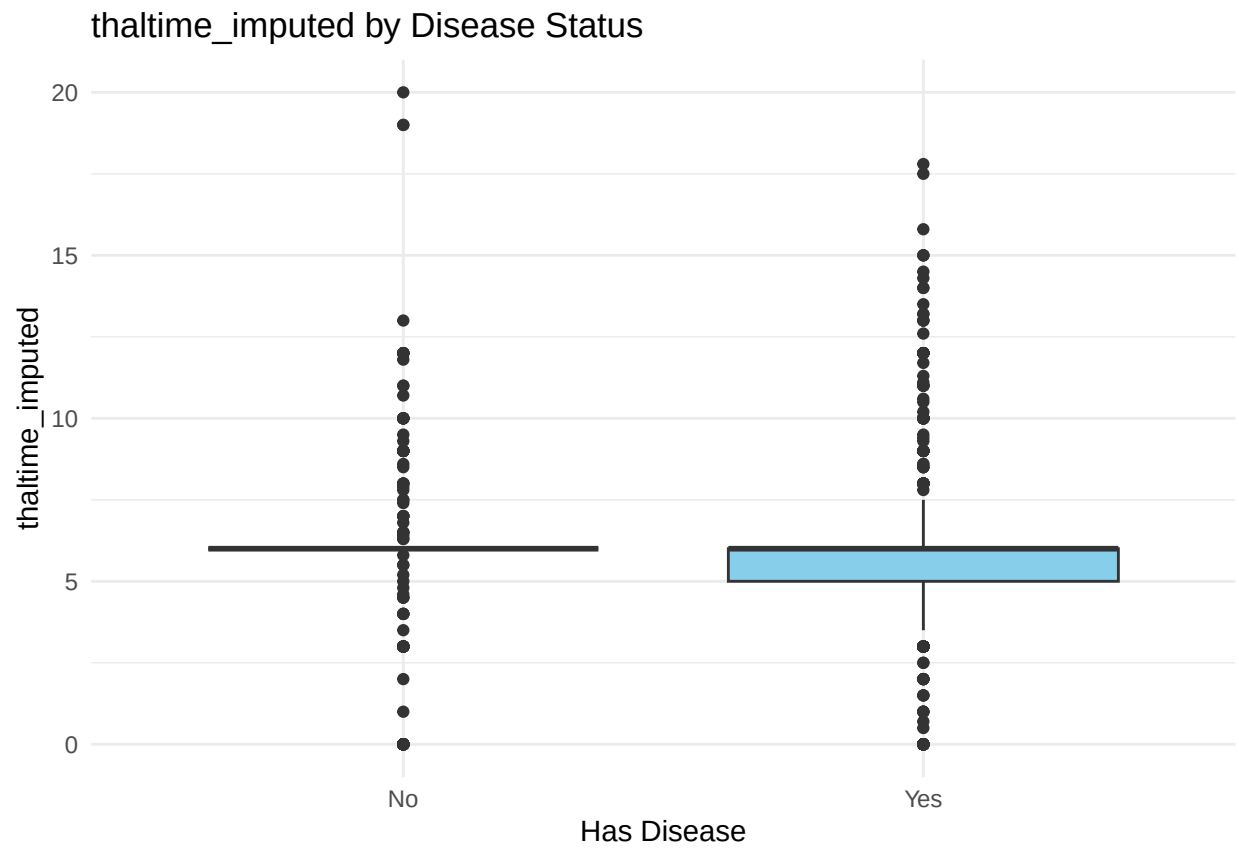
```
plot_bivariate_num <- function(var) {  
  ggplot(df, aes(x = factor(has_disease), y = .data[[var]])) +  
    geom_boxplot(fill = "skyblue") +  
    labs(title = paste(var, "by Disease Status"), x = "Has Disease", y = var) +  
    theme_minimal()  
}
```

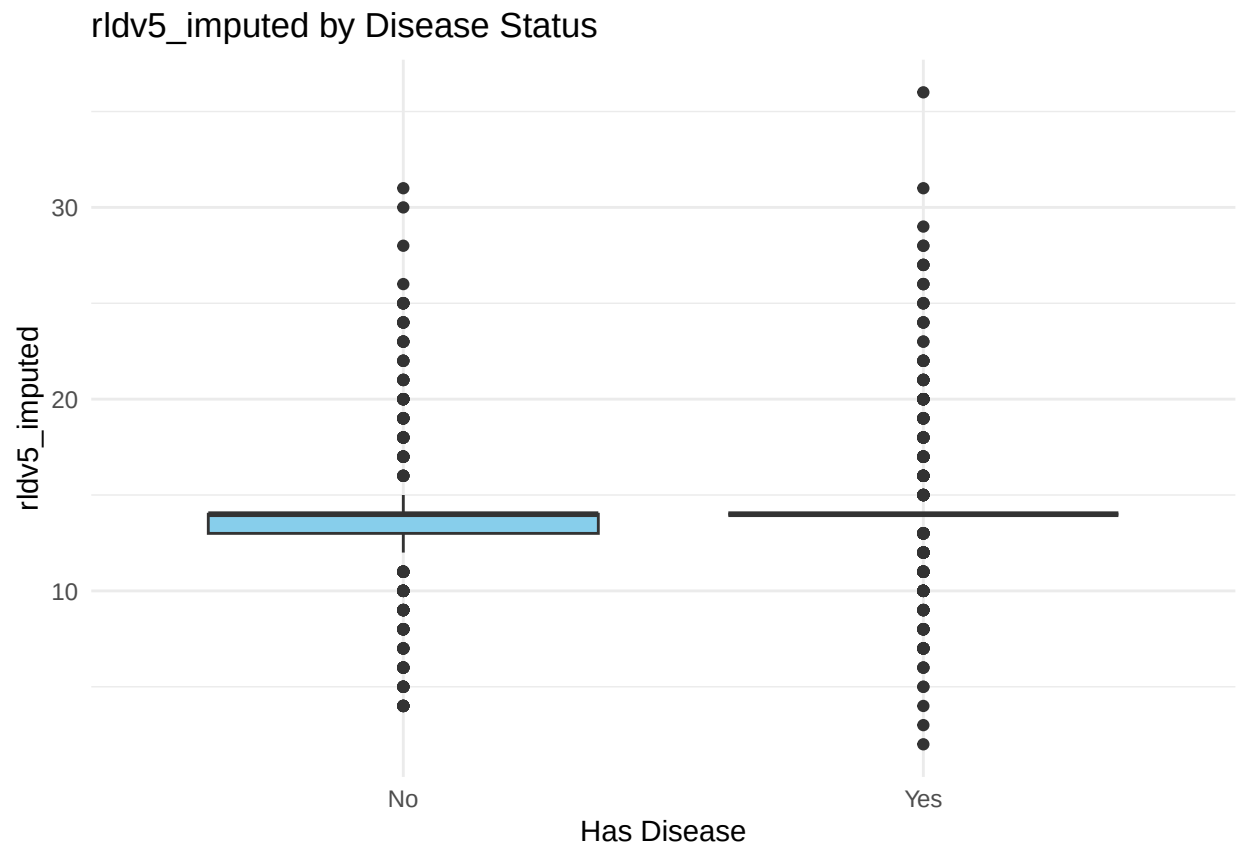
```
for (var in num_vars){  
  print(plot_bivariate_num(var))  
}
```

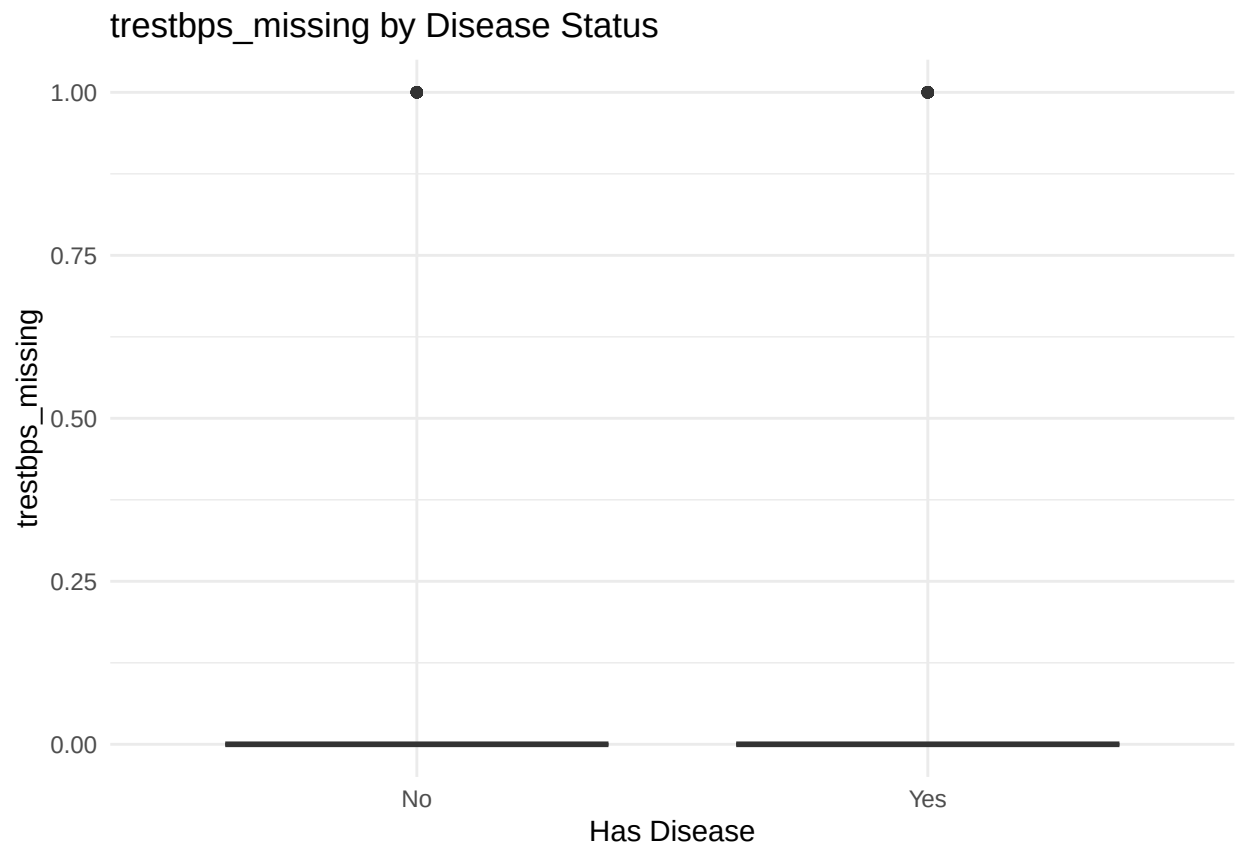


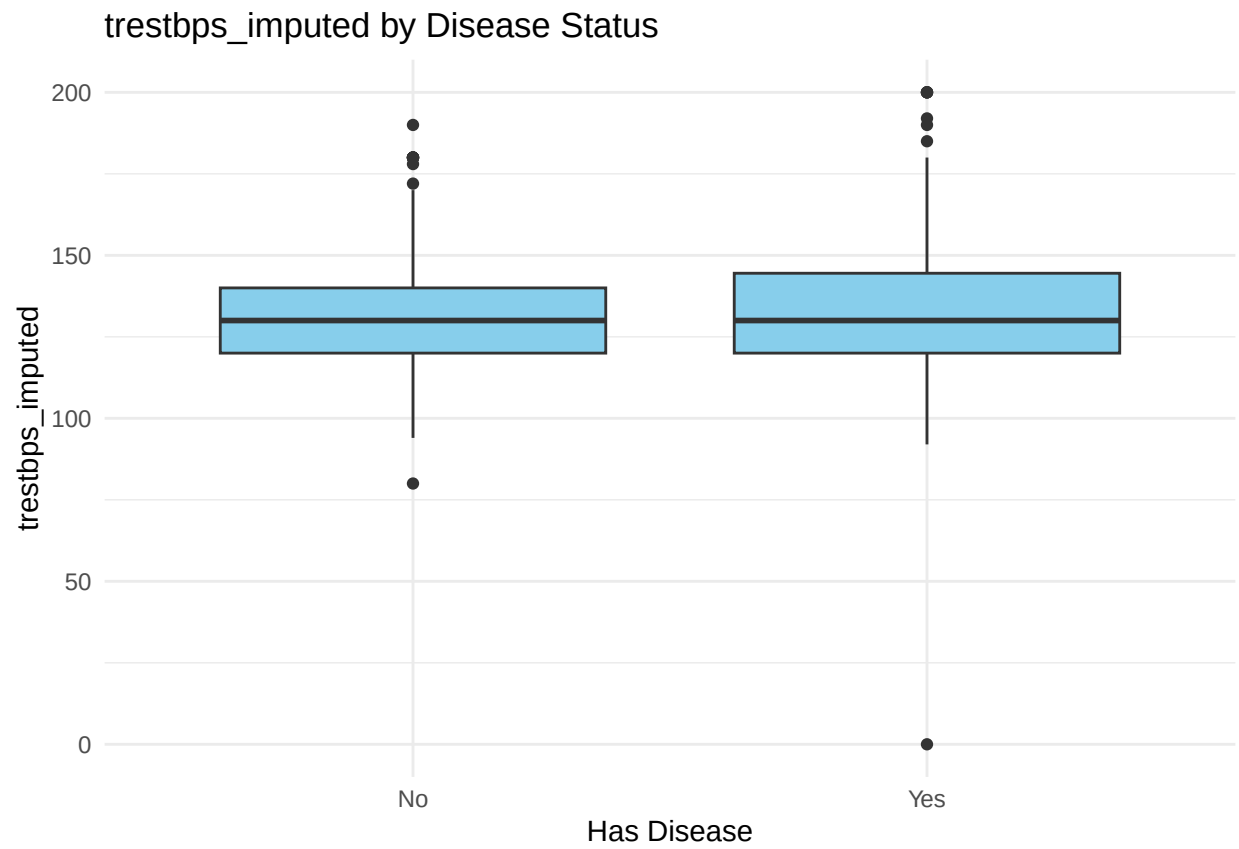


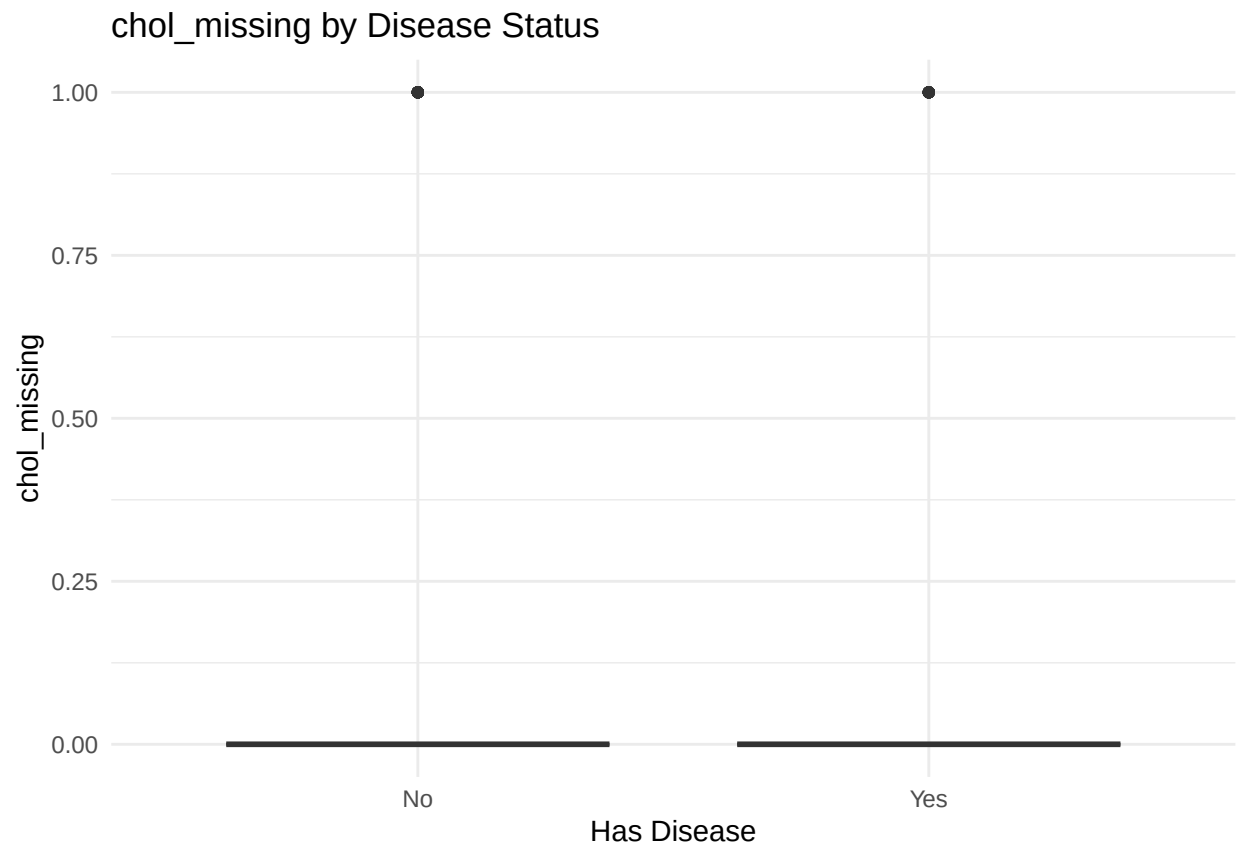


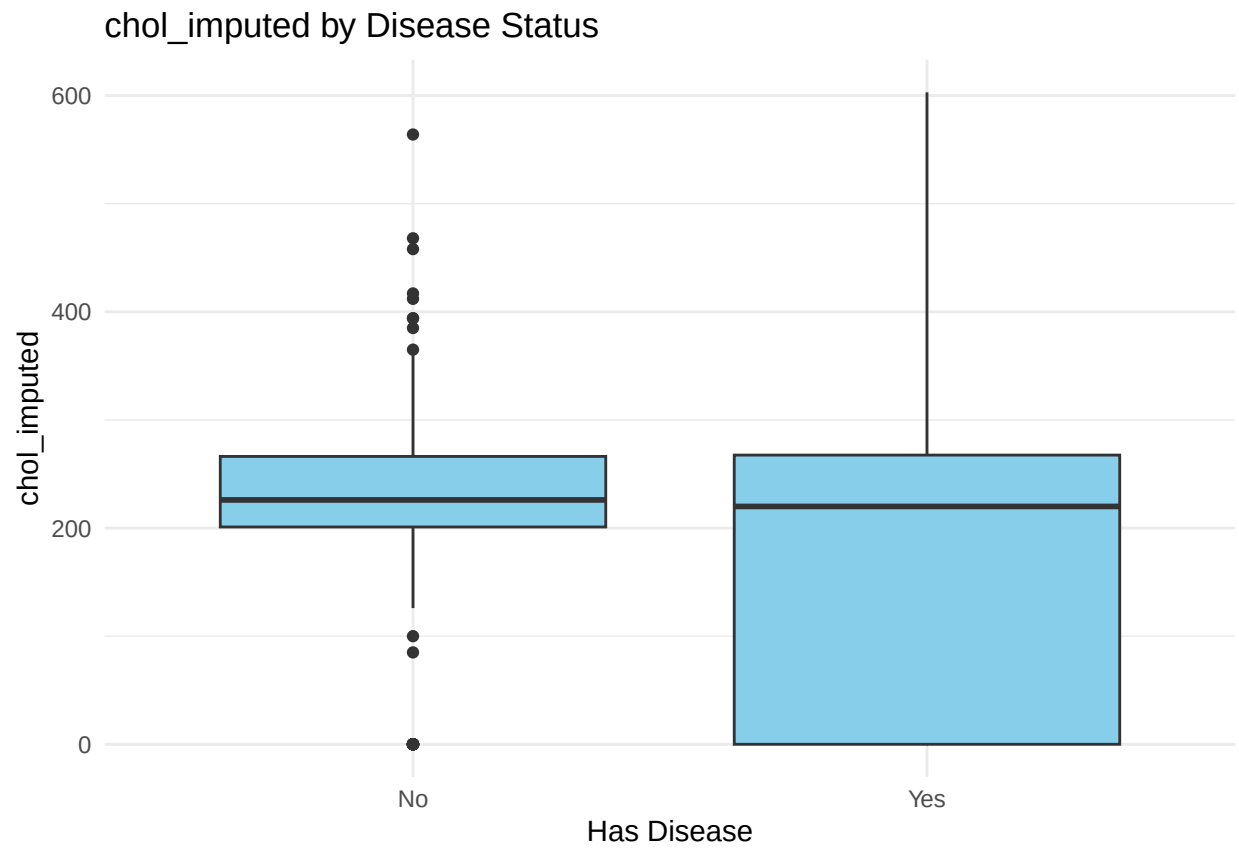


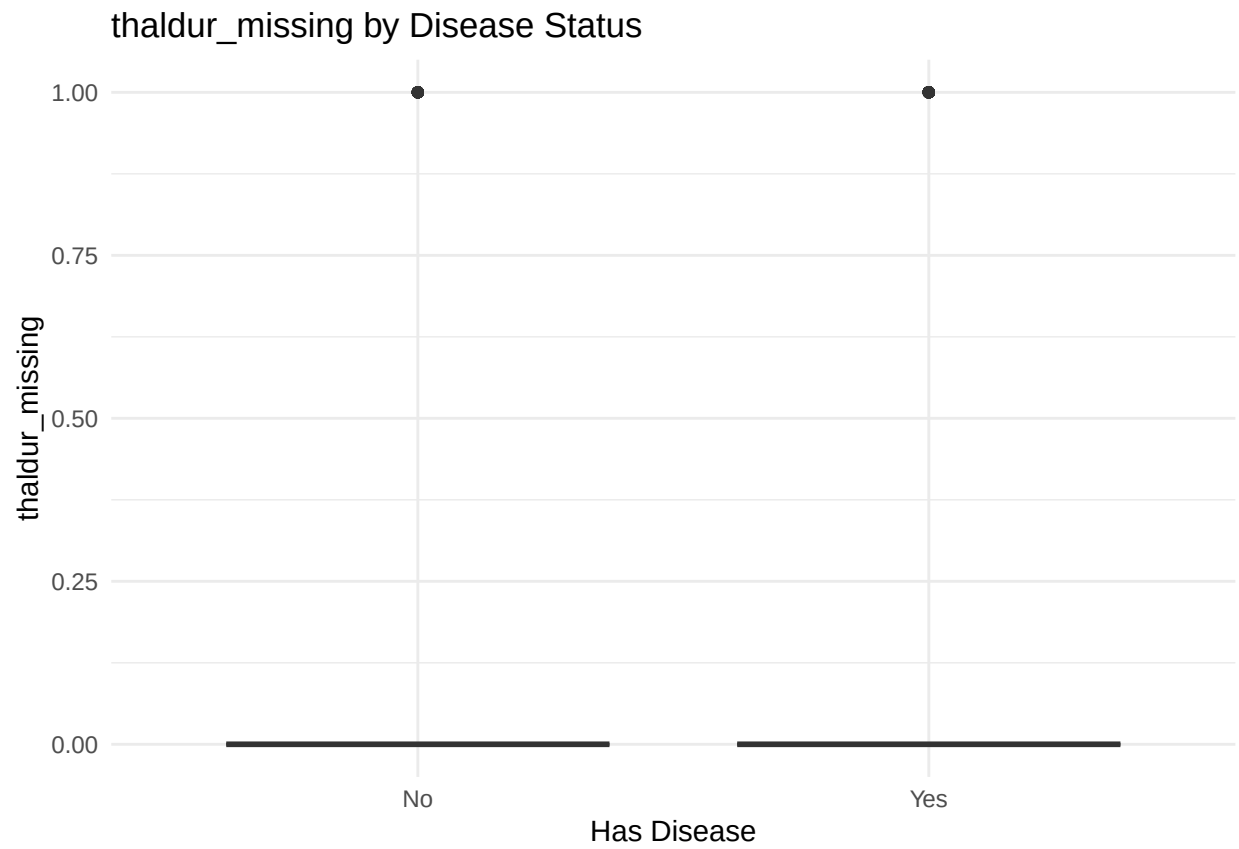


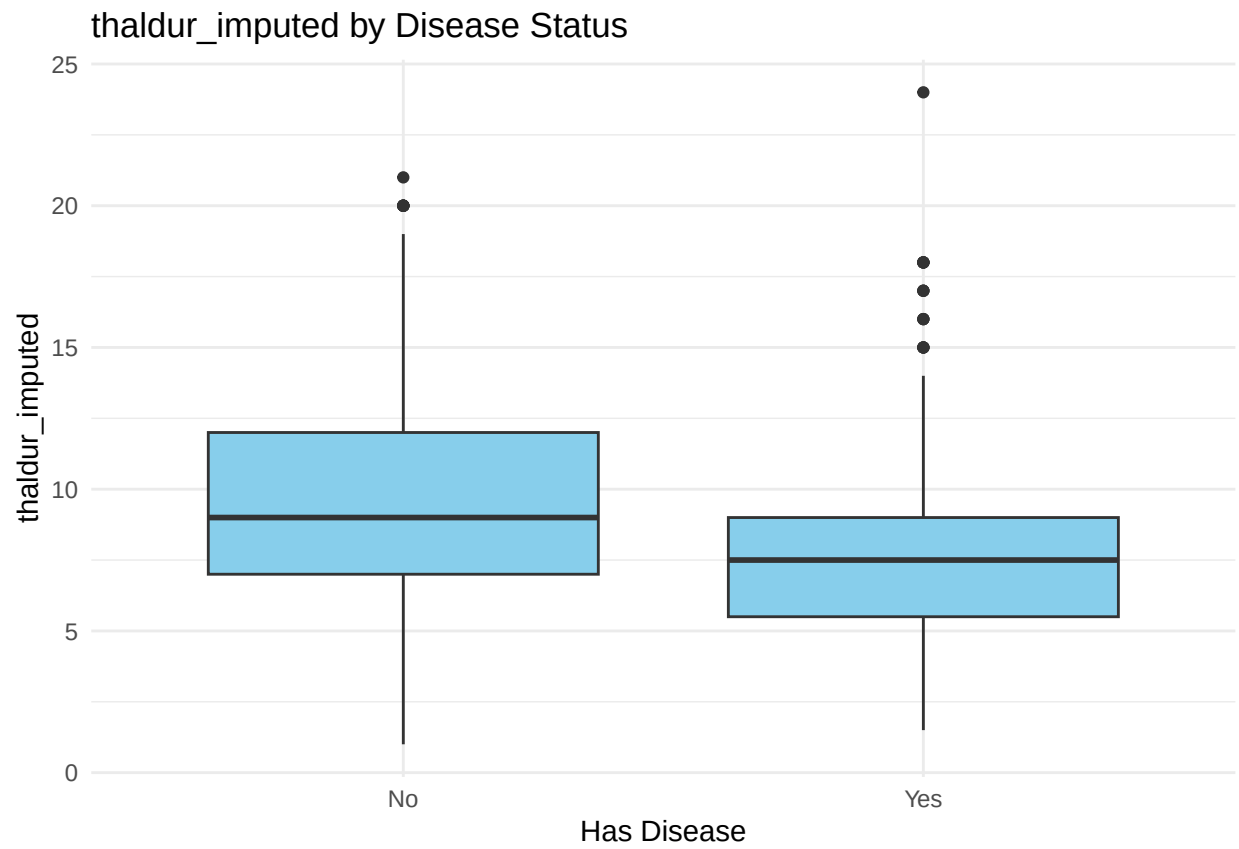


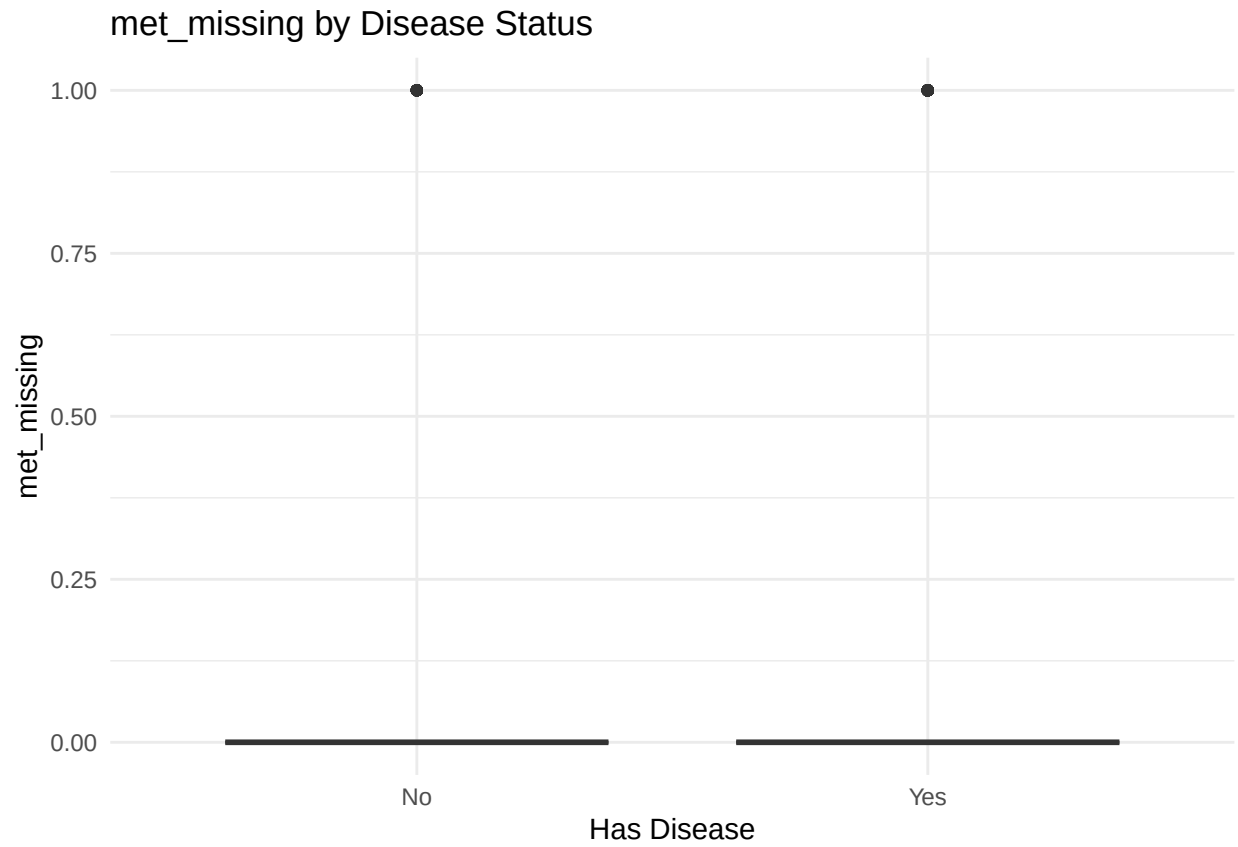


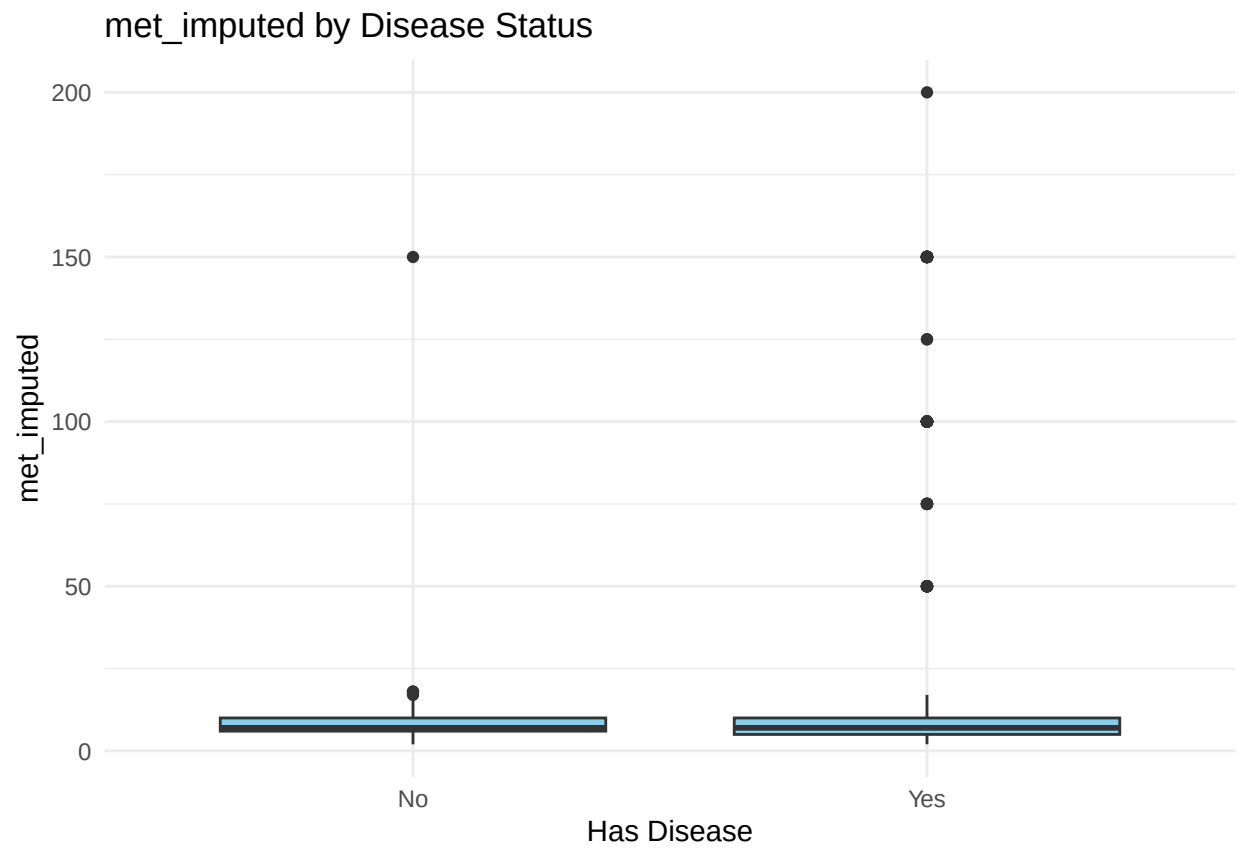


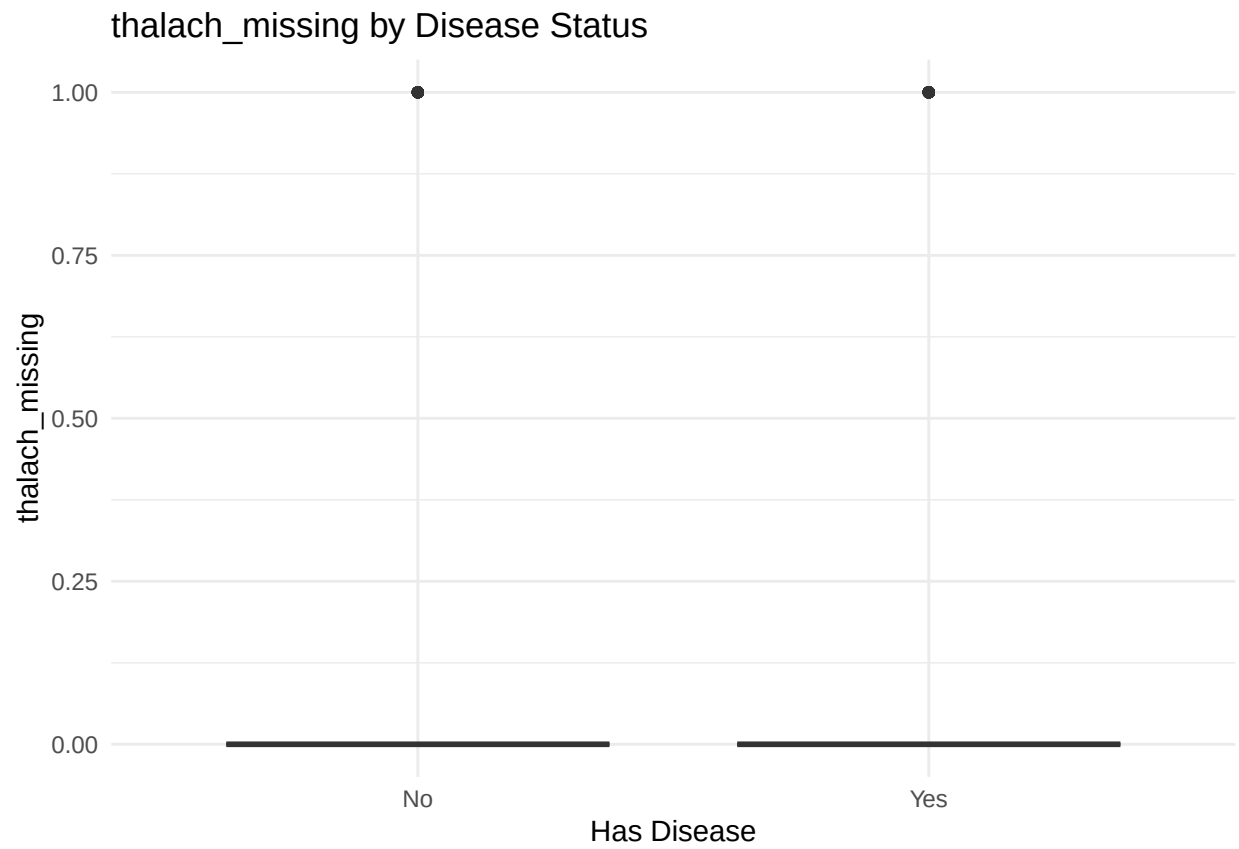


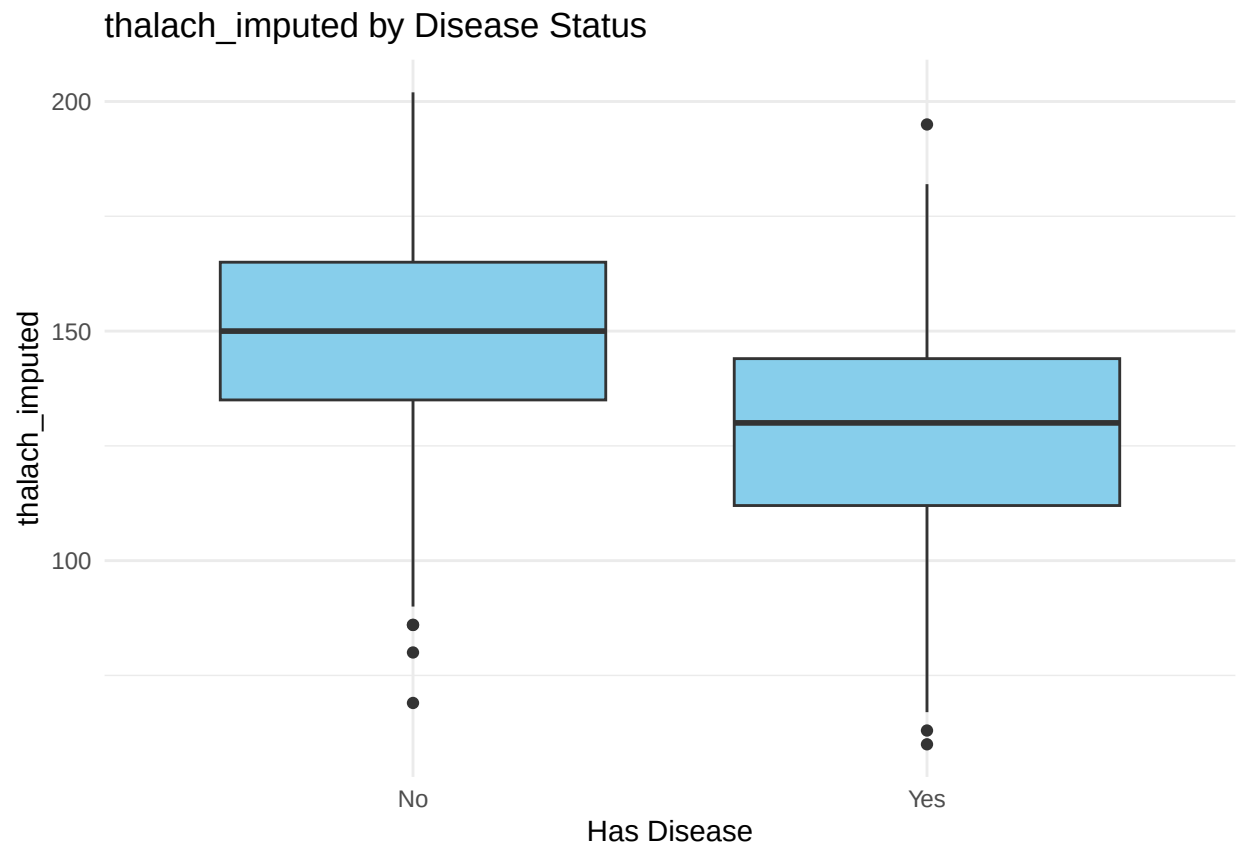


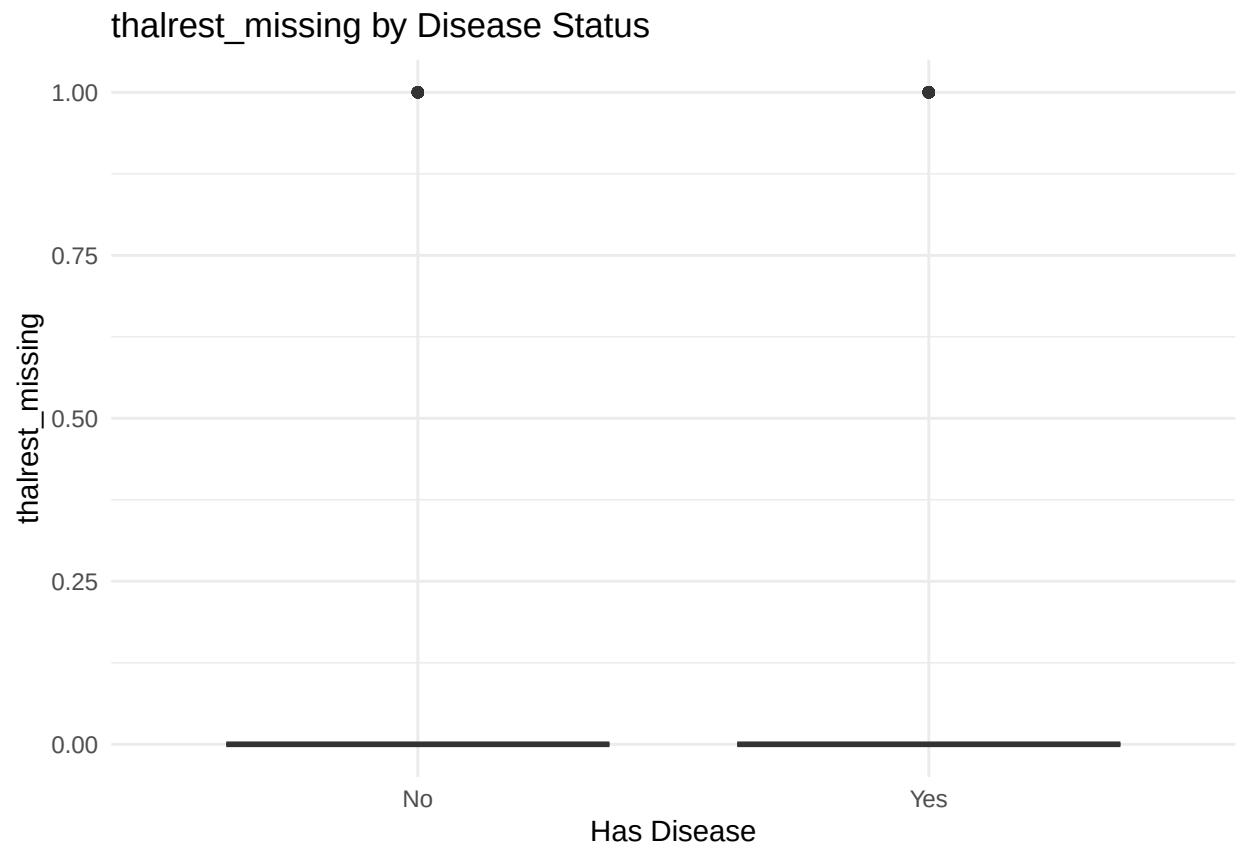


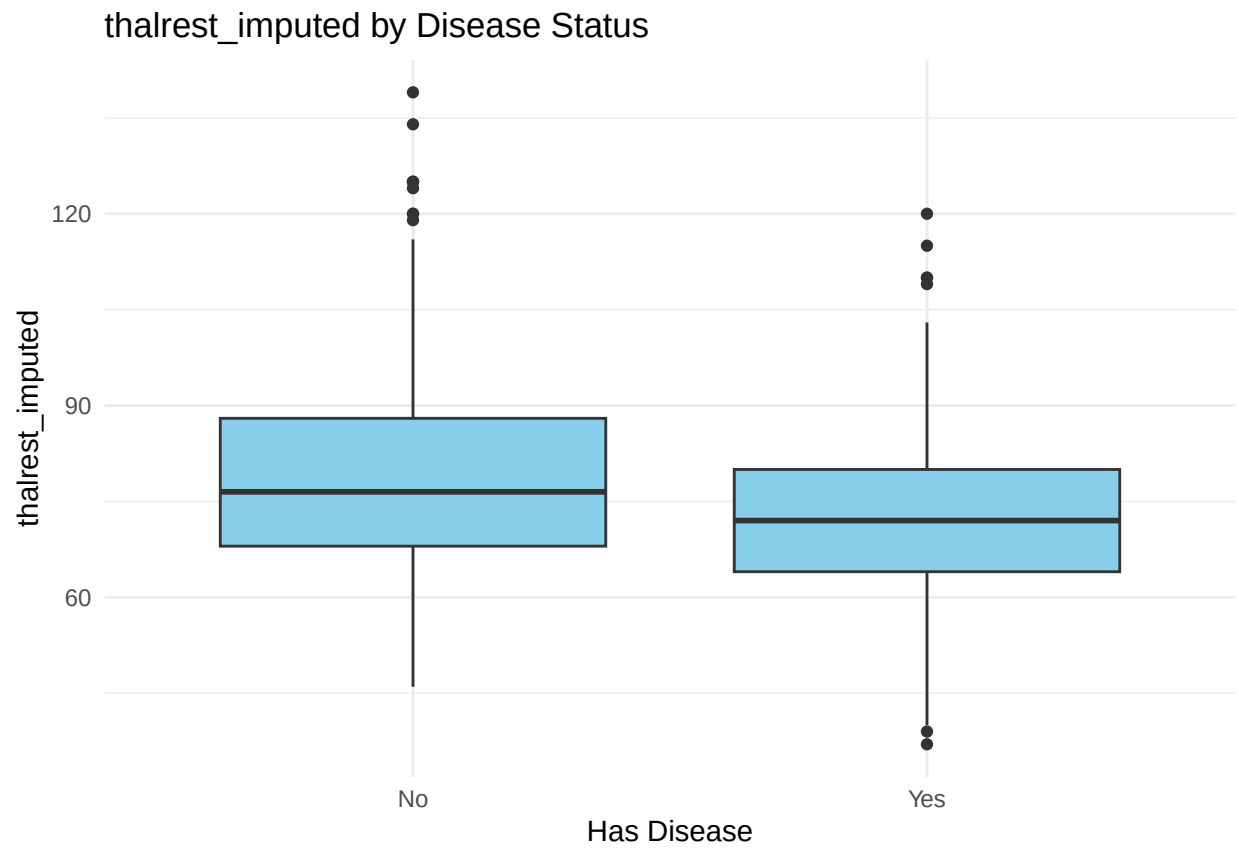


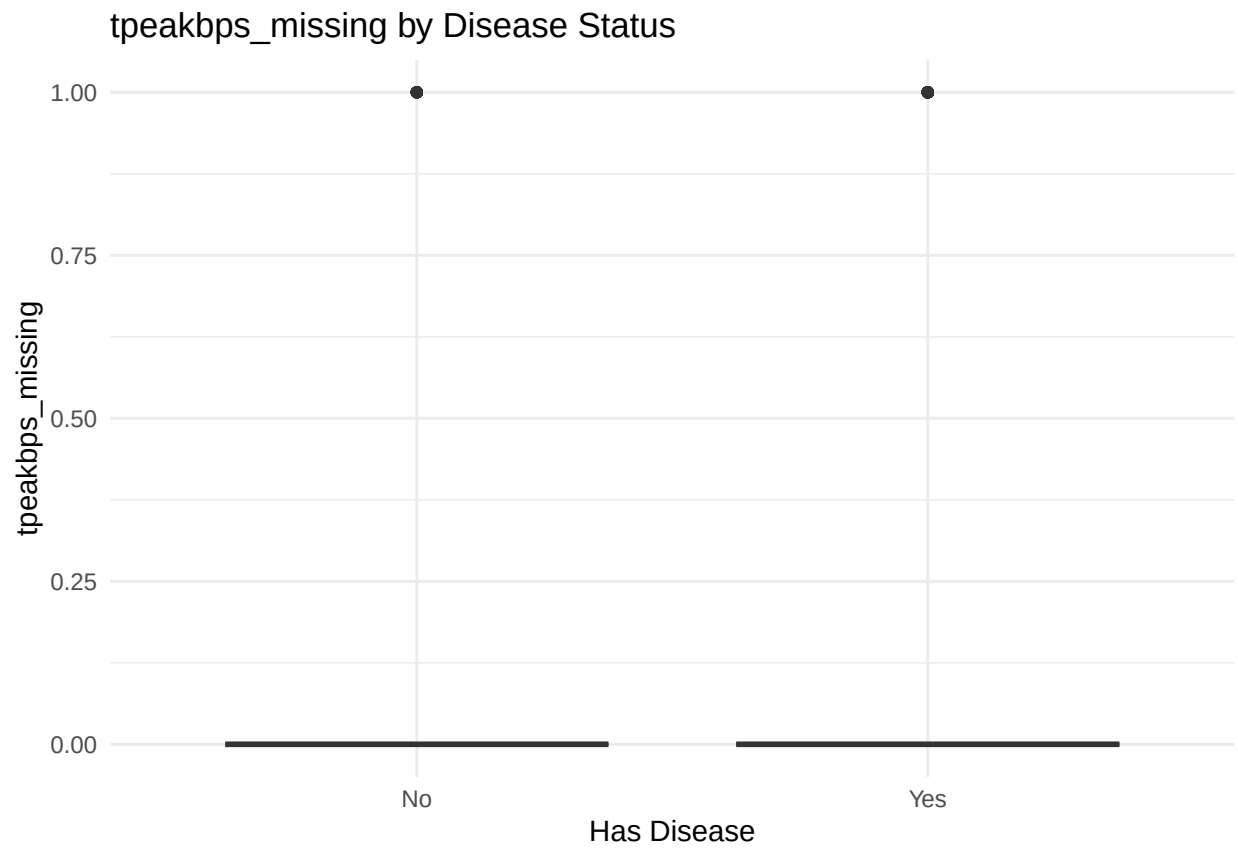


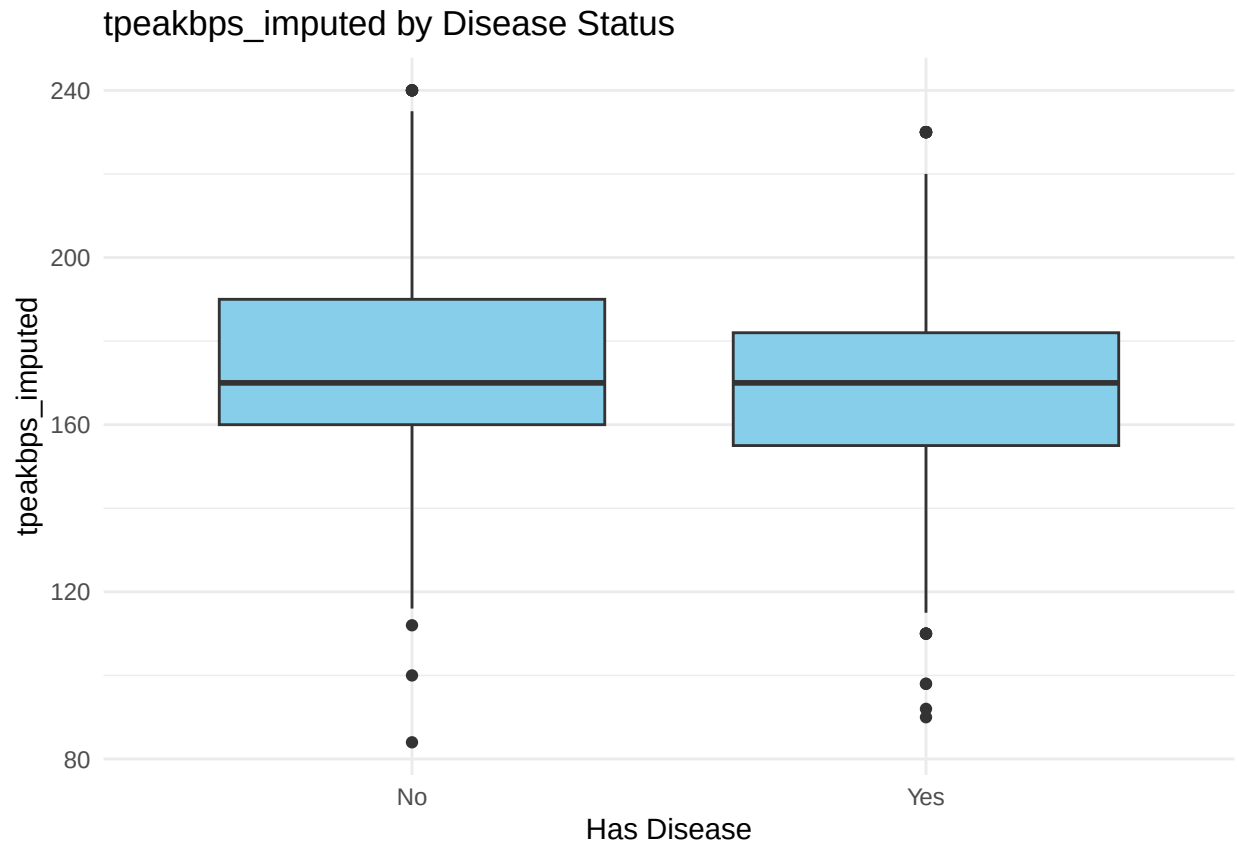


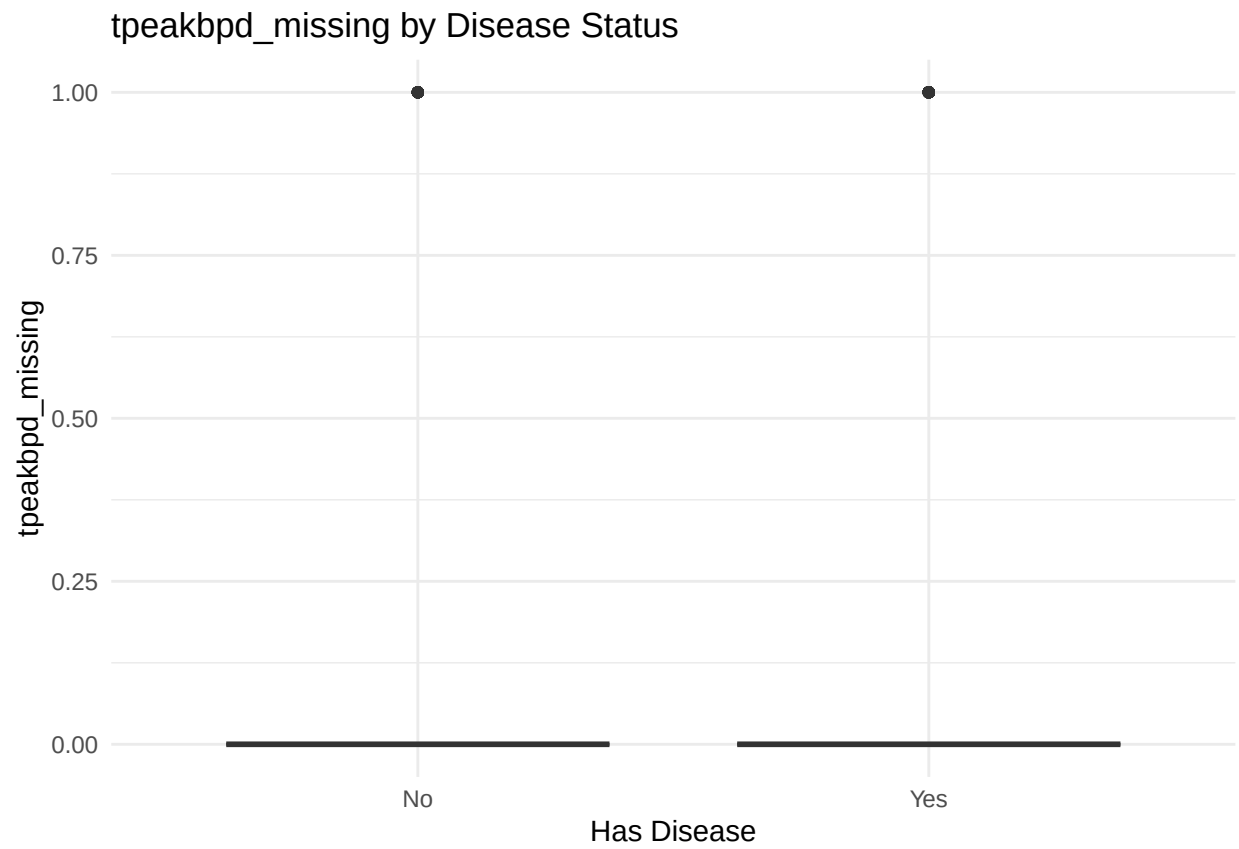


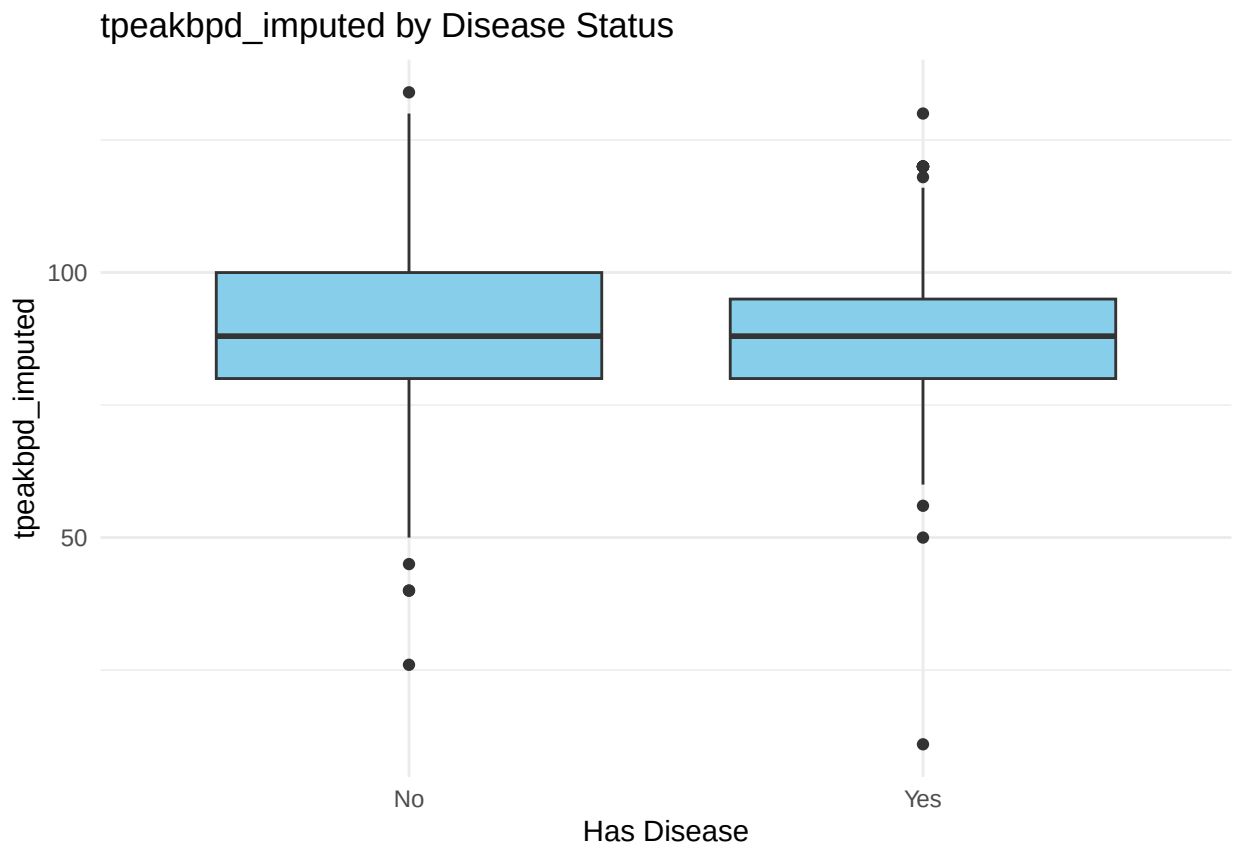


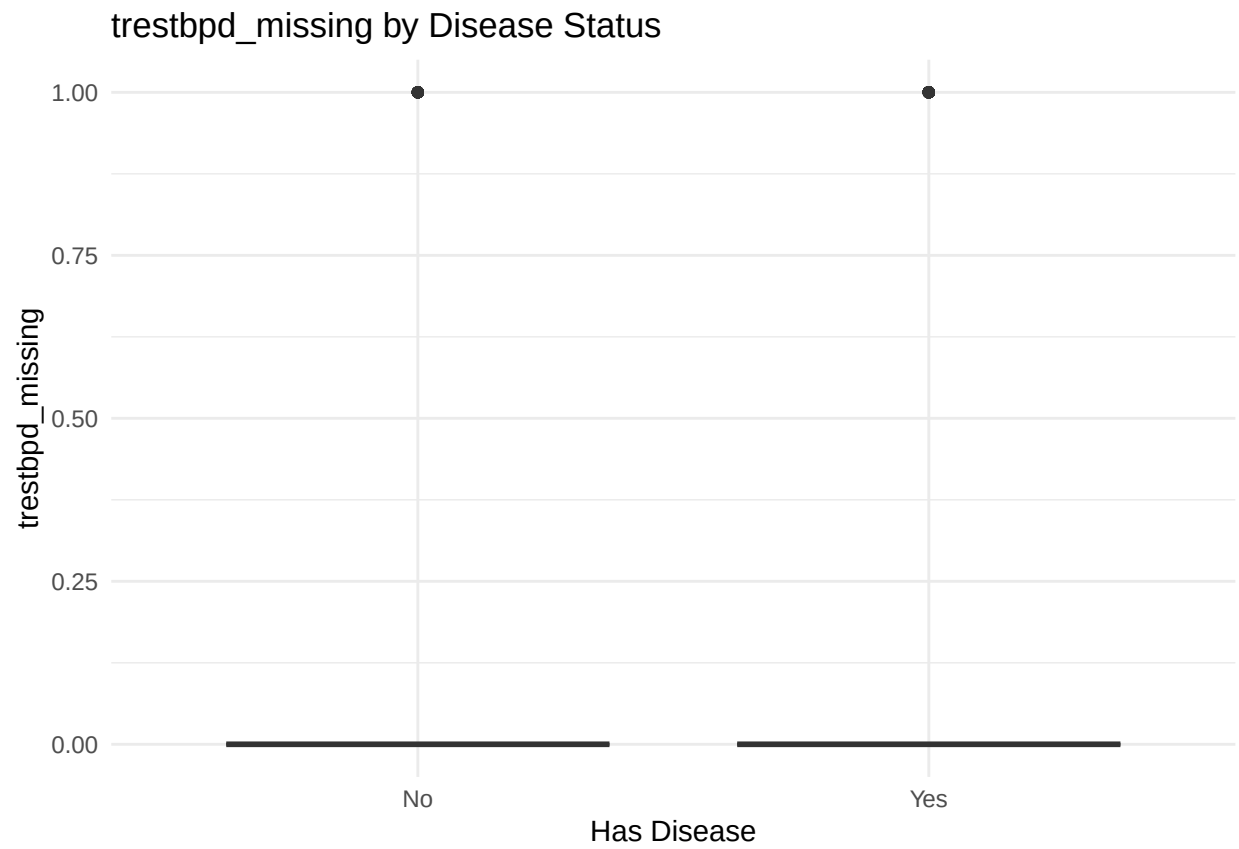


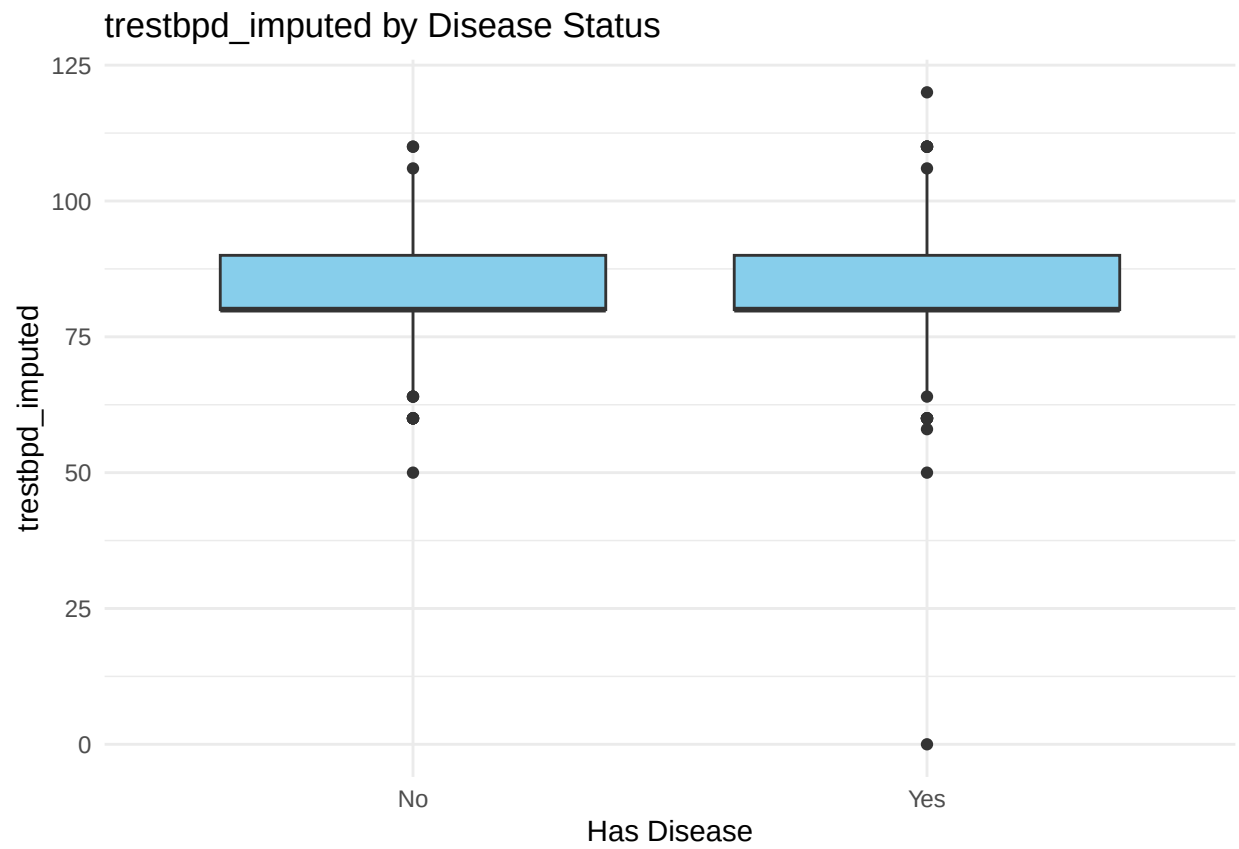


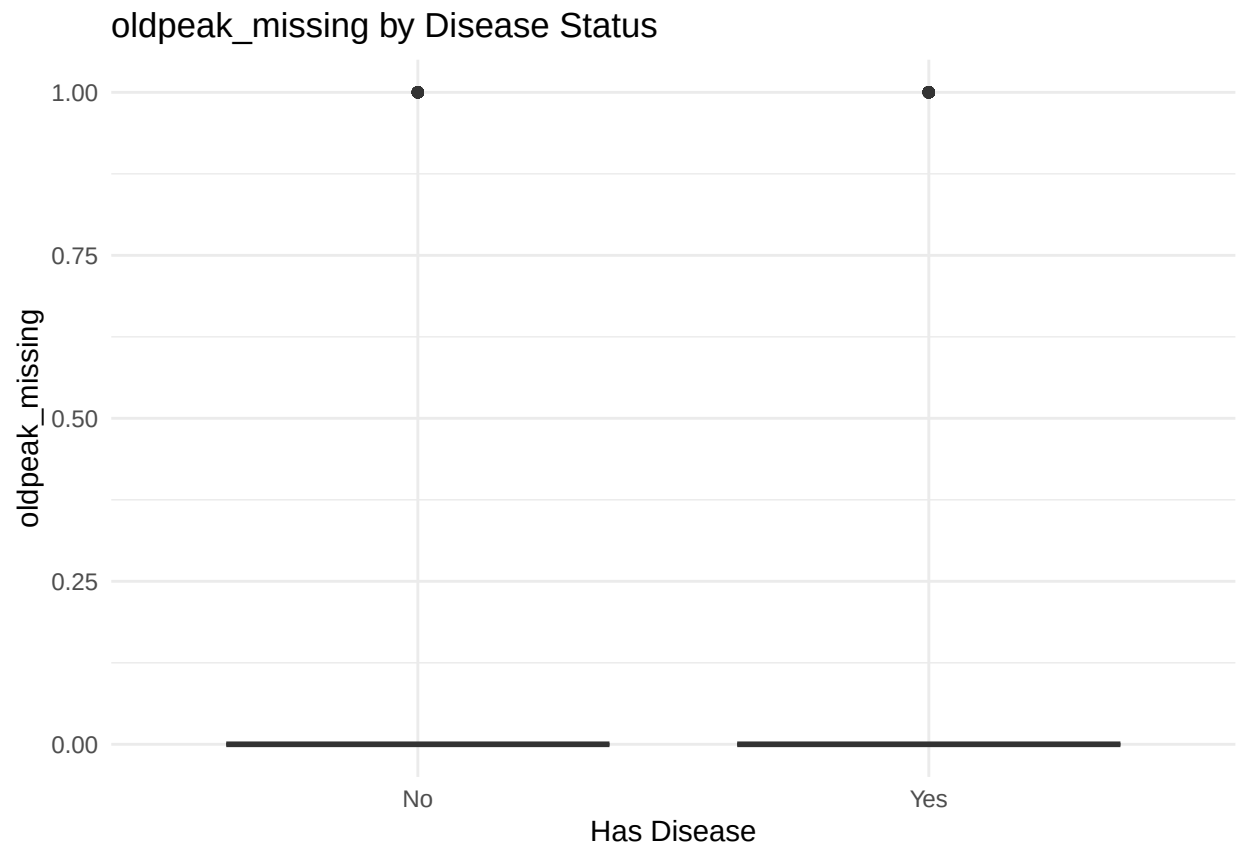


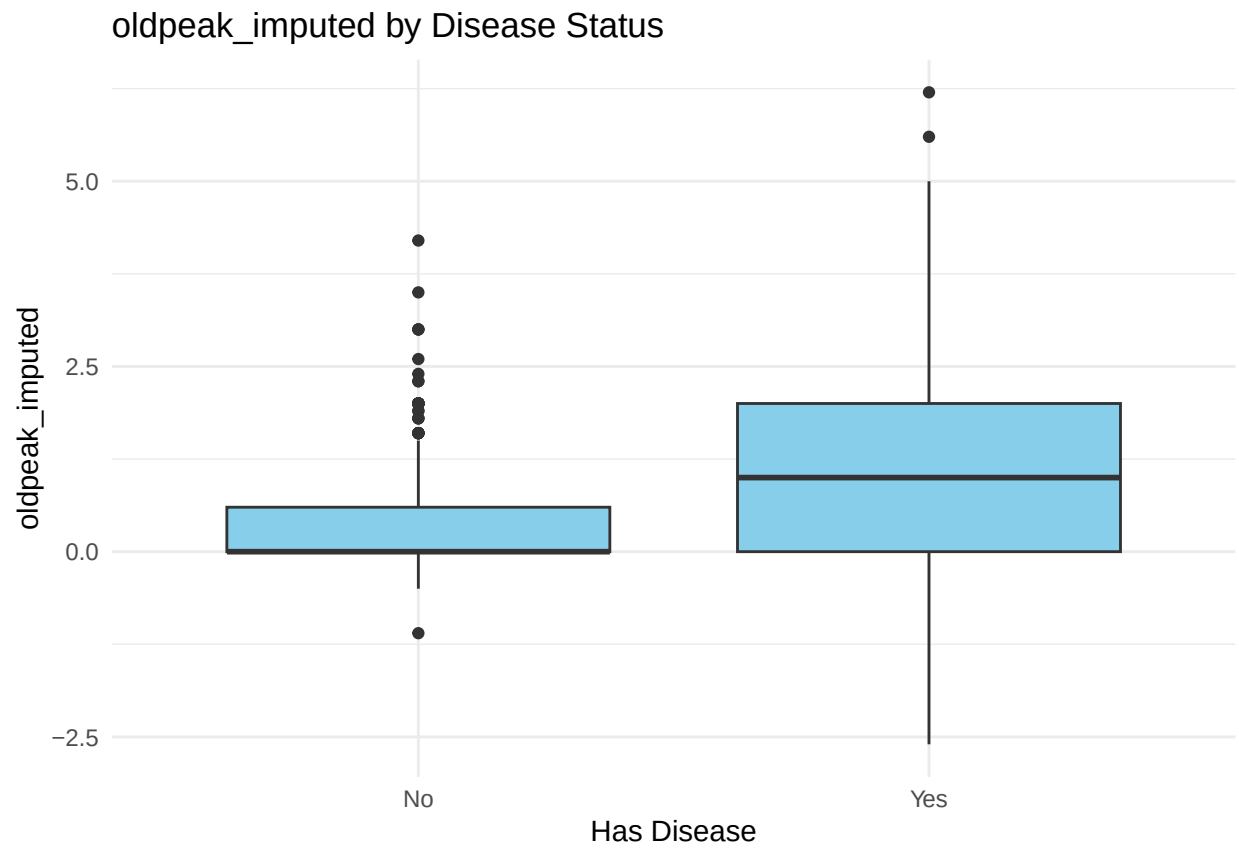


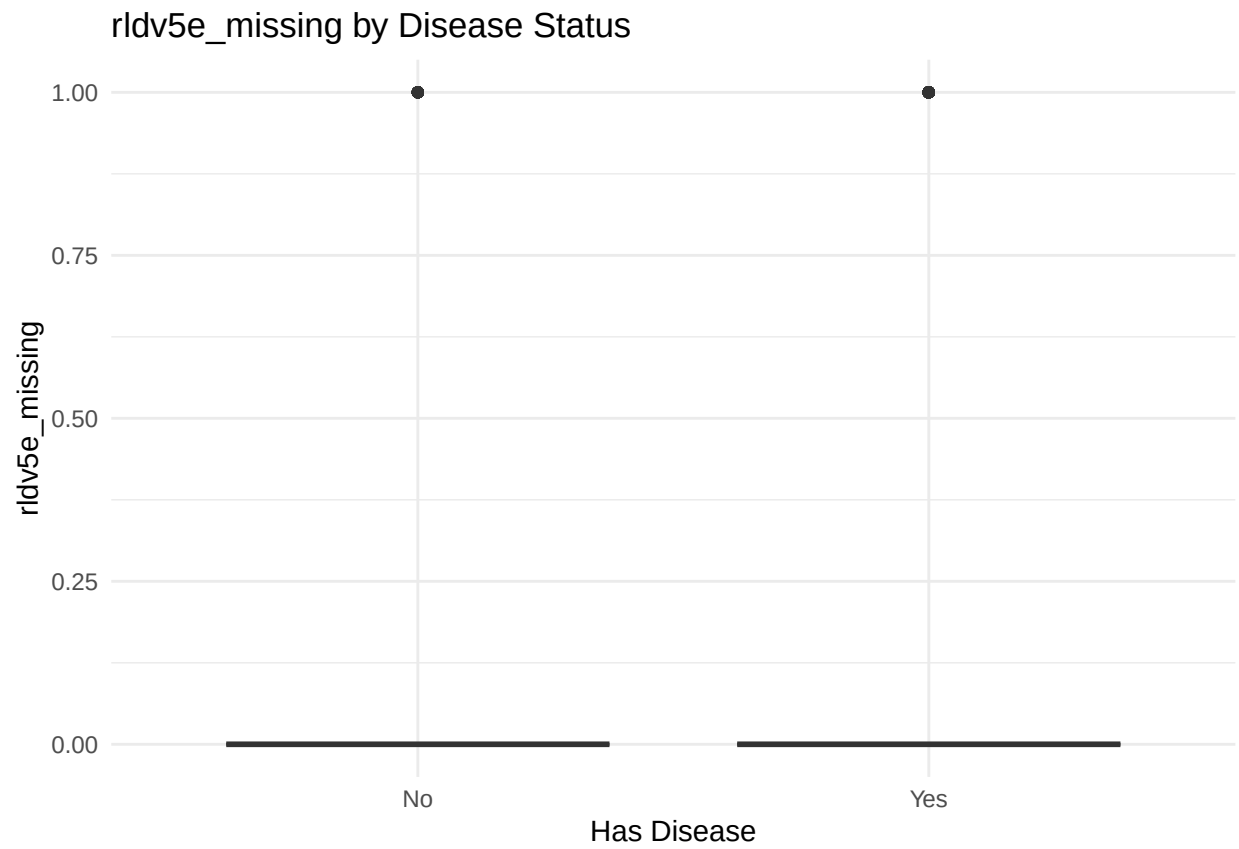


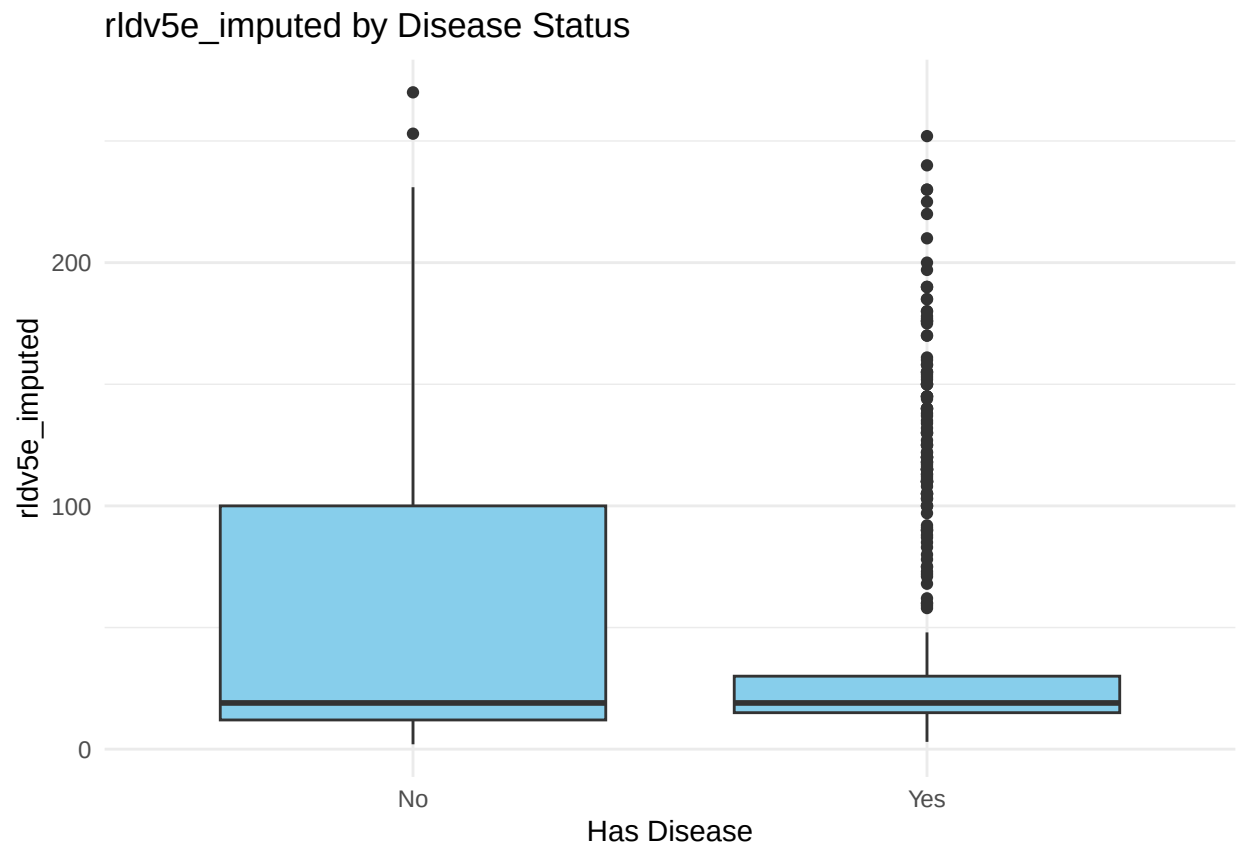


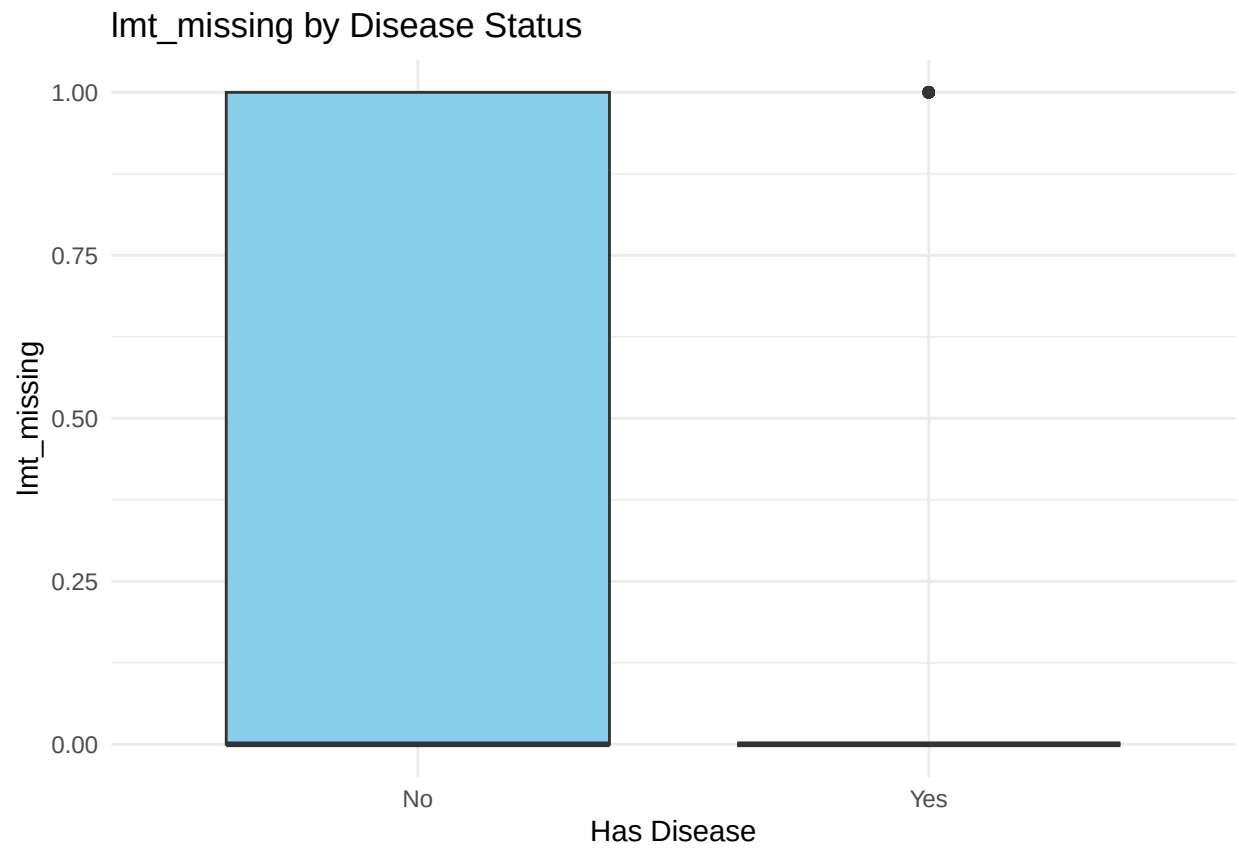


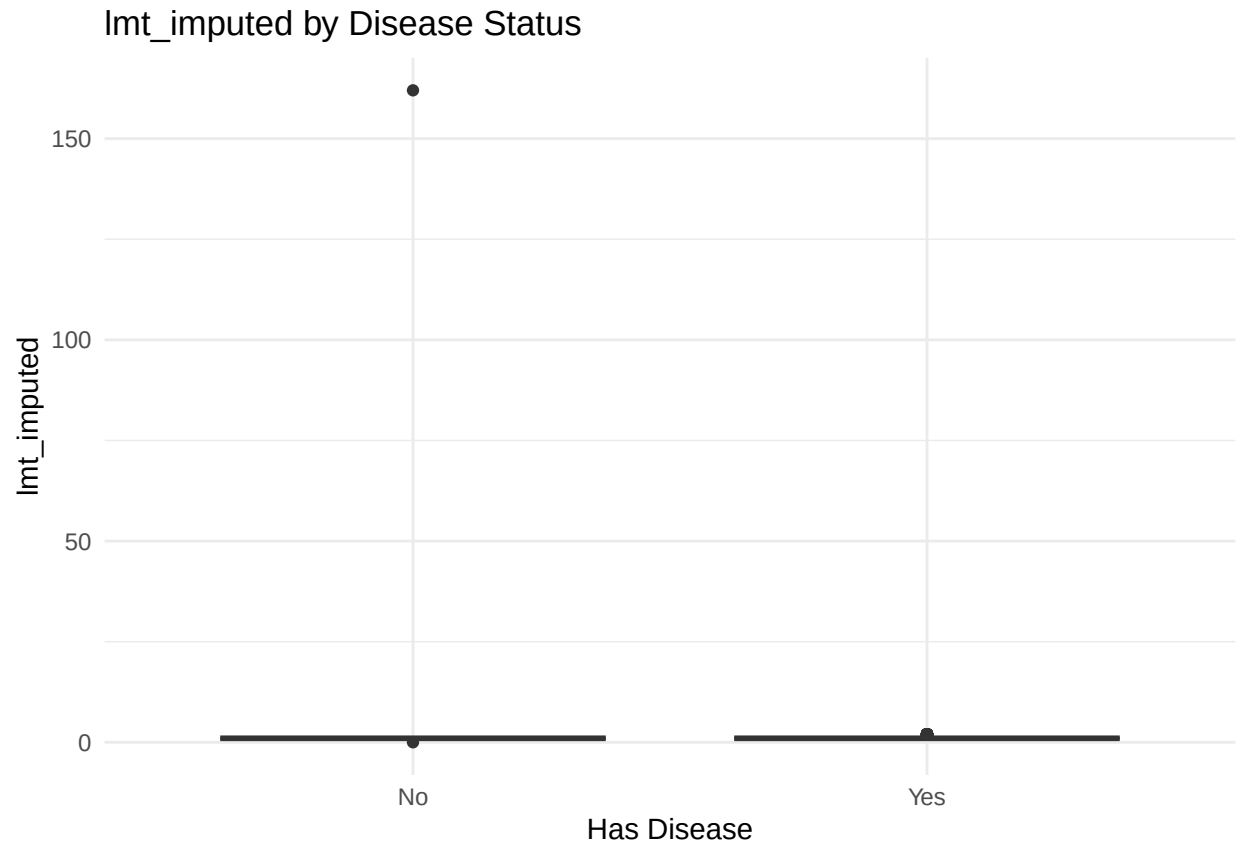












Welch's two-sample t-test

Statistical testing to determine if there are significant differences between the means of two features.

```
for (var in num_vars) {
  formula <- reformulate("has_disease", response = var)
  test_result <- t.test(formula, data = df)
  cat("=== T-test for", var, "===\n")
  print(test_result)
  cat("\n\n")
}
```

```
## === T-test for age ===
##
##  Welch Two Sample t-test
##
## data:  age by has_disease
## t = -8.5853, df = 833.36, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
##   -6.462916 -4.057656
## sample estimates:
##  mean in group No mean in group Yes
##       50.58416      55.84444
##
##
##
```

```

## === T-test for ca_imputed ===
##
## Welch Two Sample t-test
##
## data: ca_imputed by has_disease
## t = -3.6443, df = 895.1, p-value = 0.0002836
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.25055510 -0.07514748
## sample estimates:
## mean in group No mean in group Yes
## 0.1361386 0.2989899
##
##
##
## === T-test for thal_imputed ===
##
## Welch Two Sample t-test
##
## data: thal_imputed by has_disease
## t = -9.0385, df = 860.57, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -1.1314140 -0.7277019
## sample estimates:
## mean in group No mean in group Yes
## 3.438119 4.367677
##
##
##
## === T-test for thaltime_imputed ===
##
## Welch Two Sample t-test
##
## data: thaltime_imputed by has_disease
## t = -0.82499, df = 890.94, p-value = 0.4096
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.5200975 0.2122537
## sample estimates:
## mean in group No mean in group Yes
## 5.761634 5.915556
##
##
##
## === T-test for rldv5_imputed ===
##
## Welch Two Sample t-test
##
## data: rldv5_imputed by has_disease
## t = -1.7981, df = 828.98, p-value = 0.07252
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -1.05248435 0.04609371

```

```

## sample estimates:
## mean in group No mean in group Yes
##      13.93317      14.43636
##
##
##
## === T-test for trestbps_missing ===
##
## Welch Two Sample t-test
##
## data:  trestbps_missing by has_disease
## t = -1.8033, df = 896.85, p-value = 0.07167
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
##  -0.061152495  0.002586638
## sample estimates:
## mean in group No mean in group Yes
##      0.04950495      0.07878788
##
##
##
## === T-test for trestbps_imputed ===
##
## Welch Two Sample t-test
##
## data:  trestbps_imputed by has_disease
## t = -2.977, df = 896.52, p-value = 0.002989
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
##  -5.998366 -1.231847
## sample estimates:
## mean in group No mean in group Yes
##      129.9728      133.5879
##
##
##
## === T-test for chol_missing ===
##
## Welch Two Sample t-test
##
## data:  chol_missing by has_disease
## t = 1.9913, df = 696, p-value = 0.04684
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
##  0.0003482239 0.0492667376
## sample estimates:
## mean in group No mean in group Yes
##      0.04702970      0.02222222
##
##
##
## === T-test for chol_imputed ===
##
## Welch Two Sample t-test

```

```

##
## data: chol_imputed by has_disease
## t = 7.5093, df = 816.55, p-value = 1.559e-13
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## 37.88176 64.69433
## sample estimates:
## mean in group No mean in group Yes
## 227.8416 176.5535
##
##
##
## === T-test for thaldur_missing ===
##
## Welch Two Sample t-test
##
## data: thaldur_missing by has_disease
## t = -1.1721, df = 893.81, p-value = 0.2415
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.05008523 0.01263149
## sample estimates:
## mean in group No mean in group Yes
## 0.05198020 0.07070707
##
##
##
## === T-test for thaldur_imputed ===
##
## Welch Two Sample t-test
##
## data: thaldur_imputed by has_disease
## t = 8.7171, df = 775.94, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## 1.608559 2.543589
## sample estimates:
## mean in group No mean in group Yes
## 9.764356 7.688283
##
##
##
## === T-test for met_missing ===
##
## Welch Two Sample t-test
##
## data: met_missing by has_disease
## t = -4.1789, df = 877.82, p-value = 3.222e-05
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.12675602 -0.04574123
## sample estimates:
## mean in group No mean in group Yes
## 0.06930693 0.15555556

```

```

##
##
##
## === T-test for met_imputed ===
##
## Welch Two Sample t-test
##
## data: met_imputed by has_disease
## t = -7.6036, df = 544.71, p-value = 1.27e-13
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -16.563228 -9.762245
## sample estimates:
## mean in group No mean in group Yes
##      8.12797      21.29071
##
##
##
## === T-test for thalach_missing ===
##
## Welch Two Sample t-test
##
## data: thalach_missing by has_disease
## t = -1.3416, df = 895.8, p-value = 0.1801
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.052219591 0.009815351
## sample estimates:
## mean in group No mean in group Yes
##      0.04950495      0.07070707
##
##
##
## === T-test for thalach_imputed ===
##
## Welch Two Sample t-test
##
## data: thalach_imputed by has_disease
## t = 12.322, df = 865.57, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## 16.15533 22.27721
## sample estimates:
## mean in group No mean in group Yes
##      148.0446      128.8283
##
##
##
## === T-test for thalrest_missing ===
##
## Welch Two Sample t-test
##
## data: thalrest_missing by has_disease
## t = -1.4592, df = 896.5, p-value = 0.1449

```

```

## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.05445650 0.00801186
## sample estimates:
## mean in group No mean in group Yes
## 0.04950495 0.07272727
##
##
##
## === T-test for thalrest_imputed ===
##
## Welch Two Sample t-test
##
## data: thalrest_imputed by has_disease
## t = 6.3454, df = 773.7, p-value = 3.774e-10
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## 4.183520 7.931421
## sample estimates:
## mean in group No mean in group Yes
## 78.73020 72.67273
##
##
##
## === T-test for tpeakbps_missing ===
##
## Welch Two Sample t-test
##
## data: tpeakbps_missing by has_disease
## t = -2.2436, df = 894.11, p-value = 0.0251
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.070047808 -0.004679664
## sample estimates:
## mean in group No mean in group Yes
## 0.04950495 0.08686869
##
##
##
## === T-test for tpeakbps_imputed ===
##
## Welch Two Sample t-test
##
## data: tpeakbps_imputed by has_disease
## t = 3.1634, df = 860.4, p-value = 0.001614
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## 1.988506 8.489841
## sample estimates:
## mean in group No mean in group Yes
## 174.4109 169.1717
##
##
##

```



```

## === T-test for tpeakbpd_missing ===
##
## Welch Two Sample t-test
##
## data: tpeakbpd_missing by has_disease
## t = -2.2436, df = 894.11, p-value = 0.0251
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.070047808 -0.004679664
## sample estimates:
## mean in group No mean in group Yes
## 0.04950495 0.08686869
##
##
##
## === T-test for tpeakbpd_imputed ===
##
## Welch Two Sample t-test
##
## data: tpeakbpd_imputed by has_disease
## t = 0.38958, df = 814.22, p-value = 0.6969
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -1.517519 2.269054
## sample estimates:
## mean in group No mean in group Yes
## 87.54950 87.17374
##
##
##
## === T-test for trestbpd_missing ===
##
## Welch Two Sample t-test
##
## data: trestbpd_missing by has_disease
## t = -1.8033, df = 896.85, p-value = 0.07167
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.061152495 0.002586638
## sample estimates:
## mean in group No mean in group Yes
## 0.04950495 0.07878788
##
##
##
## === T-test for trestbpd_imputed ===
##
## Welch Two Sample t-test
##
## data: trestbpd_imputed by has_disease
## t = 0.20883, df = 879.12, p-value = 0.8346
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -1.163256 1.440273

```

```

## sample estimates:
## mean in group No mean in group Yes
##      83.36881      83.23030
##
##
##
## === T-test for oldpeak_missing ===
##
## Welch Two Sample t-test
##
## data: oldpeak_missing by has_disease
## t = -1.8566, df = 896.85, p-value = 0.06369
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.063456965 0.001760795
## sample estimates:
## mean in group No mean in group Yes
##      0.05198020      0.08282828
##
##
##
## === T-test for oldpeak_imputed ===
##
## Welch Two Sample t-test
##
## data: oldpeak_imputed by has_disease
## t = -12.186, df = 832.96, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.8839527 -0.6387035
## sample estimates:
## mean in group No mean in group Yes
##      0.4257426      1.1870707
##
##
##
## === T-test for rldv5e_missing ===
##
## Welch Two Sample t-test
##
## data: rldv5e_missing by has_disease
## t = -8.4101, df = 779.1, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
## -0.2318428 -0.1440948
## sample estimates:
## mean in group No mean in group Yes
##      0.05445545      0.24242424
##
##
##
## === T-test for rldv5e_imputed ===
##
## Welch Two Sample t-test

```

```
##
## data:  rldv5e_imputed by has_disease
## t = 2.8953, df = 816.16, p-value = 0.003889
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
##    3.581025 18.657808
## sample estimates:
## mean in group No mean in group Yes
##      55.36386      44.24444
##
##
##
## === T-test for lmt_missing ===
##
## Welch Two Sample t-test
##
## data:  lmt_missing by has_disease
## t = 9.4361, df = 742.88, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
##    0.2257806 0.3444065
## sample estimates:
## mean in group No mean in group Yes
##      0.4628713      0.1777778
##
##
##
## === T-test for lmt_imputed ===
##
## Welch Two Sample t-test
##
## data:  lmt_imputed by has_disease
## t = 0.78046, df = 403.8, p-value = 0.4356
## alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
## 95 percent confidence interval:
##   -0.4726466  1.0950288
## sample estimates:
## mean in group No mean in group Yes
##      1.396040      1.084848
```

Summary for numerical variables

```
missing_flags <- colnames(df)[str_detect(colnames(df), "_missing$")]

# recommendation code generated with the help of ChatGPT.
# Create an empty data frame to store results
bivariate_summary <- data.frame(
  Variable = character(),
  p_value = numeric(),
  Missingness_Flag = logical(),
  Recommendation = character(),
  stringsAsFactors = FALSE
)
```

```

# Loop through and compute t-tests
for (var in num_vars) {
  formula <- reformulate("has_disease", response = var)
  p_val <- tryCatch(t.test(formula, data = df)$p.value, error = function(e) NA)

  is_missing_flag <- paste0(var, "_missing") %in% colnames(df) | var %in% missing_flags

  recommendation <- if (!is.na(p_val) && p_val < 0.05) {
    "Include (predictive)"
  } else if (is_missing_flag) {
    "Include (missingness informative)"
  } else {
    "Optional (evaluate with model)"
  }

  bivariate_summary <- rbind(
    bivariate_summary,
    data.frame(
      Variable = var,
      p_value = round(p_val, 5),
      Missingness_Flag = is_missing_flag,
      Recommendation = recommendation
    )
  )
}

# View result
bivariate_summary

```

##	Variable	p_value	Missingness_Flag	Recommendation
## 1	age	0.00000	FALSE	Include (predictive)
## 2	ca_imputed	0.00028	FALSE	Include (predictive)
## 3	thal_imputed	0.00000	FALSE	Include (predictive)
## 4	thaltime_imputed	0.40960	FALSE	Optional (evaluate with model)
## 5	rldv5_imputed	0.07252	FALSE	Optional (evaluate with model)
## 6	trestbps_missing	0.07167	TRUE	Include (missingness informative)
## 7	trestbps_imputed	0.00299	FALSE	Include (predictive)
## 8	chol_missing	0.04684	TRUE	Include (predictive)
## 9	chol_imputed	0.00000	FALSE	Include (predictive)
## 10	thaldur_missing	0.24149	TRUE	Include (missingness informative)
## 11	thaldur_imputed	0.00000	FALSE	Include (predictive)
## 12	met_missing	0.00003	TRUE	Include (predictive)
## 13	met_imputed	0.00000	FALSE	Include (predictive)
## 14	thalach_missing	0.18008	TRUE	Include (missingness informative)
## 15	thalach_imputed	0.00000	FALSE	Include (predictive)
## 16	thalrest_missing	0.14486	TRUE	Include (missingness informative)
## 17	thalrest_imputed	0.00000	FALSE	Include (predictive)
## 18	tpeakbps_missing	0.02510	TRUE	Include (predictive)
## 19	tpeakbps_imputed	0.00161	FALSE	Include (predictive)
## 20	tpeakbpd_missing	0.02510	TRUE	Include (predictive)
## 21	tpeakbpd_imputed	0.69695	FALSE	Optional (evaluate with model)
## 22	trestbpd_missing	0.07167	TRUE	Include (missingness informative)
## 23	trestbpd_imputed	0.83463	FALSE	Optional (evaluate with model)

```
## 24 oldpeak_missing 0.06369 TRUE Include (missingness informative)
## 25 oldpeak_imputed 0.00000 FALSE Include (predictive)
## 26 rldv5e_missing 0.00000 TRUE Include (predictive)
## 27 rldv5e_imputed 0.00389 FALSE Include (predictive)
## 28 lmt_missing 0.00000 TRUE Include (predictive)
## 29 lmt_imputed 0.43558 FALSE Optional (evaluate with model)

# create list of variables recommended as Include
selected_num_vars <- bivariate_summary %>%
  filter(str_detect(Recommendation, "Include")) %>%
  pull(Variable)
```

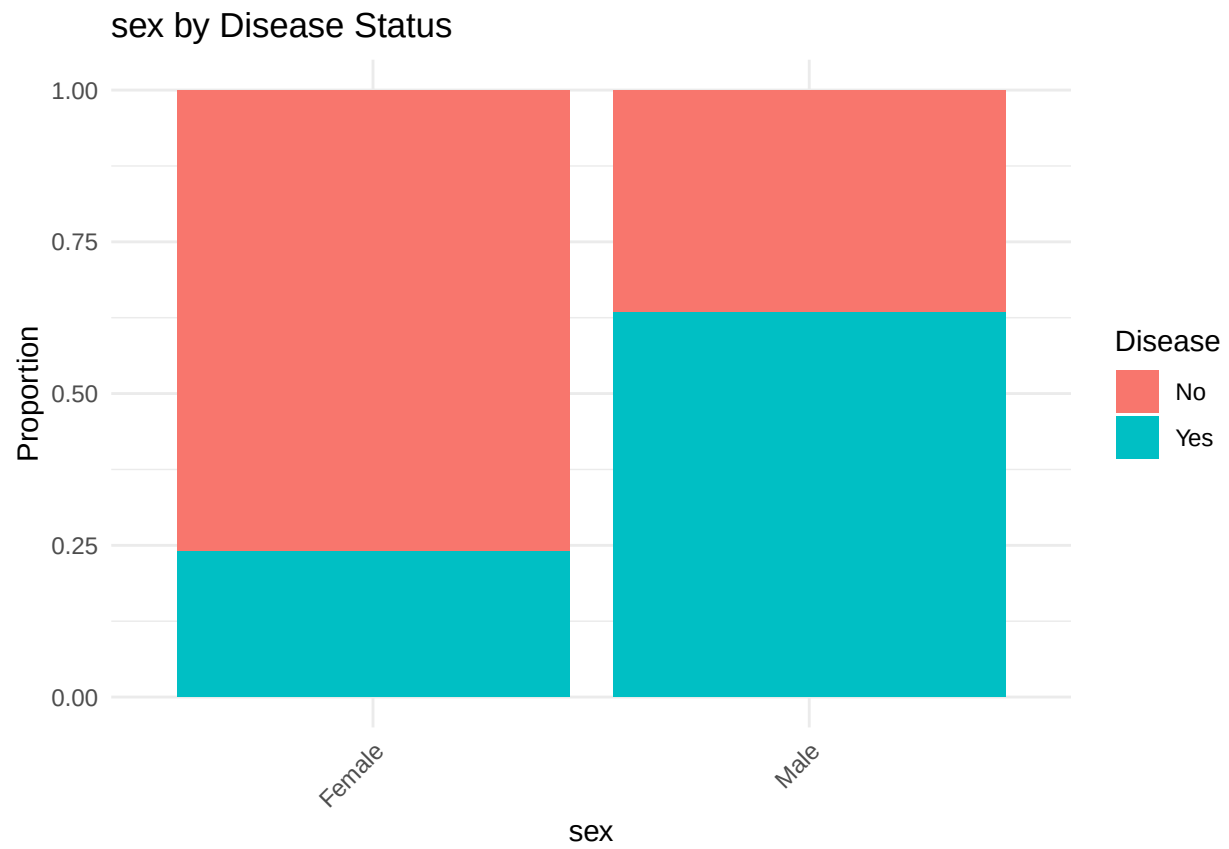
Based on recommendation, all variables recommended as 'Include' will be kept.

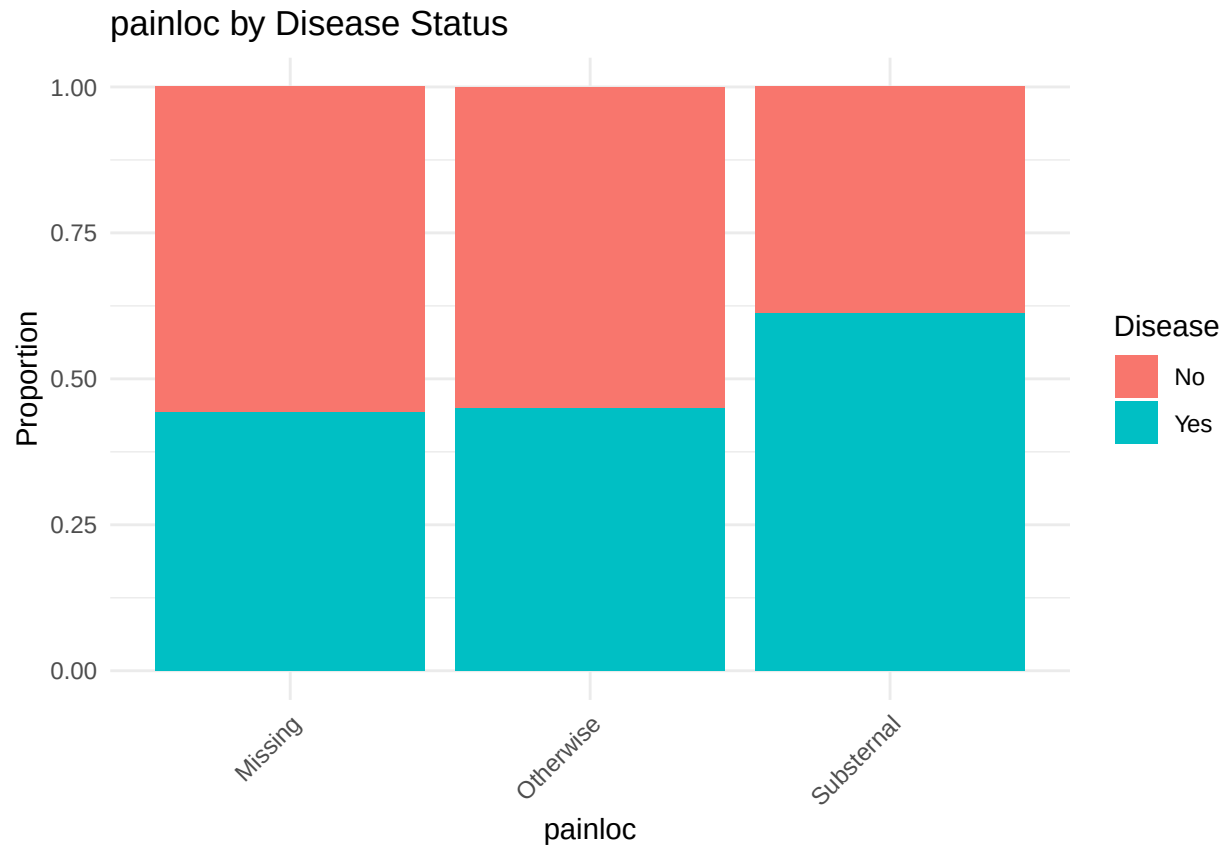
Categorical Vars

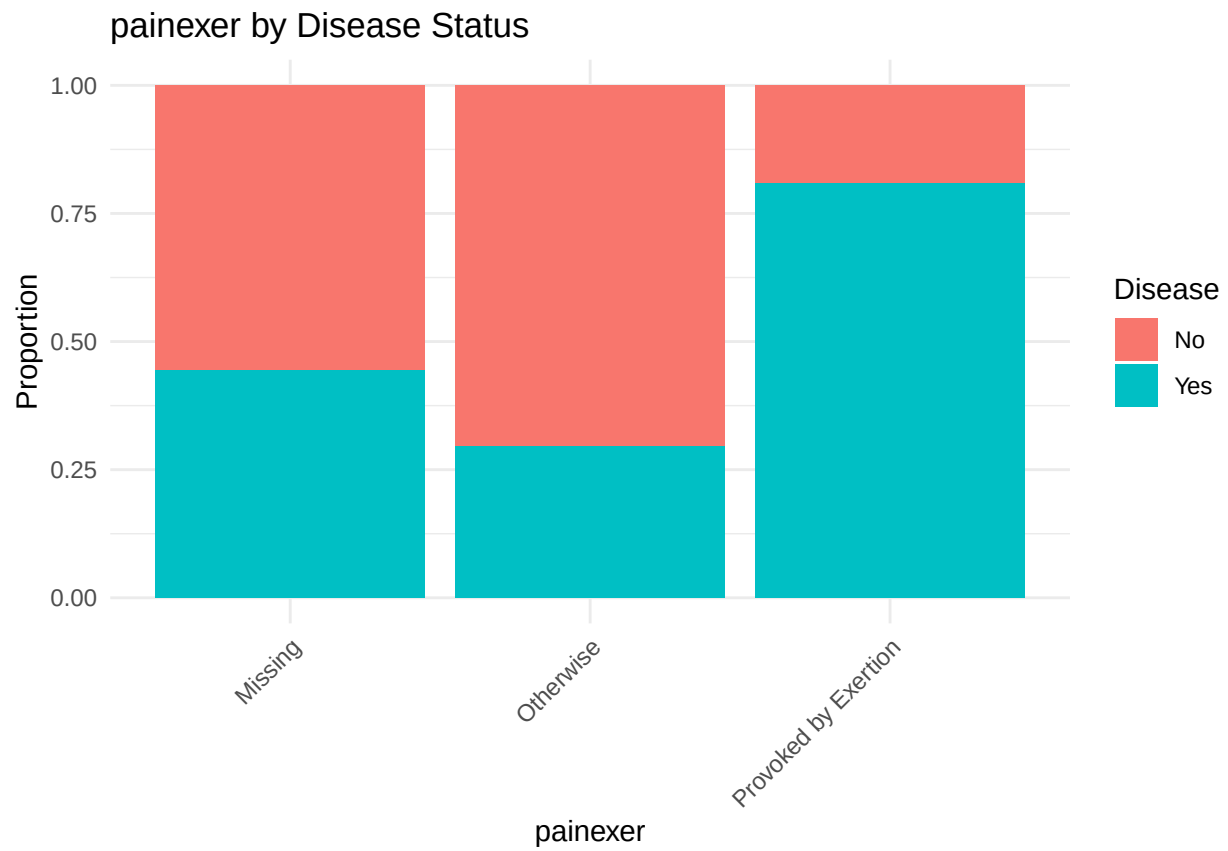
Bar Plots

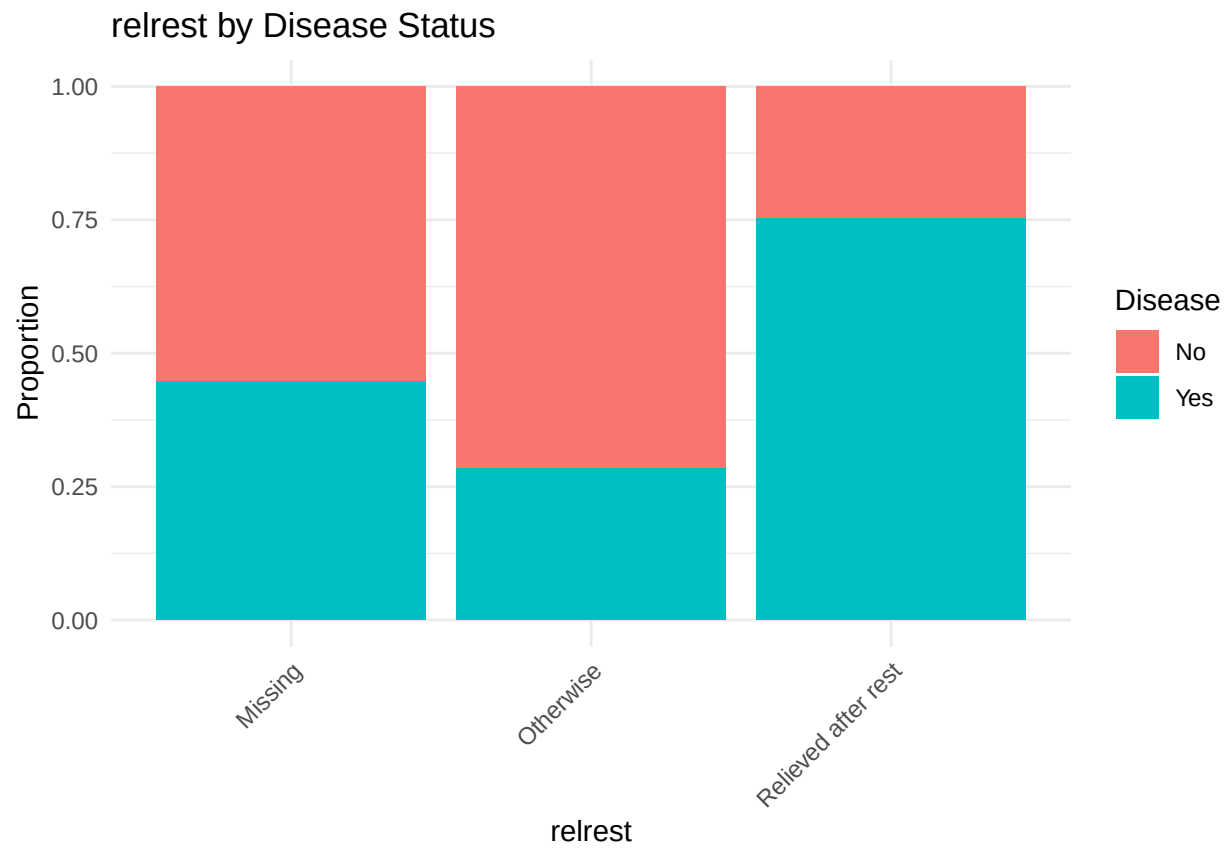
```
plot_bivariate_cat <- function(var) {
  ggplot(df, aes(x = .data[[var]], fill = factor(has_disease))) +
    geom_bar(position = "fill") + # proportion stacked
    labs(title = paste(var, "by Disease Status"), y = "Proportion", fill = "Disease") +
    theme_minimal() +
    theme(axis.text.x = element_text(angle = 45, hjust = 1))
}

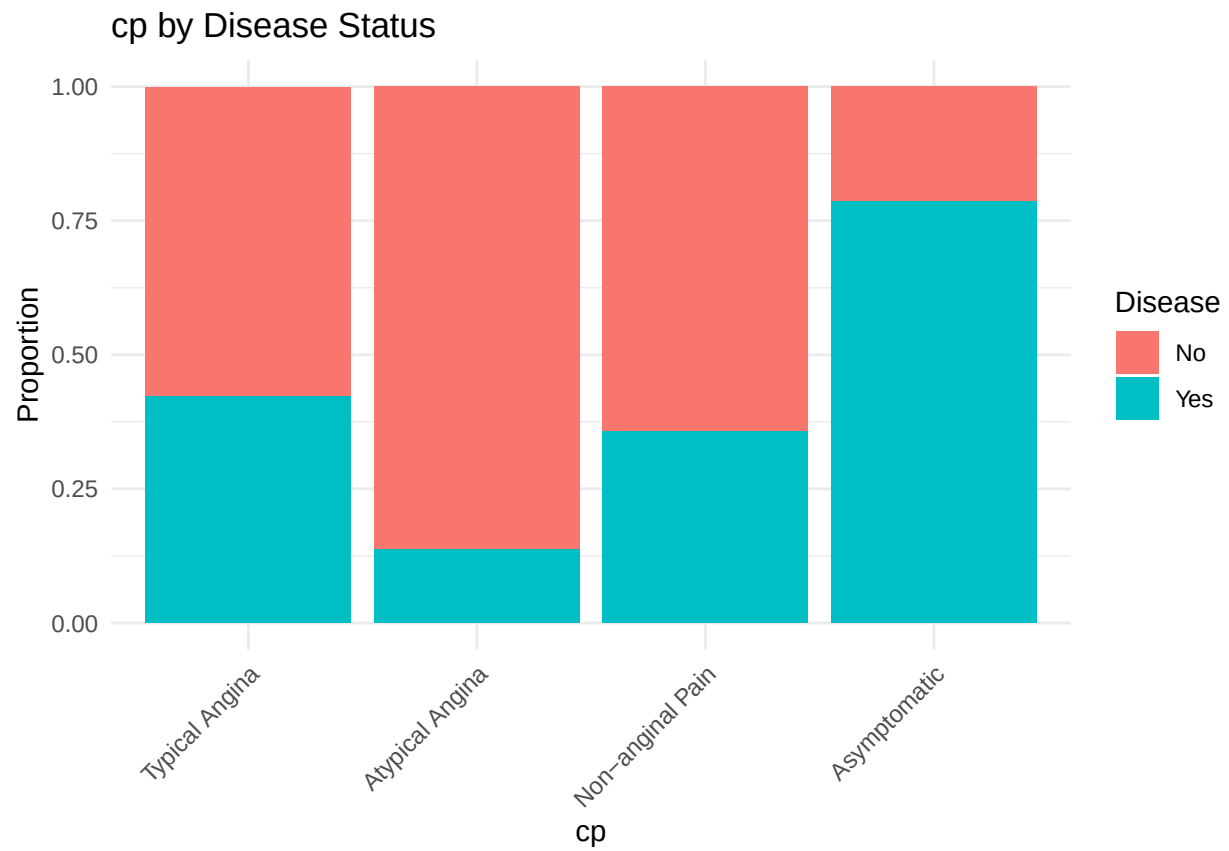
for (var in cat_vars) {
  print(plot_bivariate_cat(var))
}
```

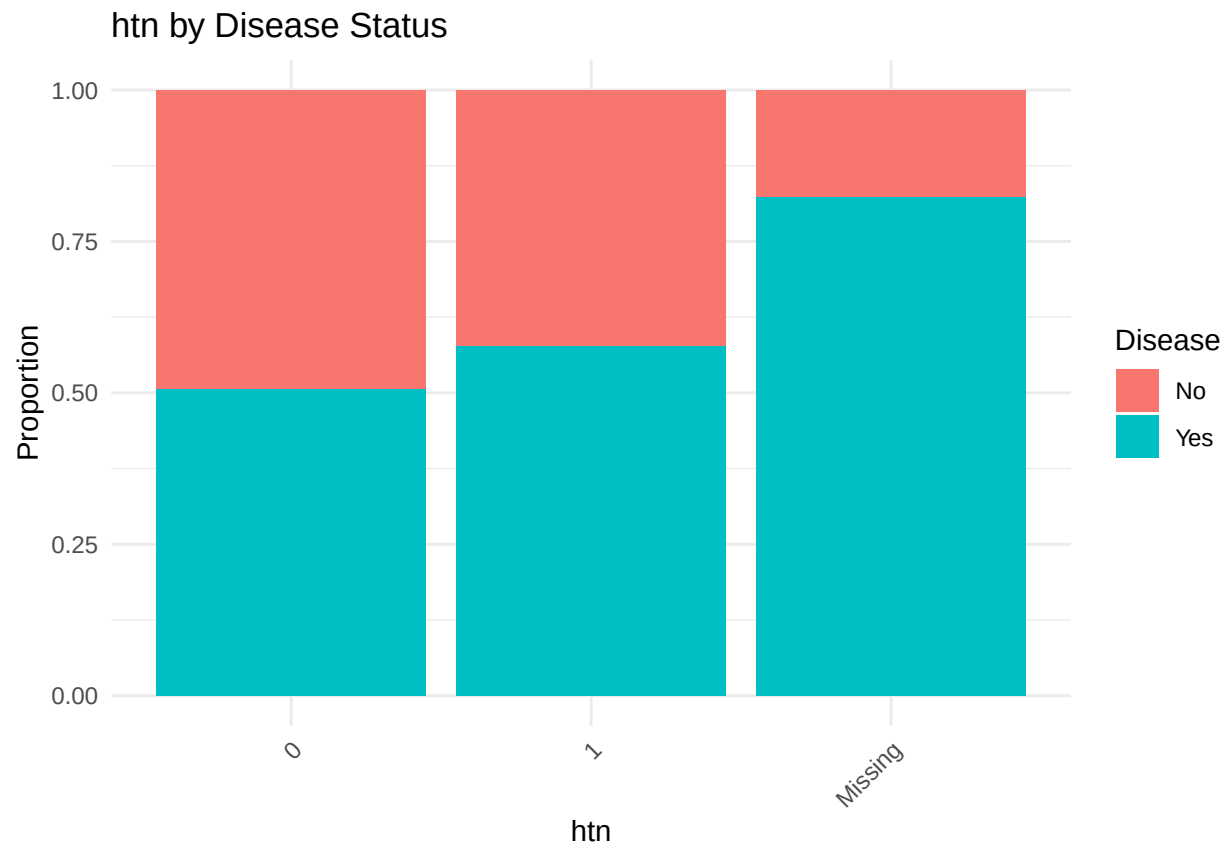


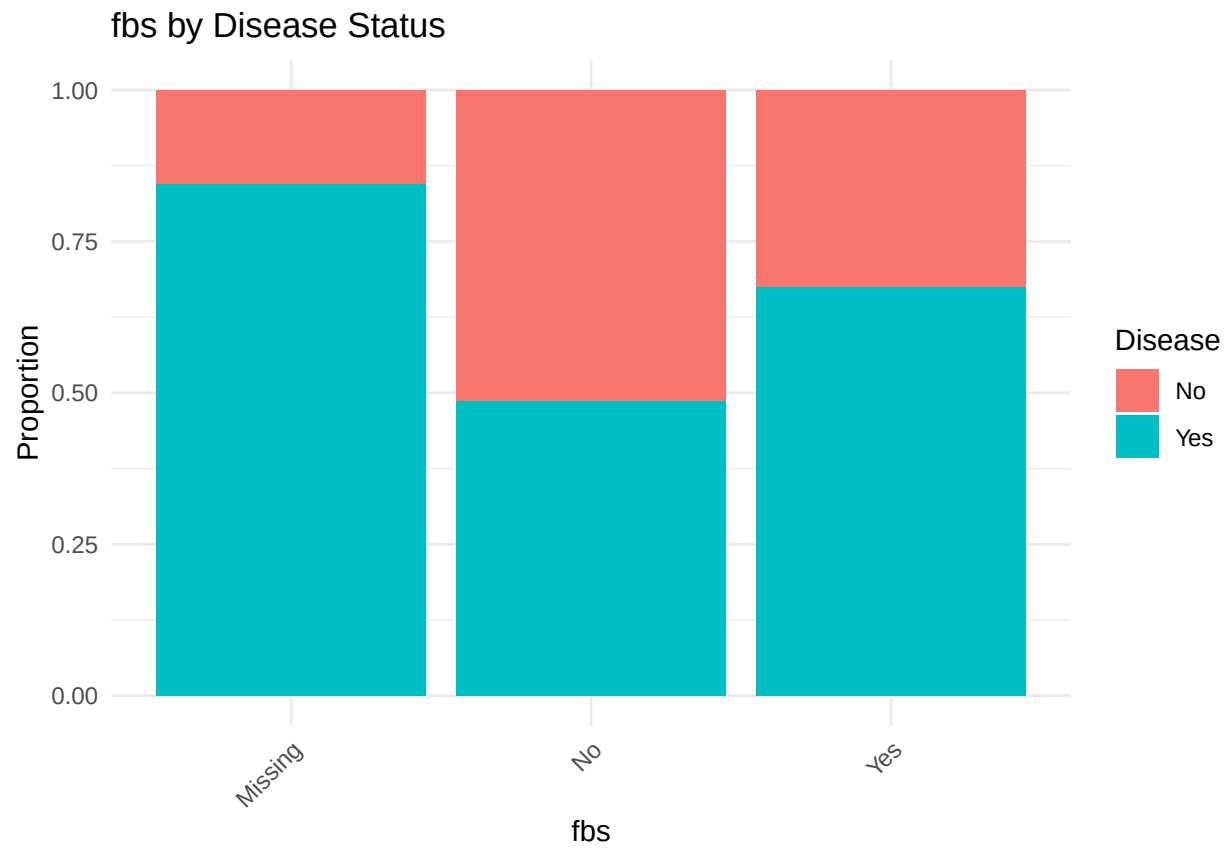


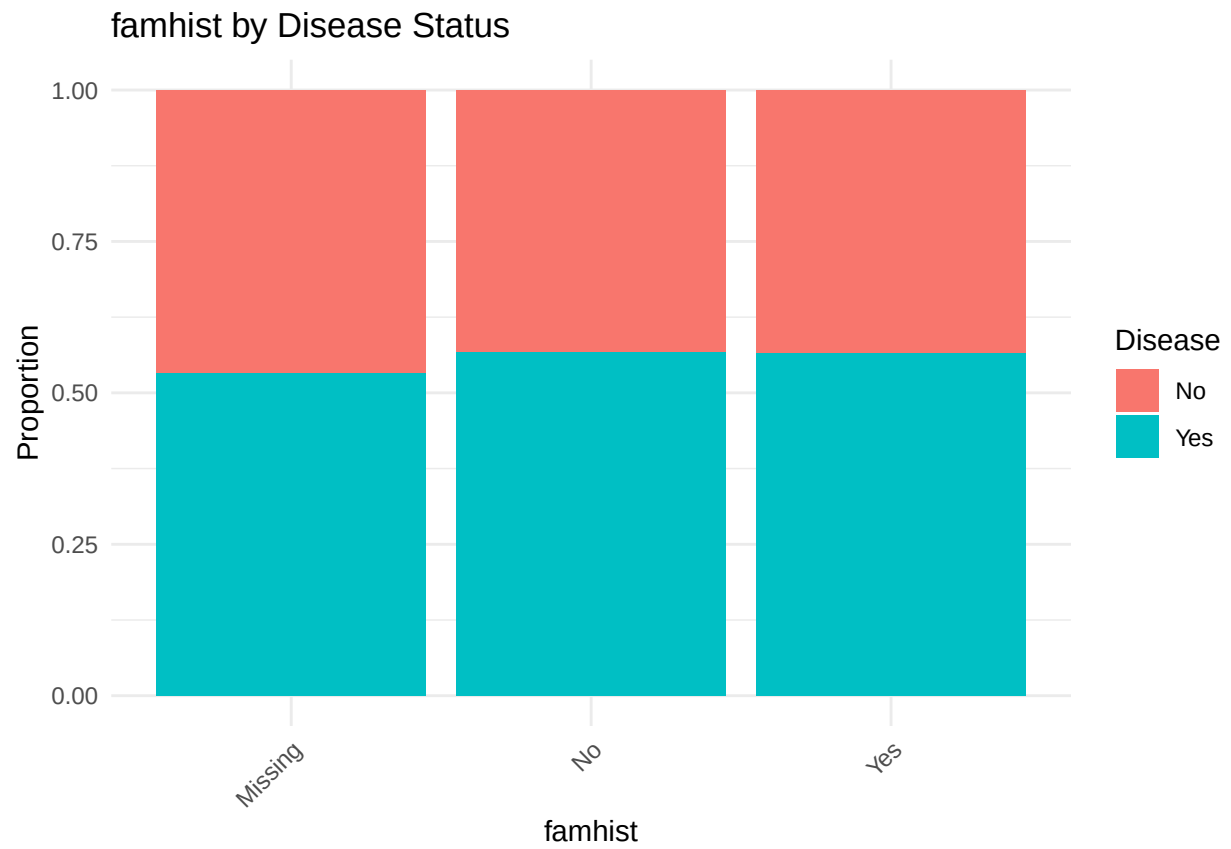


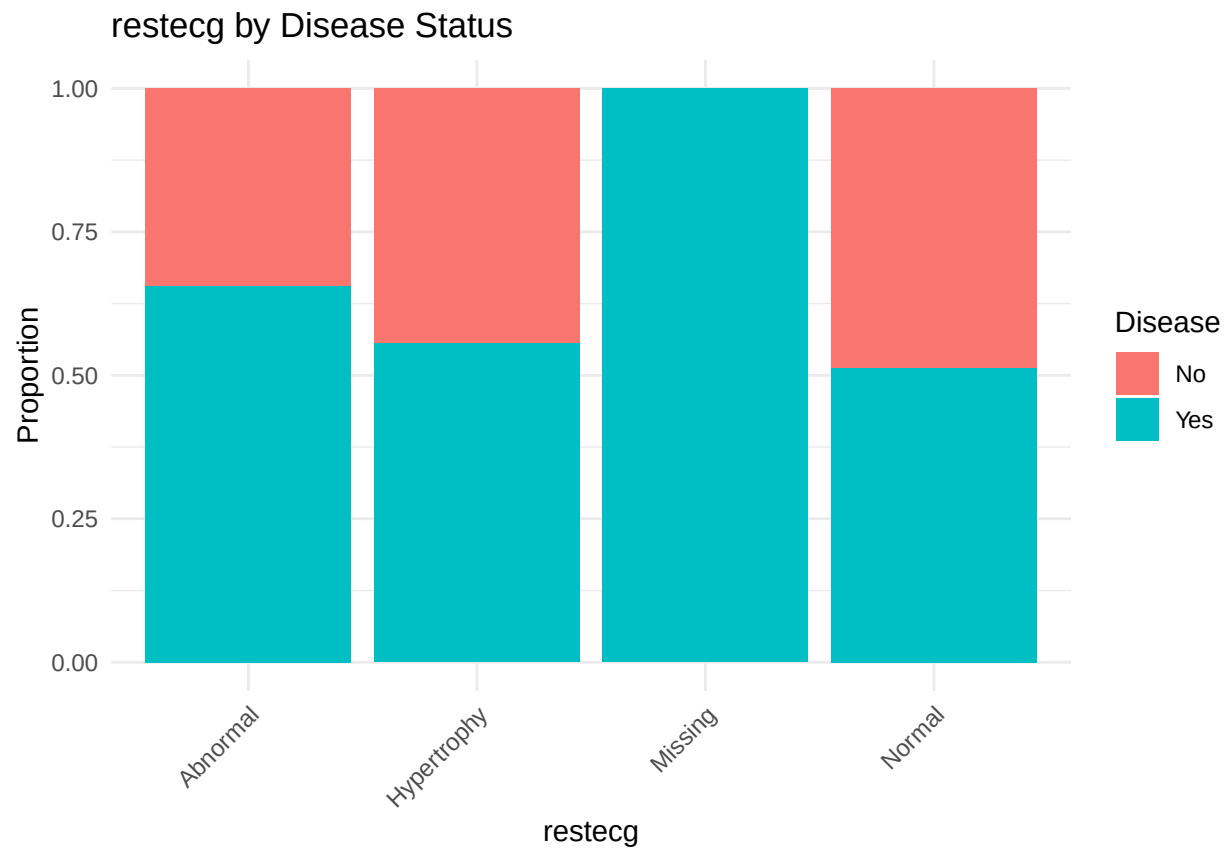


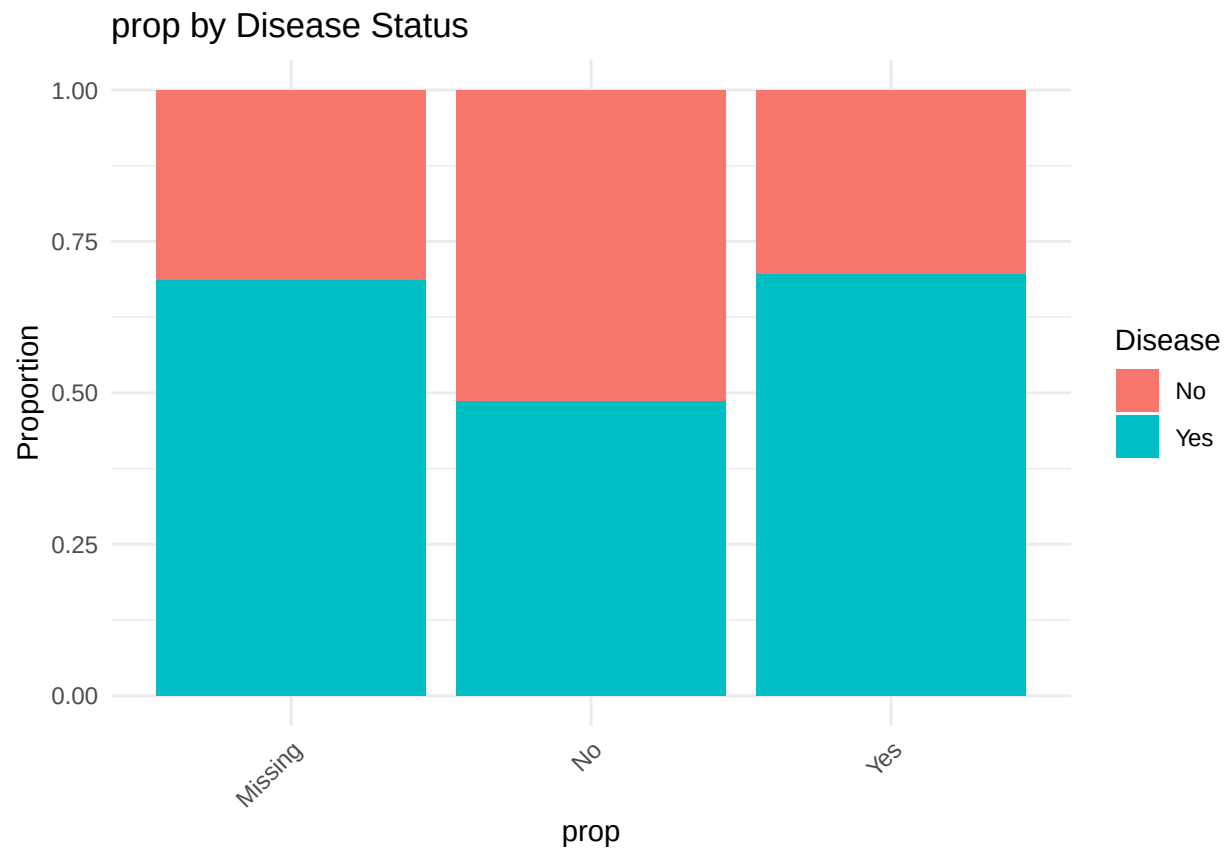


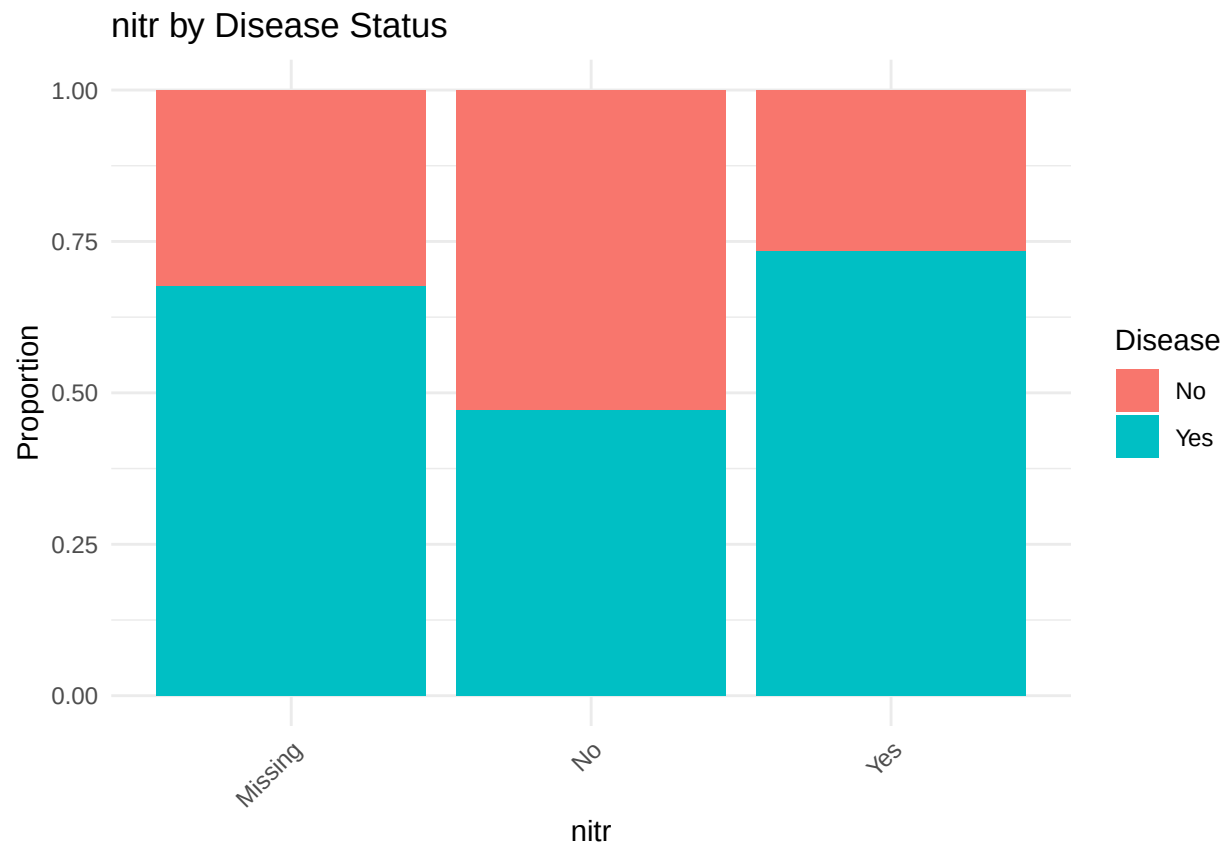


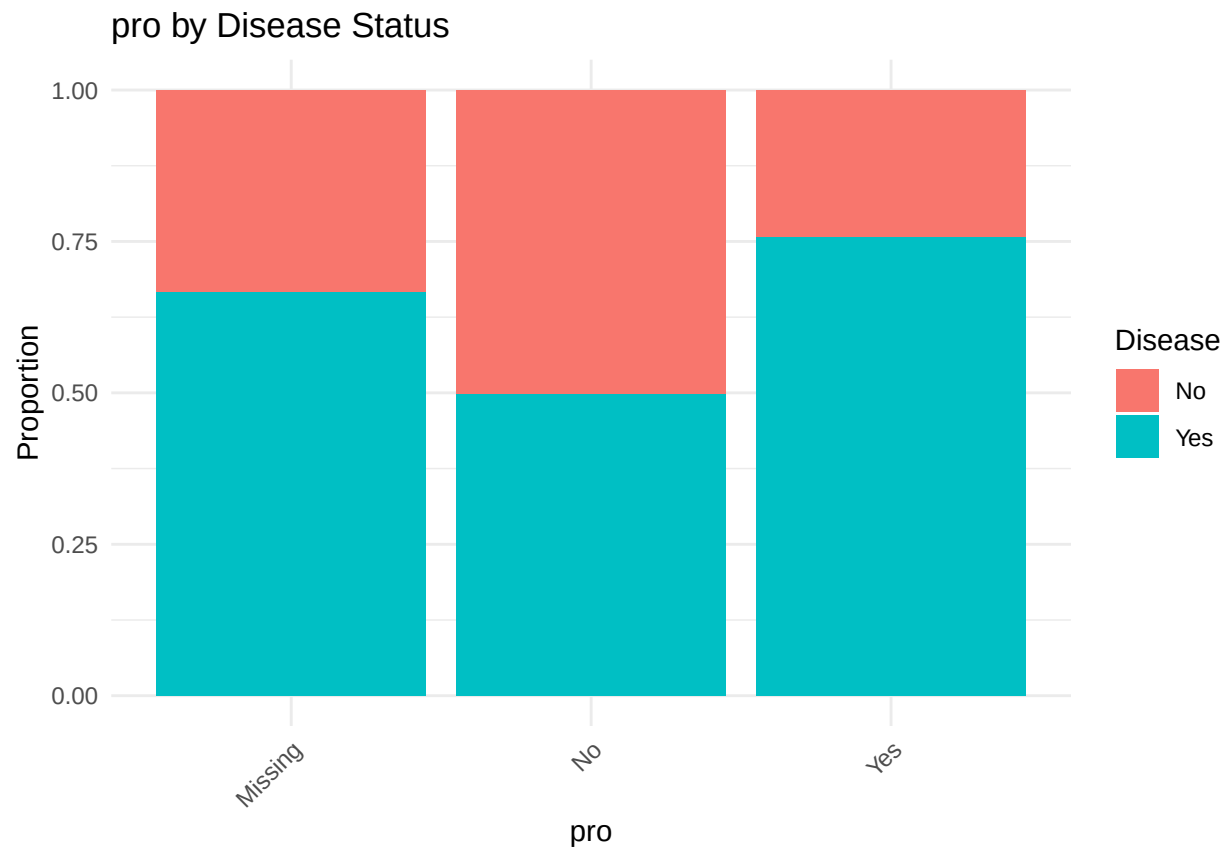


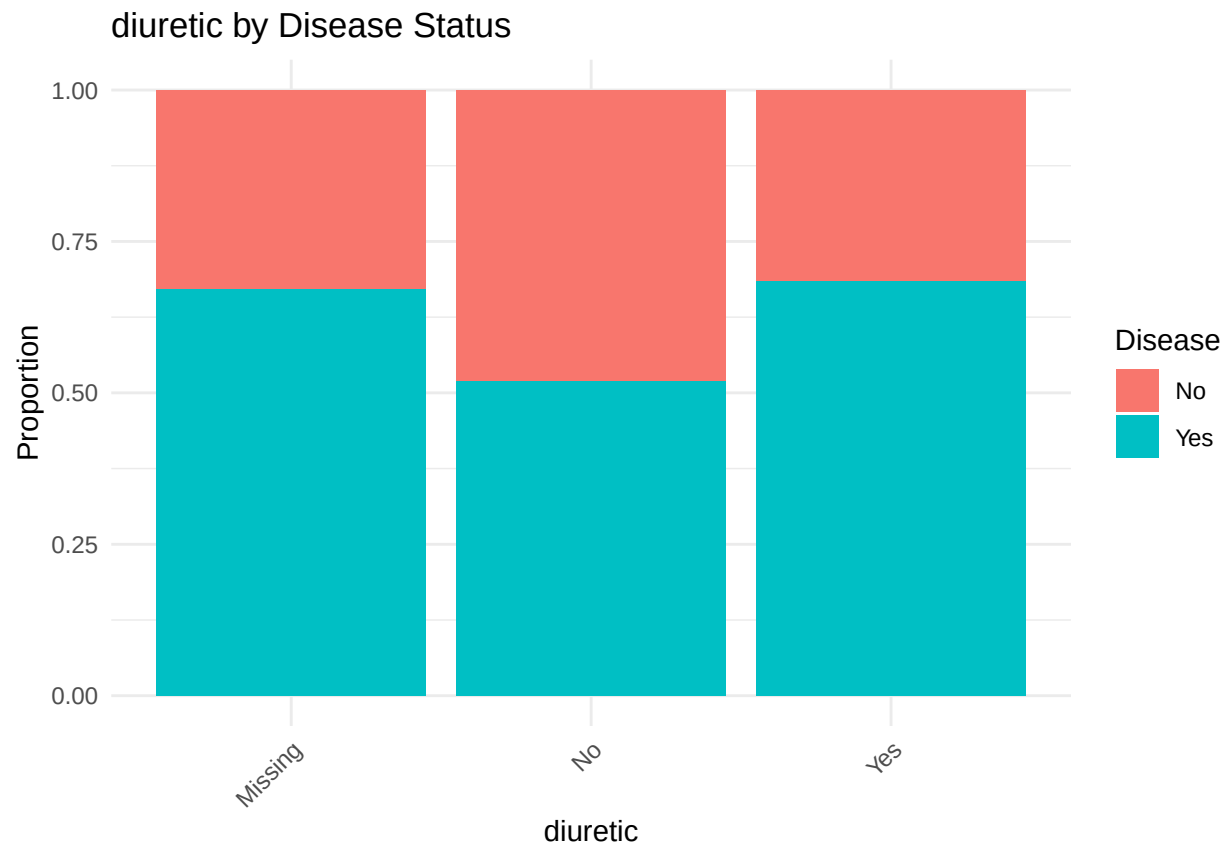


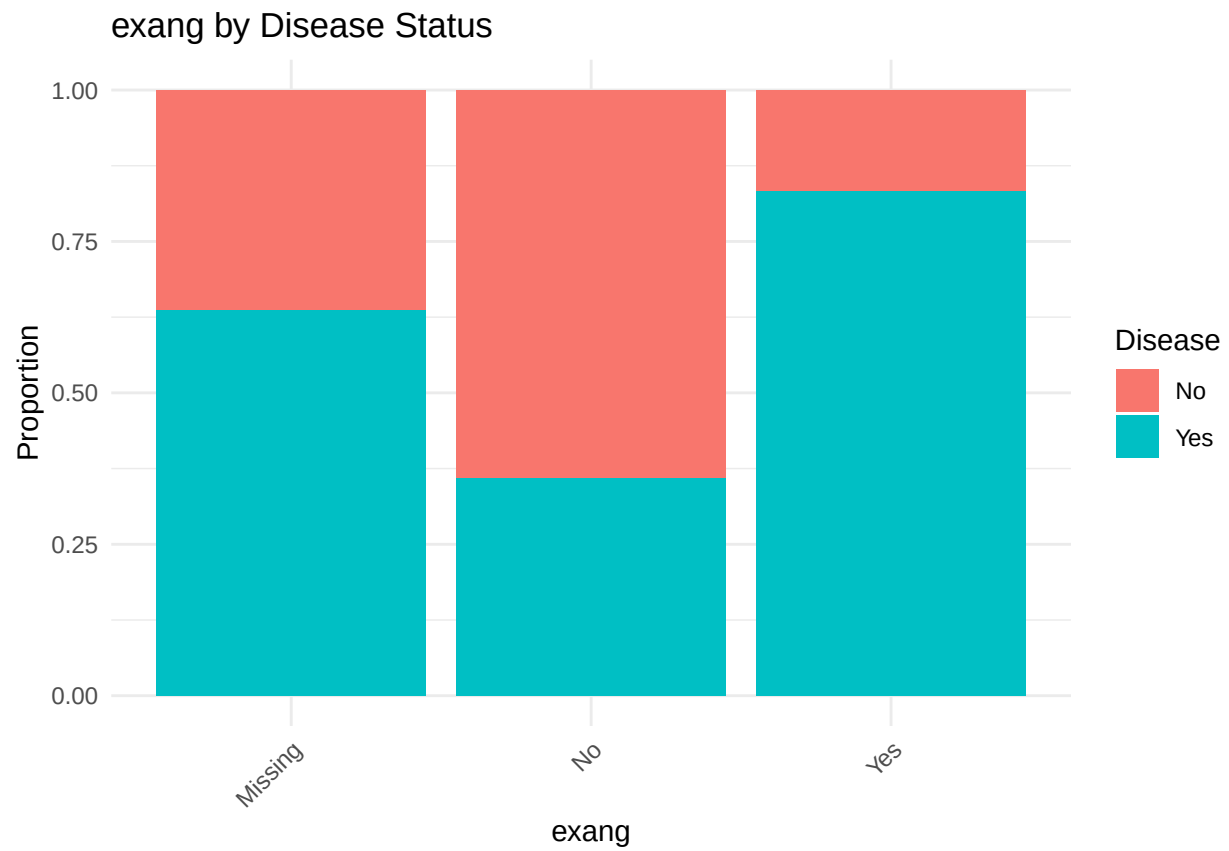


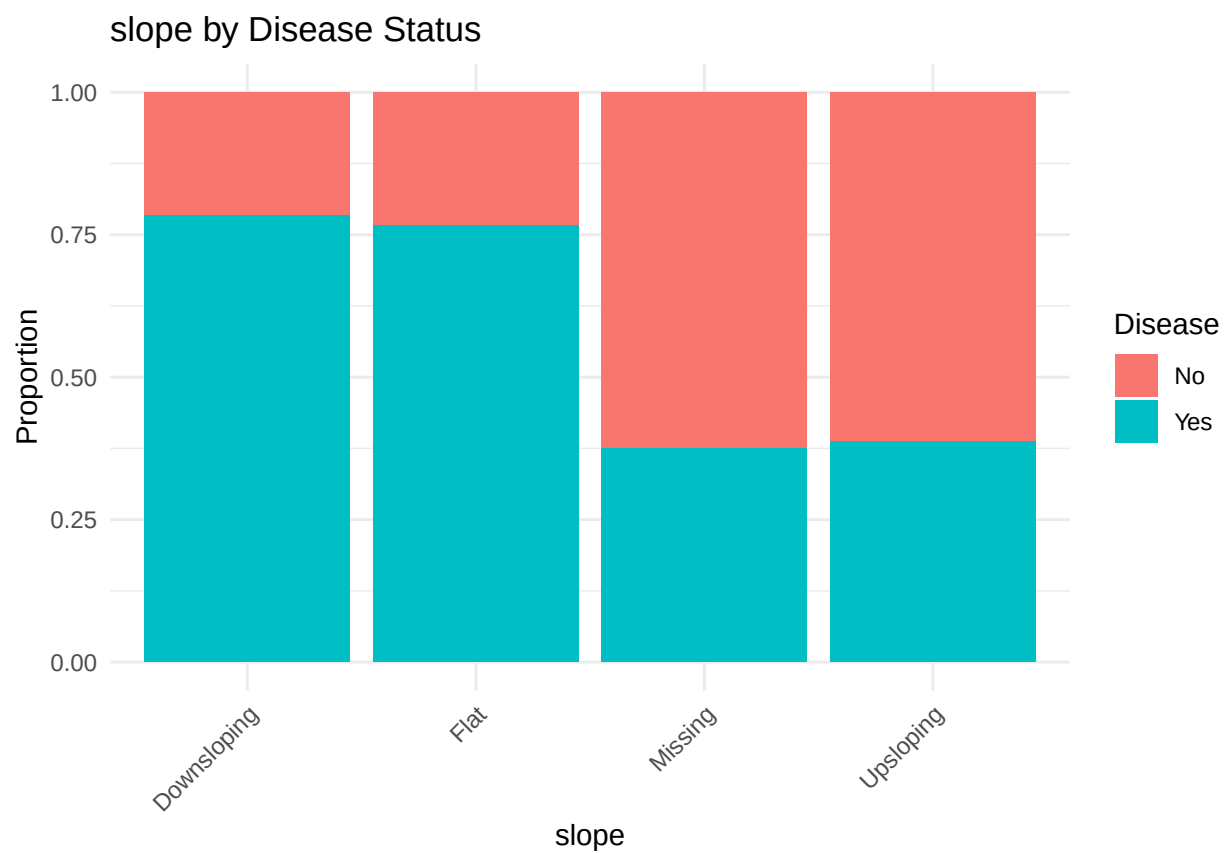


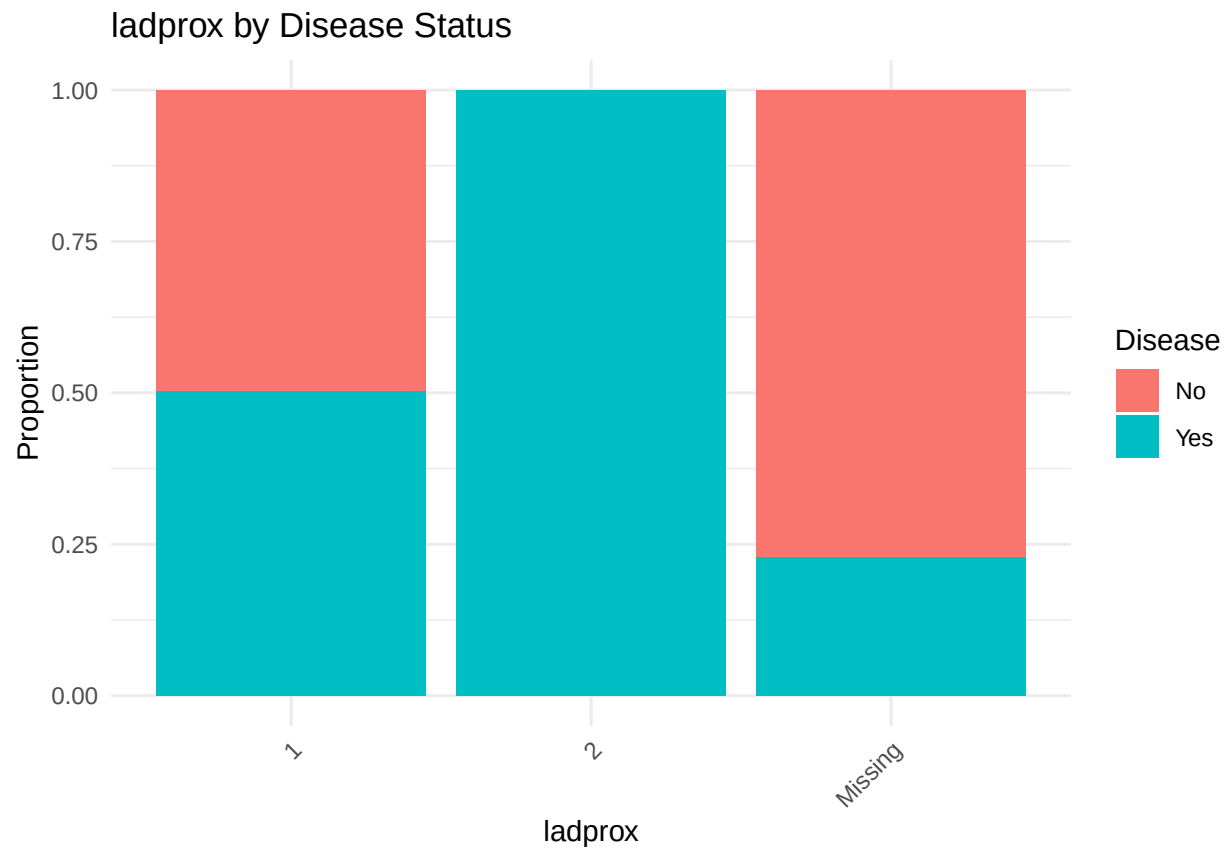


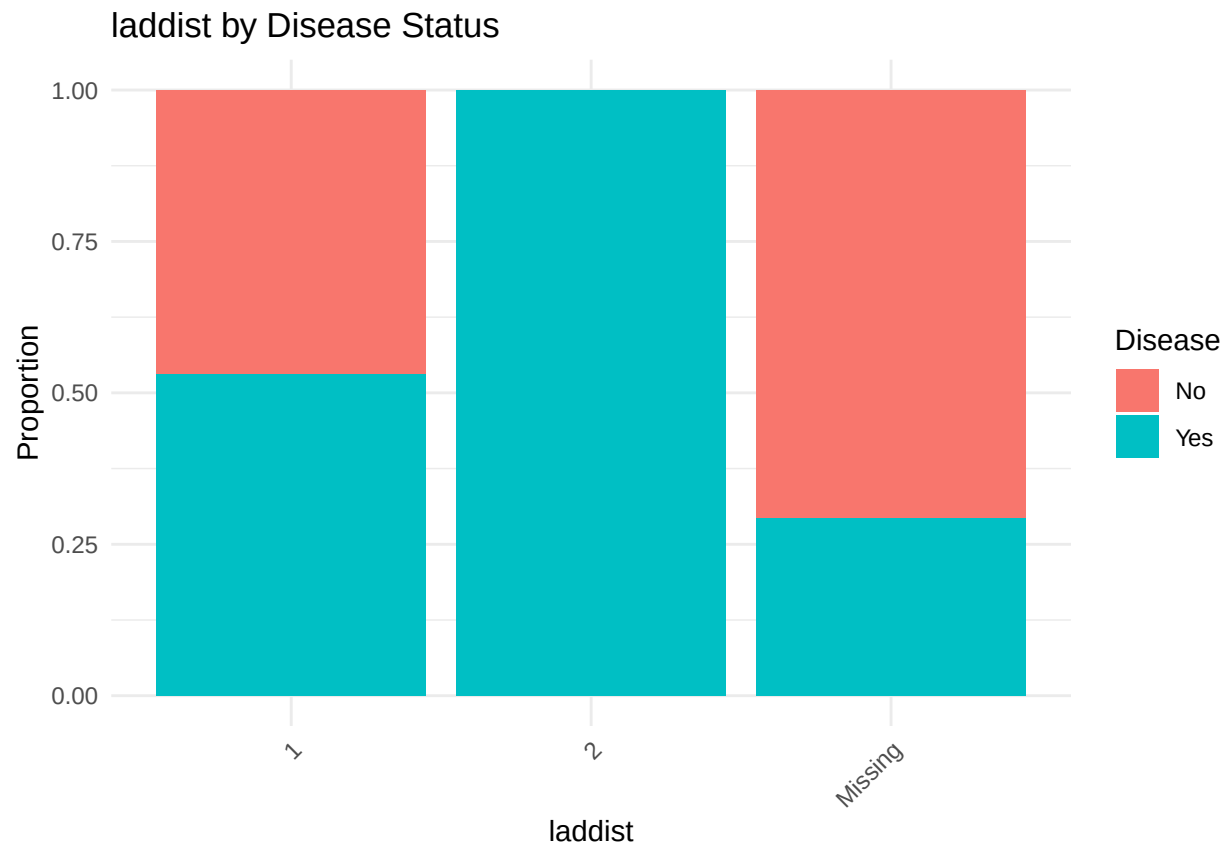


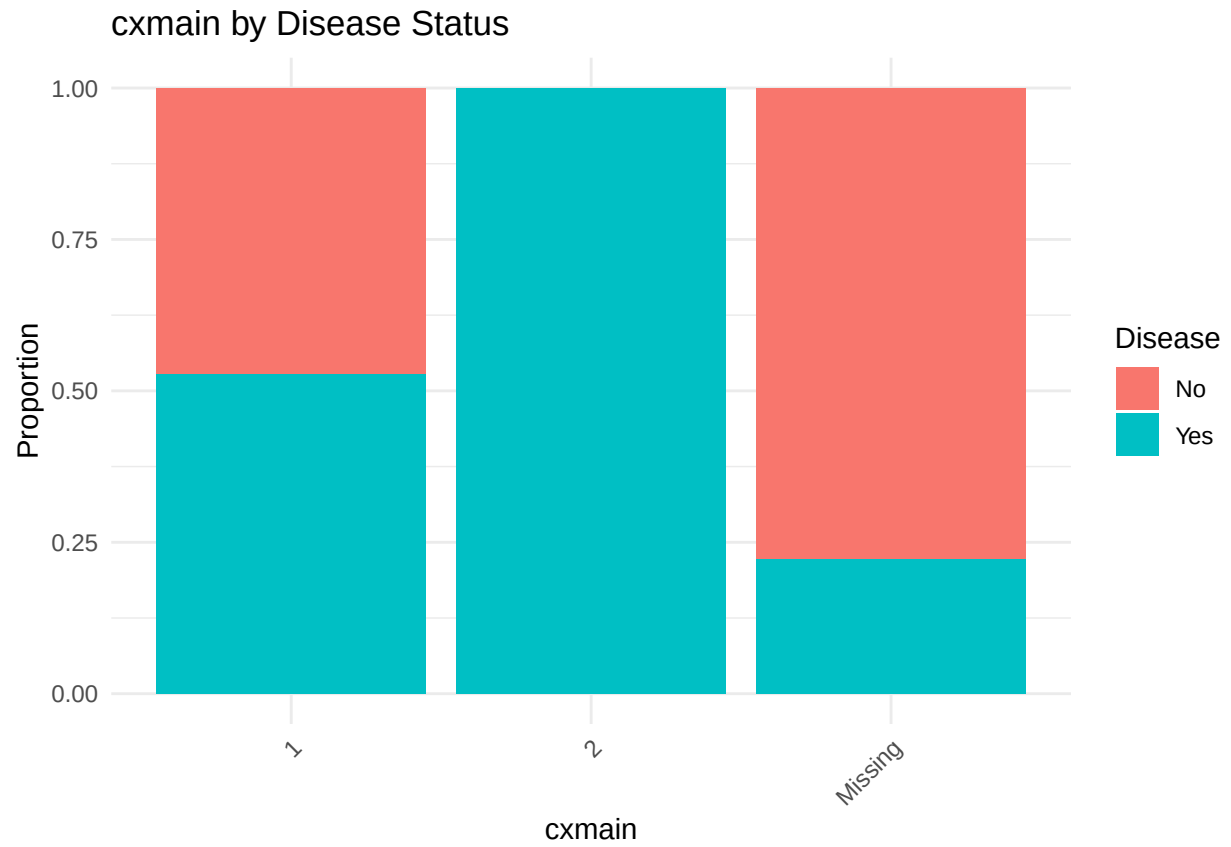


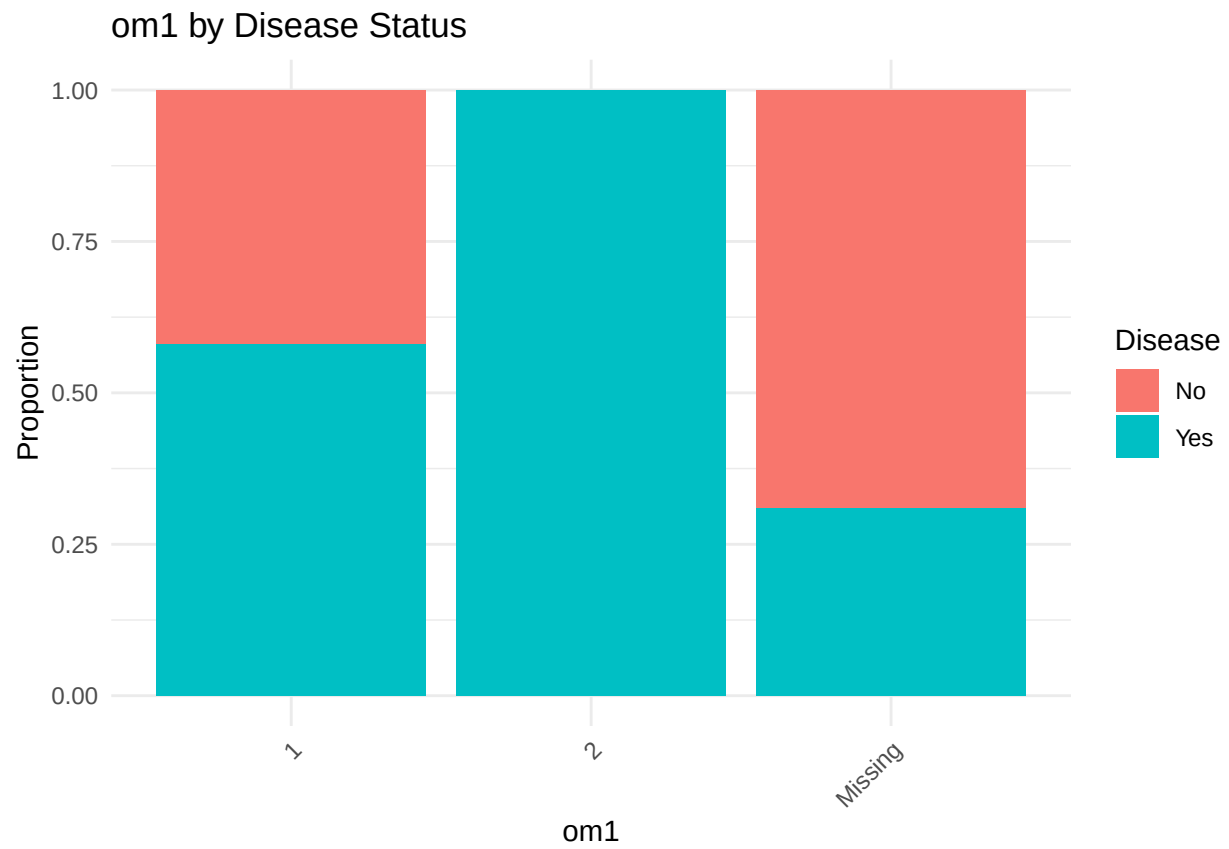


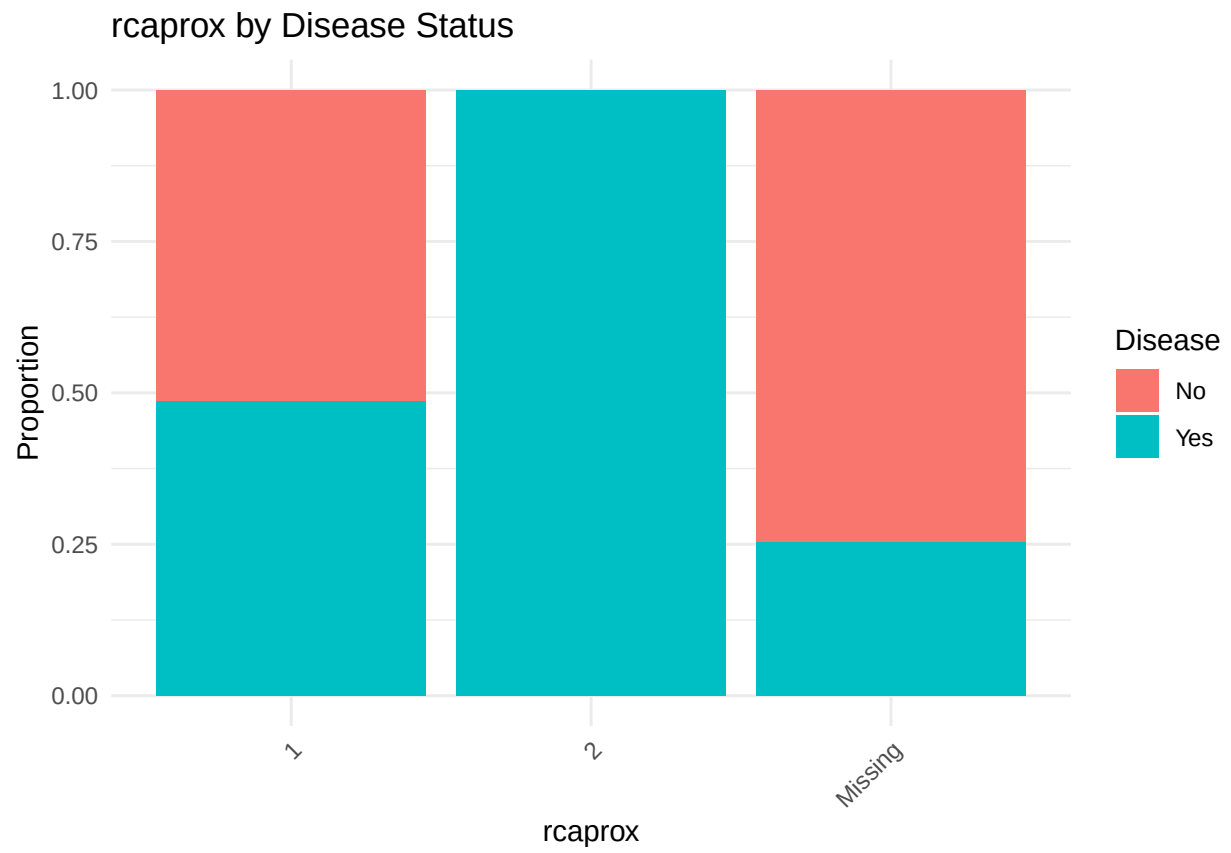


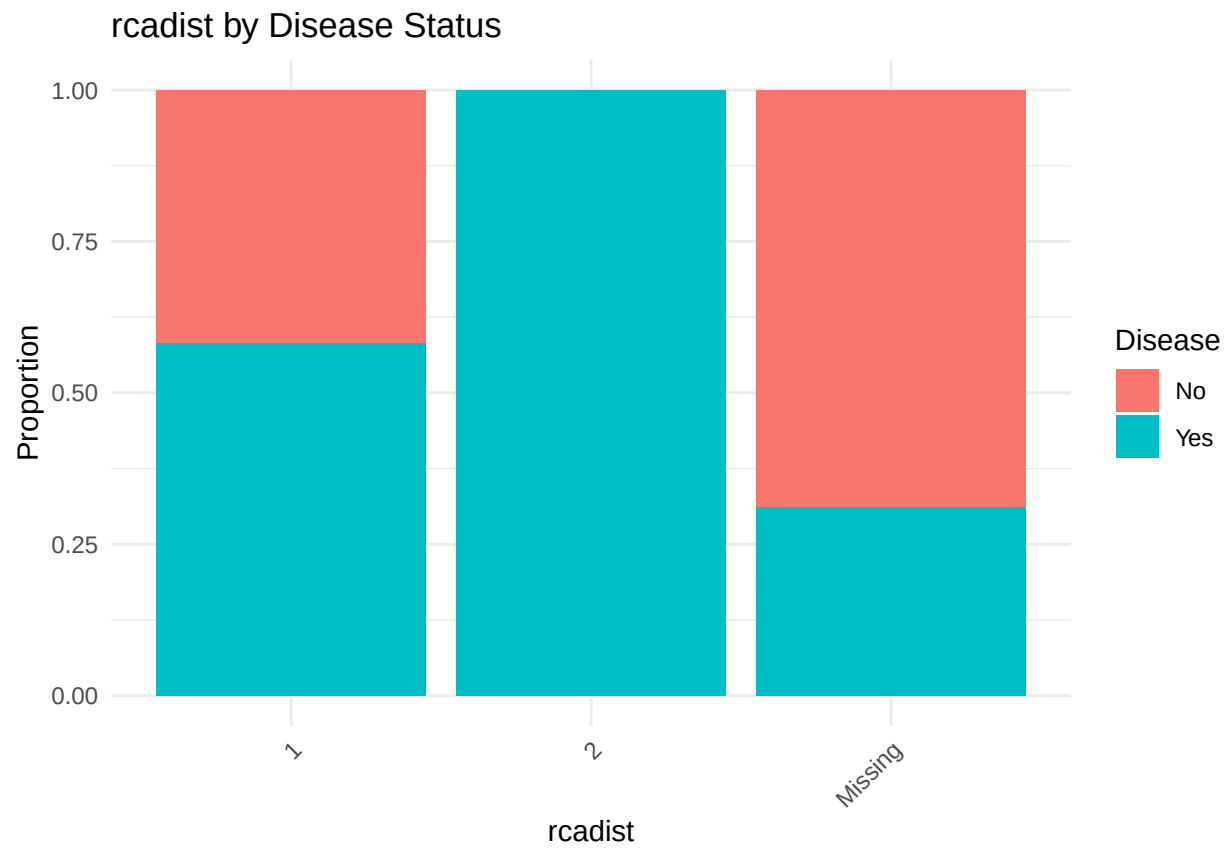


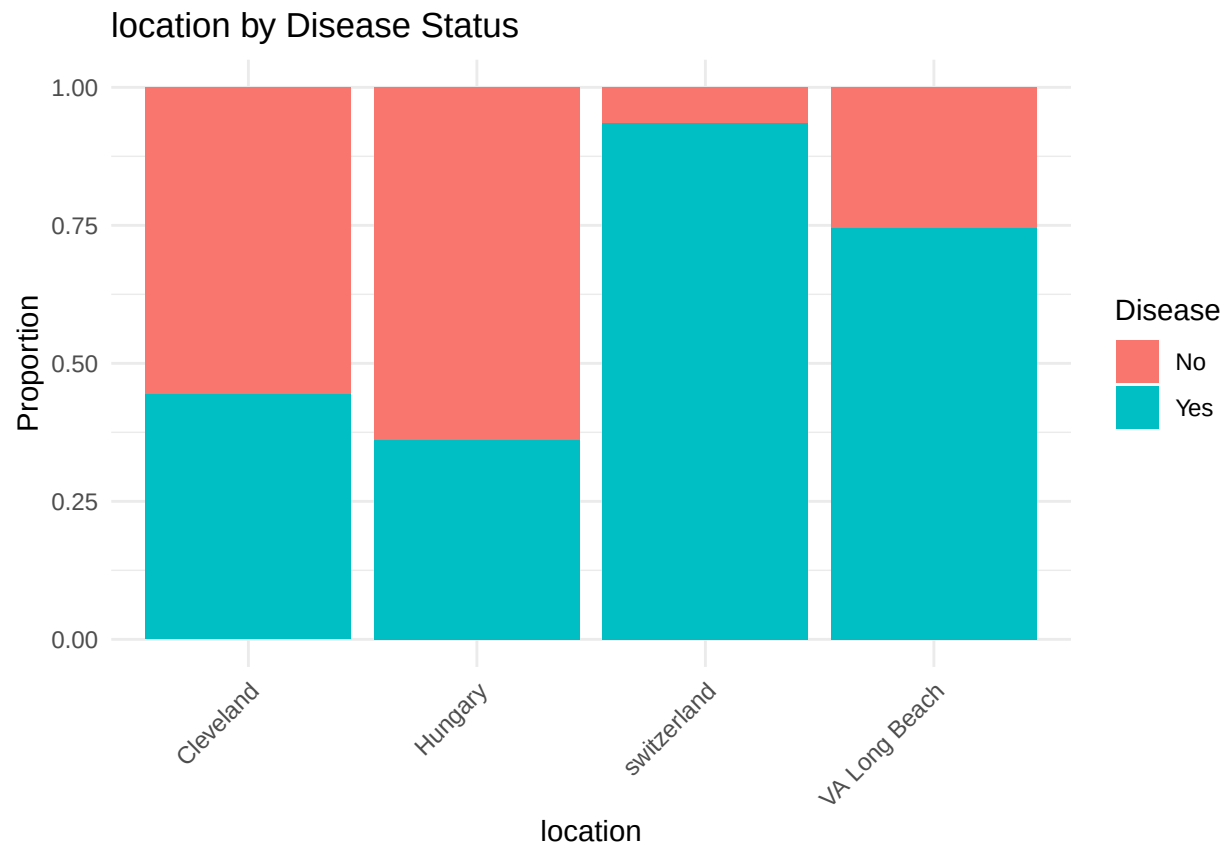


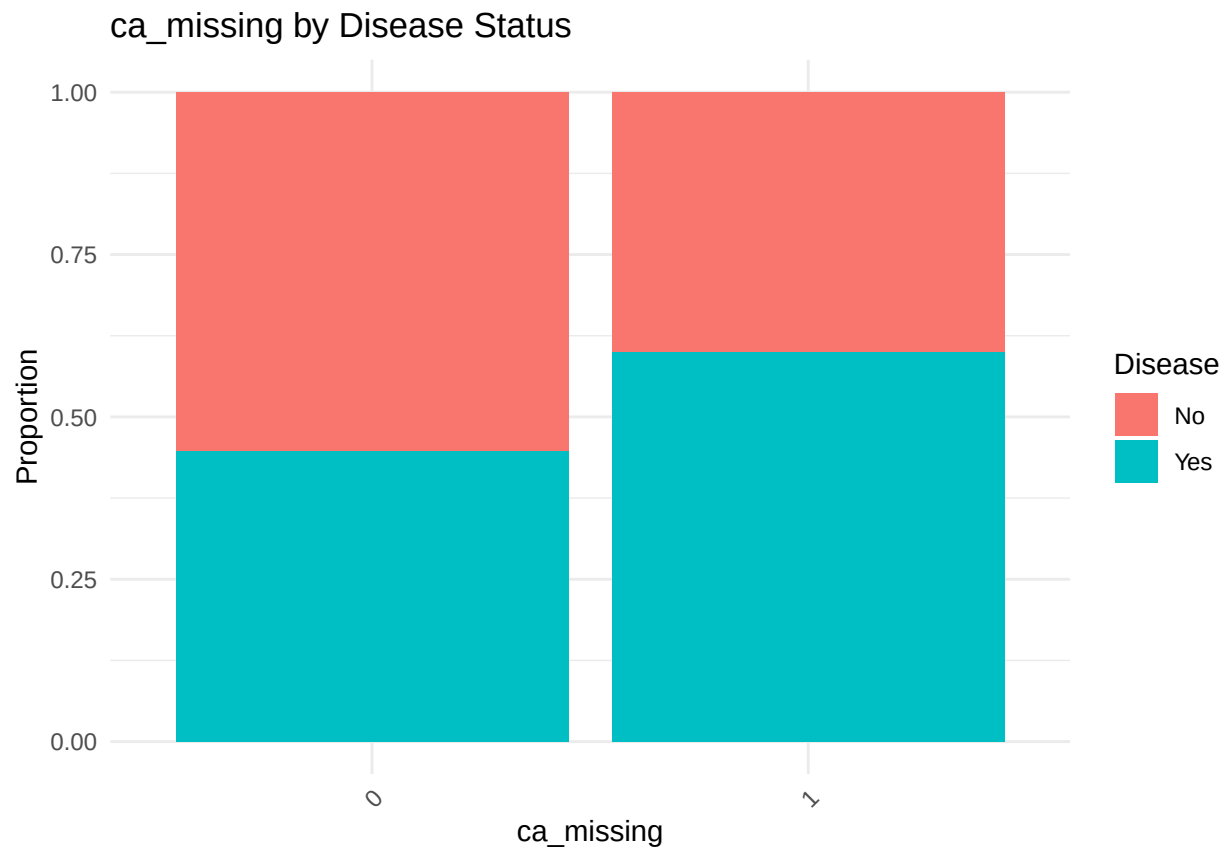


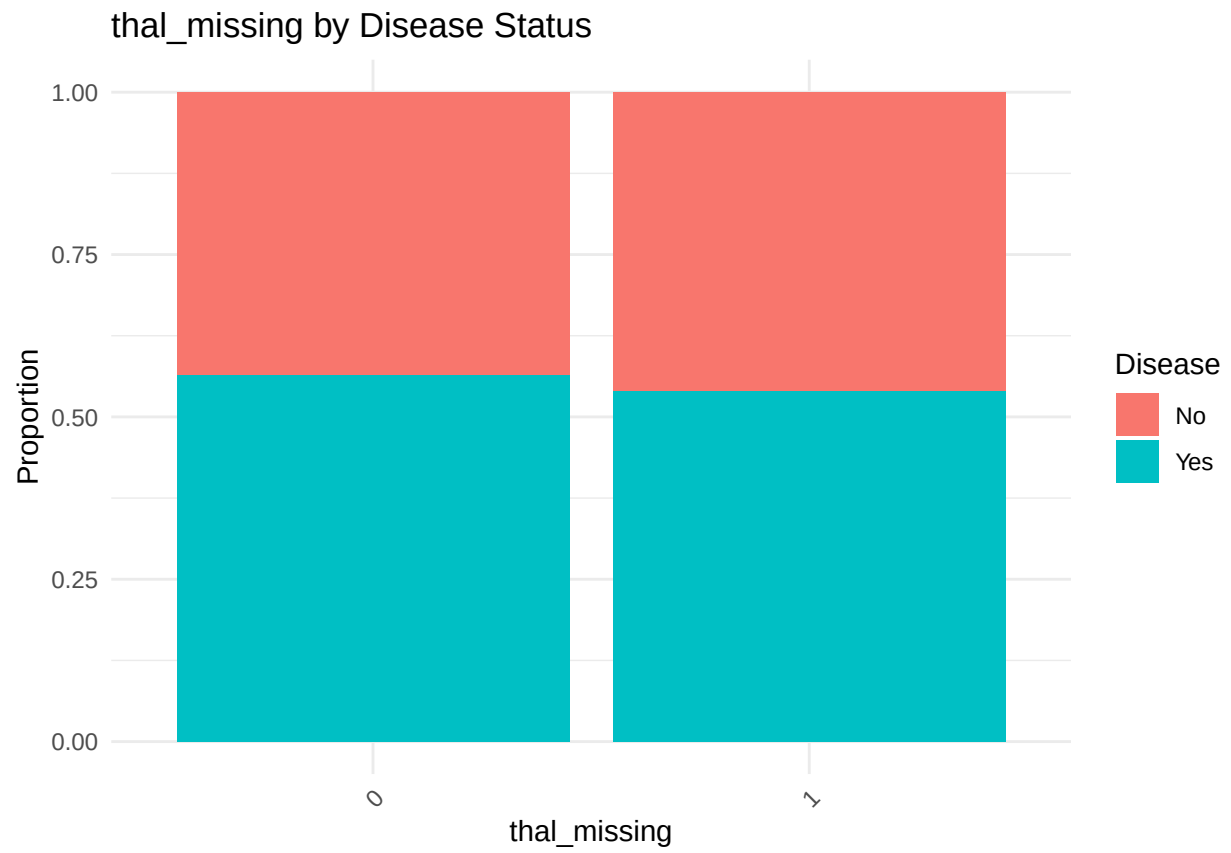




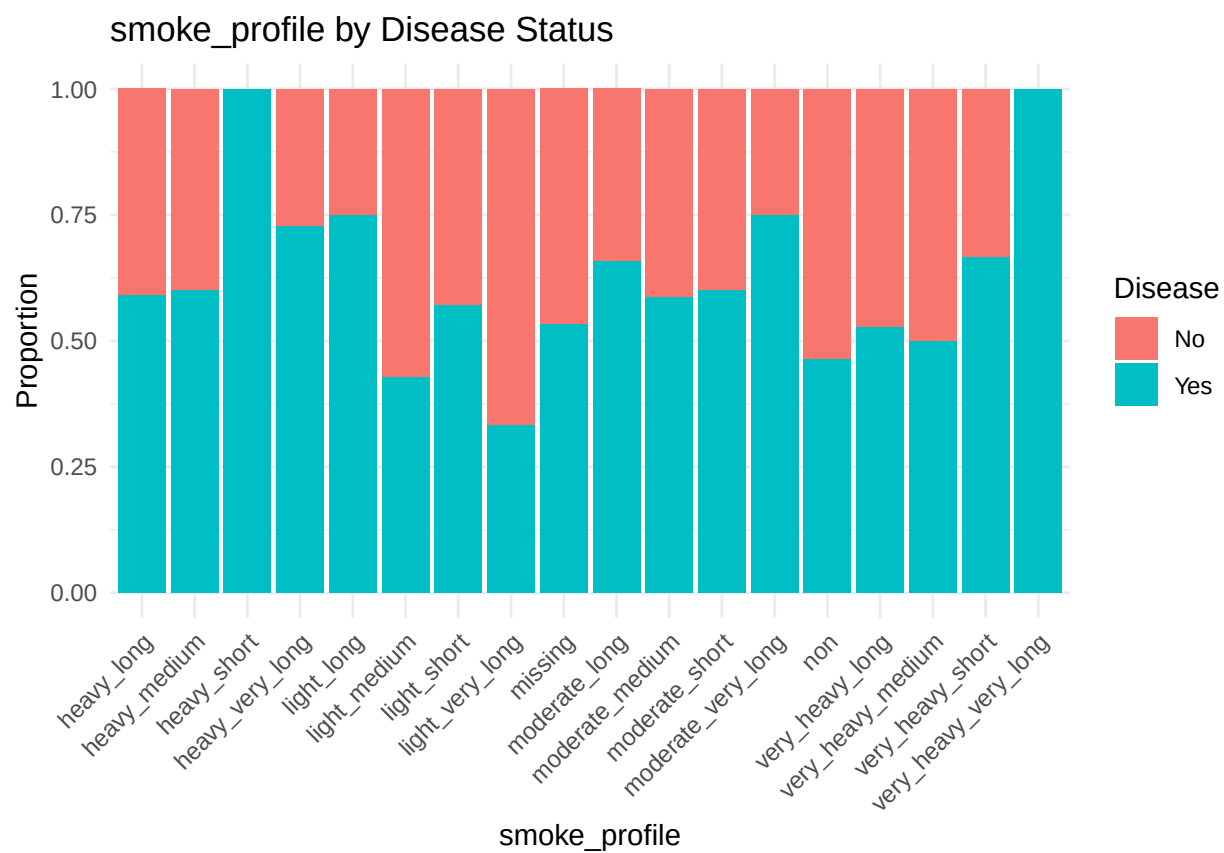


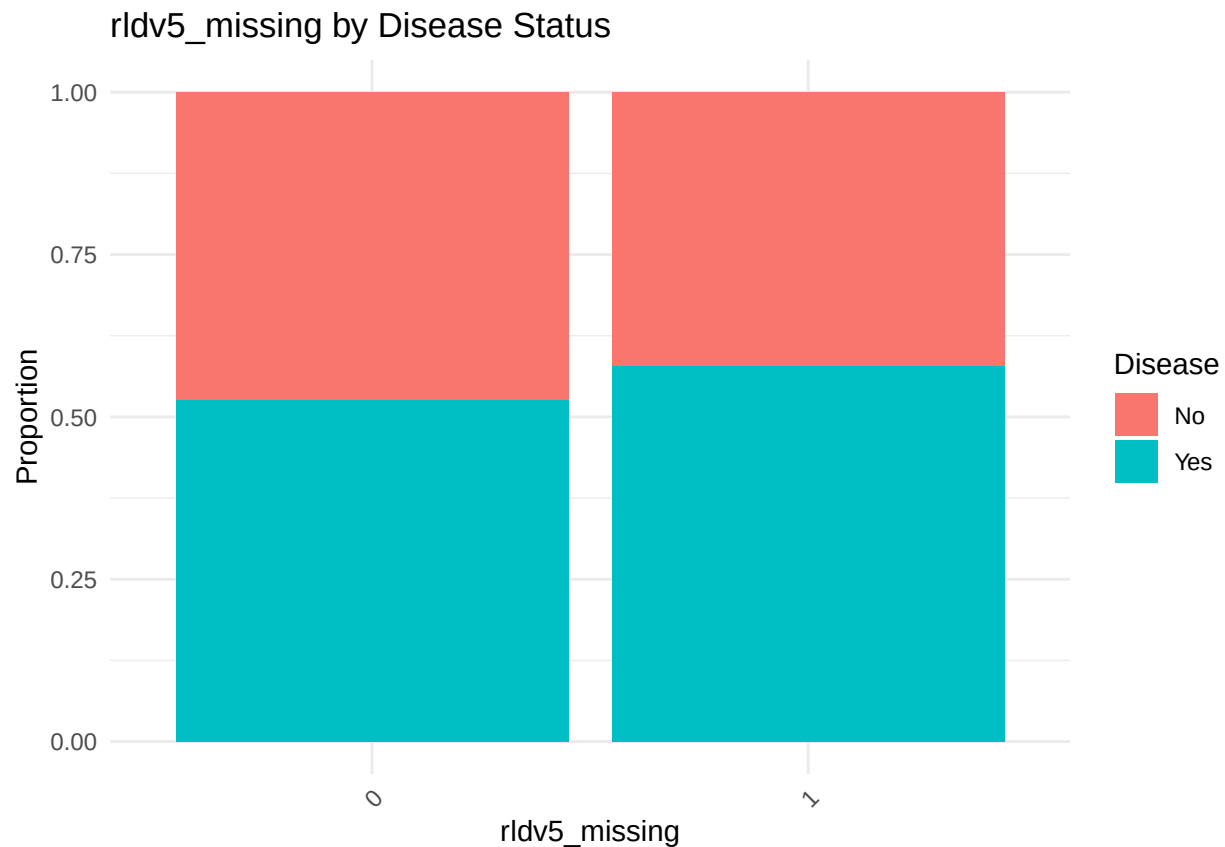












The following groups of variables have similar distributions which means they are possibly correlated:

Group 1: “prop”, “nitr”, “pro”, “diuretic”, “exang” Group 2: “ladprox”, “laddist”, “cxmain”, “om1”, “rcaprox”, “readist”

```
cat_vars_to_keep <- c("sex", "painexer", "relrest", "cp", "htn", "fbs", "restecg", "exang", "slope", "l
```

```
# cramer v
cramerv <- function(dataframe, var_list){
  cramerV_matrix <- matrix(NA, nrow = length(var_list), ncol = length(var_list),
    dimnames = list(var_list, var_list))

  for (i in seq_along(var_list)) {
    for (j in seq_along(var_list)) {
      tbl <- table(dataframe[[var_list[i]]], dataframe[[var_list[j]]])
      cramerV_matrix[i, j] <- suppressWarnings(cramerV(tbl))
    }
  }
  print(cramerV_matrix)
}

# Check correlation of variables that to have similar distribution
cramerv(df, c("prop", "nitr", "pro", "diuretic", "exang"))
```

Cramer’s V association test

```
##          prop   nitr   pro diuretic  exang
```



```
## prop      1.0000 0.7140 0.7072    0.6344 0.6277
## nitr      0.7140 1.0000 0.7479    0.6319 0.6450
## pro       0.7072 0.7479 1.0000    0.6259 0.6482
## diuretic  0.6344 0.6319 0.6259    1.0000 0.5616
## exang     0.6277 0.6450 0.6482    0.5616 1.0000

cramerv(df, c("ladprox", "laddist", "cxmain", "om1", "rcaprox", "rcadist"))

##          ladprox laddist cxmain    om1 rcaprox rcadist
## ladprox  1.0000  0.5490 0.5943 0.6370  0.6245  0.6277
## laddist  0.5490  1.0000 0.5914 0.6084  0.5994  0.6215
## cxmain   0.5943  0.5914 1.0000 0.6150  0.6324  0.6126
## om1      0.6370  0.6084 0.6150 1.0000  0.6385  0.6851
## rcaprox  0.6245  0.5994 0.6324 0.6385  1.0000  0.6128
## rcadist  0.6277  0.6215 0.6126 0.6851  0.6128  1.0000

drop_colinearity_cols <- c("prop", "nitr", "pro", "diuretic", "laddist",
                           "cxmain", "om1", "rcaprox", "rcadist")
```

Based on the results, each set of variables are moderate to highly correlated. To reduce multicollinearity, the following variables will be kept due to clinical significance: * **exang** - exercise-induced angina * **ladprox** - proximal LAD artery

Summary for categorical variables

```
# recommendation code generated with the help of ChatGPT.
cat_summary <- data.frame(
  Variable = character(),
  p_value = numeric(),
  Recommendation = character(),
  stringsAsFactors = FALSE
)

for (var in cat_vars) {
  tbl <- table(df[[var]], df$has_disease)
  p_val <- tryCatch(chisq.test(tbl)$p.value, error = function(e) NA)

  rec <- if (!is.na(p_val) && p_val < 0.05) {
    "Include (predictive)"
  } else {
    "Optional (evaluate with model)"
  }

  cat_summary <- rbind(
    cat_summary,
    data.frame(Variable = var, p_value = round(p_val, 5), Recommendation = rec)
  )
}

## Warning in chisq.test(tbl): Chi-squared approximation may be incorrect
## Warning in chisq.test(tbl): Chi-squared approximation may be incorrect

cat_summary
```

```
##          Variable p_value          Recommendation
## 1             sex 0.00000      Include (predictive)
```

```
## 2      painloc 0.00001      Include (predictive)
## 3      painexer 0.00000      Include (predictive)
## 4      relrest 0.00000      Include (predictive)
## 5      cp 0.00000      Include (predictive)
## 6      htn 0.00051      Include (predictive)
## 7      fbs 0.00000      Include (predictive)
## 8      famhist 0.61273 Optional (evaluate with model)
## 9      restecg 0.00567      Include (predictive)
## 10     prop 0.00000      Include (predictive)
## 11     nitr 0.00000      Include (predictive)
## 12     pro 0.00000      Include (predictive)
## 13     diuretic 0.00082      Include (predictive)
## 14     exang 0.00000      Include (predictive)
## 15     slope 0.00000      Include (predictive)
## 16     ladprox 0.00000      Include (predictive)
## 17     laddist 0.00000      Include (predictive)
## 18     cxmain 0.00000      Include (predictive)
## 19     om1 0.00000      Include (predictive)
## 20     rcaprox 0.00000      Include (predictive)
## 21     rcadist 0.00000      Include (predictive)
## 22     location 0.00000      Include (predictive)
## 23     ca_missing 0.00002      Include (predictive)
## 24     thal_missing 0.49284 Optional (evaluate with model)
## 25     thaltime_missing 0.00000      Include (predictive)
## 26     smoke_profile 0.22461 Optional (evaluate with model)
## 27     rldv5_missing 0.12282 Optional (evaluate with model)
```

```
# create list of all variables recommended as Include
```

```
selected_cat_vars <- cat_summary %>%
  filter(str_detect(Recommendation, "Include")) %>%
  pull(Variable)
```

```
selected_cat_vars
```

```
## [1] "sex"      "painloc"  "painexer" "relrest"
## [5] "cp"       "htn"      "fbs"       "restecg"
## [9] "prop"     "nitr"     "pro"       "diuretic"
## [13] "exang"    "slope"    "ladprox"   "laddist"
## [17] "cxmain"   "om1"      "rcaprox"   "rcadist"
## [21] "location" "ca_missing" "thaltime_missing"
```

Final Variable Sets

```
# combine selected variables
```

```
included_vars <- c(selected_num_vars, selected_cat_vars)
```

```
model_df <- df %>%
  select(all_of(included_vars), has_disease) %>%
  select(-all_of(drop_colinearity_cols))

str(model_df)
```

```
## tibble [899 x 39] (S3: tbl_df/tbl/data.frame)
## $ age      : num [1:899] 63 67 67 37 41 56 62 57 63 53 ...
## $ ca_imputed : num [1:899] 0 3 2 0 0 0 2 0 1 0 ...
```

```
## $ thal_imputed : num [1:899] 6 3 7 3 3 3 3 3 7 7 ...
## $ trestbps_missing: num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ trestbps_imputed: num [1:899] 145 160 120 130 130 120 140 120 130 140 ...
## $ chol_missing : num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ chol_imputed : num [1:899] 233 286 229 250 204 236 268 354 254 203 ...
## $ thaldur_missing : num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ thaldur_imputed : num [1:899] 10.5 9.5 8.5 13 7 11.3 6 9 8 5.5 ...
## $ met_missing : num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ met_imputed : num [1:899] 13 13 10 17 9 16 7 10 9 7 ...
## $ thalach_missing : num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ thalach_imputed : num [1:899] 150 108 129 187 172 178 160 163 147 155 ...
## $ thalrest_missing: num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ thalrest_imputed: num [1:899] 60 64 78 84 71 73 83 84 75 86 ...
## $ tpeakbps_missing: num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ tpeakbps_imputed: num [1:899] 190 160 140 195 160 165 180 165 120 185 ...
## $ tpeakbpd_missing: num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ trestbpd_missing: num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ oldpeak_missing : num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ oldpeak_imputed : num [1:899] 2.3 1.5 2.6 3.5 1.4 0.8 3.6 0.6 1.4 3.1 ...
## $ rldv5e_missing : num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ rldv5e_imputed : num [1:899] 172 185 150 167 40 127 122 122 75 68 ...
## $ lmt_missing : num [1:899] 0 0 0 0 0 0 0 0 0 0 ...
## $ sex : Factor w/ 2 levels "Female","Male": 2 2 2 2 1 2 1 1 2 2 ...
## $ painloc : Factor w/ 3 levels "Missing","Otherwise",...: 1 1 1 1 1 1 1 1 1 1 ...
## $ painexer : Factor w/ 3 levels "Missing","Otherwise",...: 1 1 1 1 1 1 1 1 1 1 ...
## $ relrest : Factor w/ 3 levels "Missing","Otherwise",...: 1 1 1 1 1 1 1 1 1 1 ...
## $ cp : Ord.factor w/ 4 levels "Typical Angina"<...: 1 4 4 3 2 2 4 4 4 4 ...
## $ htn : Factor w/ 3 levels "0","1","Missing": 2 2 2 1 2 2 1 2 2 1 ...
## $ fbs : Factor w/ 3 levels "Missing","No",...: 3 2 2 2 2 2 2 2 2 3 ...
## $ restecg : Factor w/ 4 levels "Abnormal","Hypertrophy",...: 2 2 2 4 2 4 2 4 2 2 ...
## $ exang : Factor w/ 3 levels "Missing","No",...: 2 3 3 2 2 2 2 3 2 3 ...
## $ slope : Factor w/ 4 levels "Downsloping",...: 1 2 2 1 4 4 1 4 2 1 ...
## $ ladprox : Factor w/ 3 levels "1","2","Missing": 1 2 1 1 1 1 2 1 2 1 ...
## $ location : chr [1:899] "Cleveland" "Cleveland" "Cleveland" "Cleveland" ...
## $ ca_missing : Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 1 1 1 ...
## $ thaltime_missing: Factor w/ 2 levels "0","1": 1 1 1 1 2 2 1 1 1 1 ...
## $ has_disease : Factor w/ 2 levels "No","Yes": 1 2 2 1 1 1 2 1 2 2 ...
```

Export dataset for modeling

```
# export modeling dataset to RDS file
saveRDS(model_df, here("Data/Clean", "heart_disease_modeling.rds"))

model_df %>% dim()
```

```
## [1] 899 39
```

The final modeling set has 899 observations and 39 features including the target variable.

3_modeling

2025-06-13

Modeling

```
knitr::opts_chunk$set(echo = TRUE)
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'caret'
```

```
##
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## lift
```

```
library(here)
```

```
## here() starts at /home/rstudio/Documents/ads503_project_g1
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
##
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## cov, smooth, var
```

```
library(pls)
```

```
##
```

```
## Attaching package: 'pls'
```

```
##
```

```
## The following object is masked from 'package:caret':
```

```
##
##      R2
##
## The following object is masked from 'package:stats':
##
##      loadings
library(gt)
```

Read and Prepare Data

```
# Read in modeling dataset
df_model <- readRDS(here("Data/Clean", "heart_disease_modeling.rds"))
# Ensure factor levels for binary outcome
df_model$has_disease <- factor(df_model$has_disease, levels = c("No", "Yes"))

# Set seed for reproducibility
seed <- 13579
set.seed(seed)
```

Train/Test Split

```
train_index <- createDataPartition(df_model$has_disease, p = 0.8, list = FALSE)
train_data  <- df_model[train_index, ]
test_data   <- df_model[-train_index, ]
```

Global Model Settings

```
# Performance metric and resampling
global_metric <- "ROC"
ctrl <- trainControl(
  method      = "cv",
  number      = 5,
  classProbs  = TRUE,
  summaryFunction = twoClassSummary,
  verboseIter = FALSE
)
# Preprocessing strategy
process_strat <- c("center", "scale")

# Tuning grids
#knn
K_grid <- data.frame(k = seq(1, 25,
                             by = 2))

#plsda
pls_grid <- expand.grid(ncomp = 1:10)
#penalized
#lasso
lasso_grid <- expand.grid(alpha = 1,
                          lambda = 10^seq(-4, 0, length = 50))

#ridge
ridge_grid <- expand.grid(alpha = 0,
                          lambda = 10^seq(-4, 0, length = 50))
```

```
#elasticnet
enet_grid <- expand.grid(alpha = seq(0.1, 0.9, by = 0.1),
                        lambda = 10^seq(-4, 0, length = 50))
```

Initialize Results Table

```
#empty results table of performance metrics
results_tbl <- tibble(
  Model      = character(),
  Accuracy   = numeric(),
  Sensitivity = numeric(),
  Specificity = numeric(),
  ROC_AUC    = numeric()
)
```

1. Logistic Regression

```
#set seed
set.seed(seed)
#train logistic model
glm_model <- train(
  has_disease ~ .,
  data      = train_data,
  method    = "glm",
  family    = "binomial",
  metric    = global_metric,
  trControl = ctrl,
  preProcess = process_strat
)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
# Testing set performance
glm_preds <- predict(glm_model, test_data)
glm_probs <- predict(glm_model, test_data, type = "prob")[, "Yes"]
roc_glm   <- roc(test_data$has_disease, glm_probs)
```

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

```
glm_cm_test <- confusionMatrix(glm_preds, test_data$has_disease,
                              positive = "Yes")
```

```
# Training set performance
```

```

glm_train_preds <- predict(glm_model, train_data)
glm_train_probs <- predict(glm_model, train_data, type = "prob")[, "Yes"]
roc_glm_train <- roc(train_data$has_disease, glm_train_probs)

## Setting levels: control = No, case = Yes
## Setting direction: controls < cases

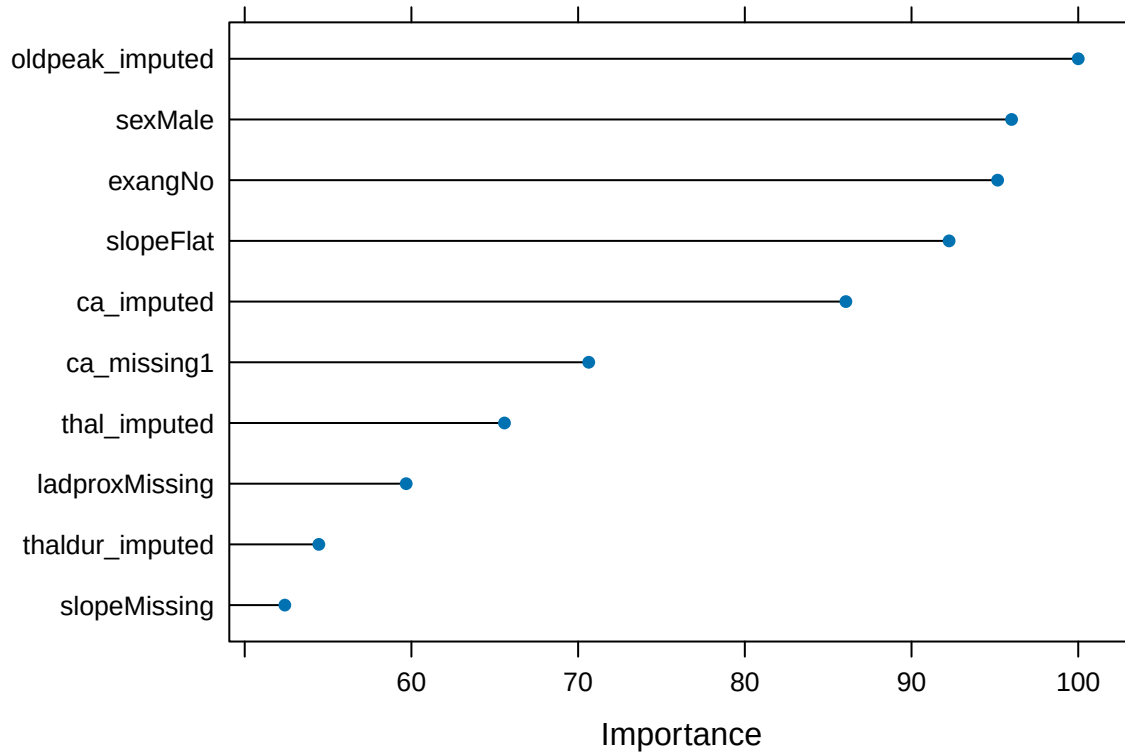
glm_cm_train <- confusionMatrix(glm_train_preds, train_data$has_disease,
                                positive = "Yes")

# Feature Importance
glm_imp <- varImp(glm_model)
# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model = "Logistic Regression",
    Accuracy = glm_cm_test$overall["Accuracy"],
    Sensitivity = glm_cm_test$byClass["Sensitivity"],
    Specificity = glm_cm_test$byClass["Specificity"],
    ROC_AUC = as.numeric(auc(roc_glm))
  )
# Display train/test AUC
cat("Logistic Regression - Train AUC:", as.numeric(auc(roc_glm_train)),
    "Test AUC:", as.numeric(auc(roc_glm)), "\n")

## Logistic Regression - Train AUC: 0.9592218 Test AUC: 0.8343434

# Plot importance
plot(glm_imp, top = 10)

```



2. Random Forest

```
#set seed
set.seed(seed)
#train model
rf_model <- train(
  has_disease ~ .,
  data      = train_data,
  method    = "rf",
  metric    = global_metric,
  trControl = ctrl,
  preProcess = process_strat,
  importance = TRUE
)

# Testing set performance
rf_preds    <- predict(rf_model, test_data)
rf_probs    <- predict(rf_model, test_data, type = "prob")[, "Yes"]
roc_rf      <- roc(test_data$has_disease, rf_probs)

## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
rf_cm_test  <- confusionMatrix(rf_preds, test_data$has_disease,
                               positive = "Yes")
```



```

# Train set performance
rf_train_preds <- predict(rf_model, train_data)
rf_train_probs <- predict(rf_model, train_data, type = "prob")[, "Yes"]
roc_rf_train   <- roc(train_data$has_disease, rf_train_probs)

## Setting levels: control = No, case = Yes
## Setting direction: controls < cases

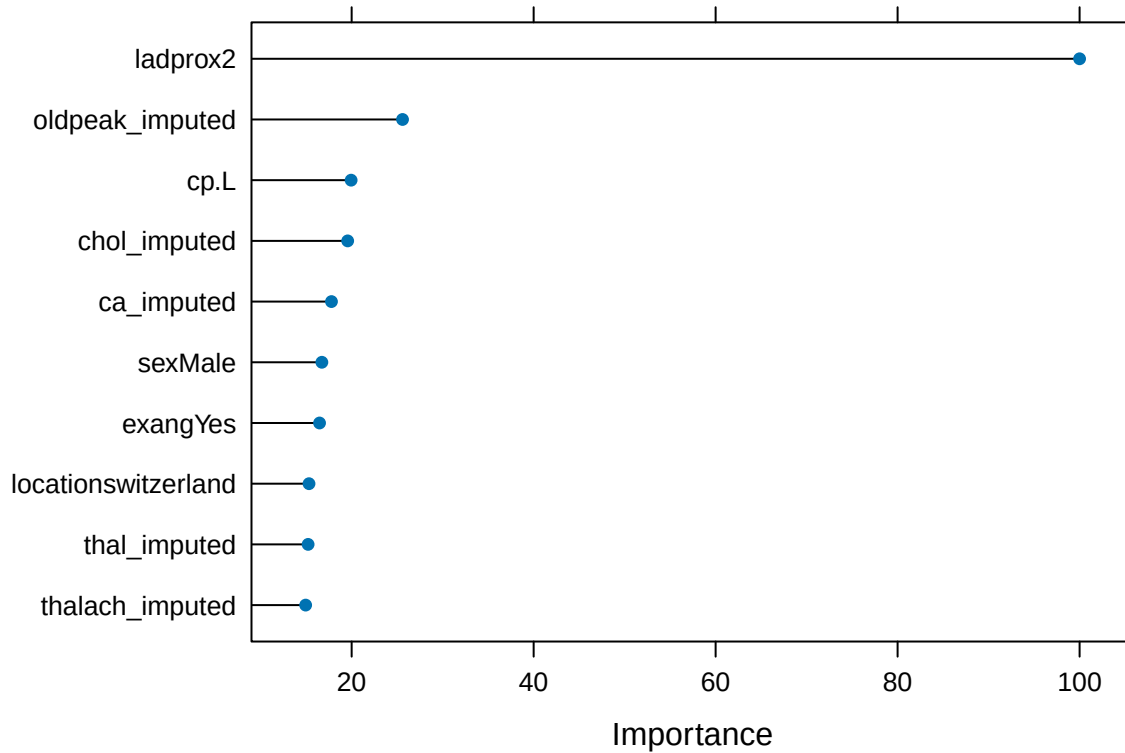
rf_cm_train    <- confusionMatrix(rf_train_preds, train_data$has_disease,
                                   positive = "Yes")

# Feature Importance
rf_imp <- varImp(rf_model)
# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model      = "Random Forest",
    Accuracy    = rf_cm_test$overall["Accuracy"],
    Sensitivity = rf_cm_test$byClass["Sensitivity"],
    Specificity = rf_cm_test$byClass["Specificity"],
    ROC_AUC     = as.numeric(auc(roc_rf))
  )
# Display train/test AUC
cat("Random Forest - Train AUC:", as.numeric(auc(roc_rf_train)),
    "Test AUC:", as.numeric(auc(roc_rf)), "\n")

## Random Forest - Train AUC: 1 Test AUC: 0.9066919

# Plot importance
plot(rf_imp, top = 10)

```



3. PLS Discriminant Analysis

```
#set seed
set.seed(seed)
#train model
pls_model <- train(
  has_disease ~ .,
  data       = train_data,
  method     = "pls",
  tuneGrid   = pls_grid,
  preProcess = process_strat,
  metric     = global_metric,
  trControl  = ctrl
)

# Testing set performance
pls_preds <- predict(pls_model, test_data)
pls_probs <- predict(pls_model, test_data, type = "prob")[, "Yes"]
roc_pls   <- roc(test_data$has_disease, pls_probs)

## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
pls_cm_test <- confusionMatrix(pls_preds, test_data$has_disease,
                               positive = "Yes")
```

```

# Training set performance
pls_train_preds <- predict(pls_model, train_data)
pls_train_probs <- predict(pls_model, train_data, type = "prob")[, "Yes"]
roc_pls_train   <- roc(train_data$has_disease, pls_train_probs)

## Setting levels: control = No, case = Yes
## Setting direction: controls < cases

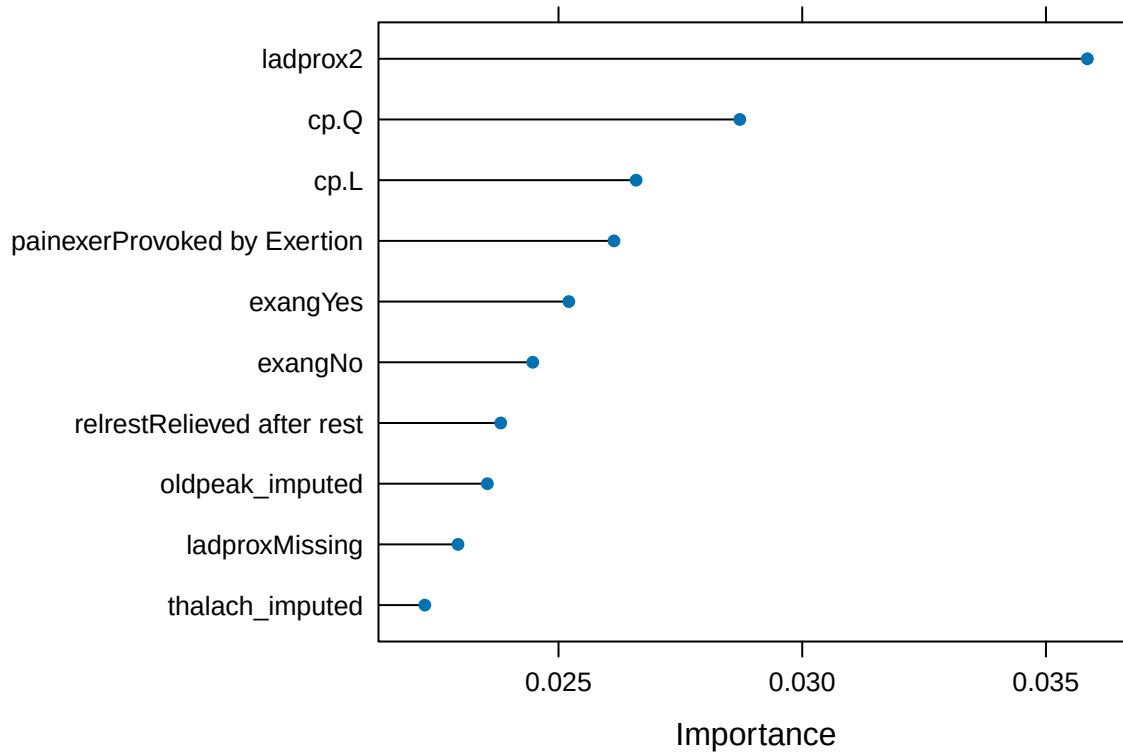
pls_cm_train    <- confusionMatrix(pls_train_preds, train_data$has_disease,
                                   positive = "Yes")

# Feature Importance
pls_imp <- varImp(pls_model, scale = FALSE)
# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model      = "PLS-DA",
    Accuracy   = pls_cm_test$overall["Accuracy"],
    Sensitivity = pls_cm_test$byClass["Sensitivity"],
    Specificity = pls_cm_test$byClass["Specificity"],
    ROC_AUC    = as.numeric(auc(roc_pls))
  )
# Display train/test AUC
cat("PLS-DA - Train AUC:", as.numeric(auc(roc_pls_train)),
    "Test AUC:", as.numeric(auc(roc_pls)), "\n")

## PLS-DA - Train AUC: 0.9441405 Test AUC: 0.9113636

# Plot importance
plot(pls_imp, top = 10)

```



4. K-Nearest Neighbors

```
# set the seed
set.seed(seed)
#build model
knn_model <- train(
  has_disease ~ .,
  data      = train_data,
  method    = "knn",
  tuneGrid  = K_grid,
  metric    = global_metric,
  trControl = ctrl,
  preProcess = process_strat
)

# Testing set performance
knn_preds  <- predict(knn_model, test_data)
knn_probs  <- predict(knn_model, test_data, type = "prob")[, "Yes"]
roc_knn    <- roc(test_data$has_disease, knn_probs)

## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
knn_cm_test <- confusionMatrix(knn_preds, test_data$has_disease,
                               positive = "Yes")
```

```

# Training set performance
knn_train_preds <- predict(knn_model, train_data)
knn_train_probs <- predict(knn_model, train_data, type = "prob")[, "Yes"]
roc_knn_train   <- roc(train_data$has_disease, knn_train_probs)

## Setting levels: control = No, case = Yes
## Setting direction: controls < cases

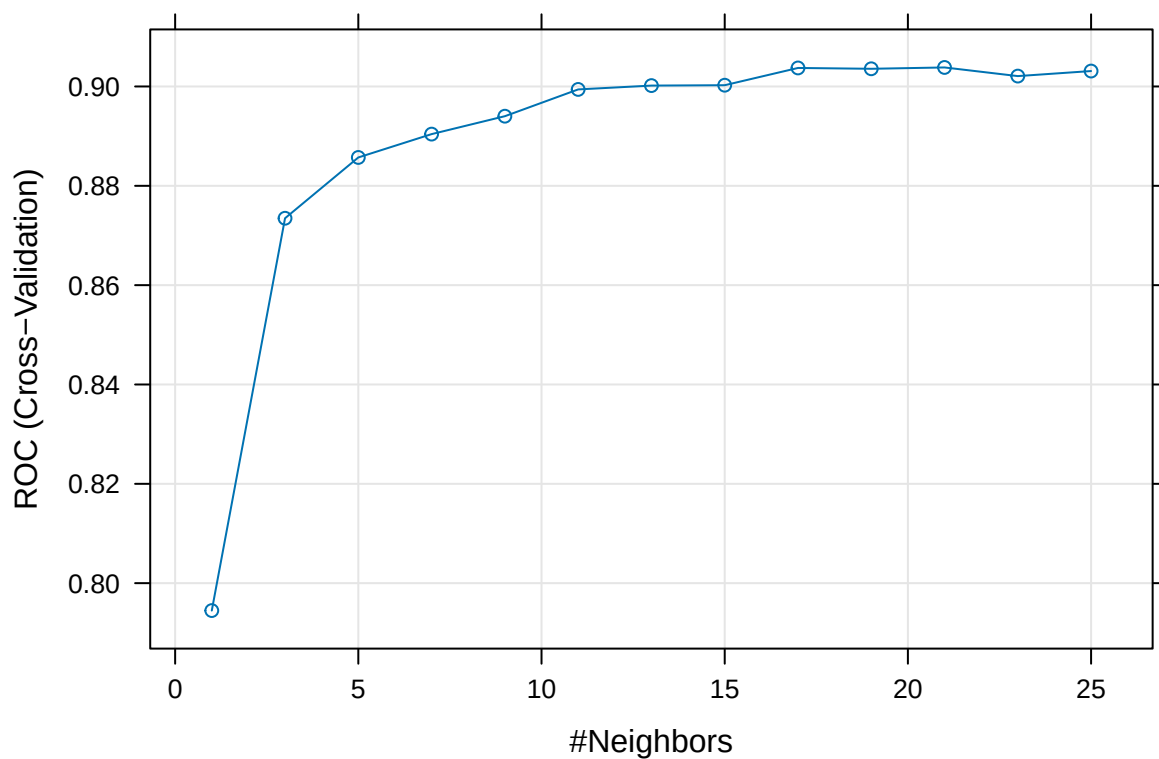
knn_cm_train    <- confusionMatrix(knn_train_preds, train_data$has_disease,
                                   positive = "Yes")

# Inspect KNN performance across K
cat("Best k:", knn_model$bestTune$k, "")

## Best k: 21

plot(knn_model, metric = "ROC")

```



```

# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model      = "K-Nearest Neighbors",
    Accuracy   = knn_cm_test$overall["Accuracy"],
    Sensitivity = knn_cm_test$byClass["Sensitivity"],
    Specificity = knn_cm_test$byClass["Specificity"],
    ROC_AUC    = as.numeric(auc(roc_knn))
  )
# Display train/test AUC

```

```
cat("KNN - Train AUC:", as.numeric(auc(roc_knn_train)),
    "Test AUC:", as.numeric(auc(roc_knn)), "\n")
```

```
## KNN - Train AUC: 0.9257623 Test AUC: 0.9016414
```

5. Penalized Regression

```
#set seed
set.seed(seed)
#train model
lasso_model <- train(
  has_disease ~ .,
  data      = train_data,
  method    = "glmnet",
  tuneGrid  = lasso_grid,
  metric    = global_metric,
  trControl = ctrl,
  preProcess = process_strat
)
# best tuning parameters
lasso_best <- lasso_model$bestTune
# Testing set performance
lasso_preds <- predict(lasso_model, test_data)
lasso_probs <- predict(lasso_model, test_data, type = "prob")[, "Yes"]
roc_lasso <- roc(test_data$has_disease, lasso_probs)
```

a. Lasso Penalization

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

```
lasso_cm <- confusionMatrix(lasso_preds, test_data$has_disease,
                             positive = "Yes")

# Training set performance
lasso_train_preds <- predict(lasso_model, train_data)
lasso_train_probs <- predict(lasso_model, train_data, type = "prob")[, "Yes"]
roc_lasso_train <- roc(train_data$has_disease, lasso_train_probs)
```

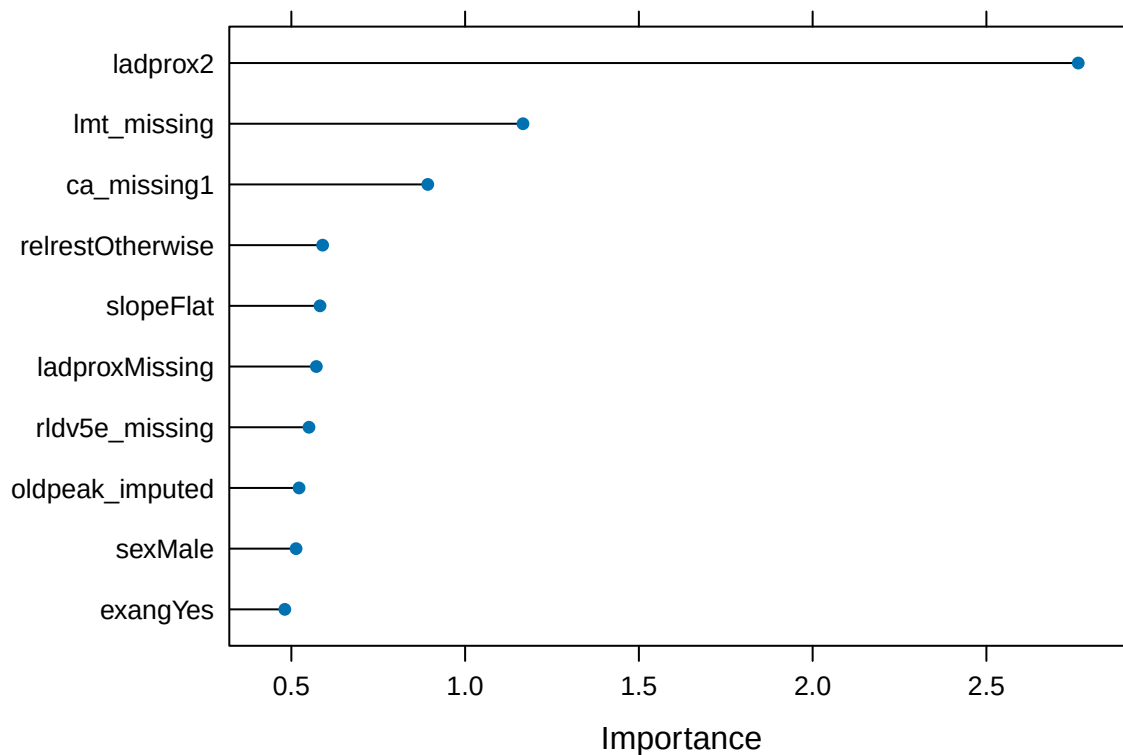
```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

```
lasso_cm_train <- confusionMatrix(lasso_train_preds, train_data$has_disease,
                                   positive = "Yes")

#lasso importance
lasso_imp <- varImp(lasso_model, scale = FALSE)
# append results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model      = "Lasso",
    Accuracy   = as.numeric(lasso_cm$overall["Accuracy"]),
    Sensitivity = as.numeric(lasso_cm$byClass["Sensitivity"]),
    Specificity = as.numeric(lasso_cm$byClass["Specificity"]),
    ROC_AUC    = as.numeric(auc(roc_lasso))
```

```
)
# Plot importance
plot(lasso_imp, top = 10)
```



```
#set seed
set.seed(seed)
# build model
ridge_model <- train(
  has_disease ~ .,
  data      = train_data,
  method    = "glmnet",
  tuneGrid  = ridge_grid,
  metric    = global_metric,
  trControl = ctrl,
  preProcess = process_strat
)
#best tuning parameters
ridge_best <- ridge_model$bestTune
# Testing set performance
ridge_preds <- predict(ridge_model, test_data)
ridge_probs <- predict(ridge_model, test_data, type = "prob")[, "Yes"]
roc_ridge   <- roc(test_data$has_disease, ridge_probs)
```

b. Ridge Penalization

```

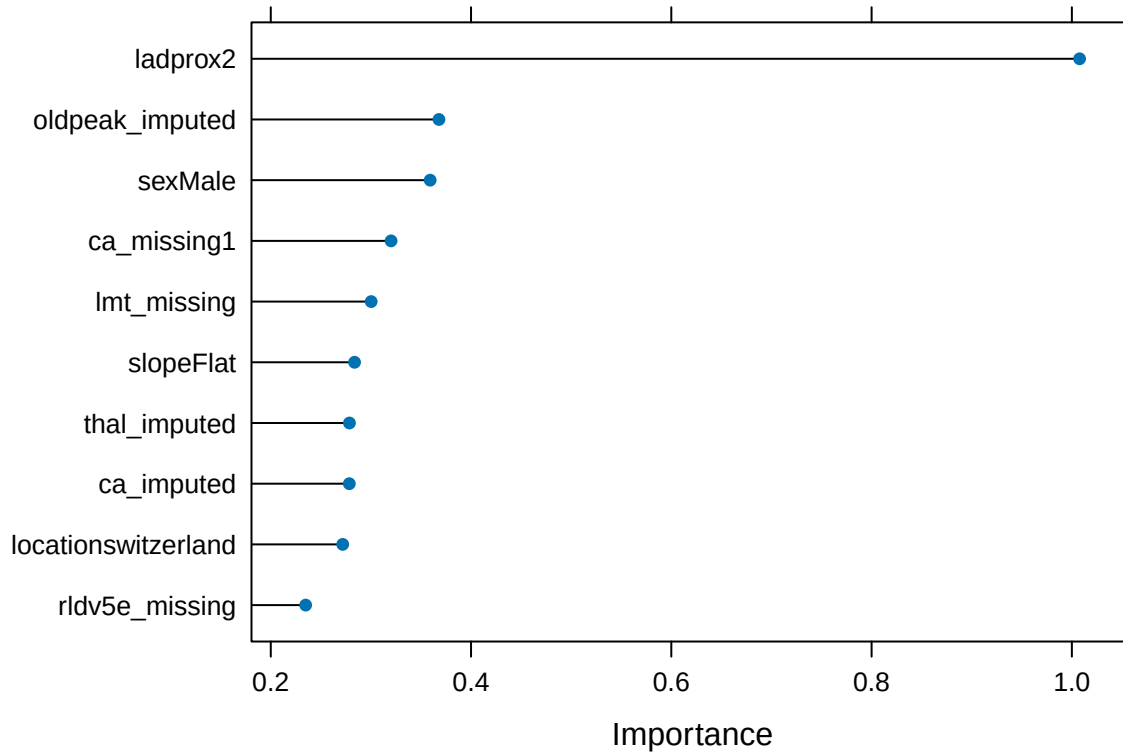
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
ridge_cm      <- confusionMatrix(ridge_preds, test_data$has_disease,
                                positive = "Yes")

# training set performance
ridge_train_preds <- predict(ridge_model, train_data)
ridge_train_probs <- predict(ridge_model, train_data, type = "prob")[, "Yes"]
roc_ridge_train  <- roc(train_data$has_disease, ridge_train_probs)

## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
ridge_cm_train  <- confusionMatrix(ridge_train_preds, train_data$has_disease,
                                positive = "Yes")

# Feature Importance
ridge_imp <- varImp(ridge_model, scale = FALSE)
# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model      = "Ridge",
    Accuracy   = ridge_cm$overall["Accuracy"],
    Sensitivity = ridge_cm$byClass["Sensitivity"],
    Specificity = ridge_cm$byClass["Specificity"],
    ROC_AUC    = as.numeric(auc(roc_ridge))
  )
# Plot importance
plot(ridge_imp, top = 10)

```

```
#set seed
set.seed(seed)
# train model
enet_model <- train(
  has_disease ~ .,
  data       = train_data,
  method     = "glmnet",
  tuneGrid   = enet_grid,
  metric     = global_metric,
  trControl  = ctrl,
  preProcess = process_strat
)
# best tuning parameters
enet_best <- enet_model$bestTune
# Testing set performance
enet_preds <- predict(enet_model, test_data)
enet_probs <- predict(enet_model, test_data, type = "prob")[, "Yes"]
roc_enet   <- roc(test_data$has_disease, enet_probs)
```

c. Elastic Net Model

```
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
```

```

enet_cm      <- confusionMatrix(enet_preds, test_data$has_disease,
                                positive = "Yes")

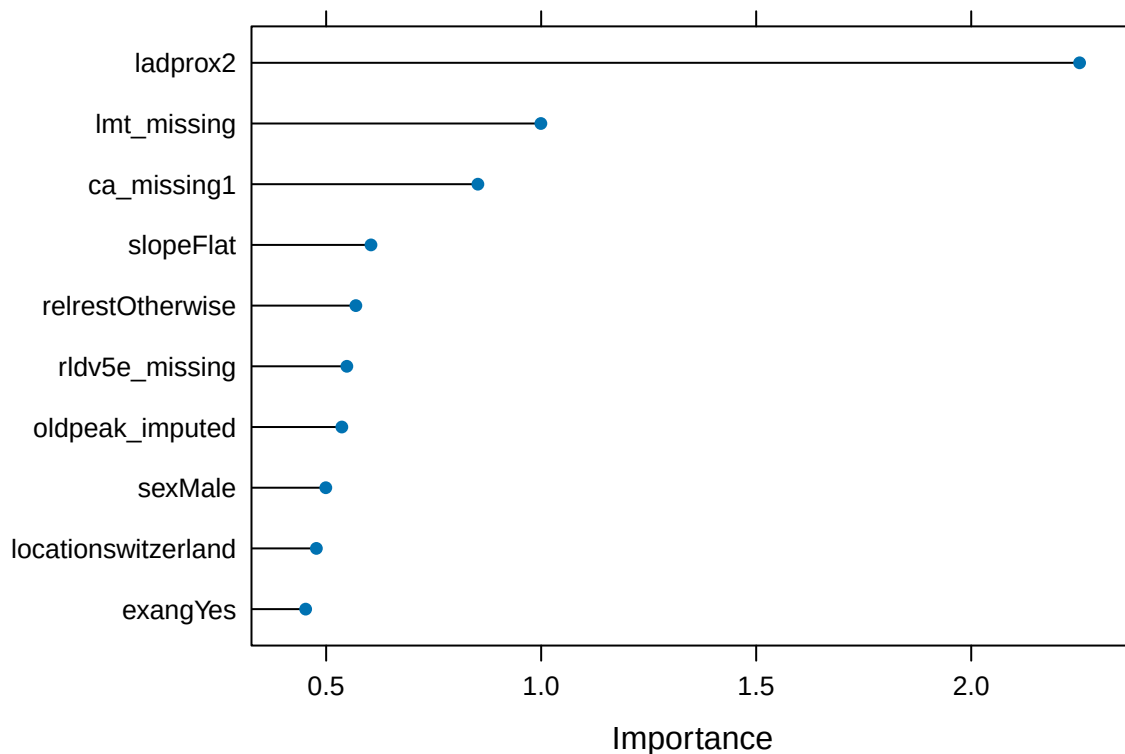
# Training set performance
enet_train_preds <- predict(enet_model, train_data)
enet_train_probs <- predict(enet_model, train_data, type = "prob")[, "Yes"]
roc_enet_train  <- roc(train_data$has_disease, enet_train_probs)

## Setting levels: control = No, case = Yes
## Setting direction: controls < cases

enet_cm_train  <- confusionMatrix(enet_train_preds, train_data$has_disease,
                                positive = "Yes")

# enet imp
enet_imp <- varImp(enet_model, scale = FALSE)
# append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model      = "Elastic Net",
    Accuracy   = enet_cm$overall["Accuracy"],
    Sensitivity = enet_cm$byClass["Sensitivity"],
    Specificity = enet_cm$byClass["Specificity"],
    ROC_AUC    = as.numeric(auc(roc_enet))
  )
#plot important features
plot(enet_imp, top = 10)

```



Results Summary and ROC Comparison

```
# ROC results comparison
knitr::kable(results_tbl, digits = 3, caption = "Model Performance Comparison")
```

Table 1: Model Performance Comparison

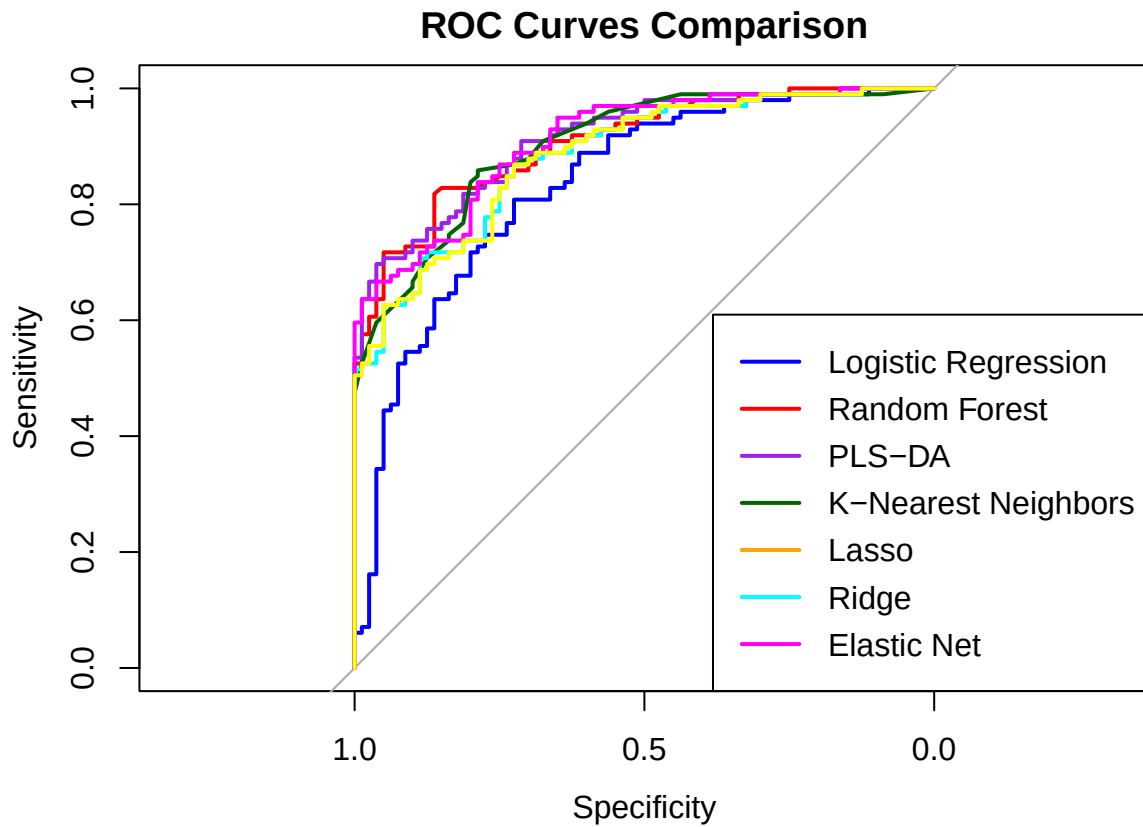
Model	Accuracy	Sensitivity	Specificity	ROC_AUC
Logistic Regression	0.749	0.727	0.775	0.834
Random Forest	0.821	0.788	0.863	0.907
PLS-DA	0.804	0.798	0.812	0.911
K-Nearest Neighbors	0.821	0.848	0.787	0.902
Lasso	0.760	0.737	0.787	0.884
Ridge	0.799	0.808	0.787	0.909
Elastic Net	0.760	0.737	0.787	0.884

```
plot(roc_glm, col = "blue", lwd = 2, main = "ROC Curves Comparison")
lines(roc_rf, col = "red", lwd = 2)
lines(roc_pls, col = "purple", lwd = 2)
lines(roc_knn, col = "darkgreen", lwd = 2)
lines(roc_lasso, col = "cyan", lwd = 2)
lines(roc_ridge, col = "magenta", lwd = 2)
lines(roc_enet, col = "yellow", lwd = 2)

legend("bottomright",
      legend = results_tbl$Model,
      col = c("blue", "red", "purple", "darkgreen", "orange", "cyan", "magenta", "yellow"),
      lwd = 2)
```

Model Performance Comparison

Model	Accuracy	Sensitivity	Specificity	ROC_AUC
Logistic Regression	0.749	0.727	0.775	0.834
Random Forest	0.821	0.788	0.863	0.907
PLS-DA	0.804	0.798	0.812	0.911
K-Nearest Neighbors	0.821	0.848	0.787	0.902
Lasso	0.760	0.737	0.787	0.884
Ridge	0.799	0.808	0.787	0.909
Elastic Net	0.760	0.737	0.787	0.884



```
#results_tbl with gt()
results_tbl %>%
  gt() %>%
  fmt_number(
    columns = c(Accuracy, Sensitivity, Specificity, ROC_AUC),
    decimals = 3) %>%
  tab_header(title = "Model Performance Comparison")
```